

IT5005

Artificial Intelligence

Lecture 5: Search Algorithms

Topics Covered

5.1 Motivation

5.2 Case Study

5.3 The Search Problem

5.4 Search Algorithms

5.5 Uninformed Search Algorithms

5.5.1 General Tree Search Algorithm

5.5.2 Breadth First Search

5.5.3 Depth First Search

5.5.4 Dijkstra's Algorithm.

5.6 Informed Search Algorithms

Topics Covered

5.6 Informed Search Algorithms

5.6.1 Greedy Best First Search

5.6.2 A* Search

5.6.3 Admissibility of Search Heuristics

Search Algorithms

MOTIVATION

5.1 Motivation

- We saw how logical agents are implemented:
 - Agents sense the world and create “percept statements” which are inserted into the KB.
 - Agents then apply inference rules to understand the world around it (e.g. is square (2, 2) safe in the Wumpus world?)
 - Agents plan accordingly to accomplish goals (explore maze, shoot Wumpus, take gold, etc.)

```
function HYBRID-WUMPUS-AGENT(percept) returns an action
inputs: percept, a list, [stench,breeze,glitter,bump,scream]
persistent: KB, a knowledge base, initially the atemporal “wumpus physics”
             t, a counter, initially 0, indicating time
             plan, an action sequence, initially empty

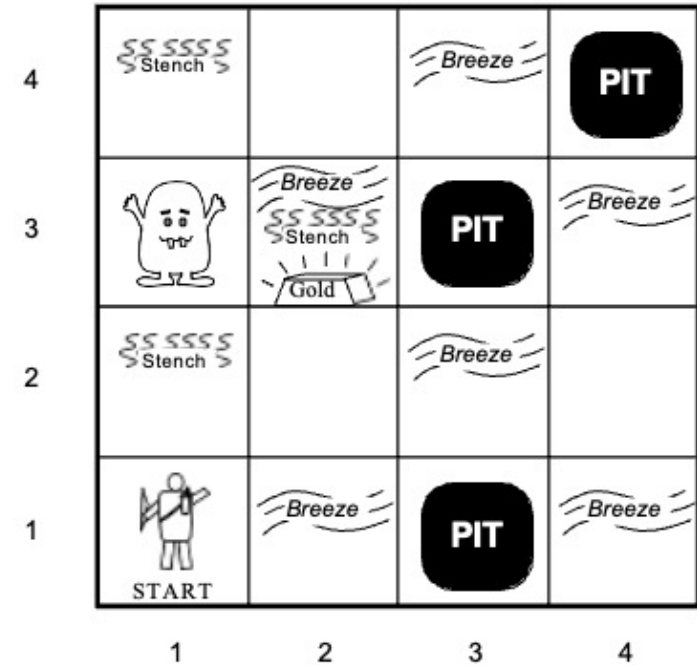
TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
TELL the KB the temporal “physics” sentences for time t
safe ← { [x, y] : ASK(KB,  $OK_{x,y}^t$ ) = true }
if ASK(KB, Glittert) = true then
    plan ← [Grab] + PLAN-ROUTE(current, {[1,1]}, safe) + [Climb]
if plan is empty then
    unvisited ← { [x, y] : ASK(KB,  $L_{x,y}^{t'}$ ) = false for all  $t' \leq t$  }
    plan ← PLAN-ROUTE(current, unvisited ∩ safe, safe)
if plan is empty and ASK(KB, HaveArrowt) = true then
    possible_wumpus ← { [x, y] : ASK(KB,  $\neg W_{x,y}$ ) = false }
    plan ← PLAN-SHOT(current, possible_wumpus, safe)
if plan is empty then // no choice but to take a risk
    not_unsafe ← { [x, y] : ASK(KB,  $\neg OK_{x,y}^t$ ) = false }
    plan ← PLAN-ROUTE(current, unvisited ∩ not_unsafe, safe)
if plan is empty then
    plan ← PLAN-ROUTE(current, {[1,1]}, safe) + [Climb]
action ← POP(plan)
TELL(KB, MAKE-ACTION-SENTENCE(action, t))
t ← t + 1
return action

function PLAN-ROUTE(current, goals, allowed) returns an action sequence
inputs: current, the agent’s current position
        goals, a set of squares; try to plan a route to one of them
        allowed, a set of squares that can form part of the route

problem ← ROUTE-PROBLEM(current, goals, allowed)
return SEARCH(problem) // Any search algorithm from Chapter 3
```

5.1 Motivation

- We immediately see a problem:
 - We can “Ask” the KB to make inferences about the world around the agent.
 - Suppose our sensors see glitter. We need to do two things:
 - Pick up the gold. Can do a rule that simply says $\forall(i,j) (Glitter_{i,j}^t \rightarrow PickGold_{i,j})$
 - Return to start.
 - Need to generate a series of *NORTH, SOUTH, EAST, WEST* commands to get from the current square to (1, 1) while avoiding squares with pits or the Wumpus.
 - Very difficult to do with logic.

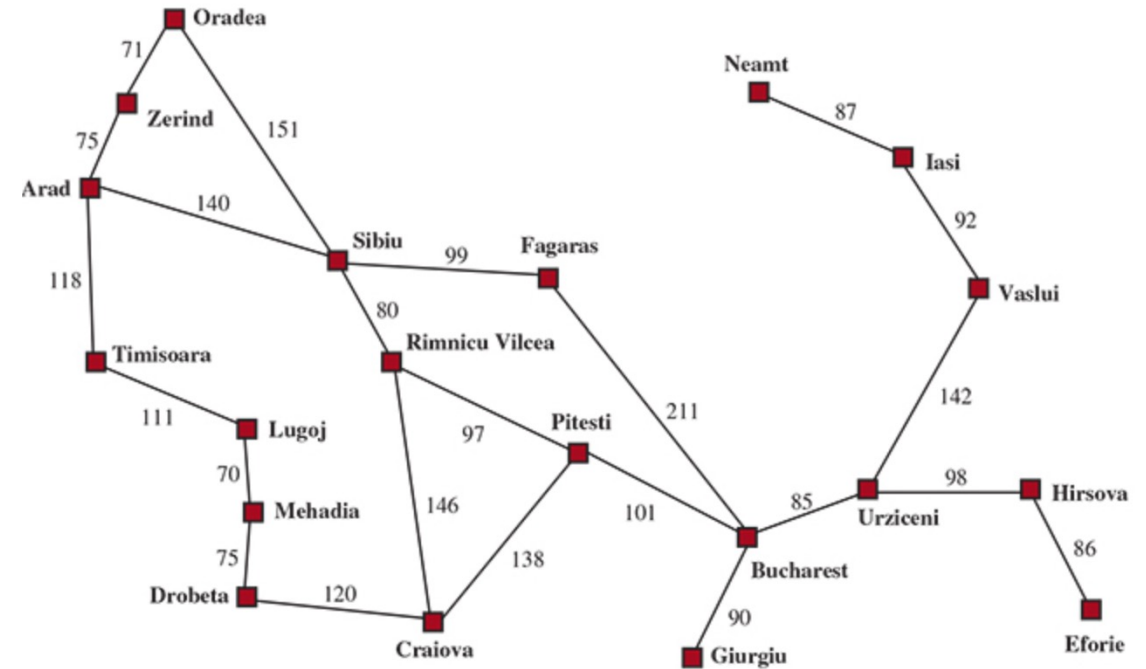


5.1 Motivation

- To perform this we need to do “planning”.
 - For now we will assume that the agent has a full view of the world, i.e. there are no hidden unknowns.
- In this lecture we will model planning as a search problem and explore two classes of search algorithms:
 - Uninformed Searches:
 - No information about how close we are to the goal, or about how good our move is.
 - Informed Searches:
 - With each step we get feedback of how close we are to the goal, or how good the move is.

5.2 Case Study

- We will consider an agent who wishes to tour Romania.
- Agent is currently in Arad, and must get to Bucharest.
- Agent has a complete map of Romania complete with distances.
 - I.e. Agent has a complete view of the “world”, in this case all the roads between major cities in Romania, and their distances.



5.2 Case Study

- We define our problem formally as follows:
 - A set of possible states S the environment can be in, called the *state space*.
 - Romania Problem: The set of possible cities
 - Wumpus World: The possible statements in the KB, the objective configuration of the world.
 - The initial state S_0 , The set of starting values in our problem.
 - Romania Problem: Arad, the starting city.
 - Wumpus World: L_{11} , the starting square, *HasArrow*, etc.
 - The goal states G .
 - Romania Problem: Bucharest, the destination city.
 - Wumpus World: $KB \models HasGold \wedge L_{11}$

5.2 Case Study

- The set of actions $A(s)$ available to the agent at each state s .
 - We say these are the actions that are relevant to state s
 - Romania: $A(\text{Arad}) = \{\text{ToSibiu}, \text{ToTimisoara}, \text{ToZerind}\}$
 - Wumpus World: $A(L_{11}) = \{\text{North}, \text{East}, \text{Shoot}\}$
- A transition function or model $T(s, a)$ that tells us the next state as a result of taking action a in state s .
 - Romania: $T(\text{Arad}, \text{ToZerind}) = \text{Zerind}$
 - Wumpus World: The transition axioms, some of which are shown below

$$\begin{aligned} & \forall i, j \left(L_{i,j}^t \rightarrow (\text{North}^t \rightarrow L_{i+1,j}^{t+1}) \right) \\ & \forall i, j \left(L_{i,j}^t \rightarrow (\text{South}^t \rightarrow L_{i-1,j}^{t+1}) \right) \\ & \forall i, j \left(L_{i,j}^t \rightarrow (\text{East}^t \rightarrow L_{i,j+1}^{t+1}) \right) \\ & \forall i, j \left(L_{i,j}^t \rightarrow (\text{West}^t \rightarrow L_{i,j-1}^{t+1}) \right) \\ & \text{HasArrow}^t \rightarrow (\text{Shoot}^t \rightarrow \sim \text{HasArrow}^{t+1}) \\ & \sim \text{HasArrow}^t \rightarrow \sim \text{HasArrow}^{t+1} \\ & \forall i, j \left(\text{FacingEast}^t \rightarrow (\text{Shoot}^t \rightarrow (W_{i,j+1} \rightarrow \sim \text{WumpusAlive}^{t+1})) \right) \\ & \sim \text{WumpusAlive}^t \rightarrow \sim \text{WumpusAlive}^{t+1} \end{aligned}$$

5.2 Case Study

- Action cost $c(s, a, s')$. The cost of applying action a in state s to reach state s' .
 - Romania: $c(\text{Arad}, \text{ToSibiu}, \text{Sibiu}) = 140$
 - Wumpus: $c(L_{11}, \text{North}, L_{21}) = 1$

5.3 The Search Problem

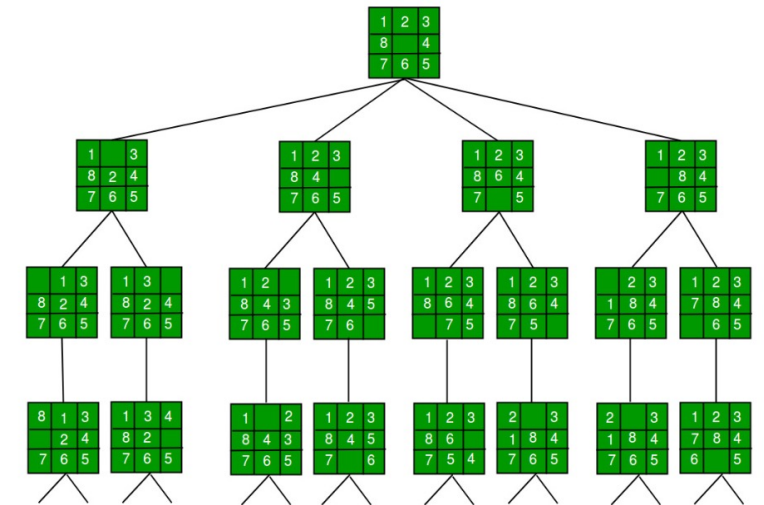
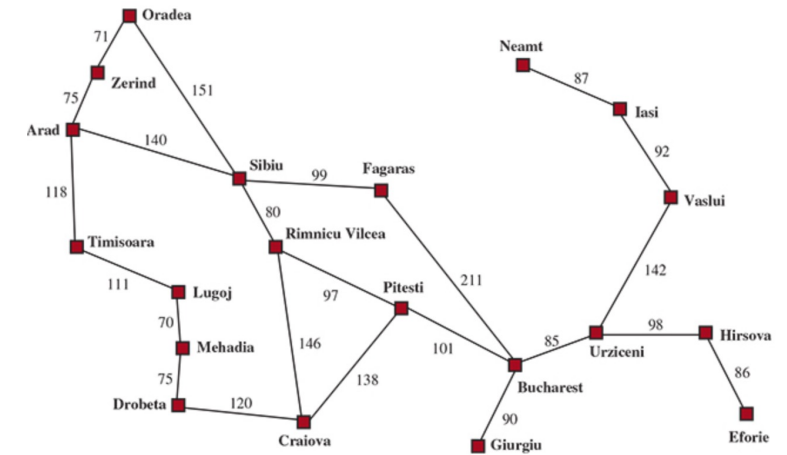
- Our "problem solving" strategy can thus be described as:
 - Starting from our initial state S_0
 - Apply a series of actions $a_0 \in A(S_0), a_1 \in A(S_1), \dots, a_n \in A(S_n)$
 - Note: A series of actions is called a "path".
 - Until we reach some state in G
 - While (hopefully) minimizing:

$$\sum_{i=0}^{n-1} c(s_i, a_i, s_{i+1})$$

5.3 The Search Problem

■ Issues:

- ❑ State space can be MASSIVE.
 - If each action opens up just 2 more options, our state space has 2^n possible states after n moves.
- ❑ Not all actions lead to the goal. E.g.
 - In Romania going to Vaslui will not bring us to Bucharest.
 - In Wumpus World, stepping into a square with a pit or a live Wumpus will kill us.
- ❑ We may run out of resources on some paths.
 - If we have 300 miles of fuel, going Arad->Sibiu->Fagaras->Bucharest will cause us to run out of fuel.



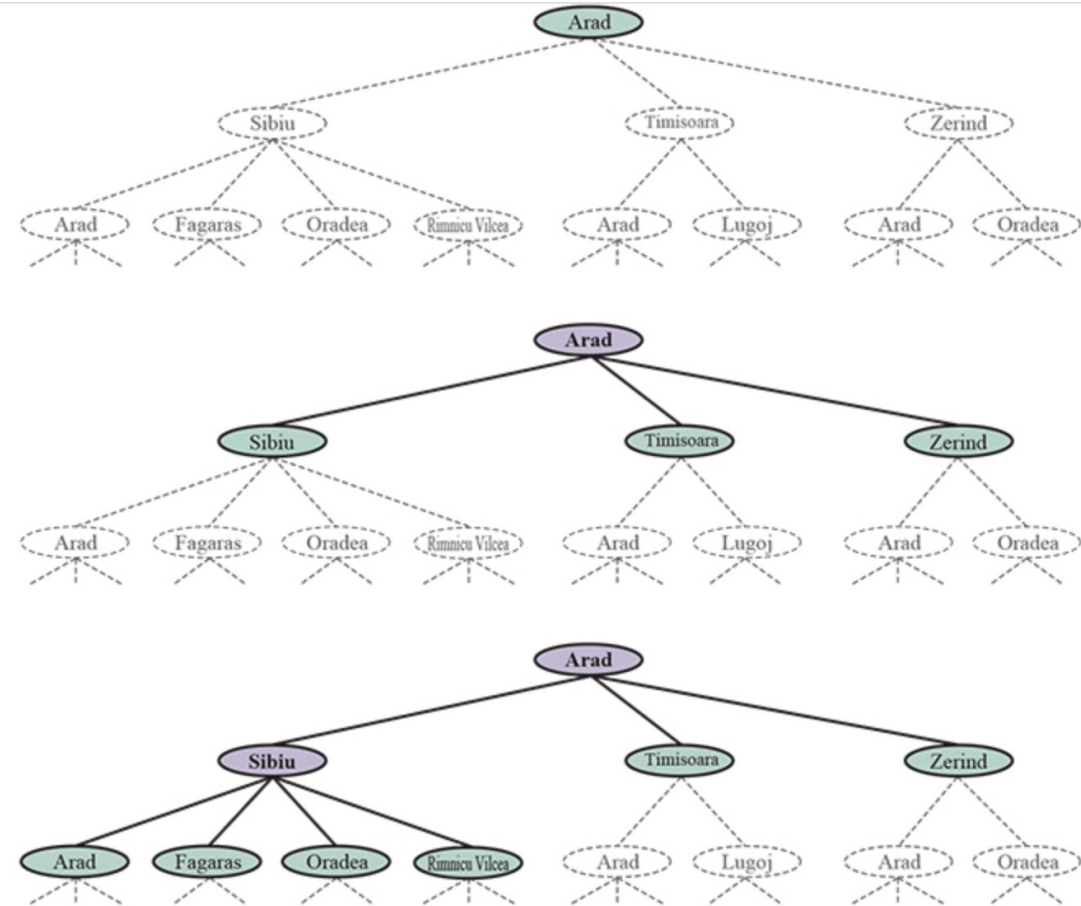
5.3 The Search Problem

- For this reason we need to use search algorithms that:
 - ❑ Sensibly expand the set of options available,
 - ❑ See if it leads to the goal state,
 - ❑ While (hopefully) minimizing costs.
- Formally, a search algorithm:
 - ❑ Takes a search problem as inputs.
 - ❑ Returns a solution in the form of a series of actions to be taken, OR
 - ❑ Returns failure if no such solution exists.

5.3 The Search Problem

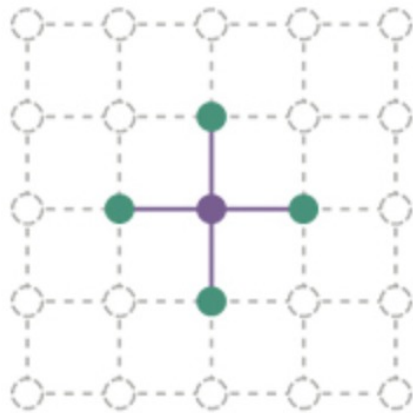
- We begin by first overlaying a tree on the problem, based on the possible set of actions at each state and their consequences:

- Some nodes have been expanded:
 - the options reachable from those nodes have been generated.
 - E.g. Arad in the second tree, Arad and Sibiu in the third tree.
- Some nodes are generated but not yet expanded (nodes in green).
- These are called “frontier nodes”
- Frontier nodes and expanded nodes are collectively called “reached nodes”.
 - Arad is visited and expanded.
 - Fagaras has been visited but not expanded.

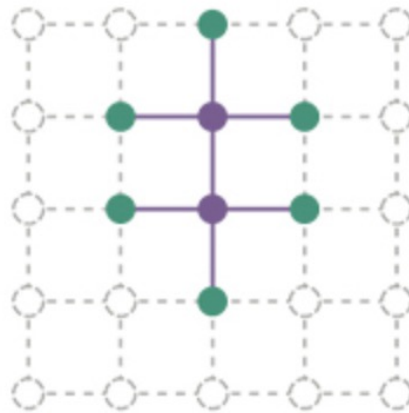


5.3 The Search Problem

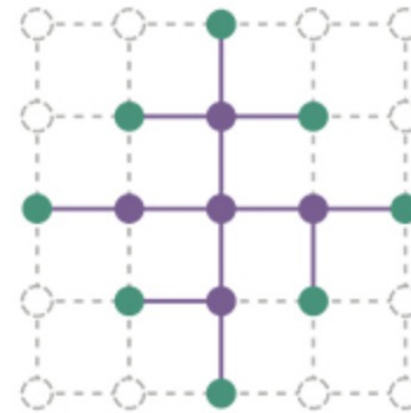
- The Separation Property of Search Algorithms
 - Frontier nodes separate interior “expanded” nodes from exterior “unreached” nodes



(a)



(b)



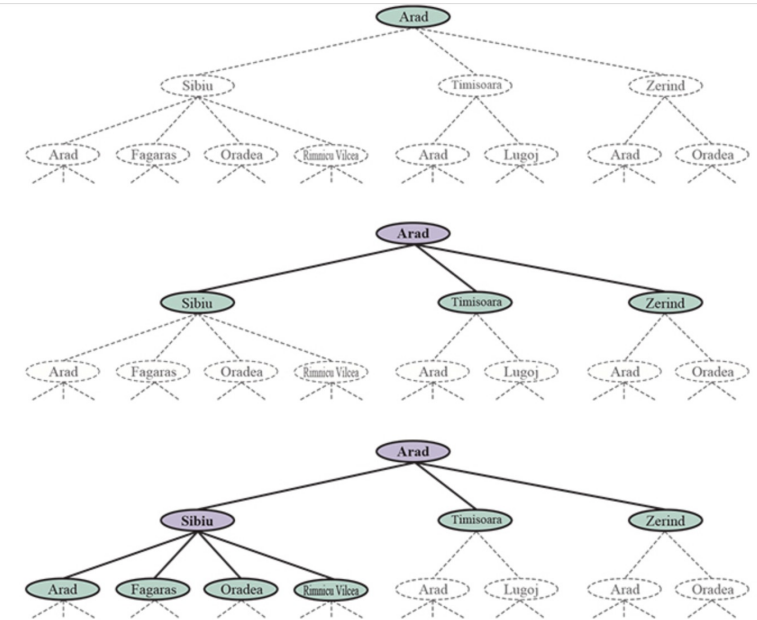
(c)

5.4 Search Algorithms

- Having seen how to model our search problem as a tree, and having seen the definition of reached, expanded and frontier nodes, we are now ready to examine search algorithms.
- Some terminology:
 - State: A representation of the physical configuration. E.g. being in square L_{11} with an arrow in the Wumpus World.
 - Node: A data structure that is part of the search tree, containing information about:
 - The parent node, children, depth, path cost.

5.4 Search Algorithms

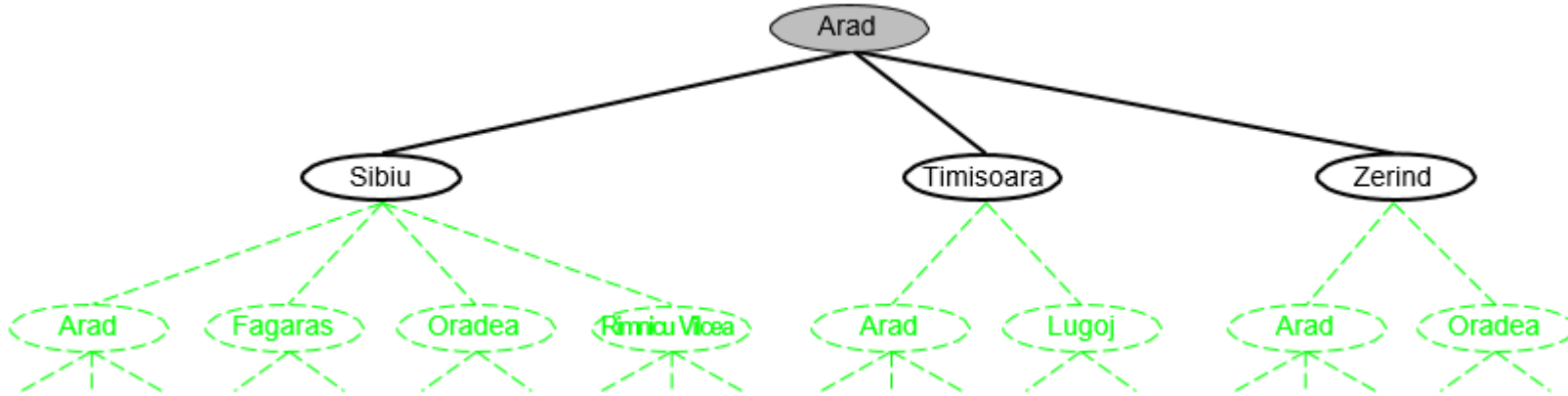
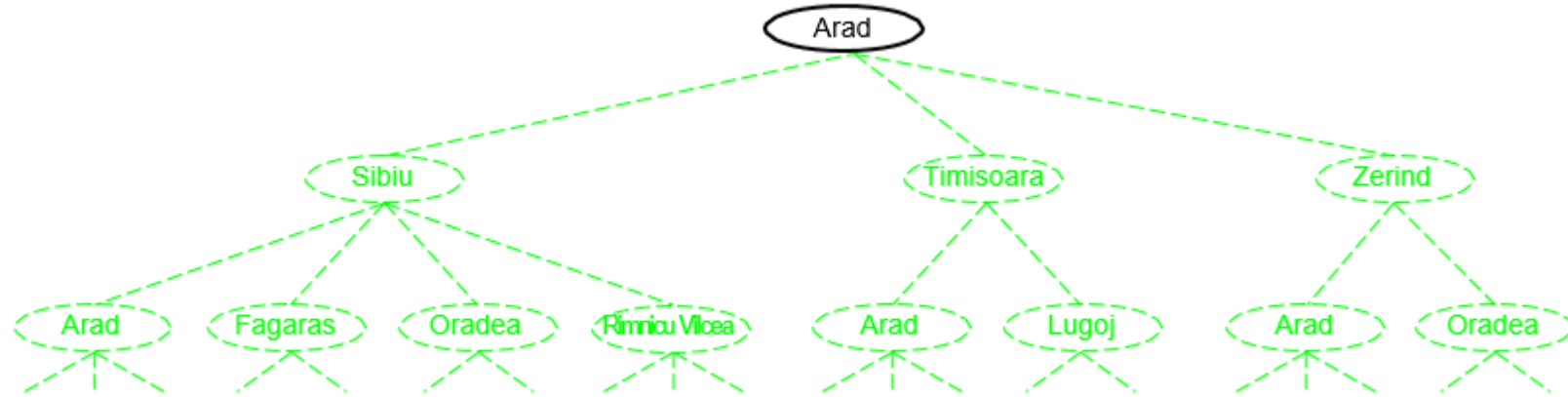
- Knowing who the parent is allows us to generate the solution.
- We start from the goal node, then work out who the parent is, who the parent of the parent is, etc. until we reach the starting node.
- The path generated shows us the actions needed to get from the starting node to the goal.
- E.g. if the parent of Bucharest is Fagaras, the parent of Fagaras is Sibiu, and the parent of Sibiu is Arad, then we have the path Arad-Sibiu-Fagaras-Bucharest.
 - We can now generate the actions $a(\text{Arad}) = \text{GoSibiu}$, $a(\text{Sibiu}) = \text{goFagaras}$, $a(\text{Fagaras}) = \text{GoBucharest}$



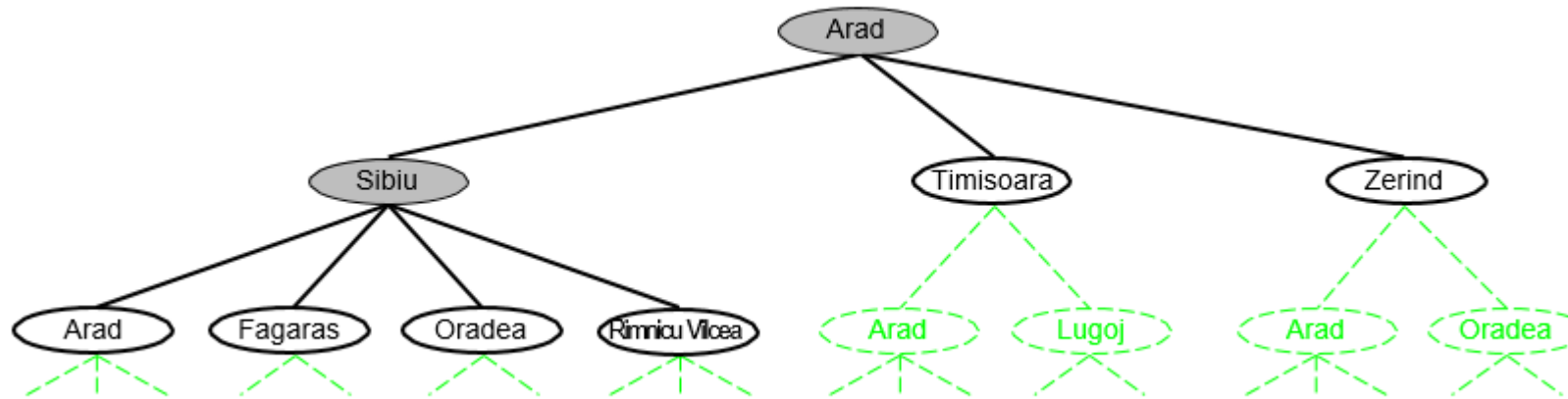
5.5 Uninformed Search Algorithms

- We will now look at some uninformed search algorithms:
 - ❑ Generalized Tree Search Algorithm (aka “Best-Fit”)
 - ❑ Breadth First Search
 - ❑ Depth First Search
 - ❑ Dijkstra’s Algorithm

5.5.1 General Tree Search



5.5.1 General Tree Search



5.5.1 General Tree Search

```
function Tree-Search(problem, fringe) returns a solution, or failure
  fringe ← Insert (Make-Node(Initial-State [problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← Remove-Front(fringe)
    if Goal-Test(problem, State(node)) then return node
    fringe ← Insert All(Expand(node, problem), fringe)
```

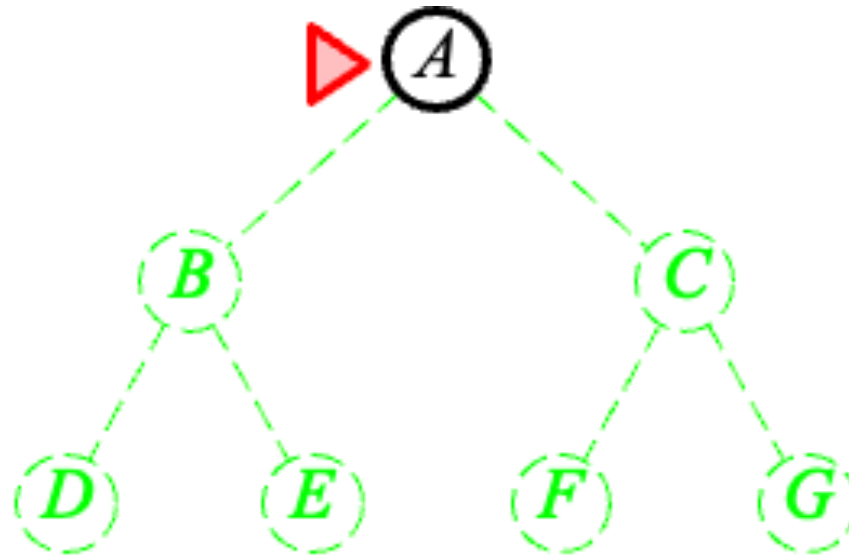
```
function Expand(node, problem) returns a set of nodes
  successors ← the empty set
  for each action, result in Successor-Fn(problem, State [node]) do
    s ← a new Node
    Parent-Node[s] ← node; Action[s] ← action; State[s] ← result
    Path-Cost[s] ← Path-Cost[node] + Step-Cost(node, action, s)
    Depth[s] ← Depth[node] + 1
    add s to successors
  return successors
```

Basic idea:

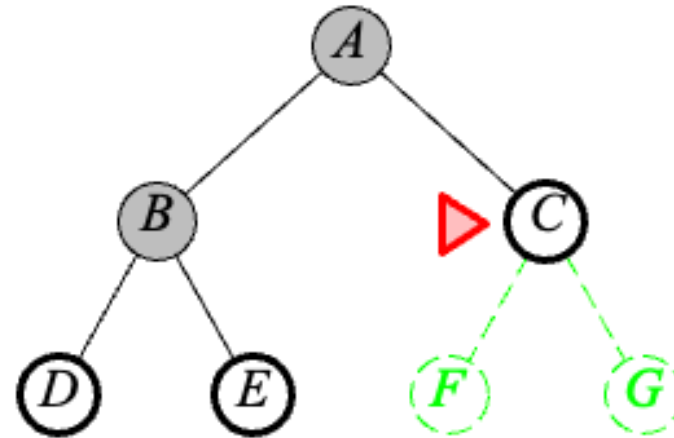
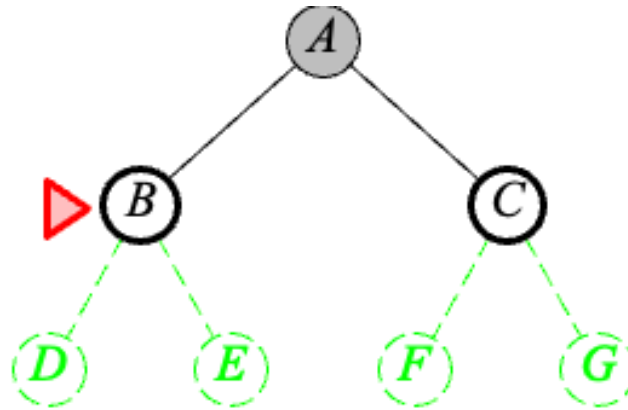
- Remove node from the fringe – the set of nodes that have been visited but not expanded.
- Apply every relevant action for that node and get the result.
 - Create a new node, store the current node as parent.
 - Store the action that allowed us to get here.
 - Store the outcome.
- Add all generated nodes to the fringe for evaluation.

5.5.2 Breadth First Search

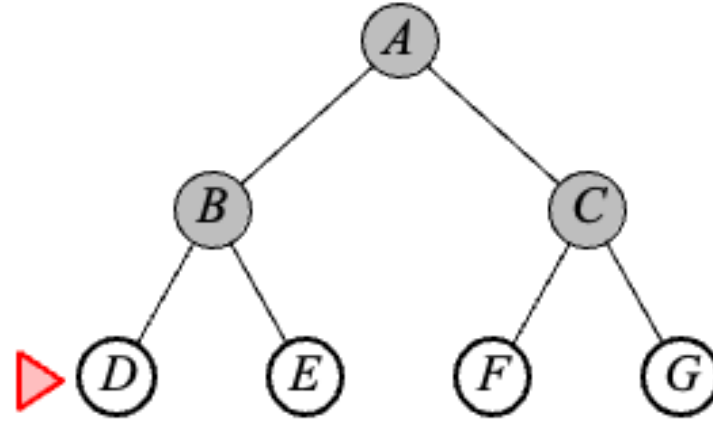
- Strategy: Expand the shallowest unexpanded node.
 - Implement the fringe as a FIFO queue. New nodes are added to the end.



5.5.2 Breadth First Search



5.5.2 Breadth First Search

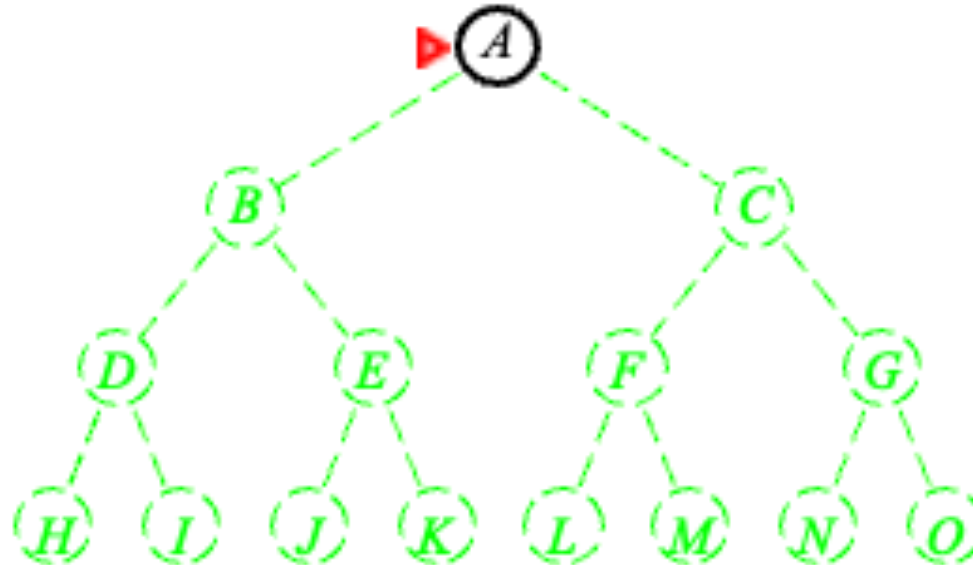


5.5.2 Breadth First Search

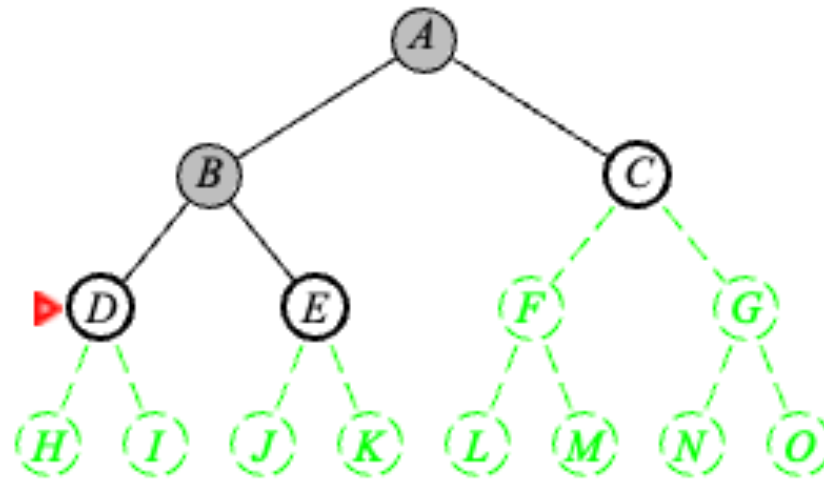
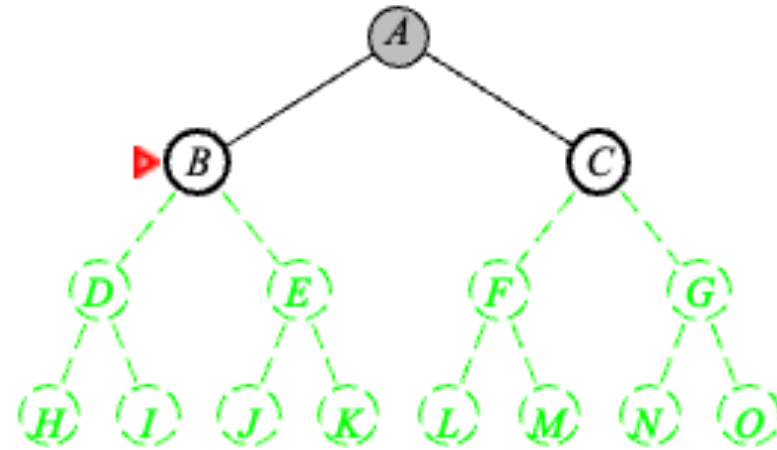
```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure  
  node  $\leftarrow$  NODE(problem.INITIAL)  
  if problem.IS-GOAL(node.STATE) then return node  
  frontier  $\leftarrow$  a FIFO queue, with node as an element  
  reached  $\leftarrow$  {problem.INITIAL}  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    for each child in EXPAND(problem, node) do  
      s  $\leftarrow$  child.STATE  
      if problem.IS-GOAL(s) then return child  
      if s is not in reached then  
        add s to reached  
        add child to frontier  
  return failure
```

5.5.3 Depth First Search

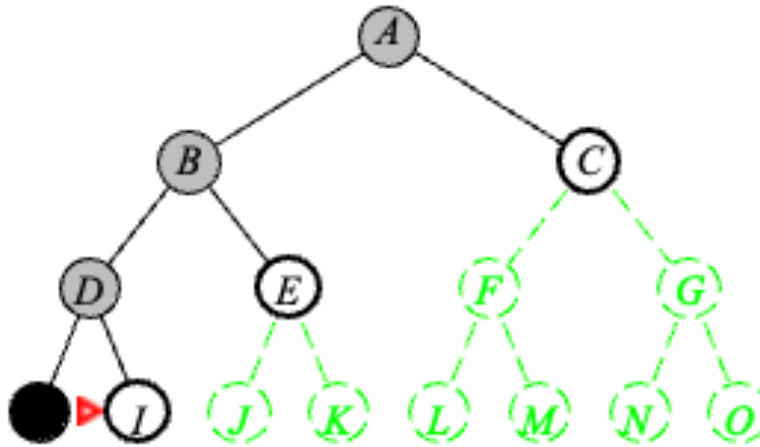
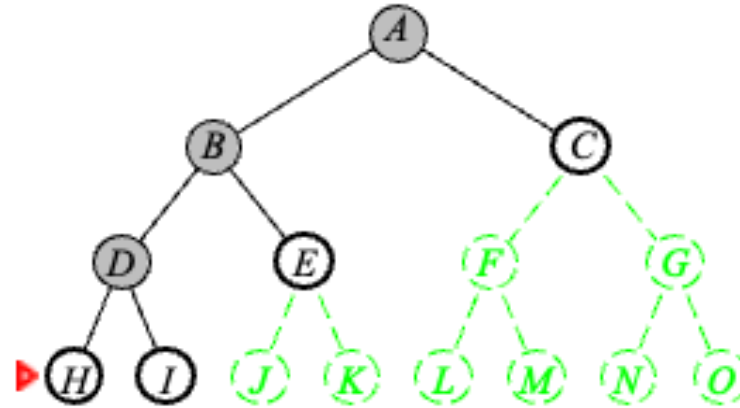
- Strategy: Expand deepest unexpanded node.
 - Implement fringe as a LIFO queue. Newly generated nodes are put in front.



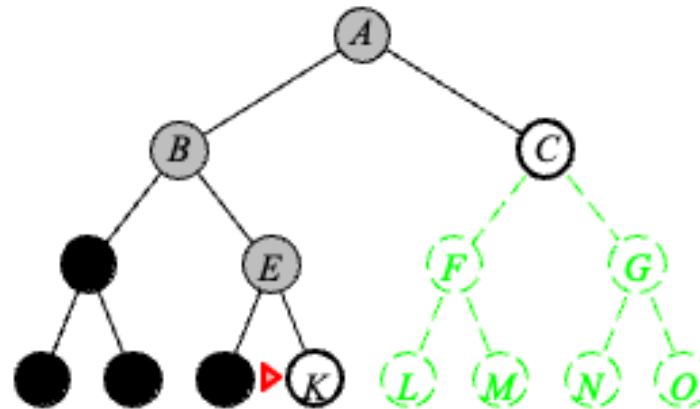
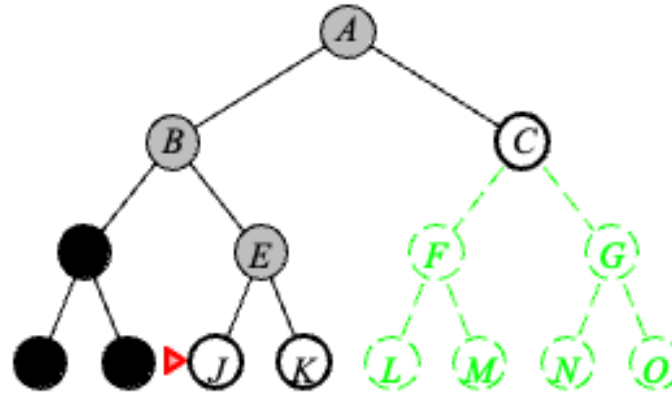
5.5.3 Depth First Search



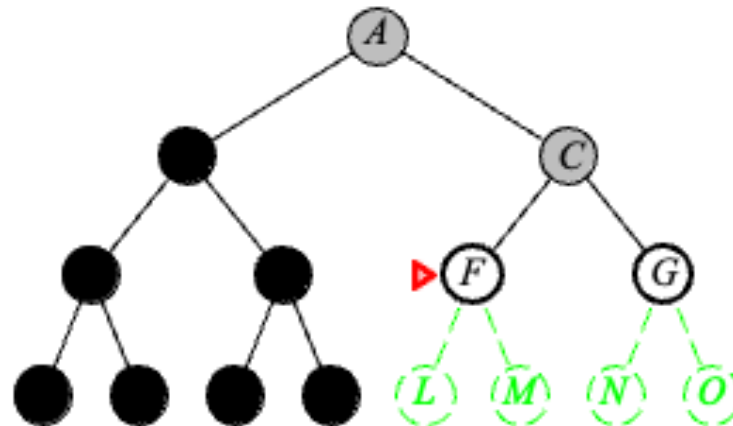
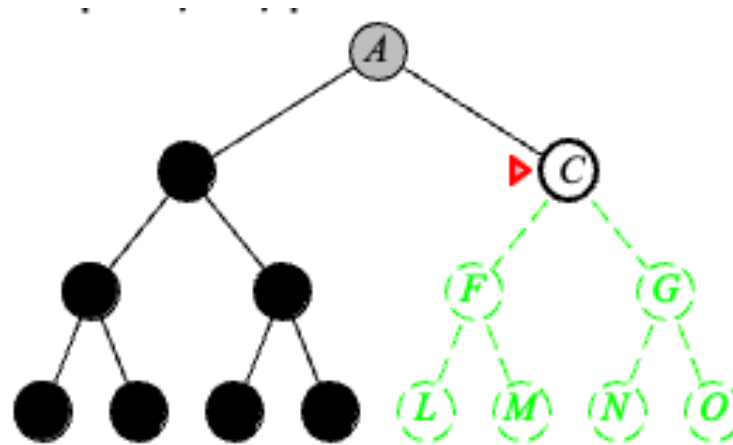
5.5.3 Depth First Search



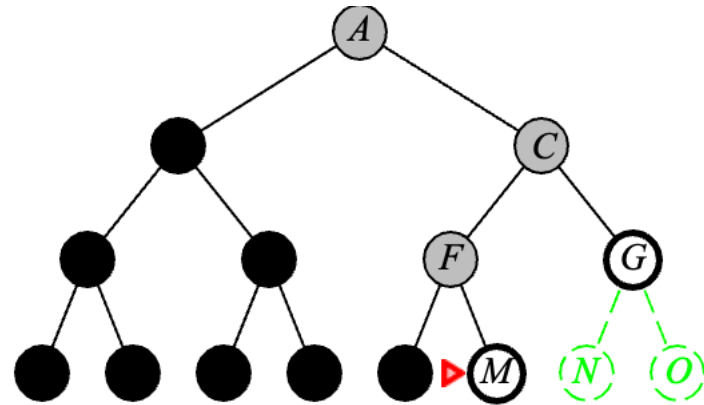
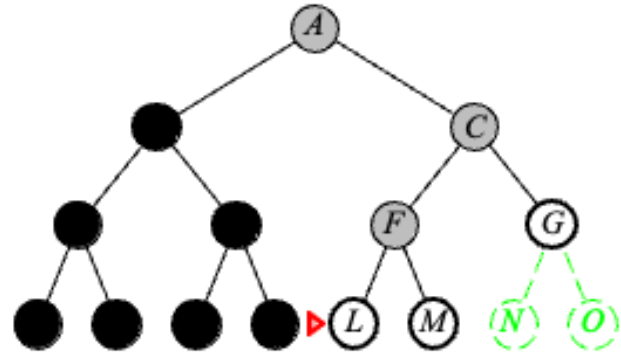
5.5.3 Depth First Search



5.5.3 Depth First Search



5.5.3 Depth First Search



5.5.3 Depth First Search

■ Issues with Depth First Search

- ❑ If there are cycles in the graph, may end up expanding nodes infinitely.
 - Must ensure that you do not visit nodes that have already been visited.
- ❑ In infinite search spaces, you can potentially traverse down an infinitely long path.
 - Does not happen in breadth first search since you would look across the width of the tree first, which is at most $2n$ nodes, where n is the number of parent nodes.

5.5.3 Depth First Search

- Use iterative deepening to solve the “infinite depth” problem.

- Idea:

- Do DFS on a tree with 2 levels, then 3 levels, then 4 levels etc till we find the goal node.

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution node or failure  
  for depth = 0 to  $\infty$  do  
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)  
    if result  $\neq$  cutoff then return result
```

```
function DEPTH-LIMITED-SEARCH(problem,  $\ell$ ) returns a node or failure or cutoff  
  frontier  $\leftarrow$  a LIFO queue (stack) with NODE(problem.INITIAL) as an element  
  result  $\leftarrow$  failure  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    if problem.IS-GOAL(node.STATE) then return node  
    if DEPTH(node) >  $\ell$  then  
      result  $\leftarrow$  cutoff  
    else if not IS-CYCLE(node) do  
      for each child in EXPAND(problem, node) do  
        add child to frontier  
  return result
```

5.5.4 Dijkstra's Algorithm

- Dijkstra's Algorithm finds the shortest path between any two points on a graph, as long as all the costs are positive.
 - Every vertex u is given a label $L(u)$ which is the current shortest path from the origin a to u . Initially $L(u) = \infty$. However $L(a) = 0$ where a is the starting node.
 - For every vertex v that is added to the path:
 - For all vertices u adjacent to v (e.g. u represents a state that is reached from v after applying action $a \in A(v)$):
 - If $L(v) + c(v, a, u) < L(u)$, then $L(u) = L(v) + c(v, a, u)$, where $L(v)$ is the least cost from the starting node to v , and $c(v, a, u)$ is the cost of applying action $a \in A(v)$ while at node v to get to u . Note that $c(v, a, u)$ is equivalent to $w(v, u)$ which is the “weight” of the edge connecting v to u .
 - The u with the smallest $L(u)$ is added to the path.
 - Intuitively, we replace the cost of getting to a node u with a lower cost when we find one, because we've effectively found a “cheaper” way to get there.

5.5.4 Dijkstra's Algorithm

Inputs:

- G [a connected simple graph with positive weight for every edge]
- ∞ [a number greater than the sum of the weights of all the edges in G]
- $w(u, v)$ [the weight of edge $\{u, v\}$]
- a [the source vertex]
- z [the destination vertex]

Algorithm:

1. Initialize T to be the graph with vertex a and no edges.
Let $V(T)$ be the set of vertices of T , and let $E(T)$ be the set of edges of T .
2. $L(a) \leftarrow 0$, and for all vertices u in G except a , $L(u) \leftarrow \infty$.
[The number $L(u)$ is called the **label** of u .]
3. Initialize $v \leftarrow a$ and $F \leftarrow \{a\}$. [The symbol v is used to denote the vertex most recently added to T .]

5.5.4 Dijkstra's Algorithm

Let $\text{Adj}(x)$ denote the set of vertices adjacent to vertex x .

4. while ($z \notin V(T)$)

a. $F \leftarrow (F - \{v\}) \cup \{\text{vertices} \in \text{Adj}(v) \text{ and } \notin V(T)\}$

[The set F is the set of fringe vertices.]

b. For each vertex $u \in \text{Adj}(v)$ and $\notin V(T)$,
if $L(v) + w(v, u) < L(u)$ then
 $L(u) \leftarrow L(v) + w(v, u)$
 $D(u) \leftarrow v$

[The notation $D(u)$ is introduced to keep track of which vertex in T gave rise to the smaller value.]

c. Find a vertex x in F with the smallest label.

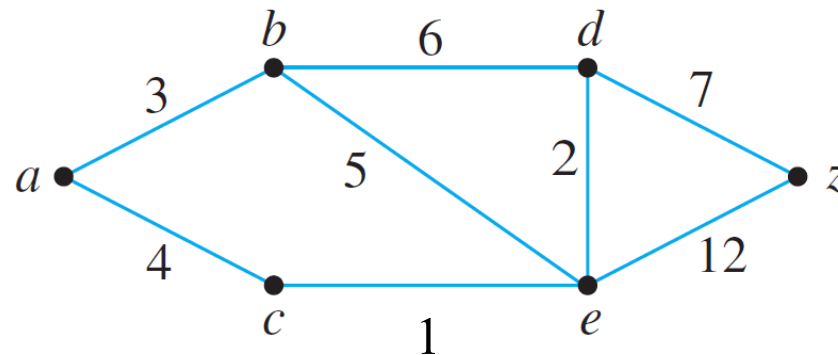
Add vertex x to $V(T)$, and add edge $\{D(x), x\}$ to $E(T)$.

$v \leftarrow x$

Output: $L(z)$ [this is the length of the shortest path from a to z .]

5.5.4 Dijkstra's Algorithm

Example: Show the steps in the execution of Dijkstra's shortest path algorithm for the graph shown below with starting vertex *a* and ending vertex *z*.



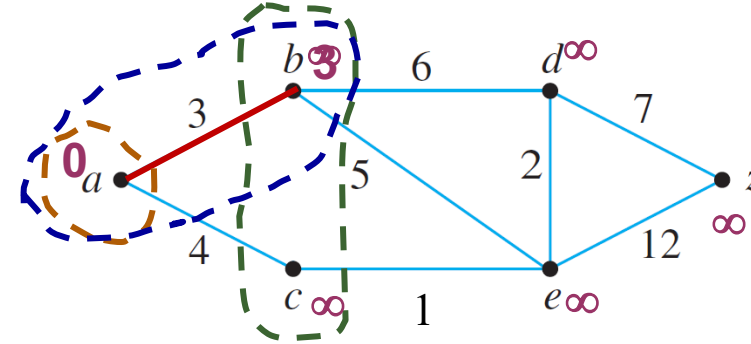
5.5.4 Dijkstra's Algorithm

Step 1: Going into the **while** loop:
 $V(T) = \{a\}$, $E(T) = \emptyset$, and $F = \{a\}$

During iteration:

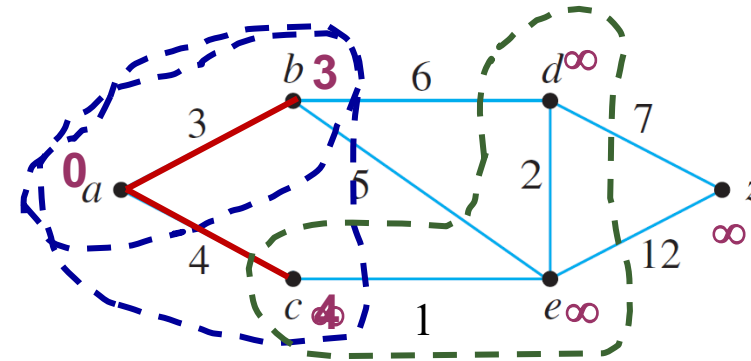
$F = \{b, c\}$, $L(b) = 3$, $L(c) = 4$.

Since $L(b) < L(c)$, b is added to $V(T)$ and $\{a, b\}$ is added to $E(T)$.



5.5.4 Dijkstra's Algorithm

Step 2: Going into the **while**
loop:
 $V(T) = \{a, b\}$, $E(T) = \{\{a, b\}\}$



During iteration:

$F = \{c, d, e\}$, $L(c) = 4$, $L(d) = 9$, $L(e) = 8$.

Since $L(c) < L(d)$ and $L(c) < L(e)$, c is added to $V(T)$ and $\{a, c\}$ is added to $E(T)$.

5.5.4 Dijkstra's Algorithm

Step 3: Going into the **while** loop:

$$V(T) = \{a, b, c\},$$

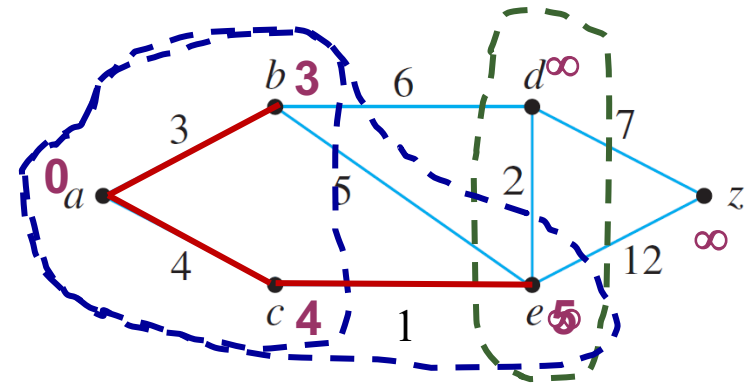
$$E(T) = \{\{a, b\}, \{a, c\}\}$$

During iteration:

$$F = \{d, e\}, L(d) = 9, L(e) = 5.$$

$L(e)$ becomes 5 because ace , which has length 5, is a shorter path to e than abe , which has length 8.

Since $L(e) < L(d)$, e is added to $V(T)$ and $\{c, e\}$ is added to $E(T)$.



5.5.4 Dijkstra's Algorithm

Step 4: Going into the **while** loop:

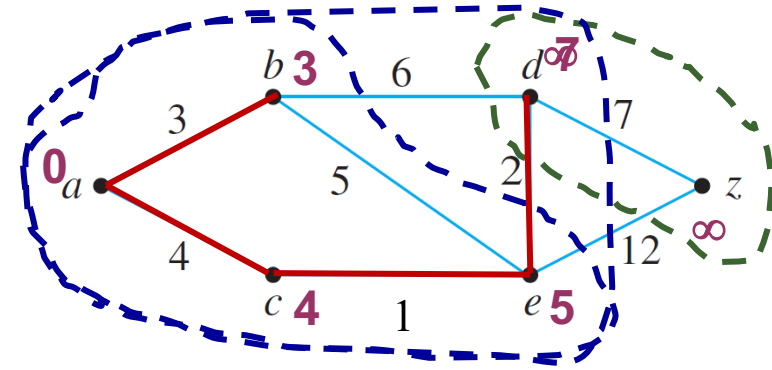
$$V(T) = \{a, b, c, e\},$$

$$E(T) = \{\{a, b\}, \{a, c\}, \{c, e\}\}$$

During iteration:

$$F = \{d, z\}, L(d) = 7, L(z) = 17.$$

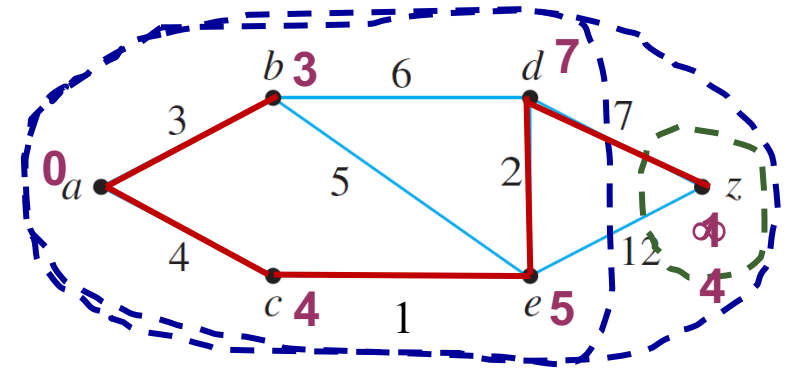
$L(d)$ becomes 7 because $aced$, which has length 7, is a shorter path to d than abd , which has length 9. Since $L(d) < L(z)$, d is added to $V(T)$ and $\{e, d\}$ is added to $E(T)$.



5.5.4 Dijkstra's Algorithm

Step 5: Going into the **while**

loop:

$$V(T) = \{a, b, c, e, d\},$$
$$E(T) = \{\{a, b\}, \{a, c\}, \{c, e\}, \{e, d\}\}$$


During iteration:

$$F = \{z\}, L(z) = 14.$$

$L(z)$ becomes 14 because $acedz$, which has length 14, is a shorter path to z than $acez$, which has length 17. Since z is the only vertex in F , its label is a minimum, and so z is added to $V(T)$ and $\{d, z\}$ is added to $E(T)$.

Algorithm terminates at this point because $z \in V(T)$.
The shortest path from a to z has length $L(z) = 14$.

5.5.4 Dijkstra's Algorithm

Keeping track of the steps in a table is a convenient way to show the action of Dijkstra's algorithm. Table 10.7.1 does this for the graph in the previous example.

Step	$V(T)$	$E(T)$	F	$L(a)$	$L(b)$	$L(c)$	$L(d)$	$L(e)$	$L(z)$
0	$\{a\}$	$\emptyset$	$\{a\}$	0	∞	∞	∞	∞	∞
1	$\{a\}$	$\emptyset$	$\{b, c\}$	0	3	4	∞	∞	∞
2	$\{a, b\}$	$\{\{a, b\}\}$	$\{c, d, e\}$	0	3	4	9	8	∞
3	$\{a, b, c\}$	$\{\{a, b\}, \{a, c\}\}$	$\{d, e\}$	0	3	4	9	5	∞
4	$\{a, b, c, e\}$	$\{\{a, b\}, \{a, c\}, \{c, e\}\}$	$\{d, z\}$	0	3	4	7	5	17
5	$\{a, b, c, e, d\}$	$\{\{a, b\}, \{a, c\}, \{c, e\}, \{e, d\}\}$	$\{z\}$	0	3	4	7	5	14
6	$\{a, b, c, e, d, z\}$	$\{\{a, b\}, \{a, c\}, \{c, e\}, \{e, d\}, \{d, z\}\}$							

Table 10.7.1

5.6 Informed Search

- Informed searches are searches that are done together with a heuristic function h that gives us an idea of how close we are to the goal.
 - In our Romanian problem, a good heuristic might be the “straight line” distance between each city and Bucharest, our goal.
- The search then proceeds using h , or using h and the cost of getting to the current node.

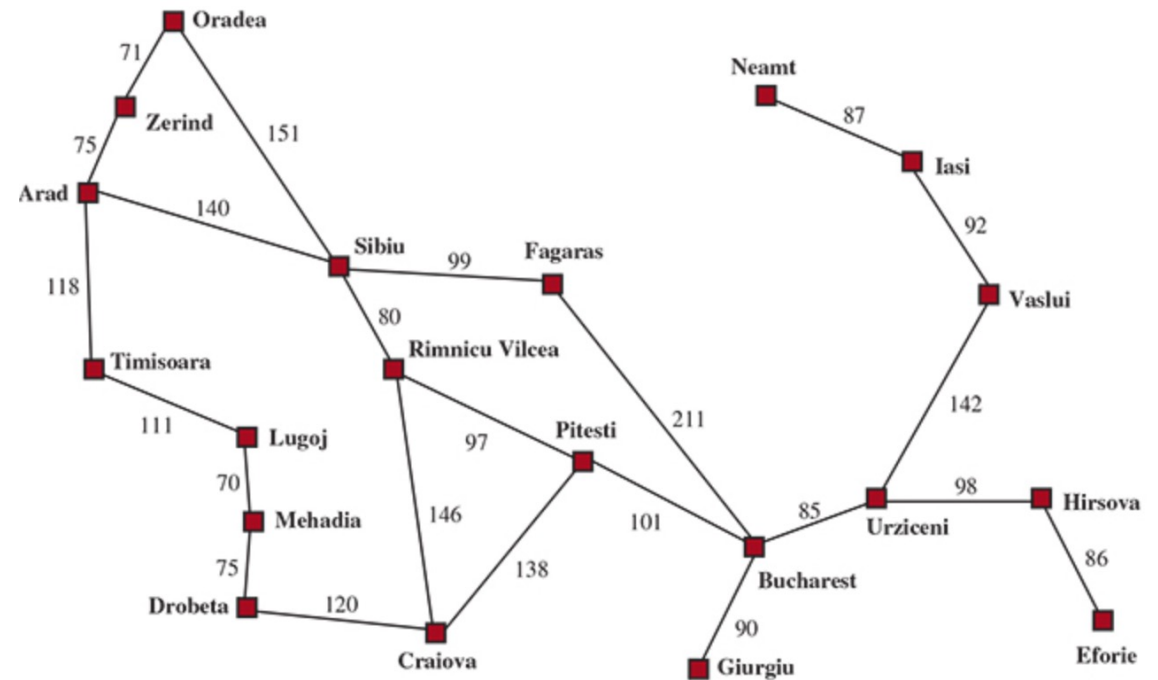
5.6 Informed Search

- We will look at two informed search algorithms:
 - Greedy Best First Search
 - A* Search

5.6.1 Greedy Best First Search

- The idea in GBFS is to choose to move as close as possible to the goal at each step – i.e. choose the step with the smallest heuristic.
- Here we use h_{SLD} , the straight-line-distance between each city and Bucharest. Our starting point is Arad.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

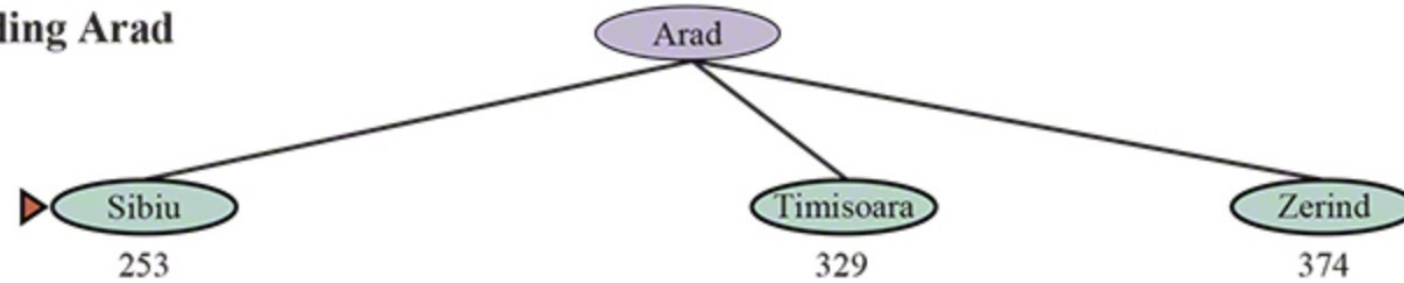


5.6.1 Greedy Best First Search

(a) The initial state

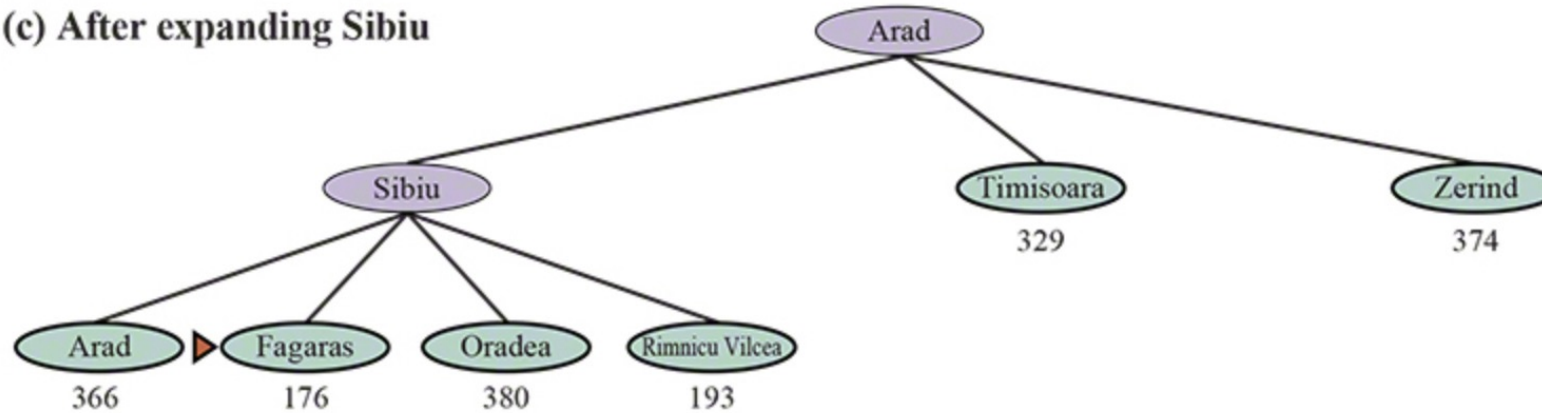


(b) After expanding Arad

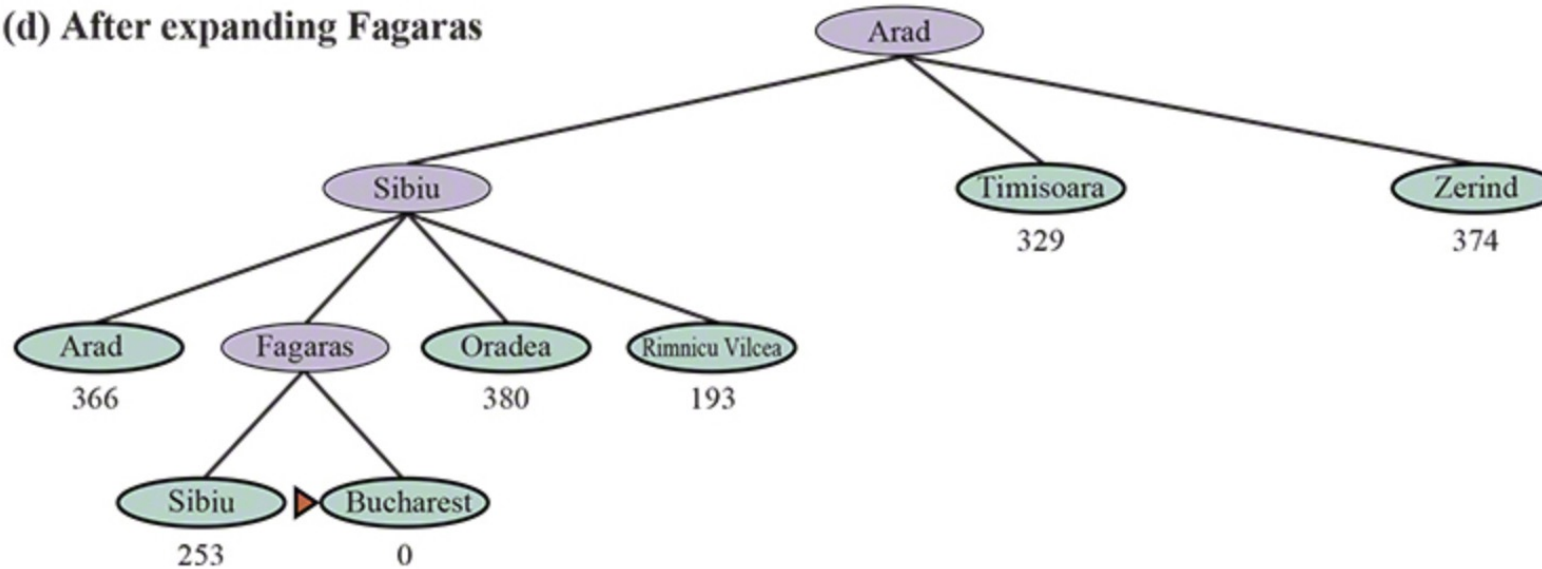


5.6.1 Greedy Best First Search

(c) After expanding Sibiu



(d) After expanding Fagaras



5.6.1 Greedy Best First Search

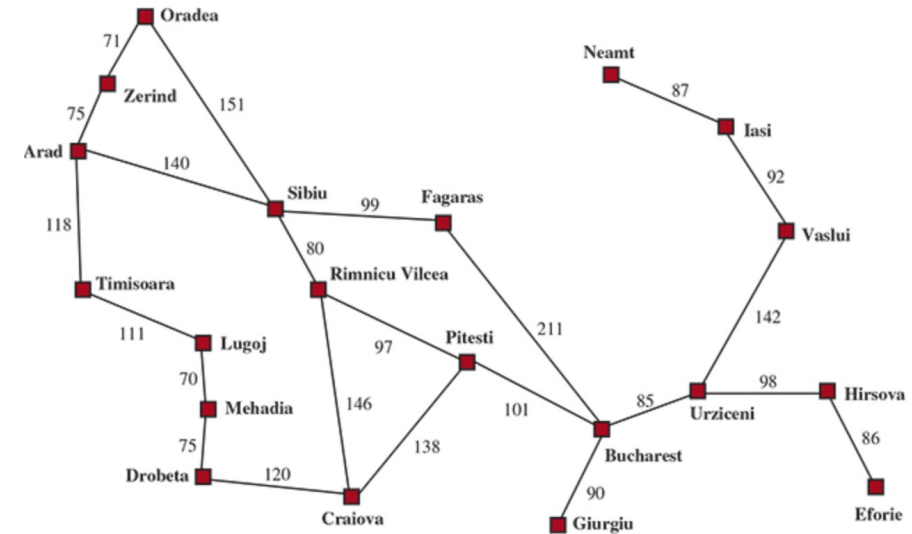
- Unfortunately GBFS does not produce an optimal solution.
 - ❑ Our current solution is Arad-Sibiu-Fagaras-Bucharest, with a total distance of $140 + 99 + 211 = 450$ miles.
 - ❑ The actual optimal solution is Arad-Sibiu-Rimincu Vilcea-Pitesti-Bulgaria, which has a distance of $140 + 80 + 97 + 101 = 418$ miles, 32 miles shorter.
- This is because greedy algorithms do not consider the “big picture”, but just greedily pick the current “best” solution.

5.6.2 A* Search

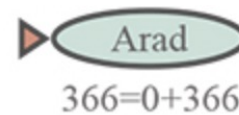
- We will now consider a search that combines both our heuristic function h and the shortest cost of getting to a node:
 - $f(n) = g(n) + h(n)$, $g(n)$ is the path cost of getting from the starting node to n , while $h(n)$ is the *estimated* cost of getting from n to the goal node.
 - We return to our Romanian Problem.

5.6.2 A* Search

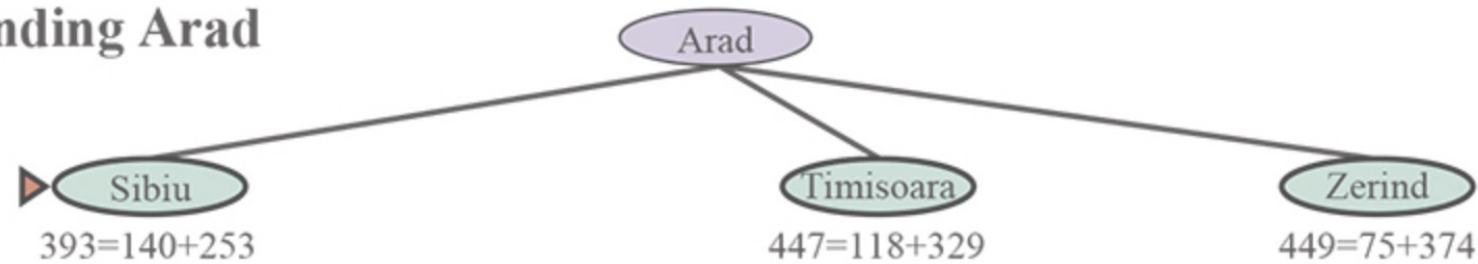
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



(a) The initial state

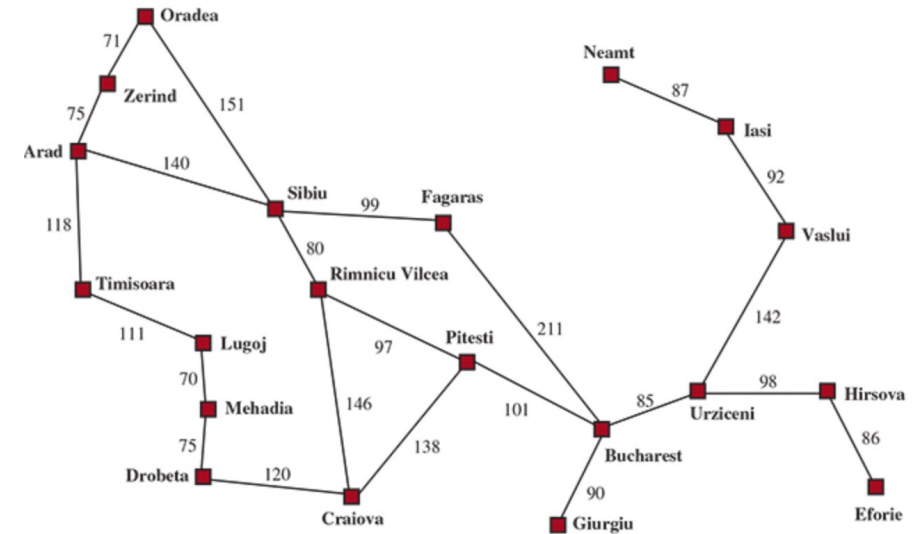


(b) After expanding Arad

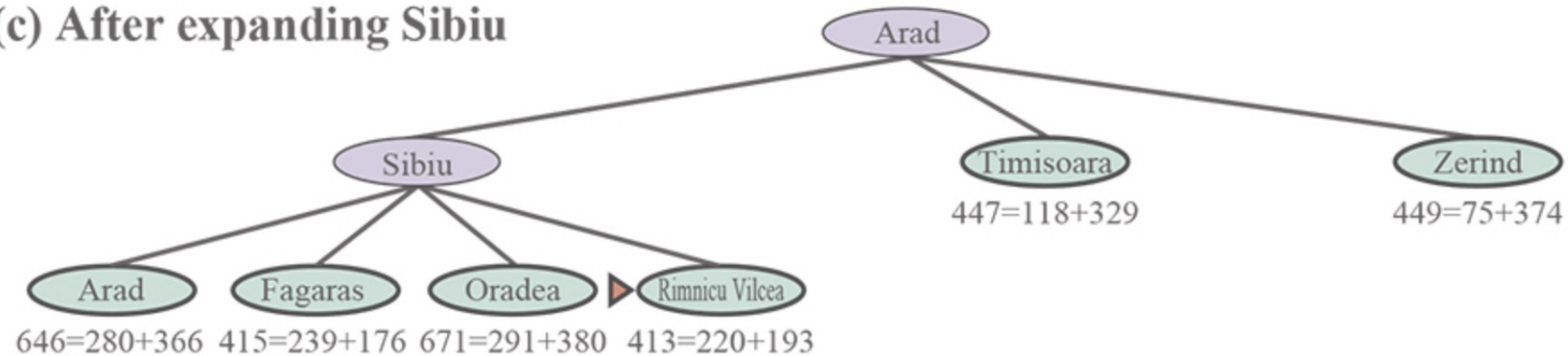


5.6.2 A* Search

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

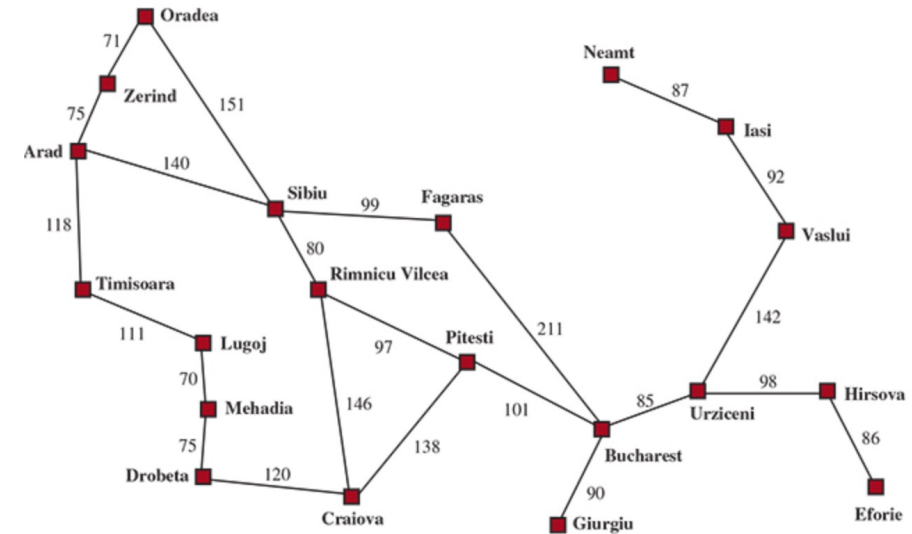


(c) After expanding Sibiu

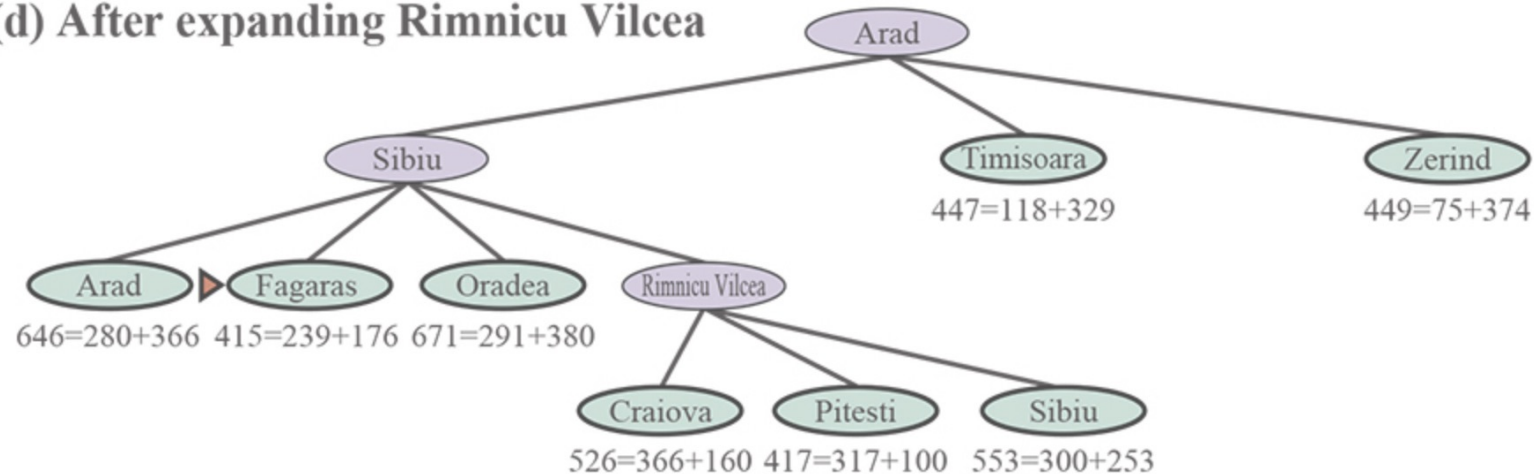


5.6.2 A* Search

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

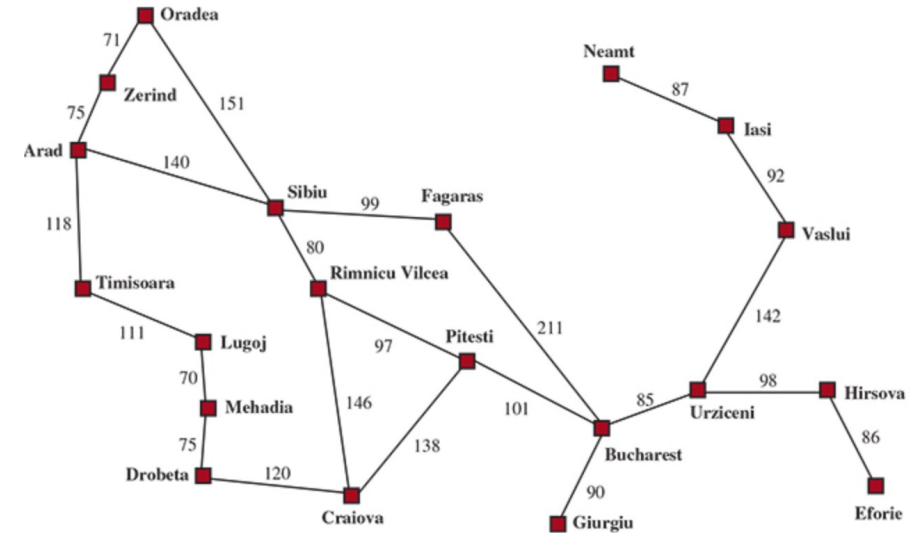


(d) After expanding Rimnicu Vilcea

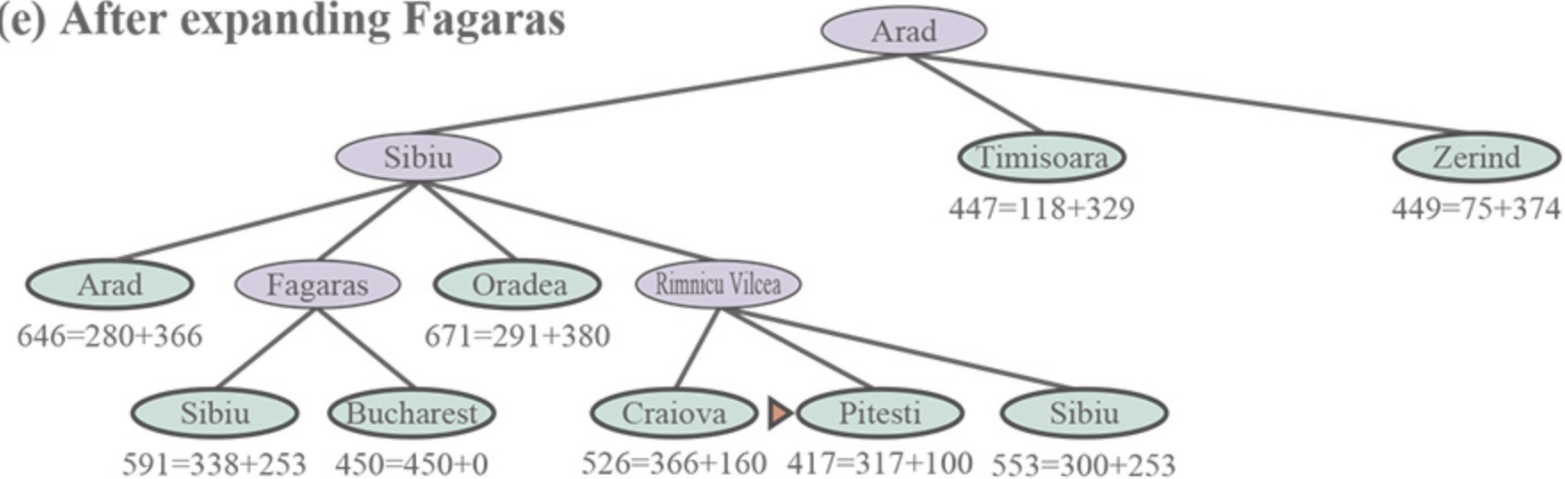


5.6.2 A* Search

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

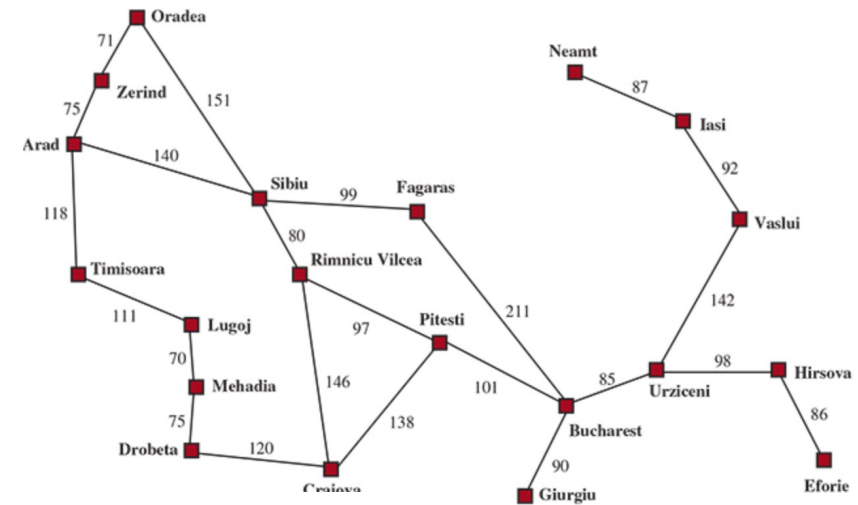


(e) After expanding Fagaras

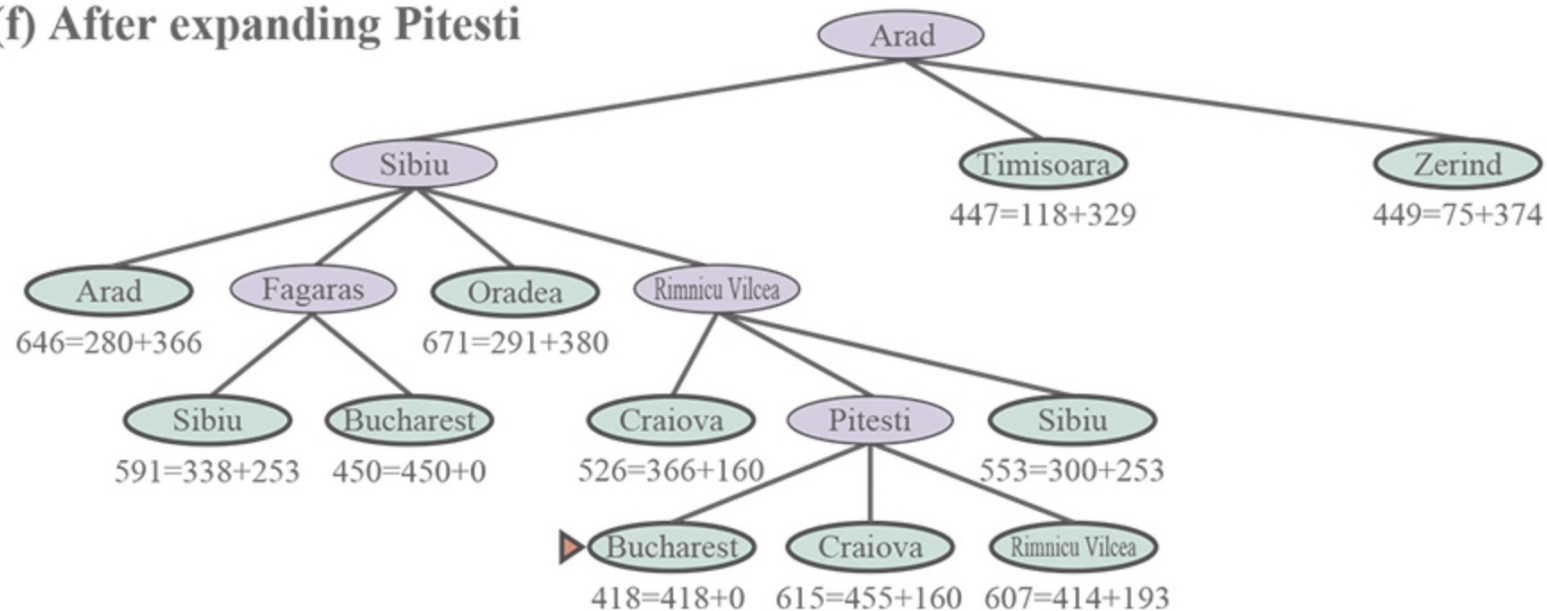


5.6.2 A* Search

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



(f) After expanding Pitesti



5.6.3 Admissibility of Heuristic Functions

- To guarantee an optimal solution, a heuristic must underestimate the actual cost of reaching the goal.
 - E.g. our straight-line distance to Bucharest will invariably be shorter than the actual distance, since roads curve, we need to go around lakes and mountains, etc.
 - A heuristic that under-estimates the true cost is called “admissible”.

5.6.3 Admissibility of Heuristic Functions

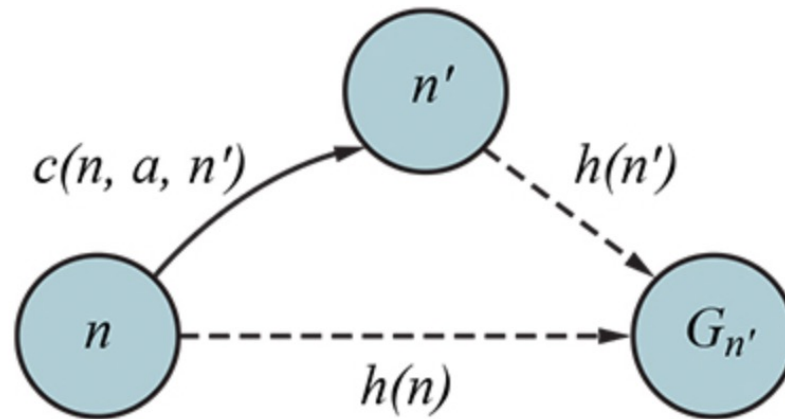
- We can prove that admissible functions always generate optimal solutions:

1. Suppose there is an optimal path with cost C^*
2. Let's suppose the A* algorithm produces a non-optimal solution with cost C . Then $C > C^*$
3. Then $\exists n$ that is on the optimal path but is unexpanded (if it was expanded it would have been chosen and we would have gotten the optimal path). Let $g^*(n)$ be the cost of the optimal path from the starting node to n , and let $h^*(n)$ be the cost of the optimal path from n to the goal.
4. Then $f(n) = g(n) + h(n) > C^*$ (otherwise n would have been chosen)
5. $f(n) = g^*(n) + h(n)$ since n is on the optimum path.
6. $f(h) \leq g^*(n) + h^*(n)$ since, by admissibility criteria, $h(n) \leq h^*(n)$
7. Then $f(h) \leq C^*$, which contradicts 4.

Thus, the assumption that it produces a sub-optimal solution is incorrect.

5.6.3 Admissibility of Heuristic Functions

- Another property is called “consistency”, which is:
$$h(n) \leq c(n, a, n') + h(n').$$
 - All consistent heuristics are admissible, but not vice-versa.
- This idea is shown below:



5.7 Conclusion

- The goal of search algorithms:
 - To find a solution to a problem by progressively searching the state space.
- Two approaches:
 - Uninformed approach: Selection is made solely based on order of expansion (breadth-first, depth-first) or on total cost (Dijkstra).
 - Informed approach: Selection is made based on a heuristic (greedy best first) or a heuristic and total cost (A^*).
- Next week we will look at adversarial algorithms.

END OF FILE

IT5005

Artificial Intelligence

Lecture 6: Adversarial Search

LOCAL SEARCH

AIMA Chapter 4.1

Local Search Algorithms

- The **path** to goal is irrelevant; the goal state itself is the solution
- State space = set of “complete” configurations
- Find final configuration satisfying constraints, e.g., n -queens
- **Local search algorithms:** maintain single “current best” state and try to improve it
- Advantages:
 - ❑ very little/constant memory
 - ❑ find reasonable solutions in large state space

Example: n -queens

- Put n queens on an $n \times n$ board with no two queens on the same row, column, or diagonal



Local search strategies

- Hill-climbing search: use of heuristic function to improve “current” state

Hill-Climbing Search

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

if *neighbor*.VALUE $\leq$ *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

“Like climbing Mt. Everest in thick fog with amnesia”

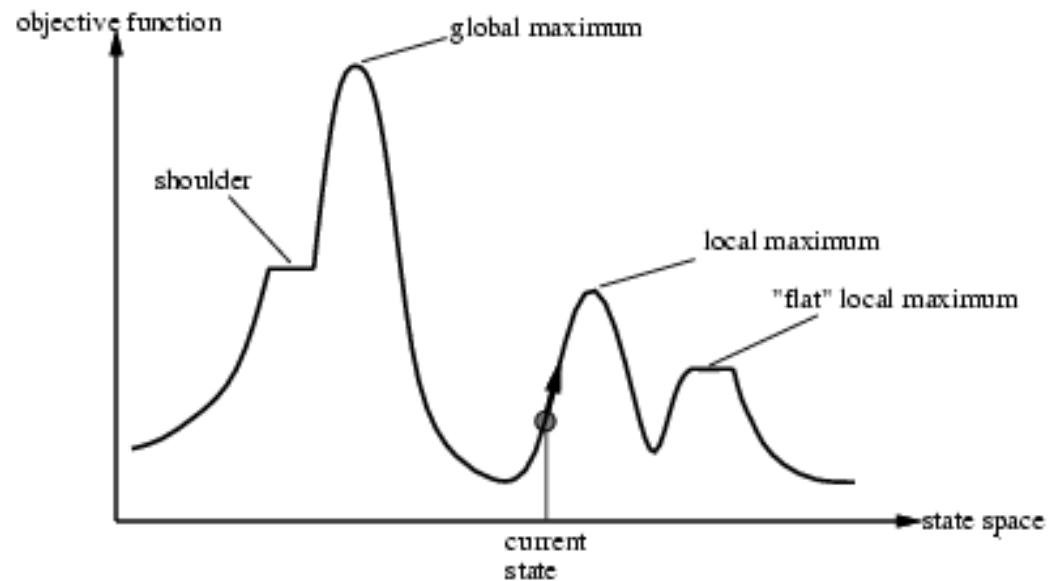
Hill-Climbing Search: 8-Queens

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14		13	16	13	16
	14	17	15		14	16	16
17		16	18	15		15	
18	14		15	15	14		16
14	14	13	17	12	14	12	18

- h = number of pairs of queens that are attacking each other, either directly or indirectly
- $h = 17$ for the above state.
 - ❑ Queens can move within the same column.
 - ❑ Squares show heuristics for each possible move.
 - ❑ Five moves have the same lowest heuristic.
 - ❑ Hill-climbing will choose any of them as the next move.

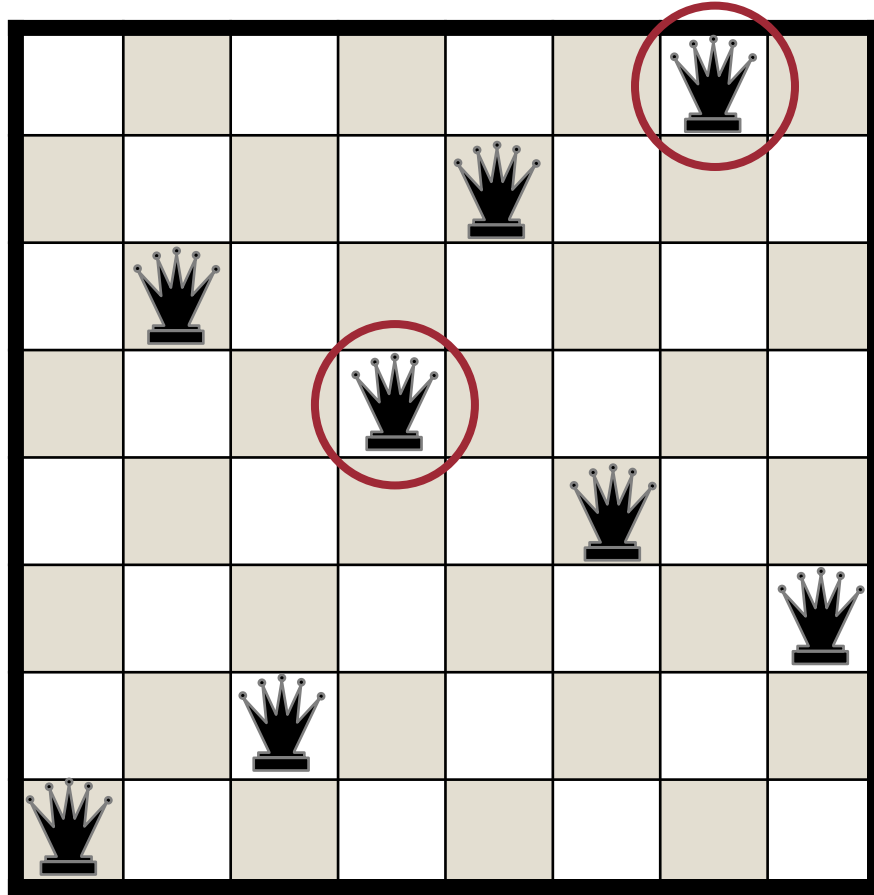
Hill-Climbing Search

- Problem: depending on initial state, can get stuck in local maxima



- Non-guaranteed fixes: sideways moves, random restarts

Hill-Climbing Search: 8-Queens



Local Minimum with $h = 1$

Adversarial Search

AIMA Chapter 5.1 – 5.5

AI vs. Human Players: the State of the Art



Kasparov vs. Deep Blue (1997)



World Poker Championship vs. Libratus (2017)



Go player vs. AlphaGo (2016)

AI vs. Human Players: the State of the Art



AI vs. Human Players: the State of the Art



OpenAI Five beats pro DOTA 2 players.



Pluribus beats multi-player poker pros



AlphaStar beats top Starcraft II players

**TO BE UPDATED
NEXT YEAR!**

Deterministic Games in Practice

- [Chinook \(Checkers, 1994\)](#). Precomputed endgame database $\Rightarrow$ perfect play for all positions with ≤ 8 pieces on the board (total of 444 billion positions).
- [Deep Blue \(Chess, 1997\)](#). Searches 200 million positions/sec + evaluation functions + secret sauce.
- [Logistello \(Othello, 1997\)](#). Human champions refuse to play against AI.
- [AlphaGo + Alphazero \(Go/everything above, 2016-2017\)](#). Learning for evaluation functions + database and efficient search + secret sauce.

Outline

- Adversarial search problems (aka games)
- Optimal (i.e., Minimax) decisions
- α - β pruning
- Imperfect, real-time decisions

Games vs. Search Problems

Utility maximizing opponent

- solution is a strategy specifying a move for every possible opponent response.

Time limit

- unlikely to find goal, must approximate

Let's Play!

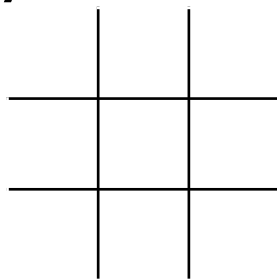
- Two players in a **zero-sum game**:
 - ❑ Winner gets paid and loser pays.
- Easy to think in terms of a **max player** and **min player**
 - ❑ Player 1 wants to maximize value (MAX player)
 - ❑ Player 2 wants to minimize value (MIN player)



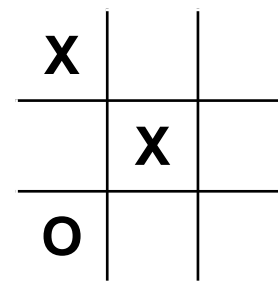
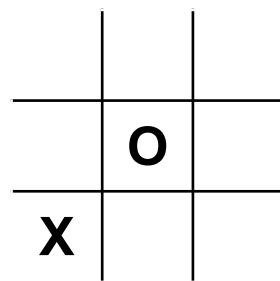
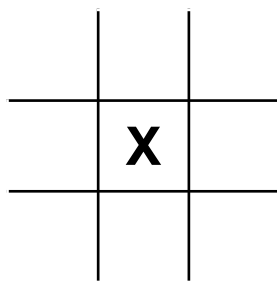
Game: Problem Formulation

A game is defined by 7 components:

1. Initial state s_0



2. States

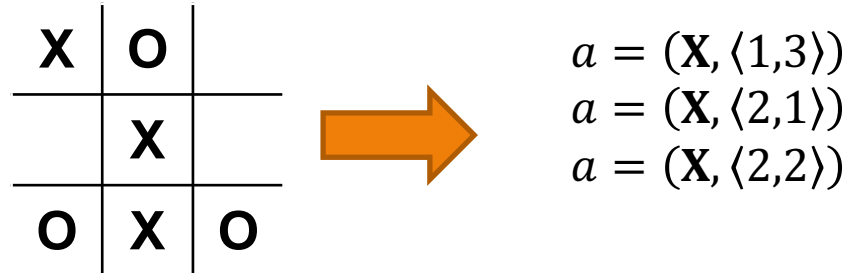


3. Players: $PLAYER(s)$ defines which player has the move in state s .

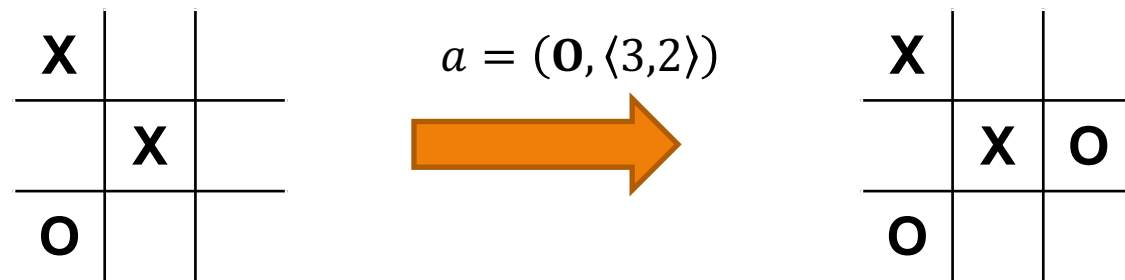
Game: Problem Formulation

A game is defined by 7 components:

4. Actions : $ACTIONS(s)$ returns set of legal moves in state s



5. Transition model : $RESULT(s, a)$ returns state that results from the move a in state s .



Game: Problem Formulation

A game is defined by 7 components:

6. Terminal test $TERMINAL(s) = true$ iff game end

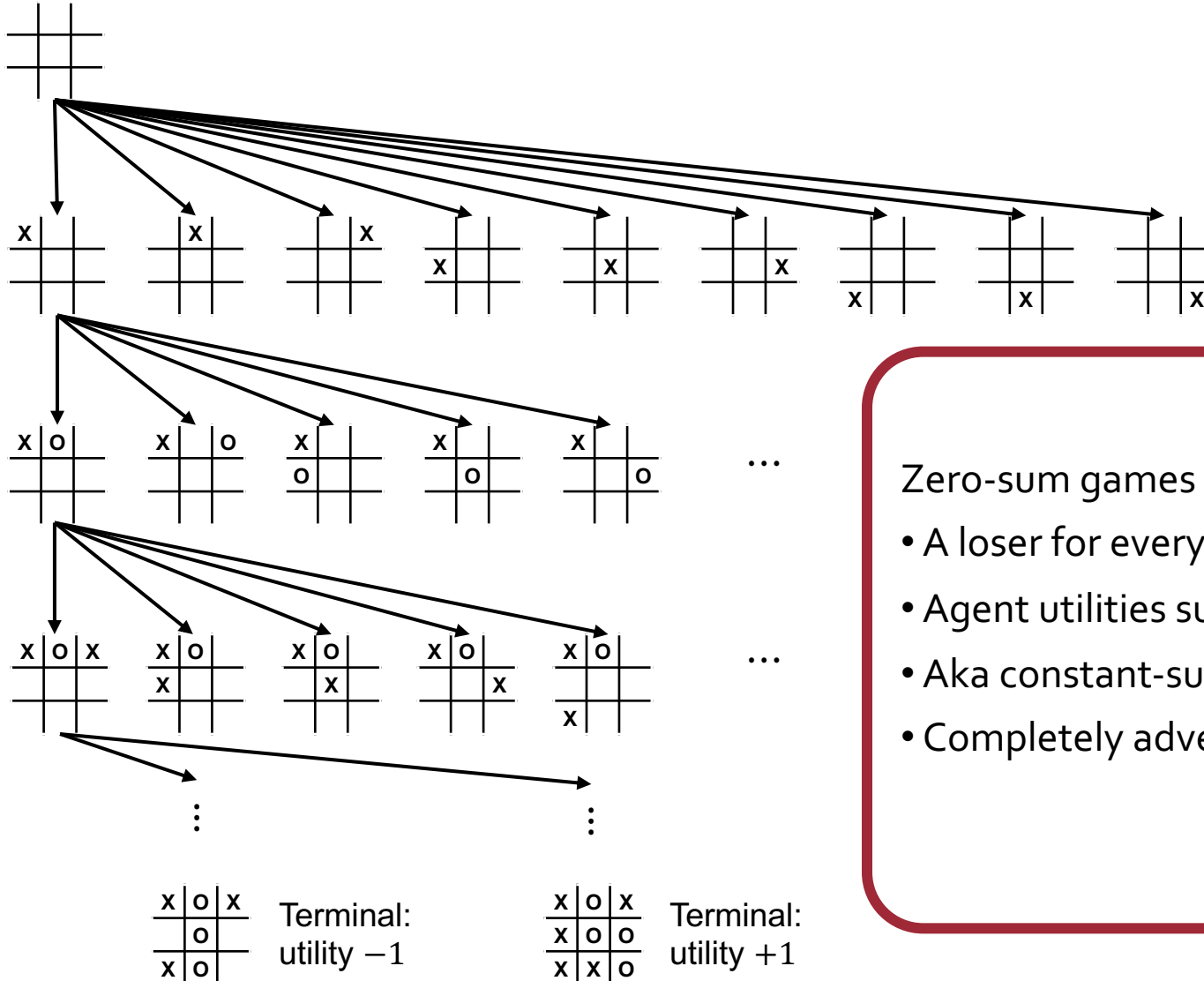
X	O	X	X		O	X	X	O	O	X	O
O	X	X	O	X			O	O	X	X	O
O	X	O	X	O	X		X	O	X		O

7. Utility function $UTILITY(s, p)$: final numeric value for a game that ends in terminal state s for a player p

■ Tic-Tac-Toe: X wins +1; O wins -1; draw 0.

Game Tree

(2-Player, Deterministic, Turn-Taking)



Zero-sum games

- A loser for every winner
- Agent utilities sum to zero
- Aka constant-sum game
- Completely adversarial game

Example : Game of NIM

Several piles of sticks are given. We represent the configuration of the piles by a monotone sequence of integers, such as $(1,3,5)$. A player may remove, in one turn, any number of sticks from one pile. Thus, $(1,3,5)$ would become $(1,1,3)$ if the player were to remove 4 sticks from the last pile. The player who takes the last stick loses.

- Represent the NIM game $(1,2,2)$ as a game tree.



Player Strategies

A strategy s for player i : what will player i do at every node of the tree that they make a move in?

Need to specify behavior in states that will never be reached!

Winning Strategy

A strategy s_1^* for player 1 is called winning if for any strategy s_2 by player 2, the game ends with player 1 as the winner.

A strategy t_1^* for player 1 is called non-losing if for any strategy s_2 by player 2, the game ends in either a tie or a win for player 1.

Theorem (Von Neumann):

in the game of chess, only one of the following is true:

1. White has a winning strategy s_w^*
2. Black has a winning strategy s_b^*
3. Each player has a non-losing strategy.

Optimal Strategy at Node - Minimax

$$\begin{aligned} &Minimax(s) \\ &= \begin{cases} Utility(s) & \text{if TerminalTest}(s) \\ \max_{a \in Actions(s)} Minimax(Result(s, a)) & \text{if Player}(s) = MAX \\ \min_{a \in Actions(s)} Minimax(Result(s, a)) & \text{if Player}(s) = MIN \end{cases} \end{aligned}$$

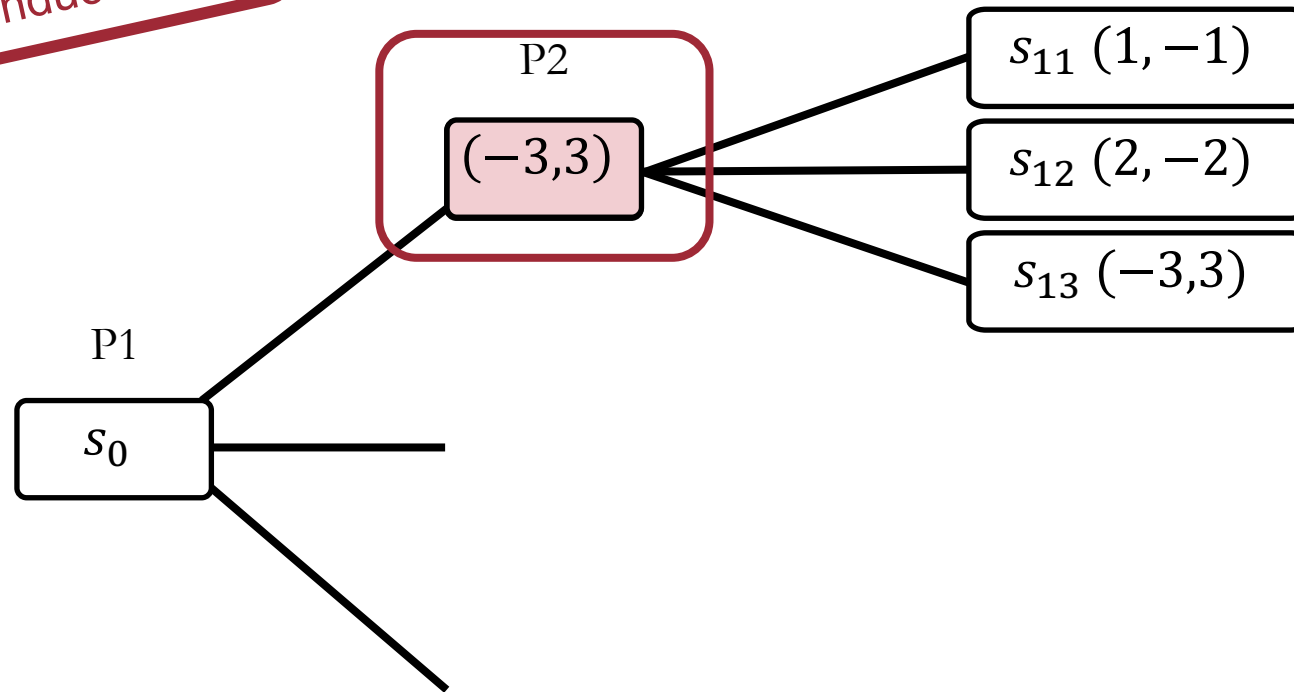
Intuitively,

- MAX chooses move to maximize the minimum payoff
- MIN chooses move to minimize the maximum payoff

Minimax Play

(Subperfect Nash Equilibrium)

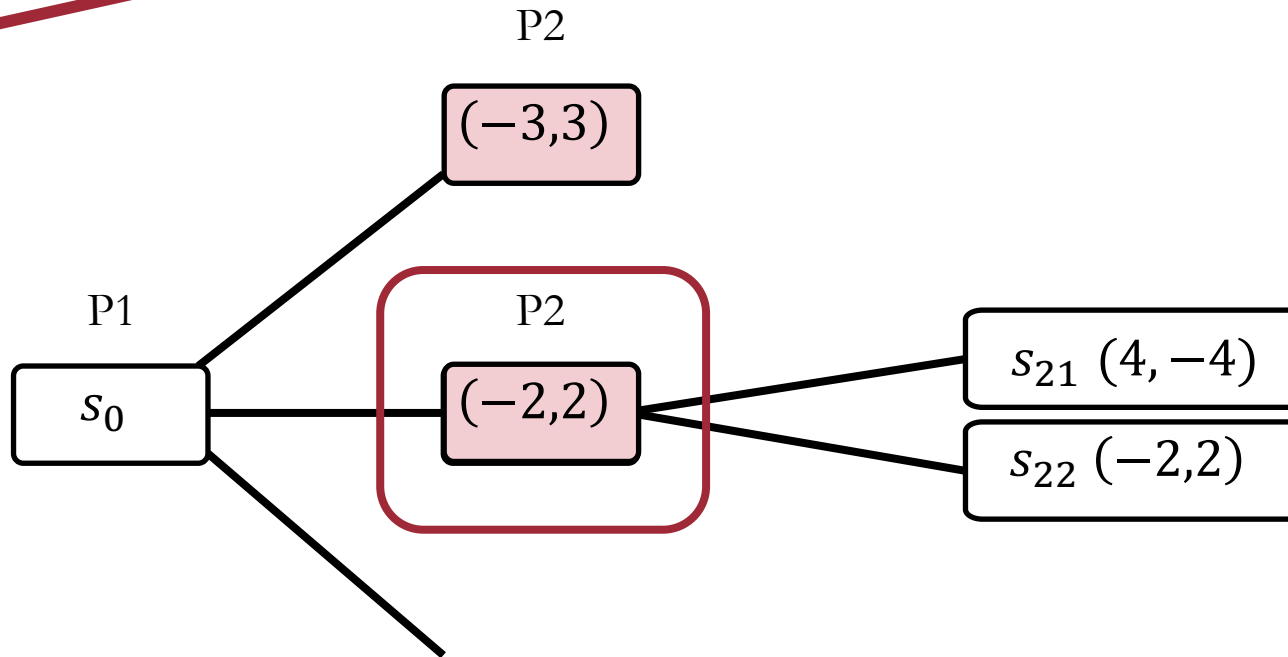
Backwards
Induction



Minimax Play

(Subperfect Nash Equilibrium)

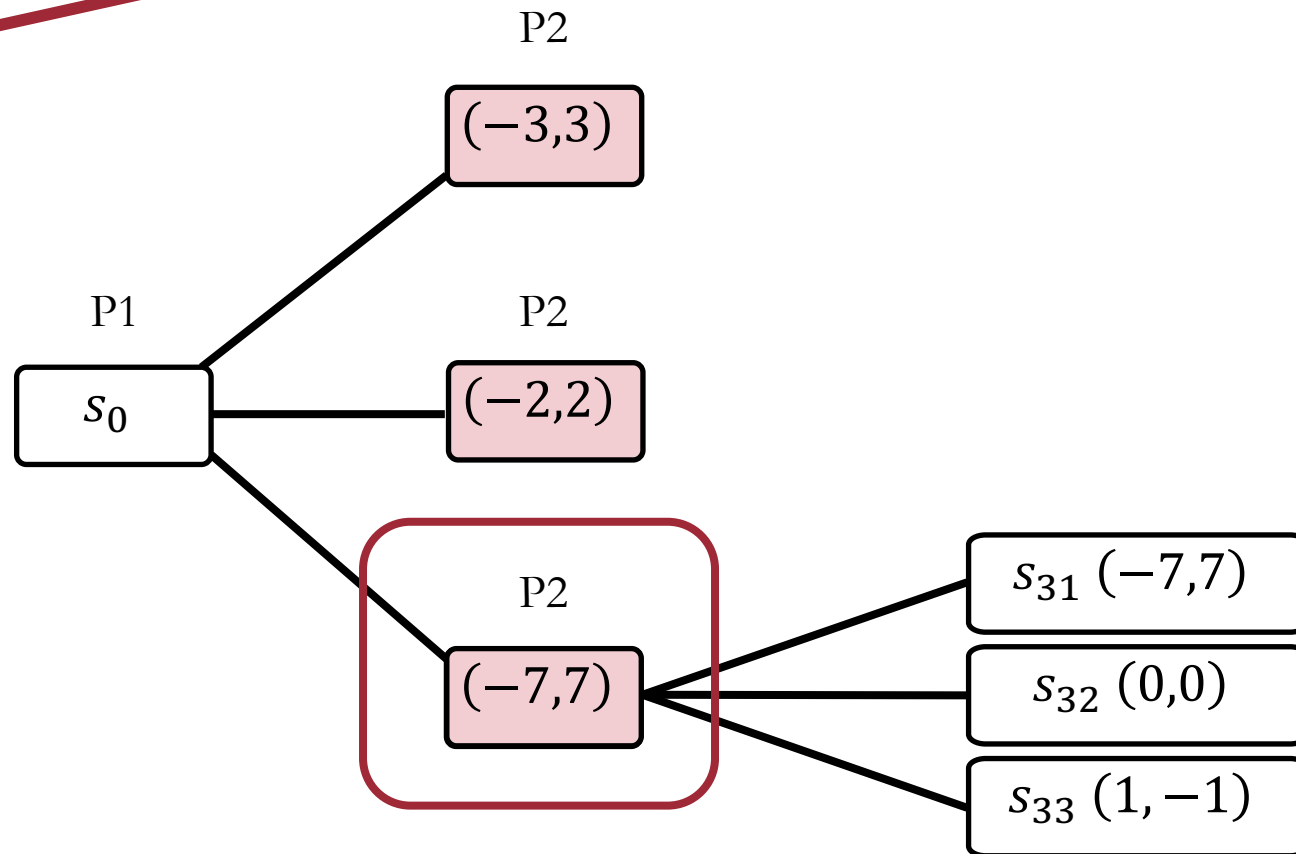
Backwards
Induction



Minimax Play

(Subperfect Nash Equilibrium)

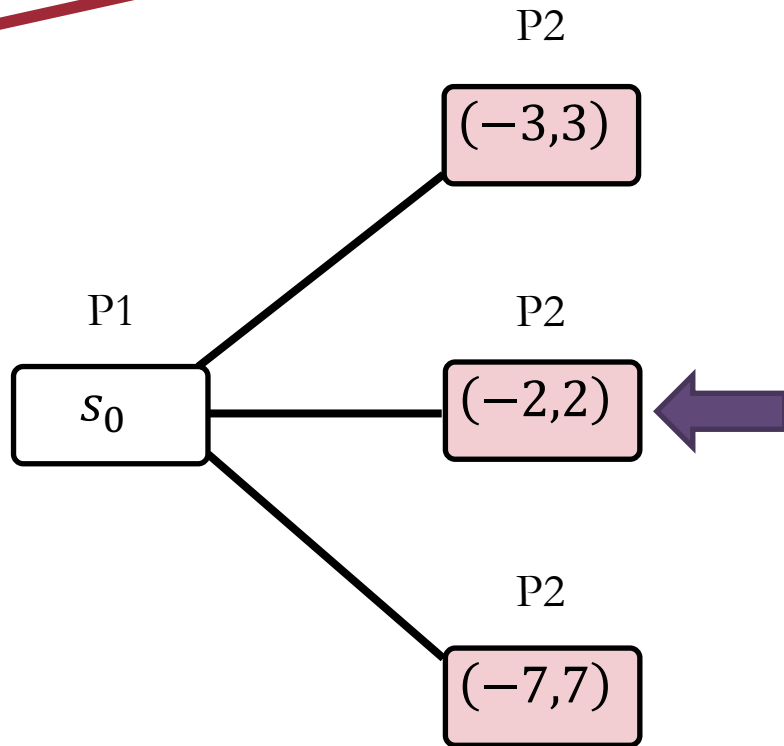
Backwards
Induction



Minimax Play

(Subperfect Nash Equilibrium)

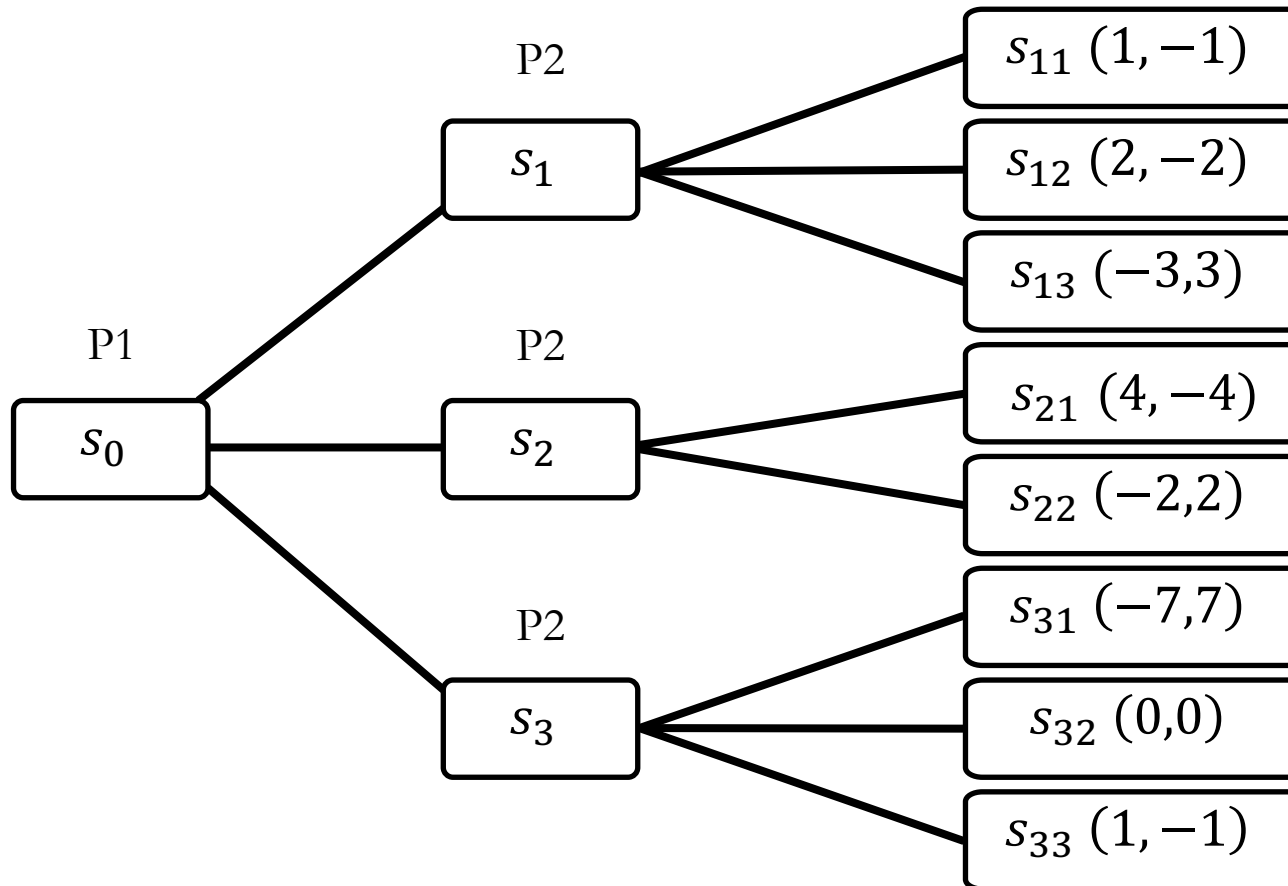
Backwards
Induction



Minimax Play

(Subperfect Nash Equilibrium)

What are the optimal strategies in this game?



Backwards Induction

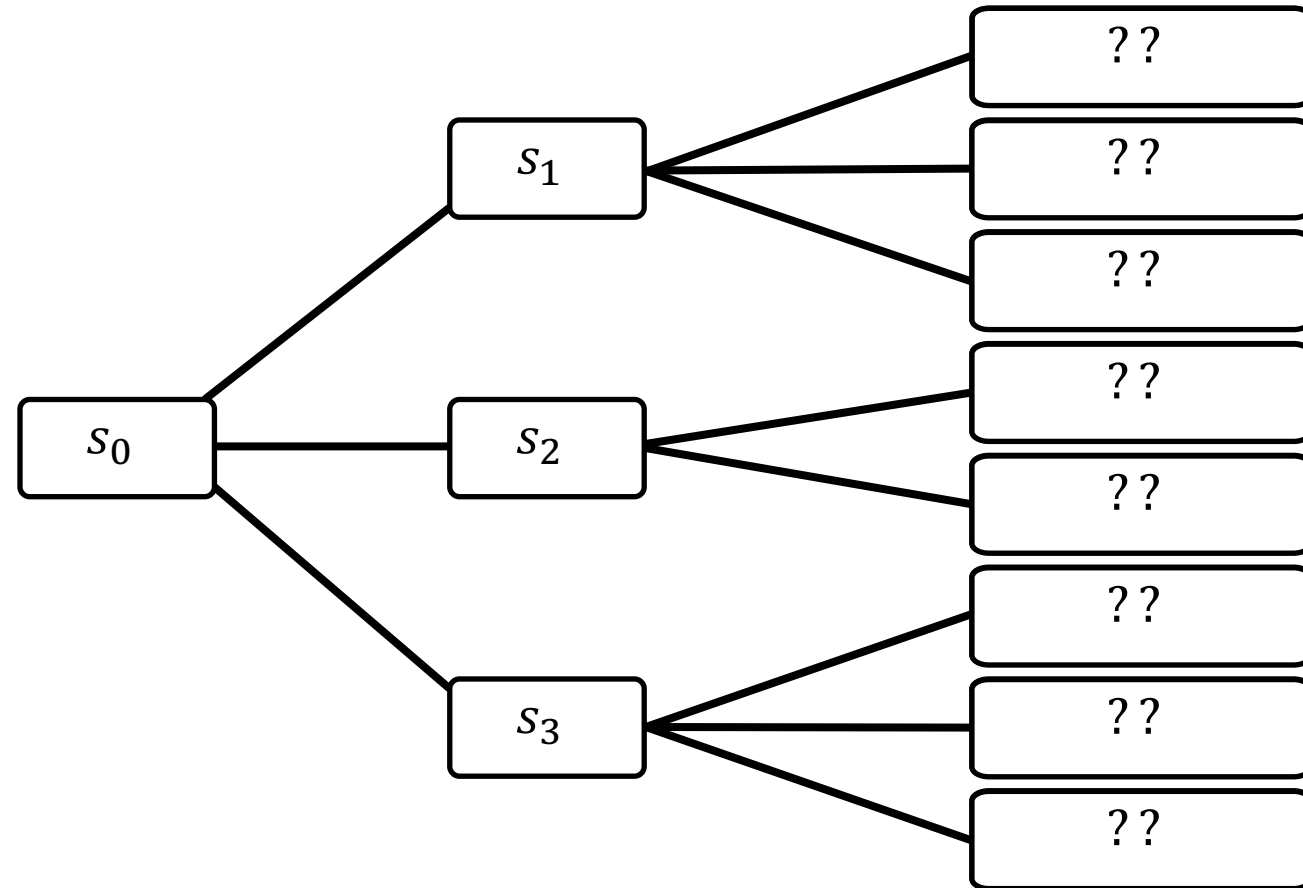
- Game trees are huge: chess has game tree with $\sim 10^{123}$ nodes (planet Earth has $\sim 10^{50}$ atoms)
- Impossible to expand the entire tree

α - β Pruning

- **Basic idea:** *“If you have an idea that is surely bad, don't take the time to see how truly awful it is.”* -- Pat Winston
- Maintain a **lower bound α** and **upper bound β** of the values of, respectively, MAX's and MIN's nodes seen thus far.
- Prune subtrees that will never affect minimax decision.

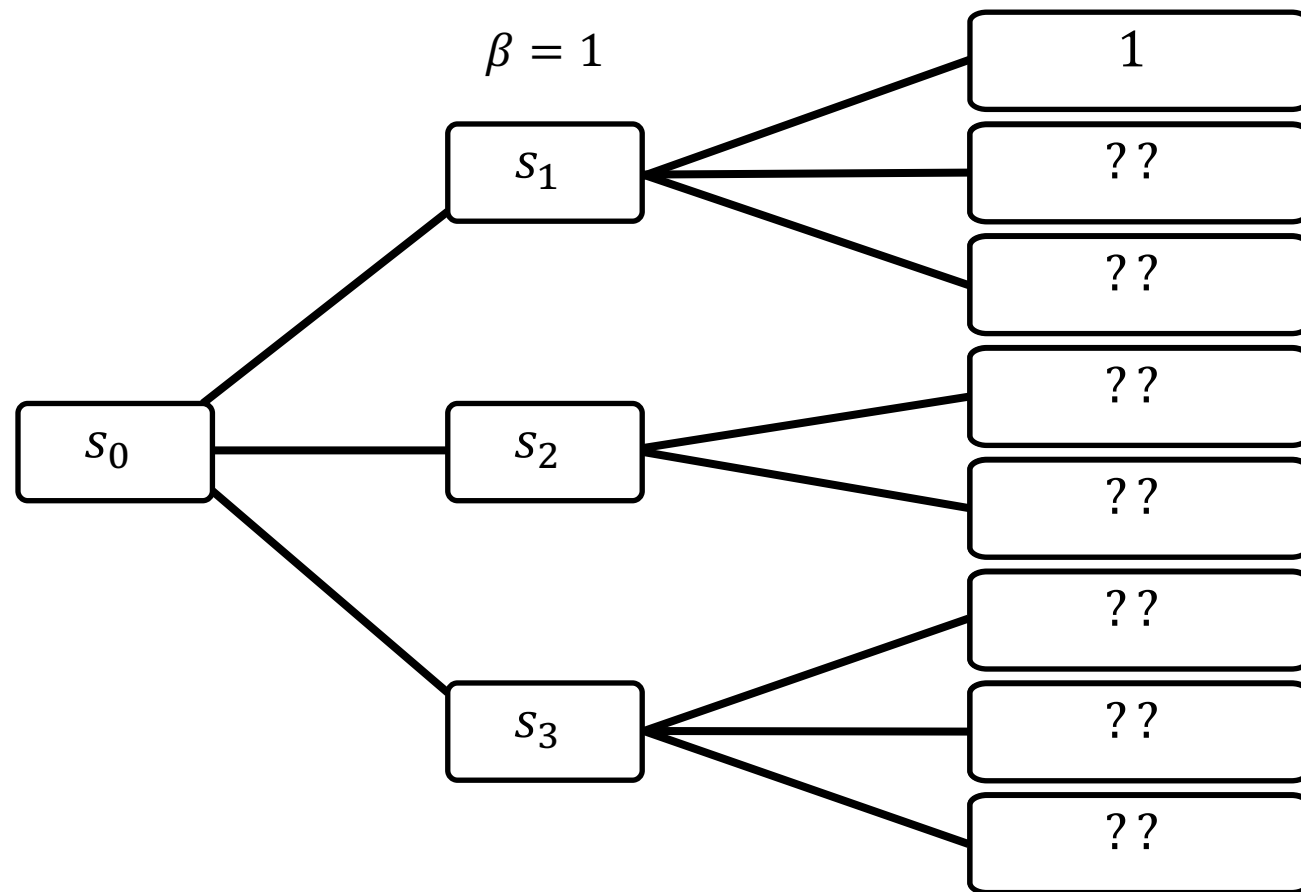
MAX

MIN



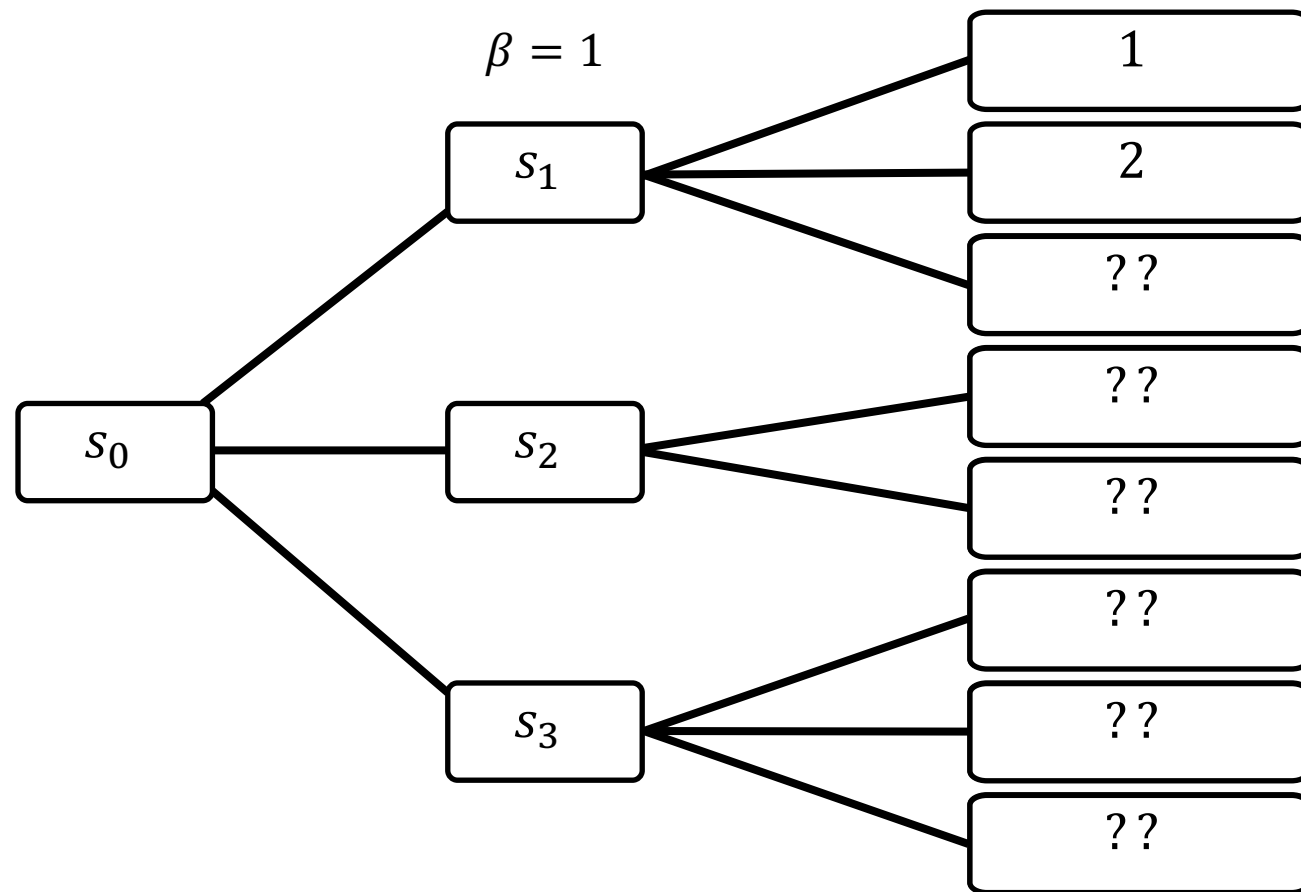
MAX

MIN



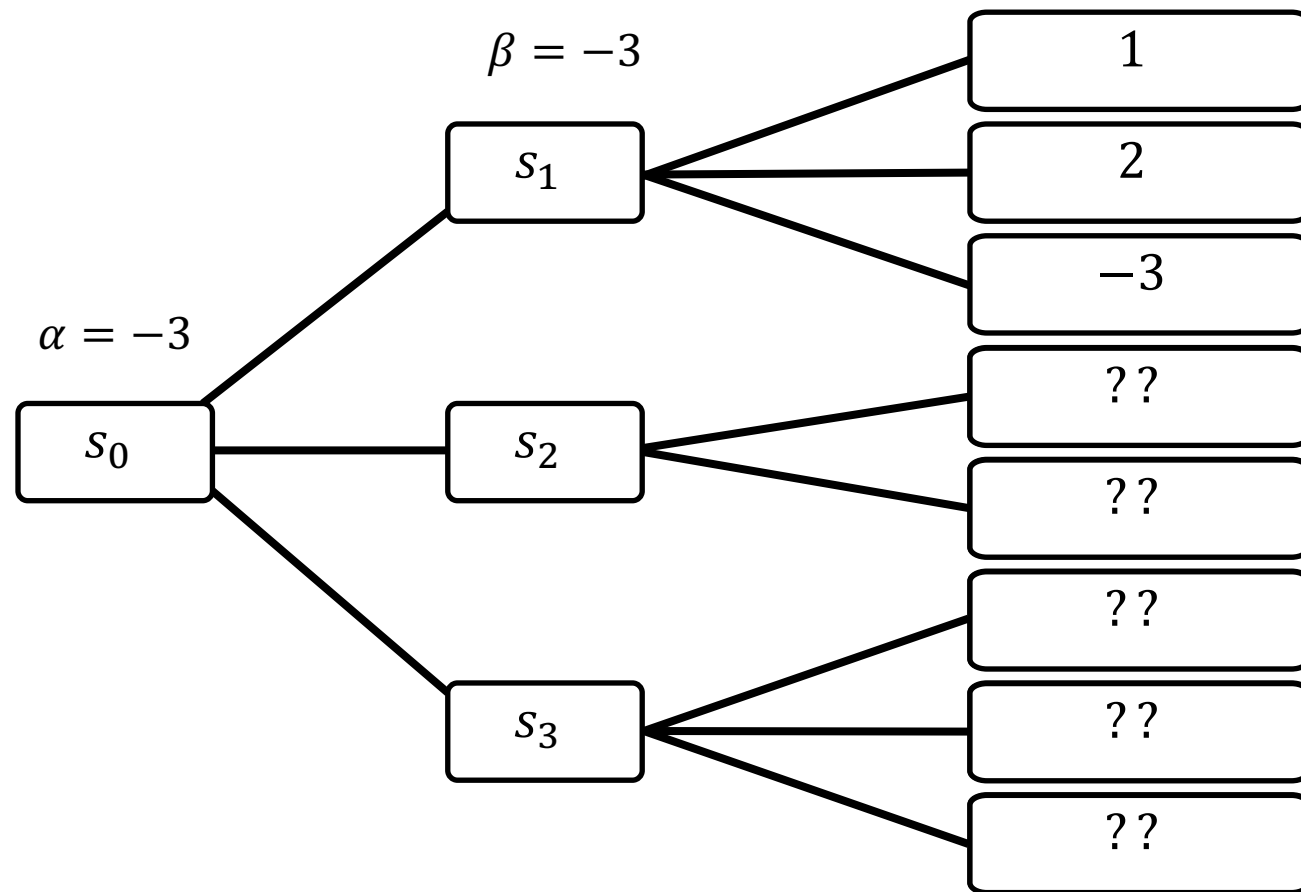
MAX

MIN



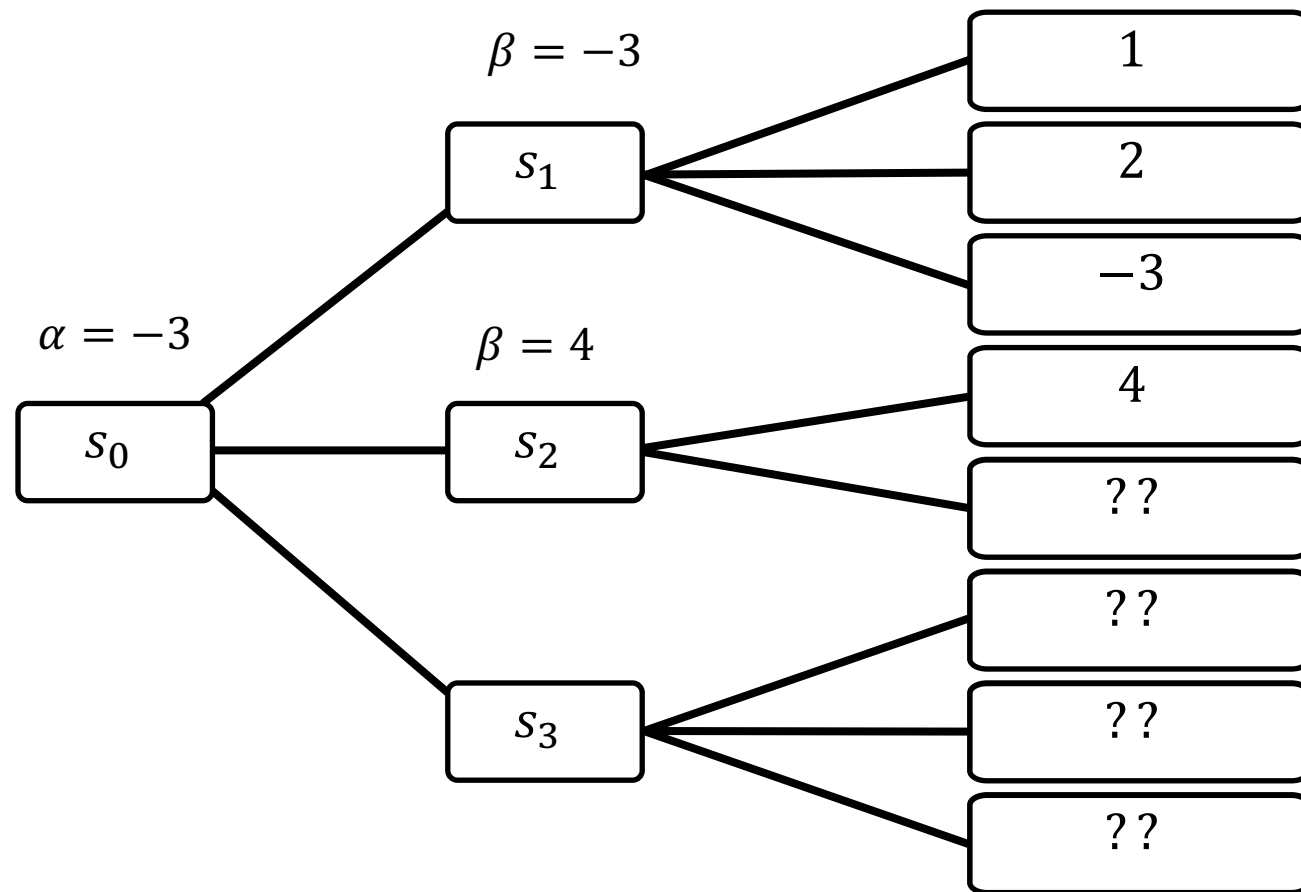
MAX

MIN



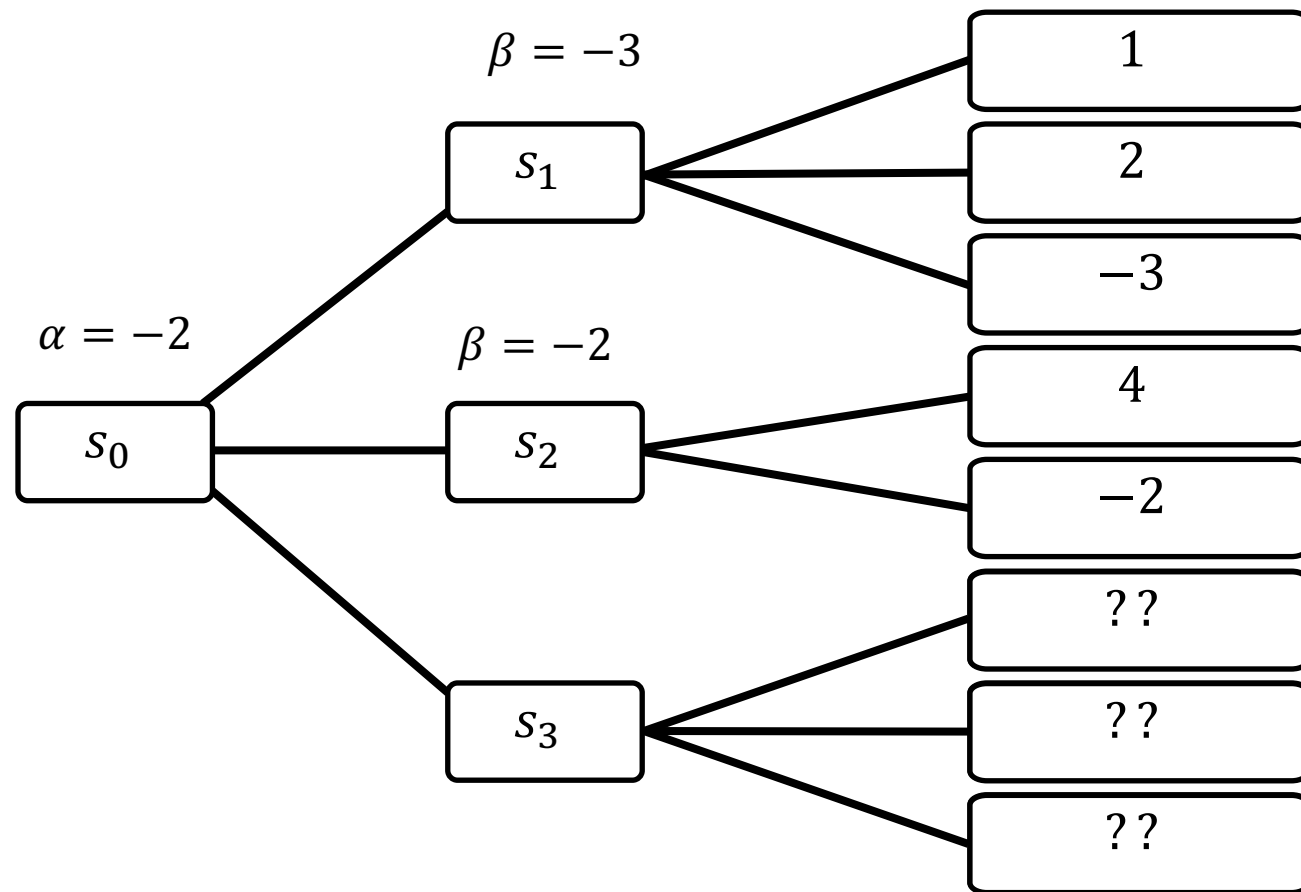
MAX

MIN



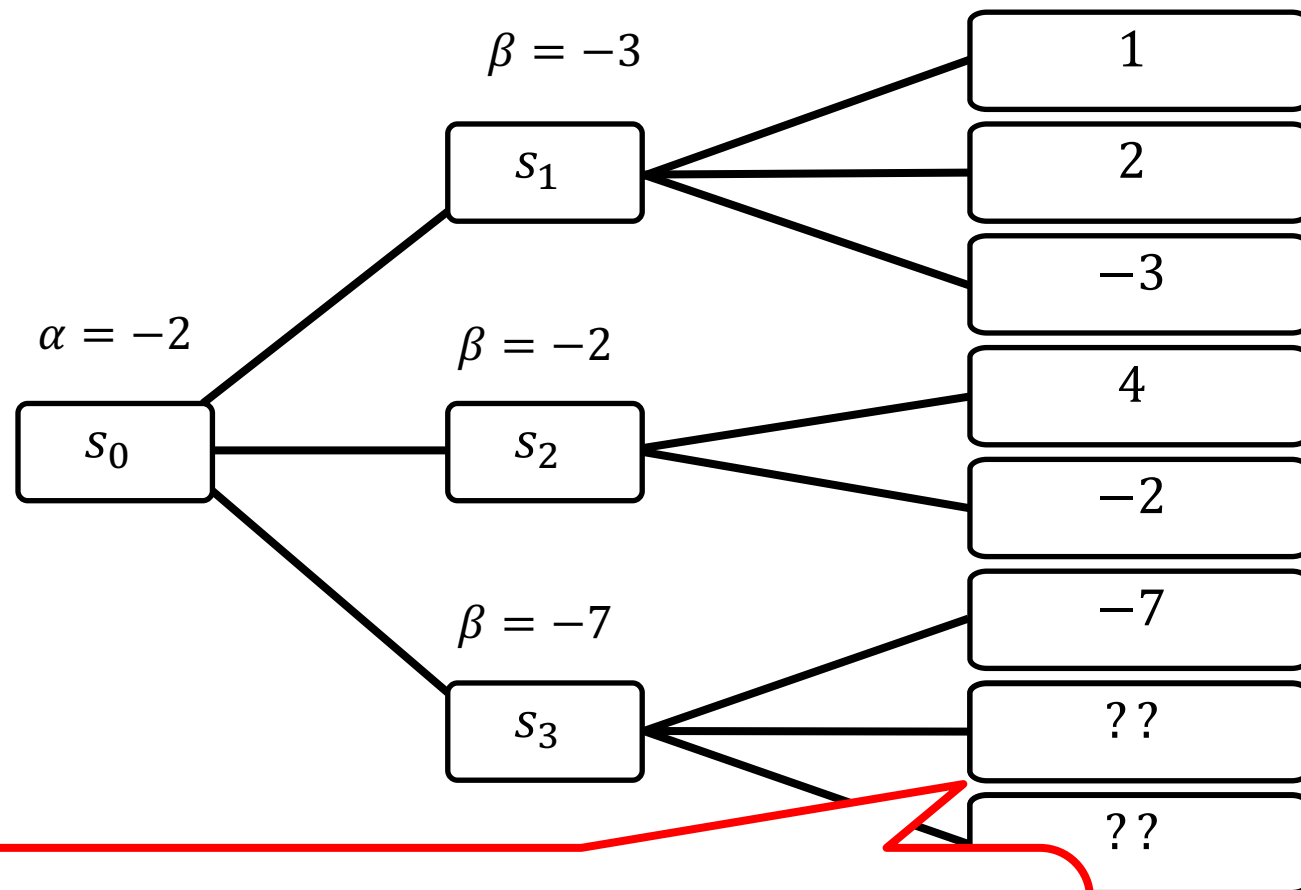
MAX

MIN

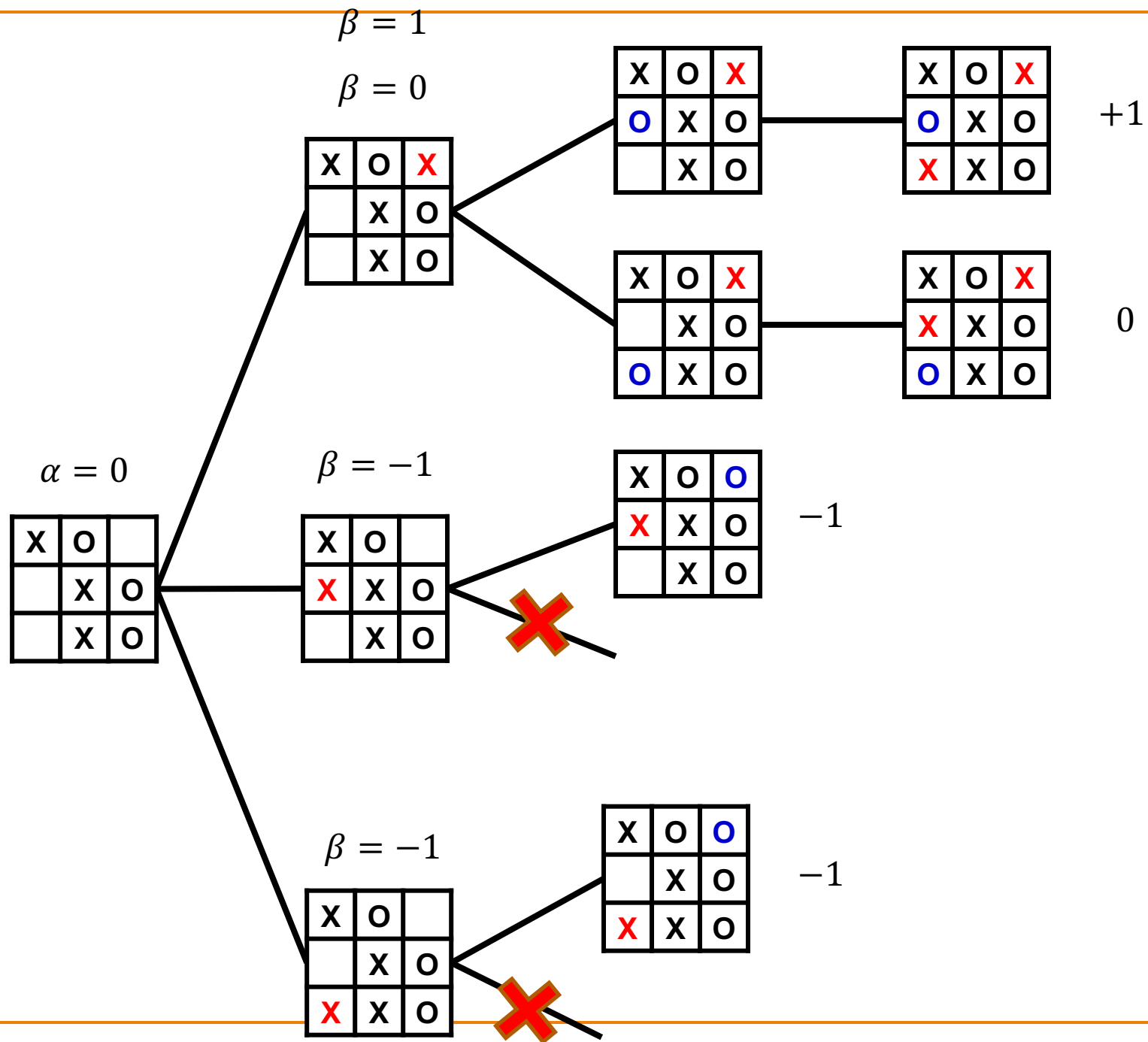


MAX

MIN



No point exploring the last two nodes! Choosing s_3 results in a loss of at least 7...



α - β Pruning

- MAX node n : $\alpha(n)$ = highest **observed** value found on path from n ; initially $\alpha(n) = -\infty$
- MIN node n : $\beta(n)$ is the lowest observed value found on path from n ; initially $\beta(n) = +\infty$
- **Pruning**:
 - given a MIN node n , stop searching below n if there is some MAX ancestor i of n with $\alpha(i) \geq \beta(n)$
 - given a MAX node n , stop searching below n if there is some MIN ancestor i of n with $\beta(i) \leq \alpha(n)$

Summary: α - β Pruning Algorithm

- Initially, $\alpha(n) = -\infty$, $\beta(n) = +\infty$
- $\alpha(n)$ is max along search path containing n
- $\beta(n)$ is min along search path containing n
- If a MIN node has value $v \leq \alpha(n)$, no need to explore further.
- If a MAX node has value $v \geq \beta(n)$, no need to explore further.

Time Limit

- **Problem:** very large search space in typical games
 - **Solution:** α - β pruning removes large parts of search space
 - Unresolved problem: Maximum depth of tree
 - Standard solutions:
 - **evaluation function** = estimated expected utility of state
 - **cutoff test:** e.g., depth limit
-

Heuristic Minimax Value

MINIMAX(s) =

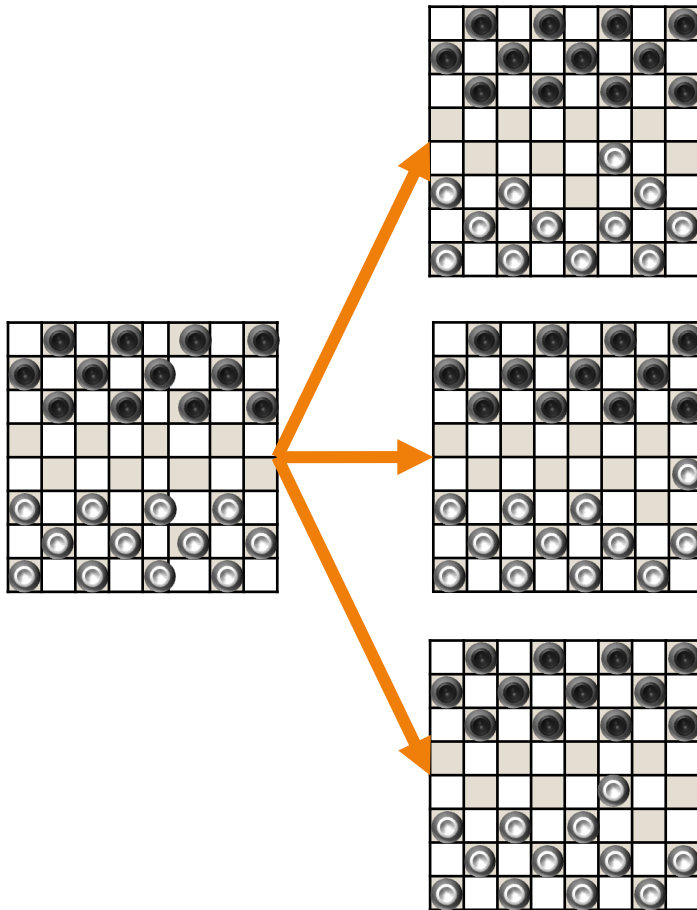
$$\begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{ACTIONS}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{ACTIONS}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

H-MINIMAX(s, d) =

$$\begin{cases} \text{EVAL}(s) & \text{if } \text{CUTOFF-TEST}(s, d) \\ \max_{a \in \text{ACTIONS}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{ACTIONS}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

Run minimax until depth d ; then start using the evaluation function to choose nodes.

Evaluation Functions



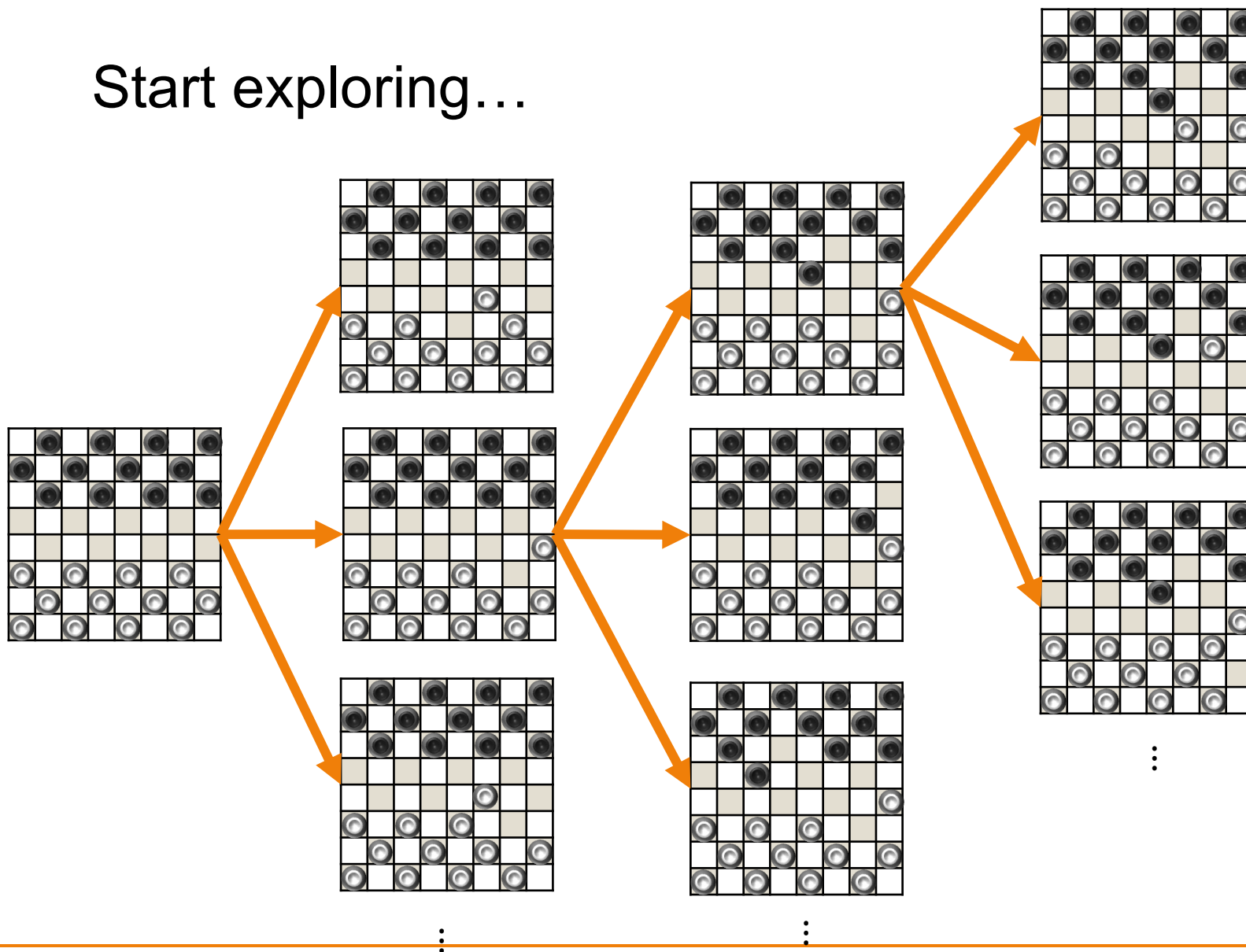
How good is this move?

To know for sure, we need to explore entire subtree up to terminal states.

Unrealistic (even for 'simple' game of checkers)

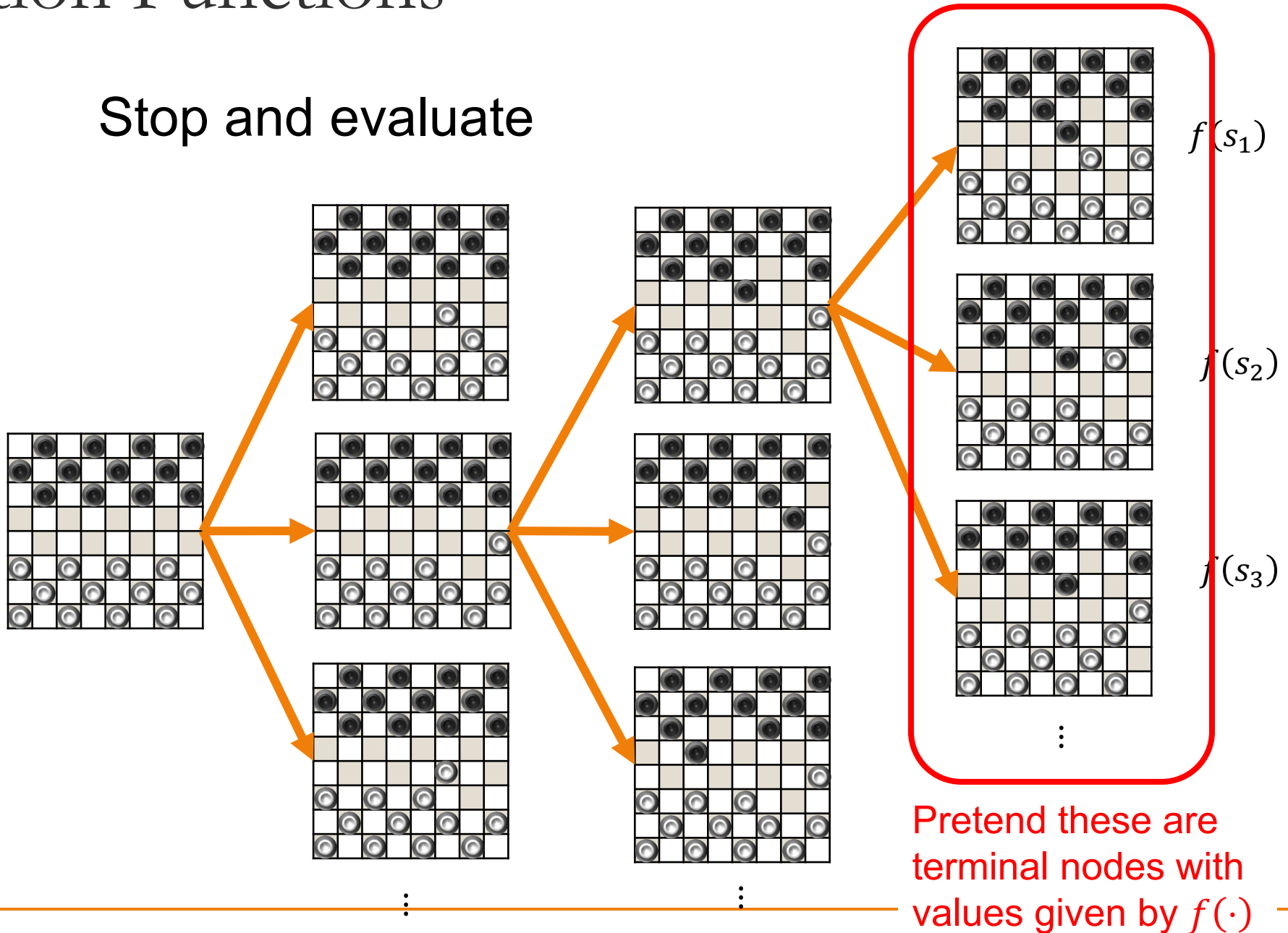
Evaluation Functions

Start exploring...



Evaluation Functions

Stop and evaluate



Evaluation Functions

- An evaluation function is a mapping from game states to real values: $f: \mathcal{S} \rightarrow \mathbb{R}$

□ So far:

$$f(s) = \begin{cases} UTILITY(s) & \text{if } TERMINAL(s) \\ 0 & \text{otherwise} \end{cases}$$

No information on quality of non-terminal nodes

- For non-terminal states, must be strongly correlated with actual chances of winning

Evaluation Functions

Important Features

- # of pieces
- # of queens
- # of controlled squares
- # of threatened opponent pieces
- ...

$$f(n) = w_1 \times (NPcs) + w_2 \times (NQns) + w_3 \times (CtlSqs) + w_4 \times (ThrPcs)$$

$w_1, \dots, w_4 = ???$

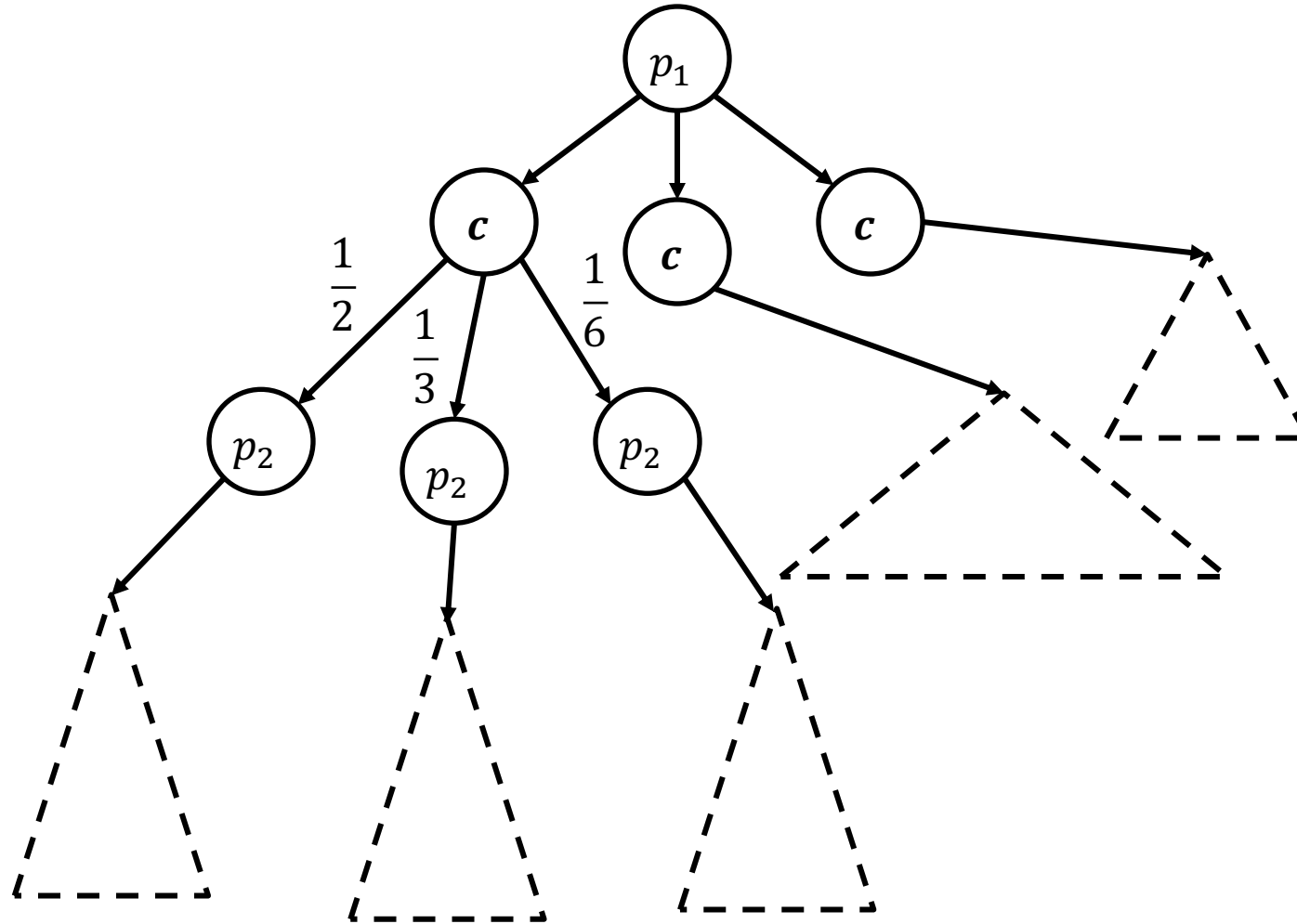
Cutting Off Search

- Modify minimax or α - β pruning algorithms by replacing
 - `TERMINALTEST( $s$ )` with `CUTOFFTEST( $s, d$ )`
 - `UTILITY( $s$ )` is replaced by `EVAL( $s$ )`
- Can also be combined with iterative deepening

Stochastic Games

- Many games have randomization:
 - ❑ Backgammon
 - ❑ Settlers of Catan
 - ❑ Poker
- How do we deal with uncertainty?
- Can we still use minimax? Yes, but search space is much bigger

Adding Chance Layers



Calculate the **expected** value of a state
(MUCH harder than deterministic games)

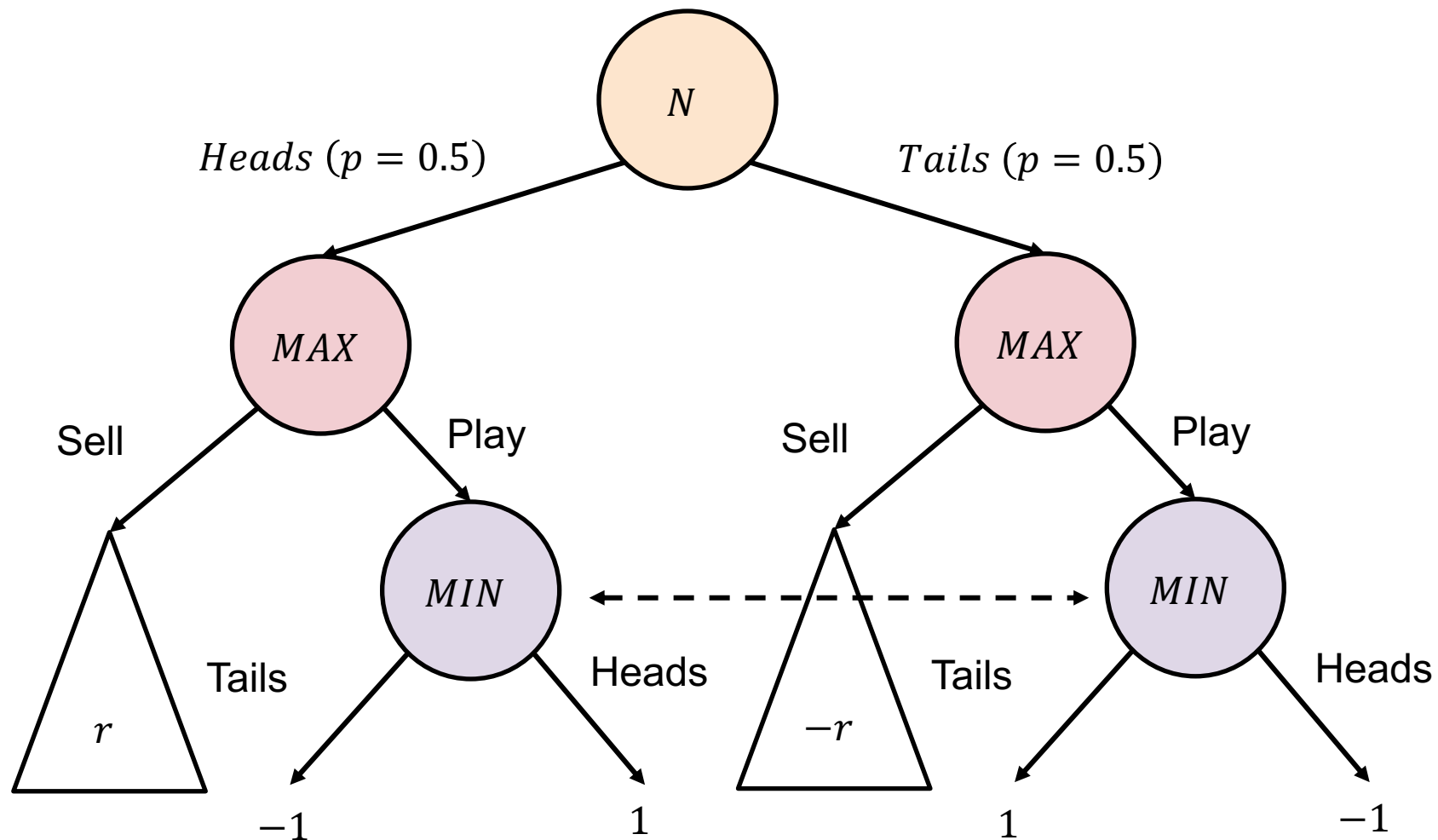
Example: Coin Toss Game

(example from Brown and Sandholm “Safe and Nested Subgame Solving for Imperfect-Information Games”, NIPS 2017)

- An unbiased coin is tossed; MAX player sees the result, MIN player doesn't.
 - MAX player can choose to:
 - Sell the coin, getting a reward of r , if the coin lands on heads and $-r$ otherwise.
 - Play, in which case it's player MIN's turn.
 - In player MIN's turn, needs to guess either “heads” or “tails”. Gets reward of 1 for correct guess.
-

Example: Coin Toss Game

(example from Brown and Sandholm “Safe and Nested Subgame Solving for Imperfect-Information Games”, NIPS 2017)



Example: Coin Toss Game

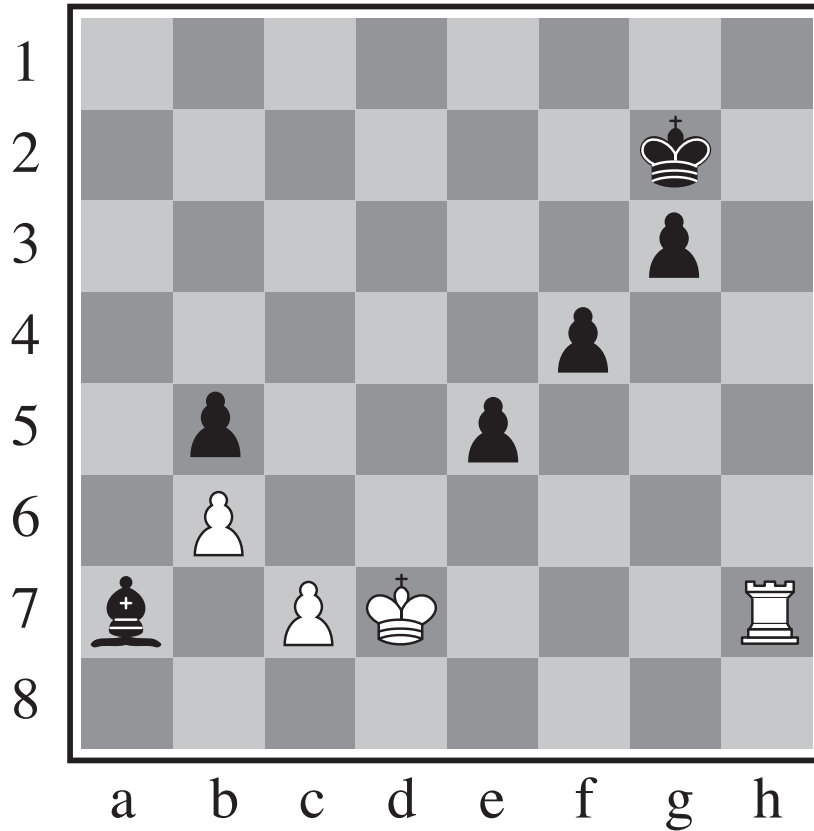
(example from Brown and Sandholm “Safe and Nested Subgame Solving for Imperfect-Information Games”, NIPS 2017)

- We allow players to choose random strategies
 - MIN's optimal strategy depends on r - a value outside MIN's subtree!
“how desperate was MAX to let me play?”
 - Computing optimal strategies in perfect information games:
no need to know what happens outside your subtree!
-

The Little Black Book

- Opening move search doesn't result in useful utility estimates
 - Applying evaluation functions on end game scenarios may not **solve** the game
 - Western chess KRK end game
 - Use a **policy** or lookup table (taken from expert knowledge or previous game history)
-

Horizon Effect



Black will lose the bishop at some point forward

- An unavoidable damaging move is beyond the current search limit
- Add **quiescence** search
 - ❑ Evaluate moves that will make evaluation function fluctuate more wildly

END OF FILE

IT5005

Artificial Intelligence

Lecture 7 – Neural Networks

colintan@nus.edu.sg



What Is Machine Learning?

In machine learning:

- ✓We present the machine with A LOT of data.
- ✓Machine learns on its own to recognize patterns in the data.

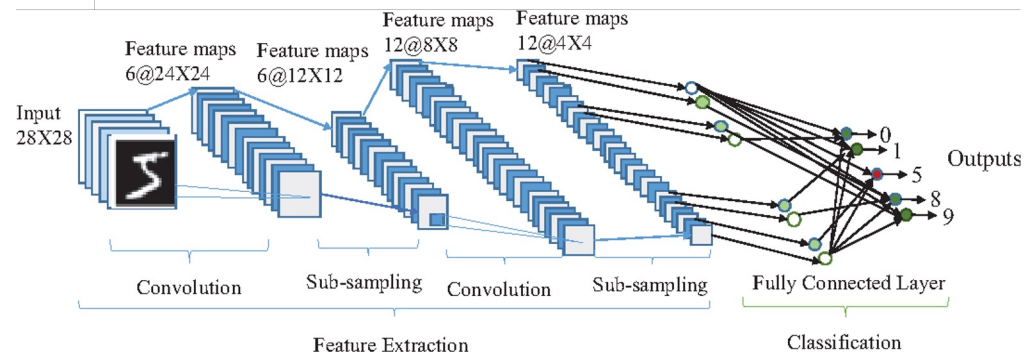
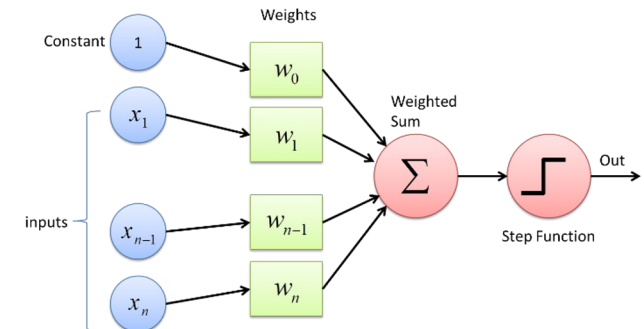
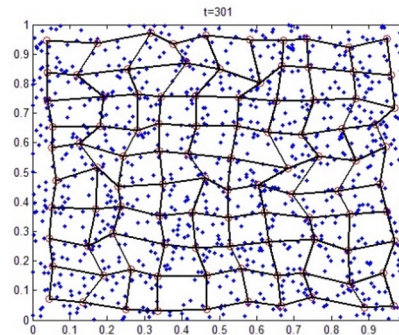
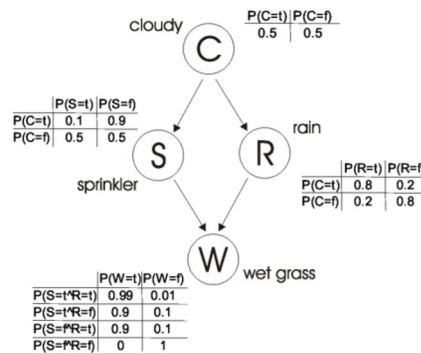
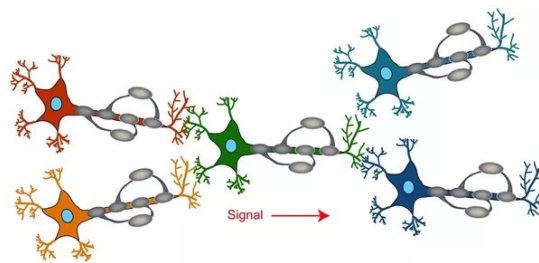
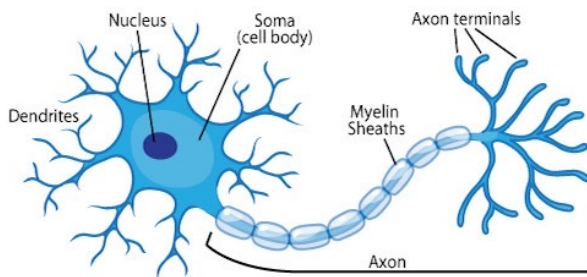


Fig. 1. CNN Block Diagram

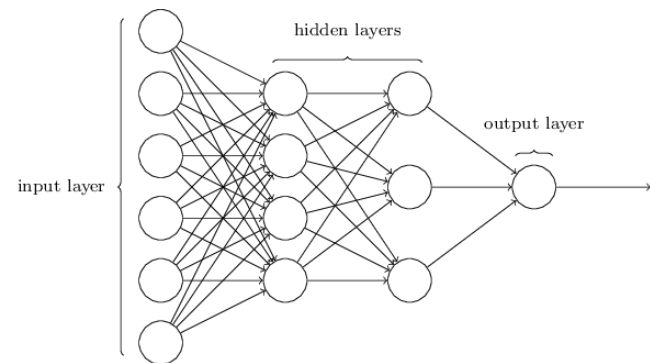
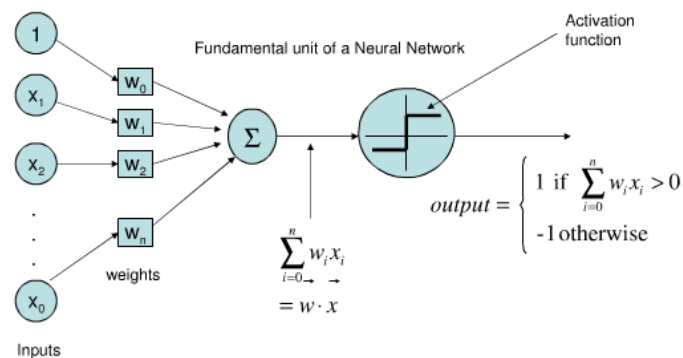
Neural Networks: Biological Inspiration

- **Neural networks are inspired by biological neurons: Units of computation that are in our brain and nervous system:**
 - Each neuron takes electrical signals from surrounding neurons.
 - Signals pass through gaps called “synapses” that pass a portion of the signal through.
 - ✓ “Strong” synapses pass more signals, “weak” synapses pass weaker signals.
 - The neuron sums these signals and fire if they exceed a “threshold”.
 - To learn, the synapses are strengthened or weakened.



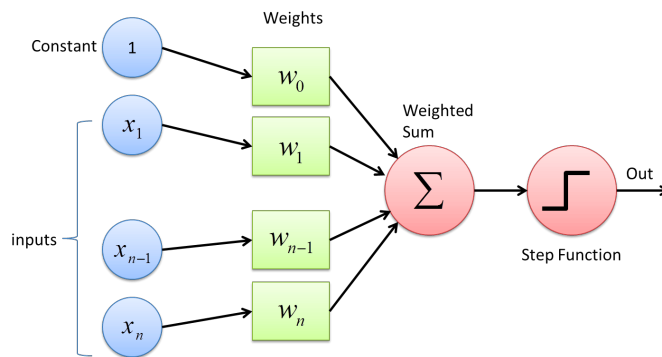
Neural Networks: Machine “Neurons”

- Many neural networks consist of computational units also called “neurons”, often called “nodes”:
 - Like the biological neurons, the “neurons” here also sum inputs through weights.
 - The neuron “fires” (outputs a “1”) if the sum exceeds a threshold, through an “activation function”.
 - The summation is essentially a dot product $w^T x$, and the neuron can be specified by $g(w^T x)$ where $g(\cdot)$ is the “activation function”.



Neural Networks

- Neural network research has been around a long, long time.



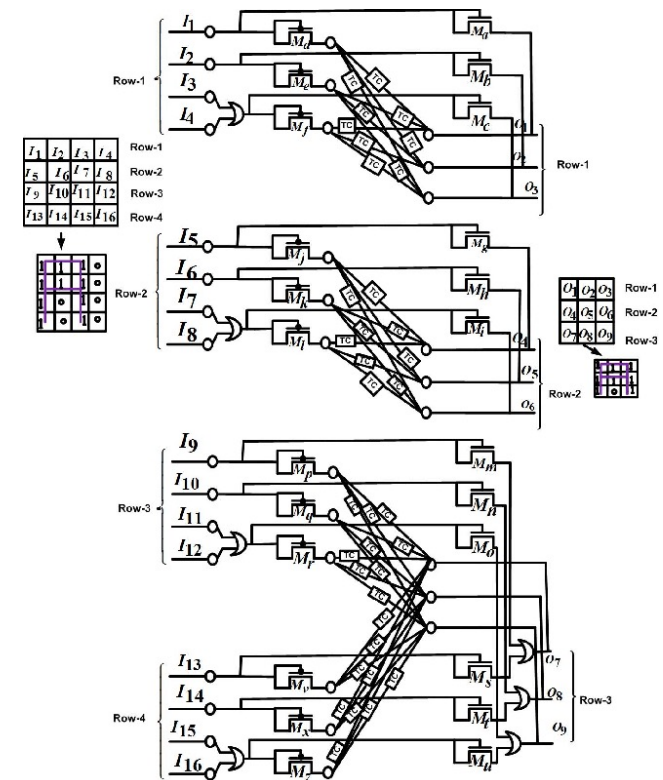
1958

Perceptron
Mark I

The Perceptron was the first artificial neural network.

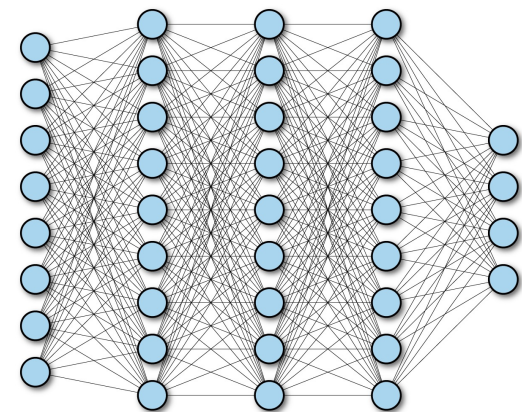
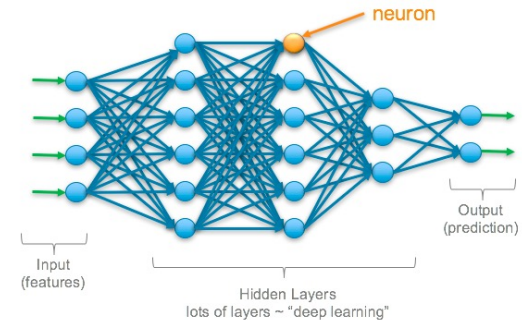
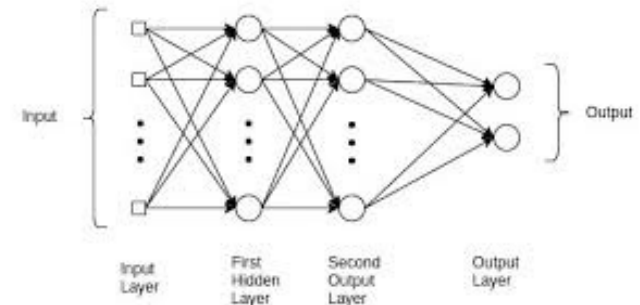
Developed by Frank Rosenblatt at the US office of Naval Research for visual recognition tasks.

The New York Times reported the perceptron to be: "the embryo of an electronic computer that will be able to walk, talk, see, write, reproduce itself and be conscious of its existence."



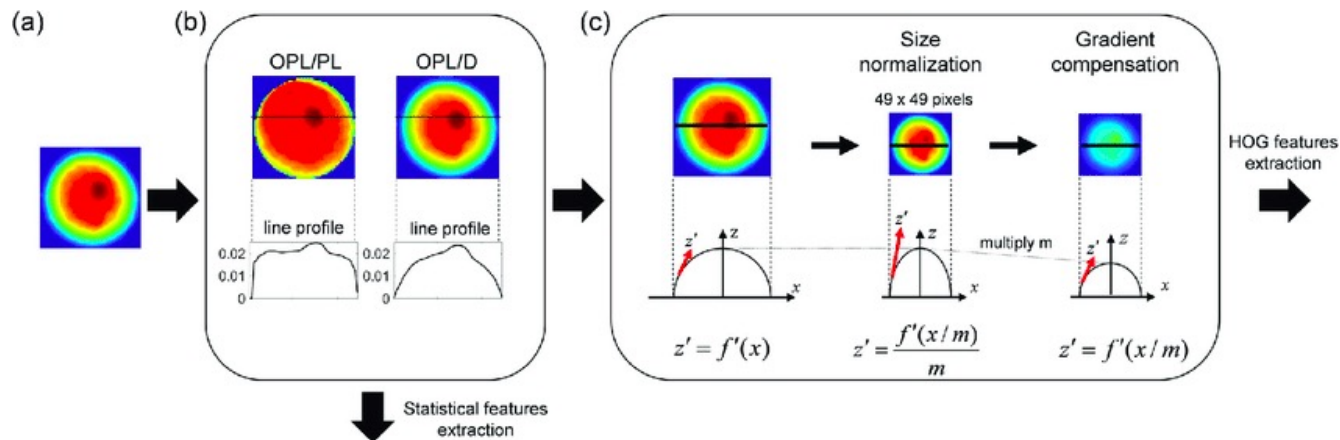
Deep Learning

- **Primary limitation of neural networks:**
 - Neural networks learn by adjusting “parameters” according to a “learning law”, based on the data presented.
 - Need large number of “parameters” to learn highly complex tasks.
 - ✓ **E.g. image classification**
 - Curse of dimensionality:
 - ✓ **As the number of parameters increases, the amount of training data required grows exponentially.**
 - ✓ **Otherwise the neural network “overfits” – It memorizes the data it is presented instead of learning from it.**



Deep Learning

- In Deep Learning we apply various techniques to “simplify” the data:
 - Extract features.
 - Make the data invariant:
 - ✓ Shift invariance.
 - ✓ Scale invariance.



Neural Networks

Introduction

- **In this lecture we will look at the basics of neural networks: Neural network learning laws.**
 - Unsupervised learning.
 - Supervised learning using gradient descent.
 - Problems with neural networks and how to solve them.
- **We will also be looking at some hands-on for programming neural networks using Keras.**

INTRODUCTION TO LEARNING LAWS

Learning Laws

- **Two classes of learning laws:**
 - **Unsupervised learning:**
 - ✓ **Tons of data is thrown at the NN.**
 - ✓ **The NN does its own inference on the structure and relationships in the data.**
 - ✓ **Not covered in this course.**
 - **Supervised learning:**
 - ✓ **We give the NN the data and the correct labels (or values we want to produce) from the data.**
 - ✓ **The NN then optimizes its weights to mimic the generator function for the data, based on what we tell it.**

SUPERVISED LEARNING

Supervised Learning

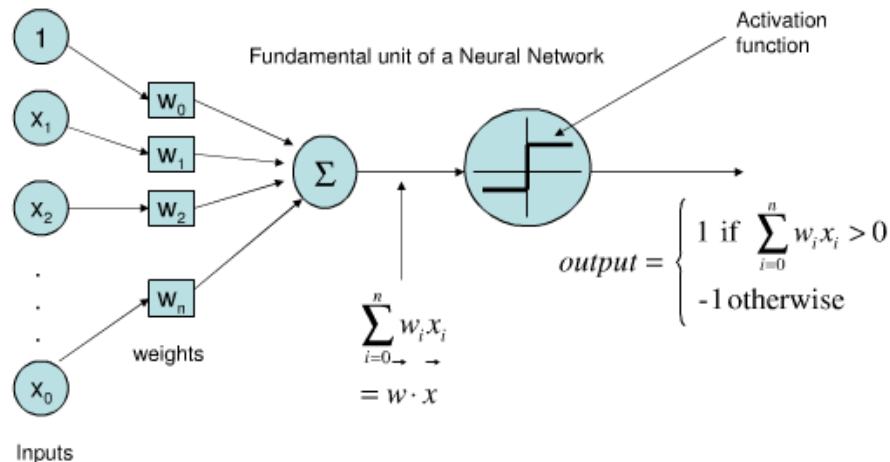
- **Unlike unsupervised learning algorithms like Kohonen SOMs, in supervised learning we must tell the network what it is looking at.**
 - Generally data is presented in the form of (x_i, y_i) , where x_i is the sample input, and y_i is some kind of target or class that the neural network needs to learn.
- **We will look at two related supervised learning algorithms:**
 - Linear Perceptrons:
 - ✓ **Simple, good for classifying linearly separable data.**
 - Multi-layer Perceptrons:
 - ✓ **More complex, good for classifying non-linearly separable data.**

SUPERVISED LEARNING : PERCEPTRONS

Supervised Learning

Perceptrons

- A single Perceptron (McCulloch – Pitts) neuron consists of:
 - A set of inputs $\mathbf{x}$.
 - A set of “weights” $\mathbf{w}$.
 - A unit that does a dot product $\mathbf{w}^T \mathbf{x}$.
 - An activation function that does outputs 1 when $\mathbf{w}^T \mathbf{x} + \mathbf{b} > 0$, and -1 otherwise. (other functions are possible, e.g. the identity function $\text{id}(x)=x$)
 - The bias $\mathbf{b}$ is normally modelled as a unit input, with the weight w_0 becoming the bias.



Supervised Learning

Perceptrons

- **The learning algorithm is very simple:**
 - Present $(\mathbf{x}_i, y_i)$. Input $\mathbf{x}_i$ may be a $(k+1)$ -element vector of elements $[x_{i1}, x_{i2}, \dots, x_{i, k+1}]$. The additional $+1$ element in the vector is the bias.
 - FEEDFORWARD: Compute:

$$y_{out} = f \left(\sum_{j=1}^{k+1} w_j x_{ij} \right)$$

Here $f(\cdot)$ is the activation function.

- UPDATE: Update each w_j with:

$$w_j = w_j + \alpha(y_i - y_{out})x_{ij}$$

Supervised Learning

Perceptrons (perceptron.xlsx)

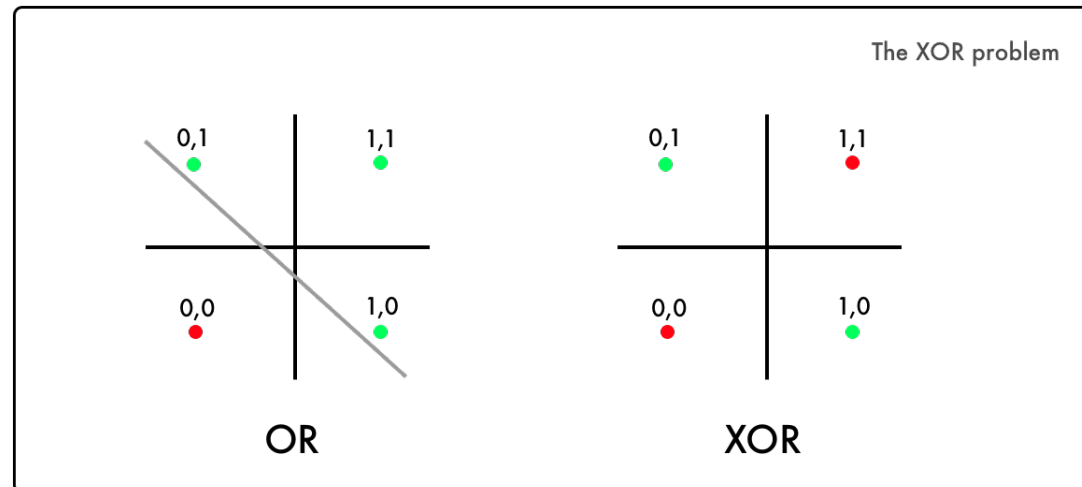
- **Function learning (Activation Function: $\text{id}(x)=x$)**
 - $y(x) = 2x$
 - $y(x)=2x+5$
 - $y(x) = 2x+5 + \text{noise}$
 - $y(x) = 2x+5 + \text{large noise}$
- **Classification:**
 - Given the length and weight of a vehicle we want to classify either as a lorry (-1) or a van (1)
 - Activation function $f(x)=1$ if $x>0$, -1 otherwise.
- **XOR Problem:**
 - Learn the XOR table.
 - Activation: $f(x)=1$ if $x>0$, 0 otherwise.

Inputs		Outputs
X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	0

Supervised Learning

Perceptrons

- **From our example we see that Perceptrons cannot learn the XOR problem.**
 - This is because it is not possible to draw a straight line to separate the samples into two classes.
 - Neural network research was killed for several years because no solution could be found for this problem.

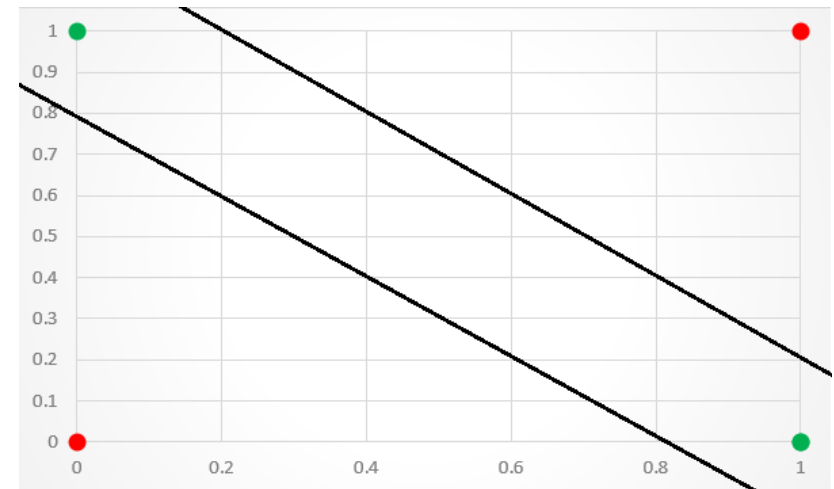
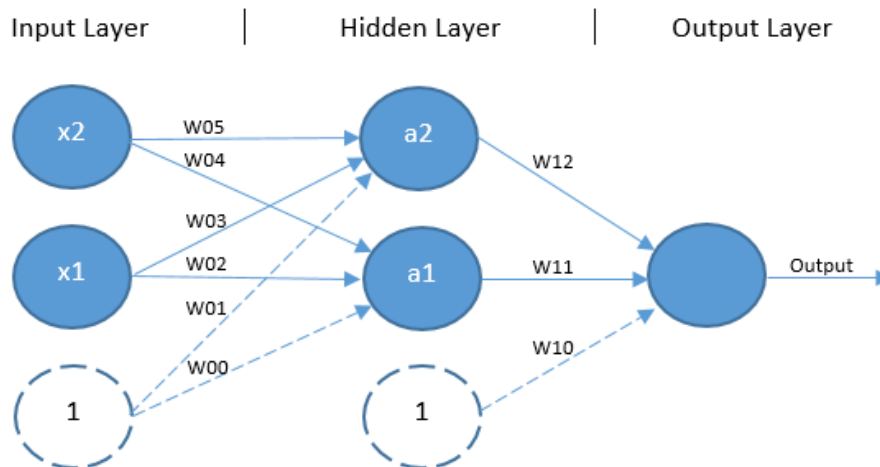


SUPERVISED LEARNING: MULTI-LAYER PERCEPTRONS

Supervised Learning

Multi-Layer Perceptrons

- Our key issue in the XOR problem with a single Perceptron is that we cannot separate the points in the XOR problem with a SINGLE linear plane.
- However if we add a second Perceptron layer, we introduce a second plane that can neatly separate the points:



Supervised Learning

Multi-Layer Perceptrons

- **An MLP consists of:**

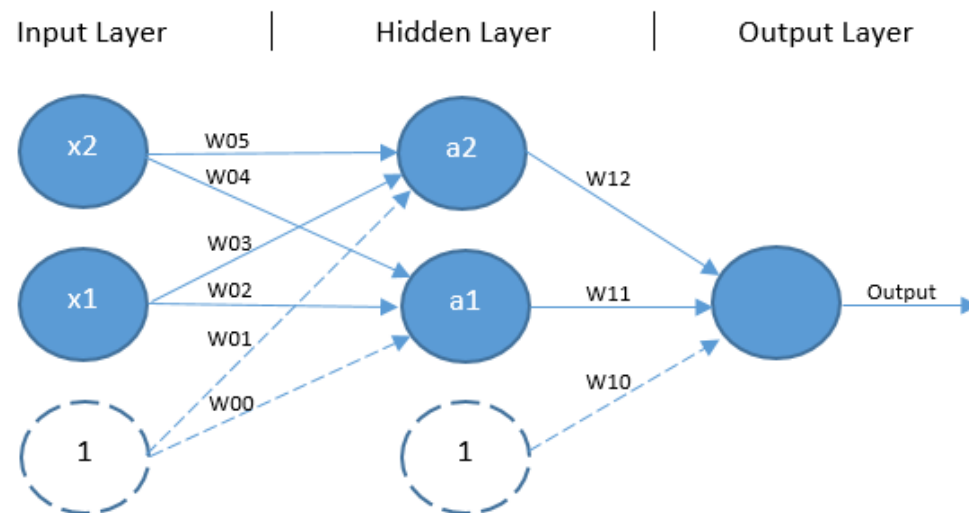
- A set of inputs $\mathbf{x}$ with $n+1$ elements $[x_1, x_2, \dots, x_n, 1]$. As before the $n+1$ item is the bias.
- A set of m Perceptron nodes called “hidden nodes”, $\mathbf{h}$. As always we add an $m+1$ term for the bias.
- A set of weights $\mathbf{w}_h$ connecting $\mathbf{x}$ to $\mathbf{h}$.
- A set of q Perceptron nodes $\mathbf{p}$ for the output.
- A set of weights $\mathbf{w}_p$ connecting $\mathbf{h}$ to $\mathbf{p}$.
- **FEEDFORWARD:**

$$h_j = f \left(\sum_{i=1}^{m+1} w_{ij} x_i \right)$$
$$p_k = g \left(\sum_{j=1}^{q+1} w_{jk} h_j \right)$$

Supervised Learning

Multi-Layer Perceptrons

- **The update step is considerably more complicated than single layer Perceptrons:**
 - In SLP there is only one “output” layer. It is relatively easy to compute the error, which is simply $y_t - y_{out}$.
 - In MLP, things get more complicated in the hidden layer. How do we assign errors to this layer?



Supervised Learning

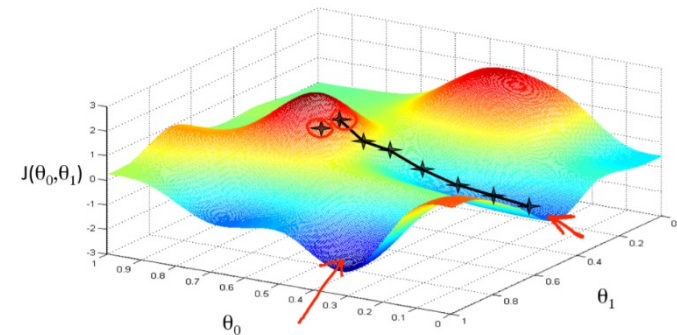
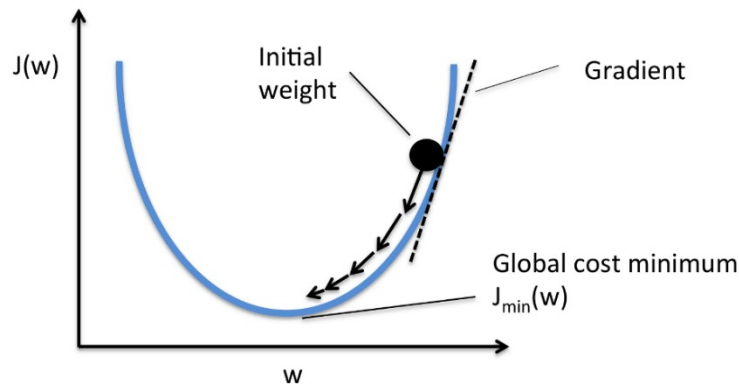
Multi-Layer Perceptrons

- In MLPs we use a technique called “gradient descent”:
 - In the output layer we find the quadratic error for output k :

$$E = \frac{1}{2} (p_k - y_t)^2$$

- We want to minimize E :

$$\frac{dE}{dw_{jk}} = 0$$

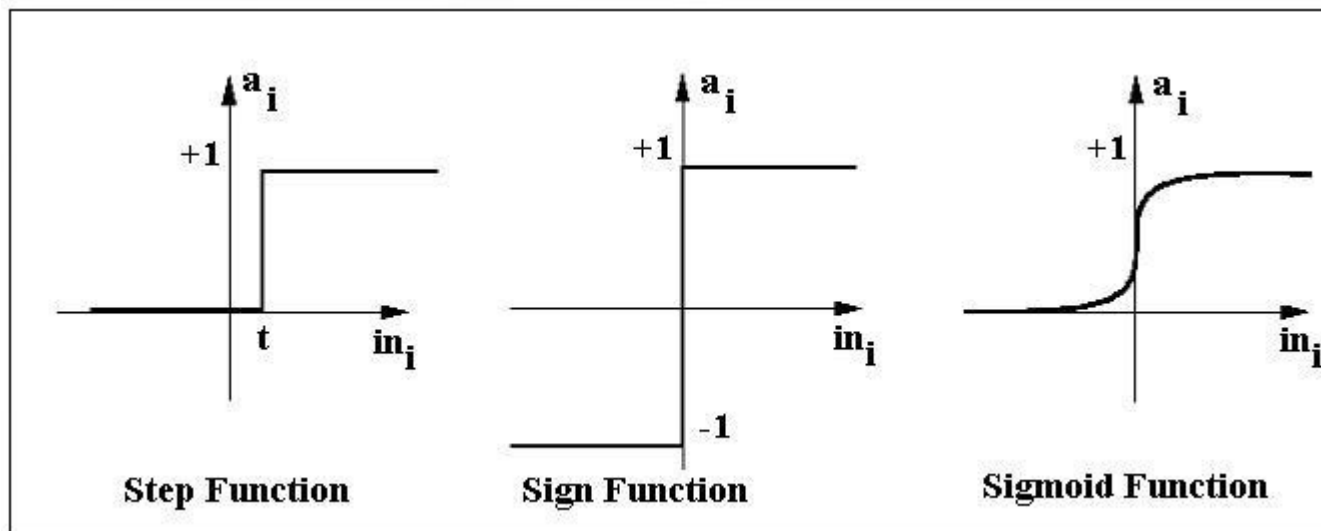


Supervised Learning

Multi-Layer Perceptrons

- **This means that the activation functions $f(.)$ and $g(.)$ must be differentiable:**
 - Our step (and sign) function has a discontinuity at 0, where the gradient is infinite. We choose instead the sigmoid function:

$$f(x) = \frac{1}{1 + e^{-x}}$$



Supervised Learning

Multi-Layer Perceptrons

- The sigmoid has a very convenient derivative:

$$f(x) = \frac{1}{1 + e^{-x}}$$
$$\frac{df(x)}{dx} = f(x)(1 - f(x))$$

- From here we can find the derivative of the quadratic error E of an output node p_k :

$$E = \frac{1}{2}(p_k - y_t)^2$$
$$\frac{dE}{dw_{jk}} = (p_k - y_t) \frac{dp_k}{dw_{jk}}$$
$$= p_k(1 - p_k)(p_k - y_t)$$

Supervised Learning

Multi-Layer Perceptrons

- Similar to what we did for SLPs, we adjust the weights connecting the hidden layer to the output layer proportional to the derivative and the “input” to the output layer (which is the output of the hidden layer):

$$\delta_{jk} = h_j p_k (1 - p_k) (p_k - y_t)$$

- We can now update the weight w_{jk}

$$w_{jk} = w_{jk} - \alpha \delta_{jk}$$

where α is the learning rate.

Supervised Learning

Multi-Layer Perceptrons

- **Now we are ready to tackle the more challenging issue: Adjusting the weights connecting the inputs to the hidden layer.**

- In the output layer we took into account the error in the output layer.
- In the hidden layer we must similarly take into account all the corrections applied to every weight connecting h_j to every output node p_k . We do this by summing over all the corrections derived from each output node p_k :

$$\sum_{k=1}^{m+1} w_{jk} \delta_{jk}$$

- We multiply this by the derivative of h_j :

$$h_j(1 - h_j) \sum_{k=1}^{m+1} w_{jk} \delta_{jk}$$

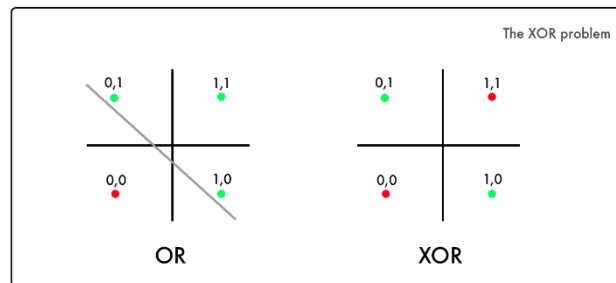
- Finally our correction to w_{ij} is:

$$w_{ij} = w_{ij} - \alpha x_i h_j(1 - h_j) \sum_{k=1}^{m+1} w_{jk} \delta_{jk}$$

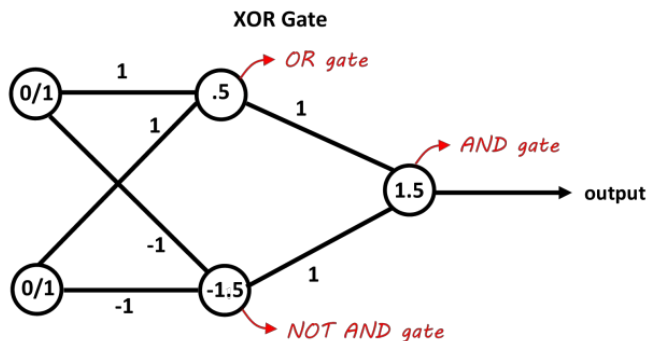
Supervised Learning

Multi-Layer Perceptrons – XOR Problem

- Recall that Perceptrons cannot solve the XOR problem:



- Adding a hidden layer solves this problem:

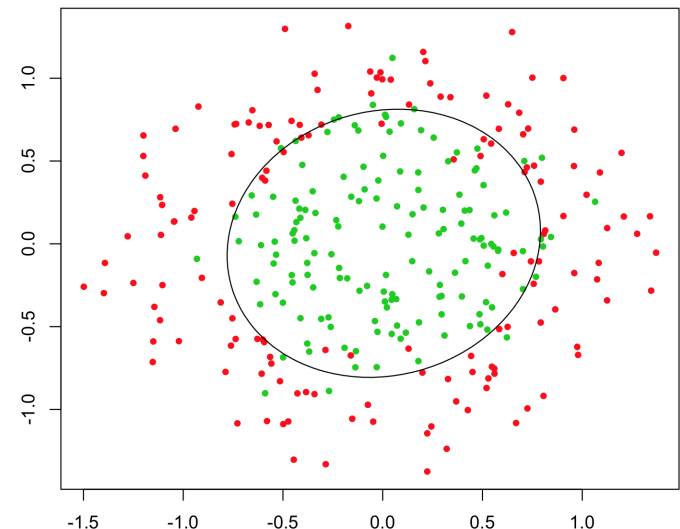
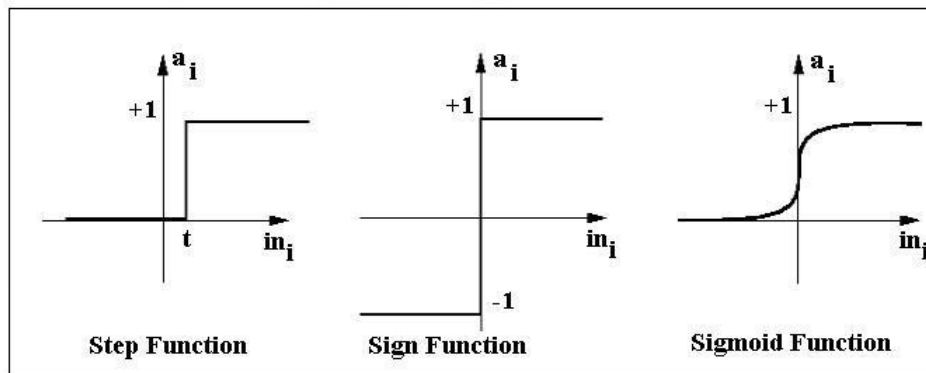


SUPERVISED LEARNING: INTRODUCING NON-LINEARITY

Supervised Learning

Non-Linearity

- **The addition of a hidden layer only partially solves the problem of classifying data points:**
 - What if the data points occur in a circle? No straight hyperplane can solve this.
- **Solution: Use non-linear decision boundaries.**
 - Popular ones: Sigmoid for $(0, 1)$ outputs, tanh for $[-1, 1]$ outputs, and ReLu.



Neural Network Hyperparameters

- **The weights that are being adjusted by our optimization algorithms are called “parameters”, and neural networks are known as “parameterized models”.**
- **There is another dimension of neural networks that must be “optimized” as well to get good results, called “hyperparameters”. This is related to the design of the neural network. Important hyperparameters include:**
 - Architecture (Dense networks, Long-Short-Term Memories, Convolutional Neural Networks, Autoencoders, Generative-Adversarial Networks, etc)
 - # of input nodes (usually constraints by the problem itself)
 - Input and output encoding.
 - # of hidden layers.
 - Size of each hidden layer.
 - Loss functions
 - Transfer functions.
 - Optimization functions and their parameters (learning rate, momentum, etc.)
 - Dropouts and Regularizers.

Neural Network Hyperparameters

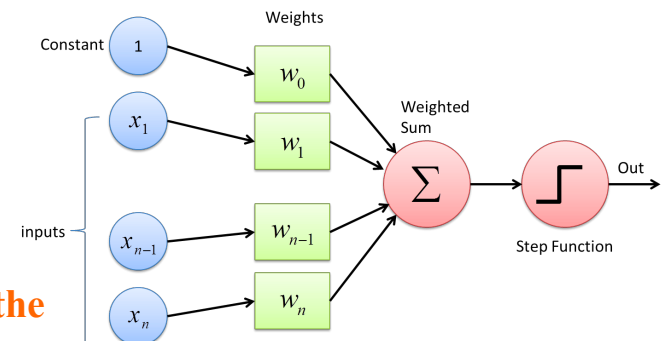
- **Some of the parameters are constrained by the problem itself (e.g. input and output size), others are found by trial-and-error (yes really).**
- **We will look at a subset of hyperparameters that are slightly more “scientific”**
 - **Transfer Functions**
 - ✓ **We’ve already seen the sigmoid and step functions.**
 - **Dropouts and Regularizers**
 - ✓ **These control the issue of “overfitting”**

Recall: Neural Networks and How They Work.

- **Generally neural network applications can be split into two classes:**
 - **Classification:**
 - ✓ Given a set of inputs $\{x_0, x_1, \dots, x_{n-1}\}$, assign labels y_j to each input x_i from the set of classes $\{y_0, y_1, \dots, y_{m-1}\}$, to mean that x_i is a member of class y_j .
 - ✓ E.g. we may classify a library of books into the genres of novels, cookbooks, self-improvement books, etc.
 - **Regression:**
 - ✓ Give a series of data of data $\{x_0, x_1, \dots, x_{n-1}\}$, can we predict x_n ?
- **In either case we are attempting to learn a function $f(x)$:**
 - **Classification:** What $f(.)$ gives $f(x_i) = y_i$?
 - **Regression:** What $g(.)$ gives $g(x_{n-1}) = x_n$?

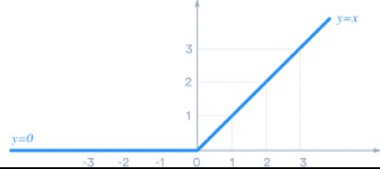
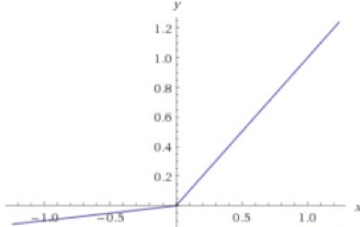
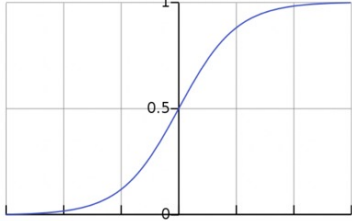
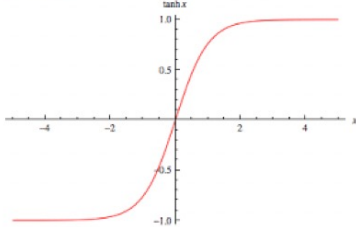
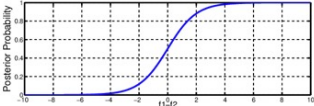
Recall: Neural Networks and How They Work.

- The figure below shows an example neuron:
 - The sum of each neuron is passed through an *activation function* to transform it in some way.
 - ✓ E.g. scale it from -1 to 1 or 0 to 1.
 - ✓ Destroy the linearity of the output.
 - ✓ Shape it into a statistical distribution.
 - The weights need to be adjusted to create $f(x)$:
 - ✓ We must know the error $\|y_i - f(x_i)\|$. This is called the “loss function”
 - ✓ We must have an algorithm to adjust the weights accordingly. This is done by “optimizers”.
- Let’s now take a closer look at each of these important parts.



Activation Functions

- **An activation function (often also called “transfer function”) transforms the dot product of the weights and inputs.**
 - Needed because this dot product is usually unbounded.
 - Also because the dot-product is a linear operation. Linearity prevents NNs from solving certain classes of problems. Explained in next section.
 - ✓ **One obvious limitation: A linear function cannot regress over non-linear functions.**
- **The table on the next slide shows the various common activation functions.**

Function	Description	Shape	Range	Primary Use
Identity	$f(x) = x$	Straight line	$[-\infty, \infty]$	Linear regression.
ReLU	$f(x) = x$ if $x > 0$ $f(x) = 0$ if $x \leq 0$		$[0, \infty]$	Linear regression, classification
Leaky ReLU	$f(x) = x$ if $x > 0$ $f(x) = \alpha x$ if $x \leq 0$, $0 < \alpha \leq 1$		$[-\infty, \infty]$	Linear regression, classification
Sigmoid (Logistic)	$S(x) = \frac{1}{1 + e^{-x}}$		$[0, 1]$	Classification, regression (scaled)
tanh	$\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$		$[-1, 1]$	Classification, regression (scaled)
Softmax	$S(x) = \frac{e^x}{\sum_j e^{x_j}}$		$[0, 1]$	Classification (Makes output a statistical distribution)

Loss Functions

- A loss function measures the error between the function $f(x_i)$ learnt by the NN and the actual target value y_i .
 - In regression y_i might just be x_{i+l} .
- The table on the next page shows the common loss functions:

Loss Function	Formulation	Use
Mean Squared Error (MSE)	$\forall t \in \{(x_i, y_i), x_i \in X, y_i \in Y\}$ $L_{MSE}(T) = \sum_{i=0}^{ T } (y_i - f(x_i))^2$	Regression. Fast convergence, sensitive to outliers.
Mean Absolute Error (MAE)	$\forall t \in \{(x_i, y_i), x_i \in X, y_i \in Y\},$ $L_{MAE}(T) = \sum_{i=0}^{ T } y_i - f(x_i) $	Regression. Less sensitive to outliers.
Binary Cross Entropy	$\forall t \in \{(x_i, y_i), x_i \in X, y_i \in Y\},$ $L_{BCE}(T) = -\frac{1}{ T } \sum_{i=0}^{ T } (y_i \log(f(x_i)) + (1 - y_i) \log(1 - f(x_i)))$	Binary classification. Use with softmax.
Hinge	$\forall t \in \{(x_i, y_i), x_i \in X, y_i \in Y\},$ $L_H(T) = \max(0, 1 - y_i f(x_i))$	Binary classification.
Categorical Cross Entropy	With m classes and n training samples: $L_{CE}(y, y') = - \sum_{j=1}^m \sum_{i=1}^n y_{ij} \log(y'_{ij})$ y'_{ij} is the jth output of $f(x_i)$, while y_{ij} is the jth output of the one-hot target vector y_i. I.e. $y_{ij} = 1$ iff x_i belongs to class j, 0 otherwise.	Multiclass classification. Use with softmax.

Optimizers

- In NNs, optimizers are algorithms that derive the best set of parameter values that minimize the loss function (see later).
- Table shows a small set of optimizers and their characteristics – No straightforward way to choose!

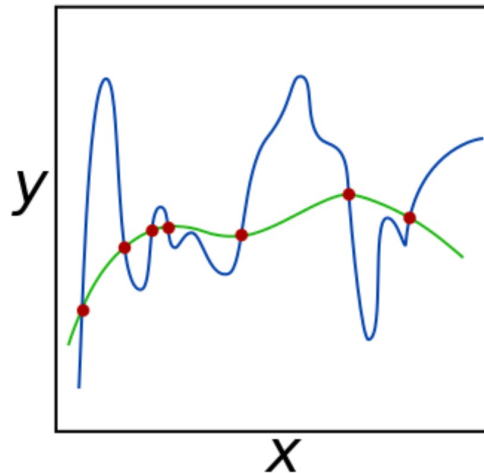
Algorithm	Characteristic
Stochastic Gradient Descent (SGD)	Classic algorithm, same learning rate is applied to all weights. Momentum may be used to speed up learning.
RProp	Each weight uses a different learning rate. Weights that have two consecutive update gradients of the same sign are updated more.
RMSProp	An improved version of RProp for mini-batches.
Adagrad	Weights that are updated more have smaller training rates (i.e. train slower). Weights that are not updated frequently have larger training rates. Good for sparse data.
Adadelata	Less aggressive form of Adagrad.
Adam	Similar to Adagrad

OVERFITTING

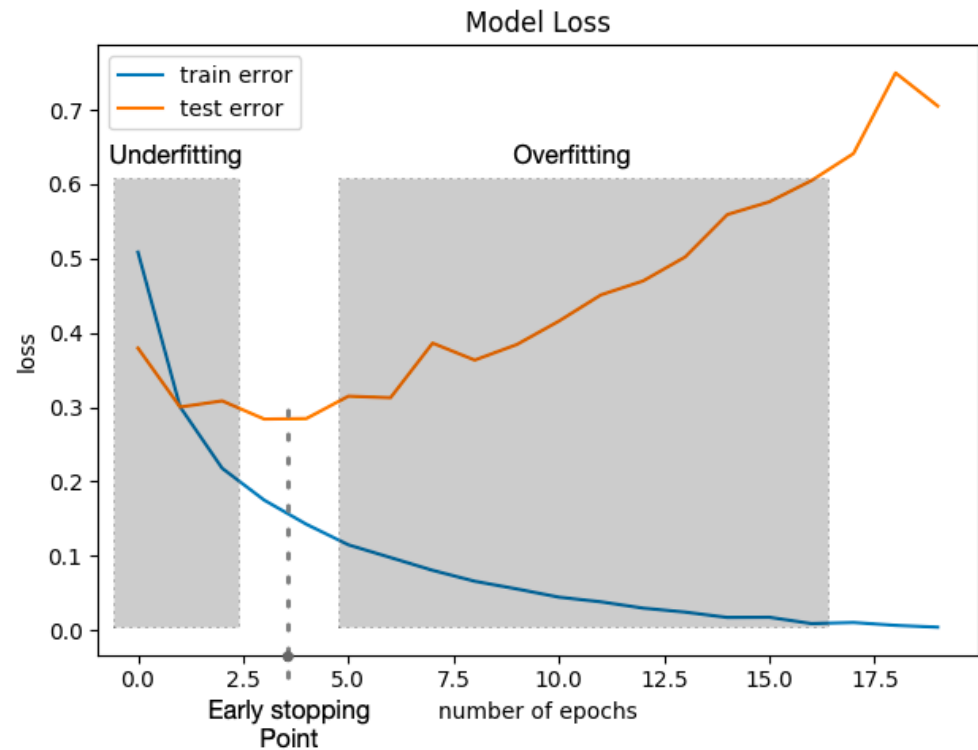
Overfitting

- **All data can be thought of as consisting of two things:**
 - Some sort of generator process $f(x)$. The input x could simply be time, or an actual input.
 - Noise
- **When we build a model, we want it to learn only the generator process $f(x)$.**
 - The noise is random and unique to the training data. If we learn the noise then our model cannot generalize well – “overfitting”.
- **Two choices to avoid learning the noise:**
 - Have a simple model (i.e. fewer parameters)
 - Have more training data so that the network can “average out” the noise.
- **Catch:**
 - A simple model may not learn $f(x)$ well – underfitting.
 - The amount of training data needed grows exponentially with the model complexity – “curse of dimensionality”

Overfitting



- **When we have too many parameters, we tend to learn the blue noisy line instead of the better green line:**
- **Marked by:**
 - Extremely low training loss.
 - Increasing validation loss.
 - Hence why we must check both!

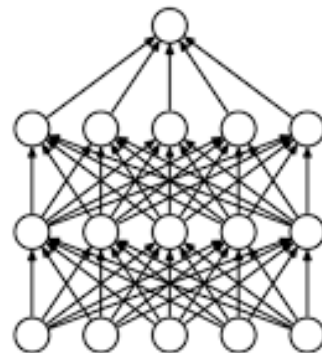


Dealing with Overfitting

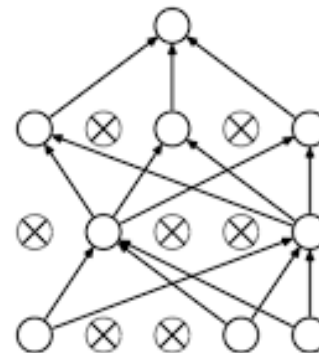
Noise Layers

- **Dropout Layers:**

- A fixed percentage of neurons in the layer are dropped from training for one more epochs.
- They are then put back in, and another percentage of neurons are dropped.
- Reduces # of training parameters:



(a) Standard Neural Net



(b) After applying dropout.

Dealing with Overfitting

Noise Layers

- **Noise layers add random noise to the outputs of the previous layer.**
 - E.g. add Gaussian noise to change the data to look like “new data” – augmentation.
 - ✓ **In Gaussian noise the values of the noise are distributed according to the Gaussian probability distribution function:**

$$p_G(z) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(z-\mu)^2}{2\sigma^2}}$$

✓ **In images this simulates sensor noise, e.g. due to low-light.**

- More details at <https://keras.io/layers/noise/>



Dealing with Overfitting

Regularizers

- **Regularizers:**

- A “regularizer” is a penalty that is applied to the loss function of a layer.
- E.g. In regression our loss function may look like this (squared error loss):

$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 .$$

- Our optimization algorithm will find values of the parameters (β) to minimize this loss function.
- If there are too many parameters (β) the model starts to learn the unique noise of the training data – overfitting or “memorizing”. Poor generalization.
- We force a reduction – simplification - in parameters:
 - ✓ **This is equivalent to forcing some of the parameters to 0.**

Dealing with Overfitting

Regularizers

- We can do this by adding the absolute values of the parameters (L1 regularization) or squares of the parameters (L2 regularization) to the loss function:

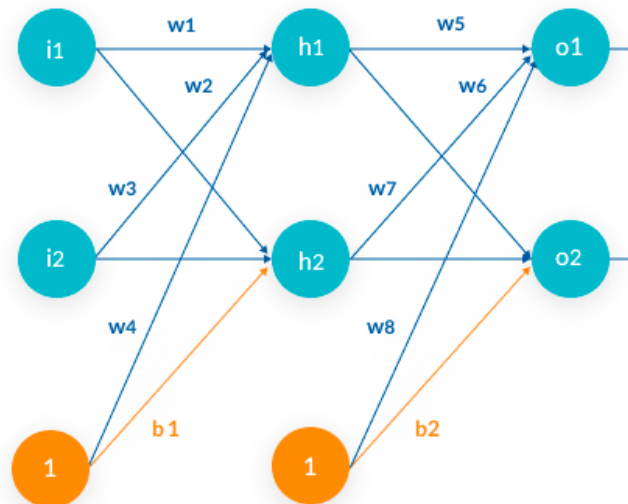
$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

- Now our optimizer will restrict the parameters themselves:
 - ✓ Some will be forced to zero while others are not allowed to grow too big.
 - ✓ Otherwise the loss function will become too much.
- The hyperparameter λ controls how much “flexibility” we give to the parameters. Higher λ means more strict control.
- L1: Eliminates less important parameters and simplifies the output.
- L2: More effective in severe overfitting since it squares the parameters.
- If λ is too high the model will underfit.

Dealing with Overfitting

Regularizers

- **Regularizers are added to individual layers:**
 - **kernel_regularizer:** Controls the main weights (lines connecting the green nodes)
 - **bias_regularizer:** Controls the bias weights (lines connecting the orange nodes)
 - **activity_regularizer:** Controls based on layer output (o1 and o2)



Dealing with Overfitting

Regularizers

- **Example (From Keras, which we will see in the hands-on)**
 - The λ is specified in the brackets of `regularizers.l1(..)` or `regularizers.l2(..)`.
 - Here it is set to 0.01.

```
from keras import regularizers
model.add(Dense(64, input_dim=64,
                  kernel_regularizer=regularizers.l2(0.01),
                  activity_regularizer=regularizers.l1(0.01)))
```

Neural Network Hands-On

Load up NN.ipynb in Jupyter Notebook

Keras Resources:

- **Sequential Model:** <https://keras.io/models/sequential/>
- **Functional API:** <https://keras.io/models/model/>
- **Core Layers (including Dropout):** <https://keras.io/layers/core/>
- **Noise Layers:** <https://keras.io/layers/noise/>
- **Convolution Layers:** <https://keras.io/layers/convolutional/>
- **Pooling Layers:** <https://keras.io/layers/pooling/>
- **Recurrent Layers (including LSTM):** <https://keras.io/layers/recurrent/>
- **Regularizers:** <https://keras.io/regularizers/>
- **Activations:** <https://keras.io/activations/>
- **Losses:** <https://keras.io/losses/>
- **Optimizers :** <https://keras.io/optimizers/>

Summary

- **In this lecture we saw:**
 - How unsupervised and supervised learning works.
 - The various activation and loss functions and optimization algorithms.
 - How to deal with overfitting.
 - NN algorithms.

IT5005

Artificial Intelligence

Lecture 8: Constraint Satisfaction Problems.

Previously...

- Minimax algorithm

- What if MIN plays sub-optimally and MAX plays MINIMAX strategy?
- Generalize to non-zero-sum, multi-player game?
See section 5.2.2 of AIMA 3rd Ed.

- α - β pruning algorithm

- Prune subtrees that won't affect minimax decision

- H -Minimax algorithm

- Use heuristic functions and impose depth limit

- Expected Minimax algorithm

- Solve stochastic games

Previously...

- Neural Networks

- Application of gradient descent to minimize errors in the estimation function.

Constraint Satisfaction Problems

AIMA Chapter 6.1 – 6.4

Outline

- Constraint satisfaction problems (CSPs)
- Backtracking search for CSPs
- Local consistency in constraint propagation
- Local search for CSPs

CSP: a Broad Modelling Framework

Linear programming

- Variables are all rational values
- Linear constraints

Combinatorial optimization

- Graph problems: vertex cover, bipartite matching, minimum spanning tree...
- Operations Research: scheduling, bin packing, planning, task allocation
- Game theory and economics: social choice, market pricing, goods allocation...

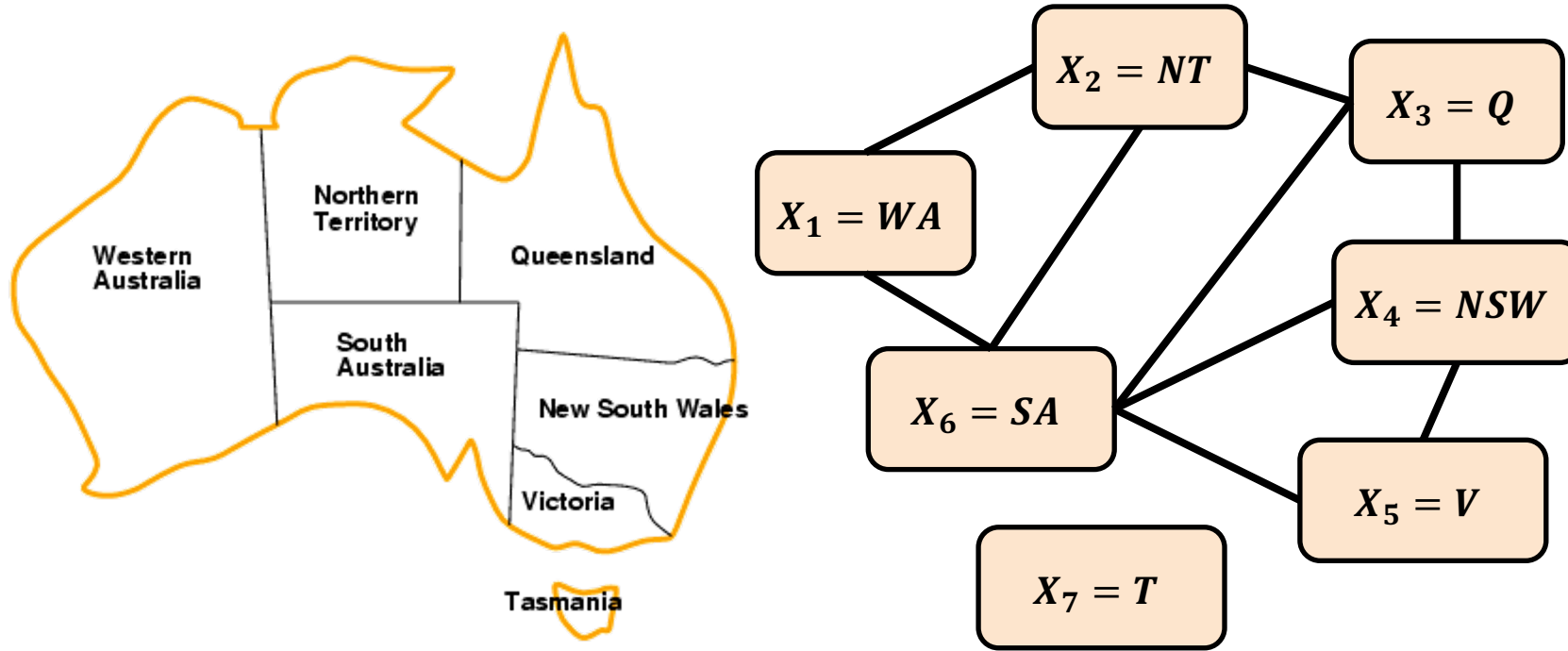
Effective CSP Solvers

- Variable dependencies reduce search space significantly
- Solve very large problem instances very quickly

Constraint Satisfaction Problems (CSPs)

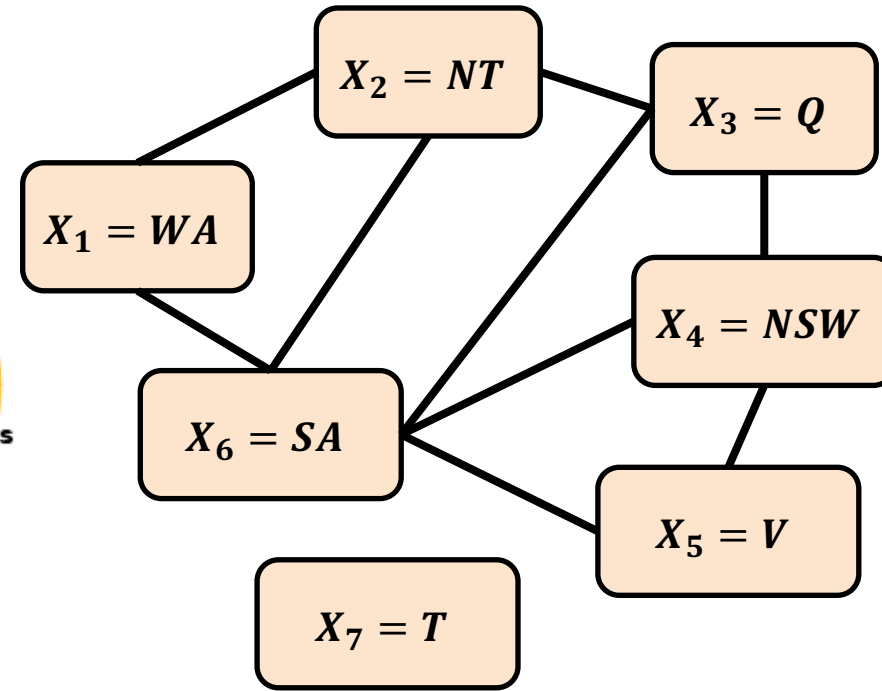
- Standard search problem:
 - **state** is atomic – any data structure that supports transition function, heuristic function, and goal test
- CSP
 - Variables $\vec{X} = X_1, \dots, X_n$; each variable X_i has a domain D_i
 - Constraints $\vec{C}$: what variable combinations are allowed?
 - Constraint scope: which variables are involved
 - Constraint relation: what is the relation between them.
 - Objective: find a legal assignment $(y_1, \dots, y_n)$ of values to variables
 - $y_i \in D_i$ for all $i \in [n]$
 - Constraints are all satisfied.

Example: Graph Coloring



Variables:	$\vec{X} = \langle WA, NT, Q, NSW, V, SA, T \rangle$
Domains:	$D_i = \{R, G, B\}$
Constraints:	If $(X_i, X_j) \in E$ then $color(X_i) \neq color(X_j)$

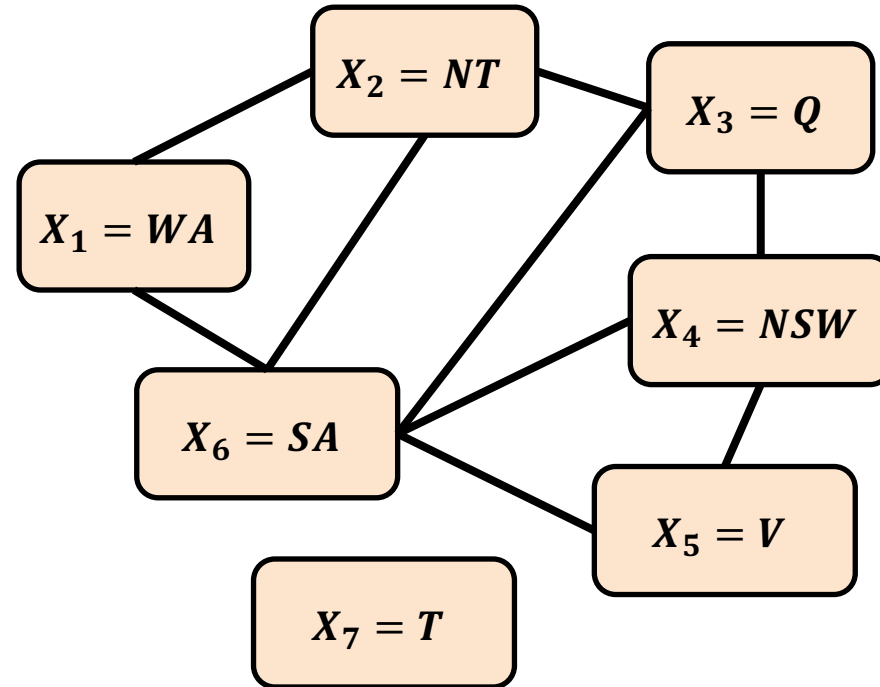
Example: Graph Coloring



Find a complete, consistent assignment

- Complete: all variables are set
- Consistent: no constraint is violated

Constraint Graph



- **Binary CSP:** each constraint relates two variables
- **Constraint graph:** nodes are variables, links are constraints

Example: Job-Shop Scheduling

- Car assembly consists of 15 tasks
- Variables: $Axle_F, Axle_B, Wheel_{LF}, Wheel_{RF}, Wheel_{LB}, Wheel_{RB}, Nuts_{LF}, Nuts_{RF}, Nuts_{LB}, Nuts_{RB}, Cap_{LF}, Cap_{RF}, Cap_{LB}, Cap_{RB}, Inspect$
- Domain: $D_i = \{1, 2, \dots, 27\}$
- Precedence constraints
 - e.g., $Axle_F + 10 \leq Wheel_{RF}$
- Disjunctive constraint
 - e.g., $(Axle_F + 10 \leq Axle_B) \text{ or } (Axle_B + 10 \leq Axle_F)$

CSP Variants

Discrete variables

- Finite domains:
 - n variables, domain size $d \rightarrow \mathcal{O}(d^n)$ complete assignments
 - e.g., 8-queens
- Infinite domains:
 - integers, strings, etc.
 - e.g., job-shop scheduling: variables are start/end days for each job
 - need a constraint language, e.g., $StartJob_3 + 5 \leq StartJob_4$

Continuous variables

- e.g., start/end times for Hubble Space Telescope observations
- Linear programming problems (i.e., with linear constraints) can be solved in polynomial time

Constraint Variants

Unary constraints

- Single variable: e.g., $SA \neq G$

Binary constraints

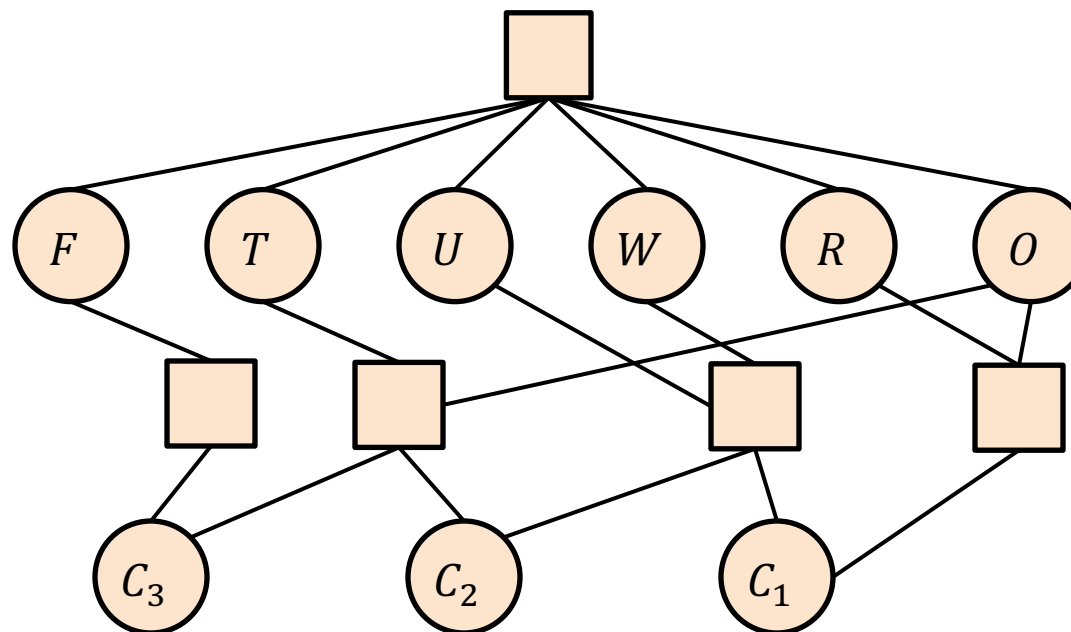
- e.g., $SA \neq WA$

Global (i.e., higher-order) constraints

- 3 or more variables e.g. $X_1 + X_2 - 4X_7 \leq 15$

Cryptarithmic Puzzles

$$\begin{array}{r} TWO \\ + TWO \\ \hline FO UR \end{array}$$



Variables:	$\vec{X} = \langle F, T, U, W, R, O, C_1, C_2, C_3 \rangle$
Domains:	$D_i = \{0, \dots, 9\}$
Constraints:	$AllDiff(F, T, U, W, R, O)$ $O + O = R + 10C_1$ $C_1 + W + W = U + 10C_2$ $C_2 + T + T = O + 10C_3$ $C_3 = F$ $T, F \neq 0$

Sudoku

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

(a)

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

(b)

Variables:	$A_1, \dots, A_9, \dots, I_1, \dots, I_9$ (81 variables)
Domains:	$D_i = \{1, \dots, 9\}$
Constraints:	$AllDiff(\dots) \times 27$ (9 columns, 9 rows, 9 boxes) e.g. $AllDiff(A_1, \dots, A_3; B_1, \dots, B_3; C_1, \dots, C_3)$ is the constraint for the top-left box.

Standard Search Formulation (Incremental)

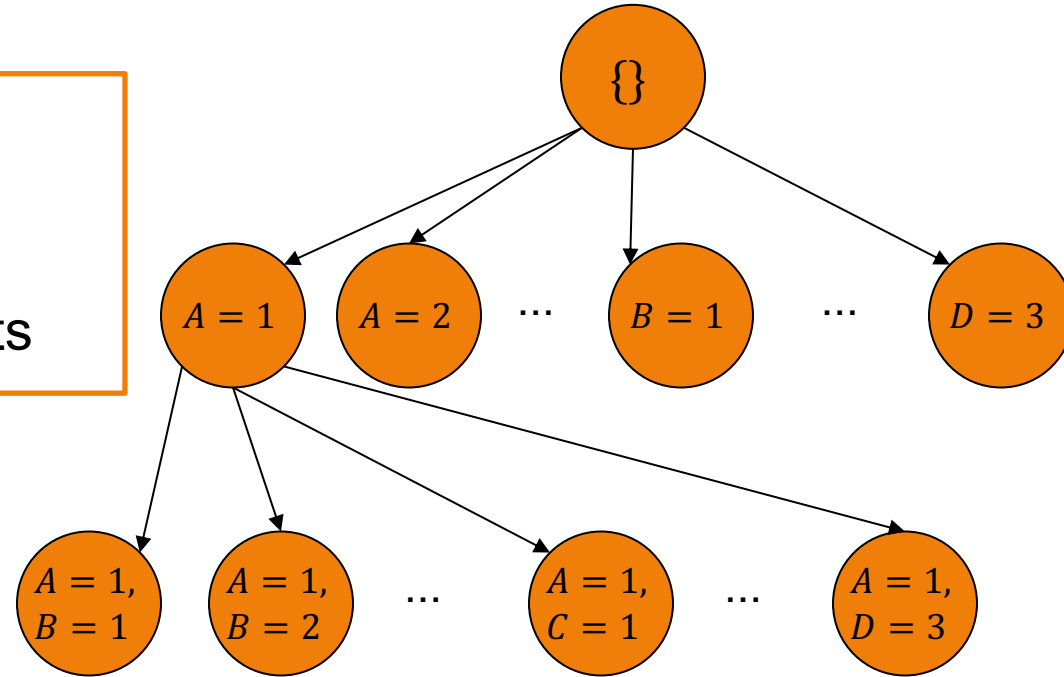
- Each state is a partial variable assignment
- **Initial state**: the empty assignment []
- **Transition function**: assign a **valid** value to an unassigned variable → fail if no **valid** assignments
- **Goal test**: all variables have been assigned.
 1. Uniform model for all CSPs; not domain-specific.
 2. Every solution appears at depth n (n variables assigned)
 3. Search path is irrelevant, can also use complete-state formulation.

What's the best search technique?

CSP Search Tree Size

CSP

- Variables: A, B, C, D
- Domains: $\{1, 2, 3\}$
- Assume no constraints



b at depth 0 = 4 variables $\times$ 3 values = 12 states

b at depth 1 = 3 variables $\times$ 3 values = 9 states

b at depth 2 = 2 variables $\times$ 3 values = 6 states

b at depth 3 = 1 variable $\times$ 3 values = 3 states

At depth ℓ : $(n - \ell)d$

Total number of leaves = $n! d^n$

Backtracking Search

Order of variable assignment is irrelevant

- $[WA = R \text{ then } NT = G]$ equivalent to $[NT = G \text{ then } WA = R]$

At every level, consider assignments to a single variable.

- Fix an order of variable assignment $\rightarrow b = d$ and there are d^n leaves

DFS for CSPs with single-variable/level assignments is called backtracking search

- Is the basic uninformed search algorithm for CSPs
- Backtracks when no legal assignments
- Can solve n -queens for $n \approx 25$

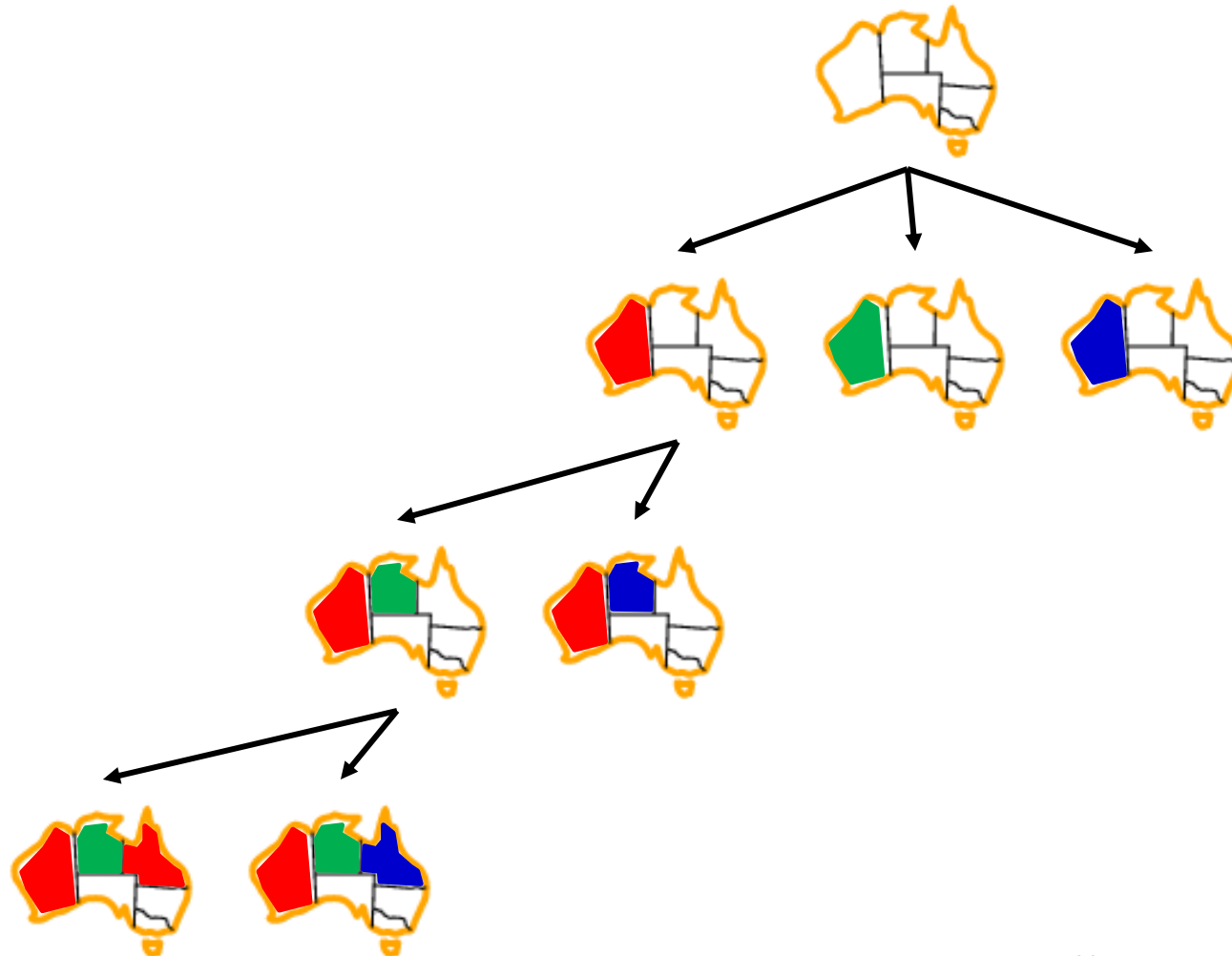
Backtracking Search

function BACKTRACKING-SEARCH(*csp*) **returns** a solution, or failure
 return BACKTRACK($\{ \}$, *csp*)

function BACKTRACK(*assignment*, *csp*) **returns** a solution, or failure
 if *assignment* is complete **then return** *assignment*
 var $\leftarrow$ SELECT-UNASSIGNED-VARIABLE(*csp*)
 for each *value* in ORDER-DOMAIN-VALUES(*var*, *assignment*, *csp*) **do**
 if *value* is consistent with *assignment* **then**
 add $\{var = value\}$ to *assignment*
 inferences $\leftarrow$ INFERENCE(*csp*, *var*, *value*)
 if *inferences* $\neq$ failure **then**
 add *inferences* to *assignment*
 result $\leftarrow$ BACKTRACK(*assignment*, *csp*)
 if *result* $\neq$ failure **then**
 return *result*
 remove $\{var = value\}$ and *inferences* from *assignment*
 return failure

Backtracking Example

Assign variables in the order of *WA*, *NT*, *Q*, ...



Improving Backtracking Efficiency

- **General-purpose** heuristics can give huge time gains:
 - ❑ SELECT-UNASSIGNED-VARIABLE: Which variable should be assigned next?
 - ❑ ORDER-DOMAIN-VALUES: In what order should its values be tried?
 - ❑ INFERENCE: How can domain reductions on unassigned variables be inferred?
 - ❑ Can we detect inevitable failure early?

Most Constrained Variable

- Most constrained variable:
 - choose the variable with fewest legal values



- a.k.a. **minimum-remaining-values (MRV)** heuristic

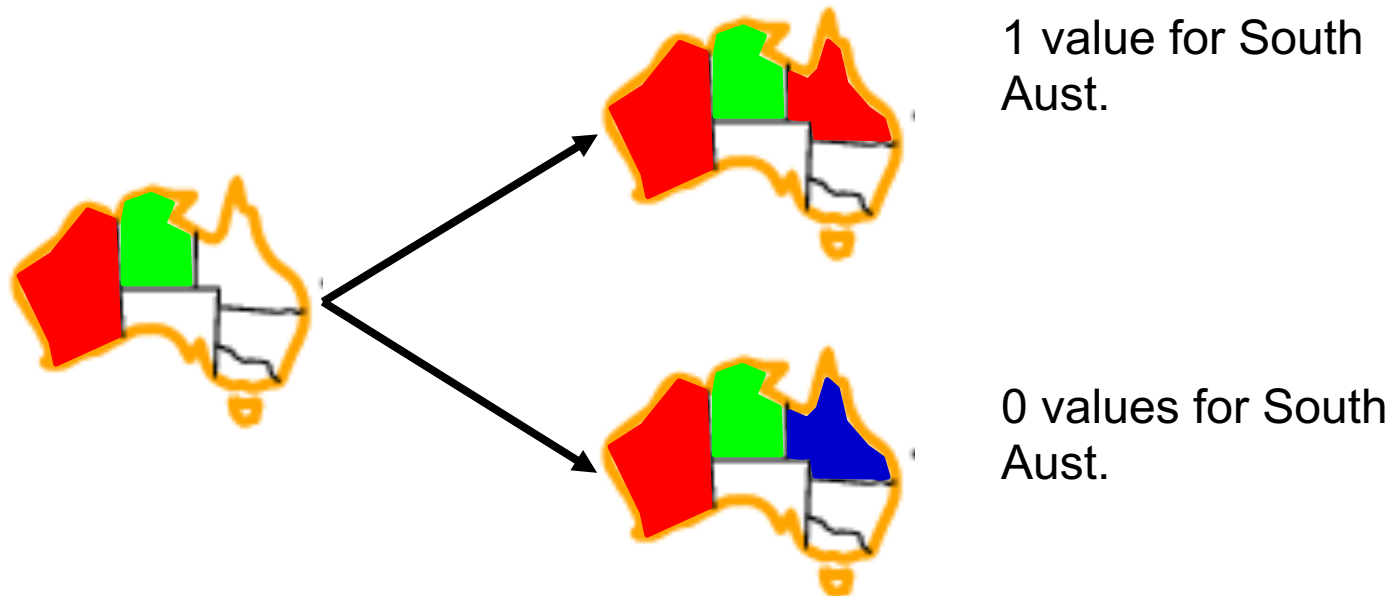
Most Constraining Variable

- Most constraining variable:
 - choose the variable with most constraints on remaining unassigned variables
- Tie-breaker among most constrained variables
- a.k.a. **degree** heuristic



Least Constraining Value

- Given a variable, choose the least constraining value:
 - the one that rules out the fewest values for neighboring unassigned variables



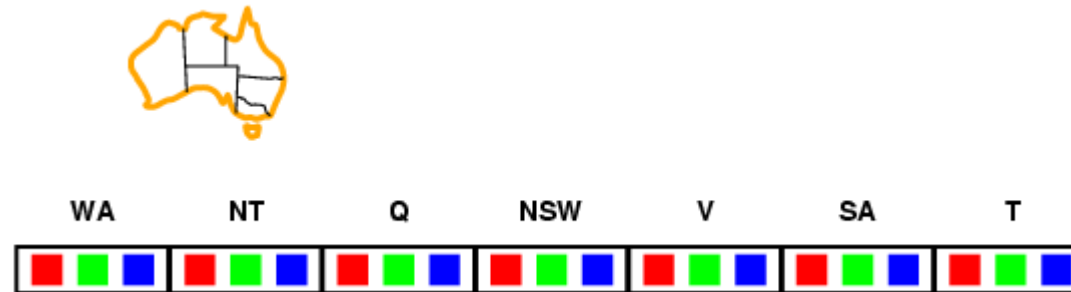
Forward Checking and Arc Consistency

INFERENCE IN CSPS

Forward Checking

Idea:

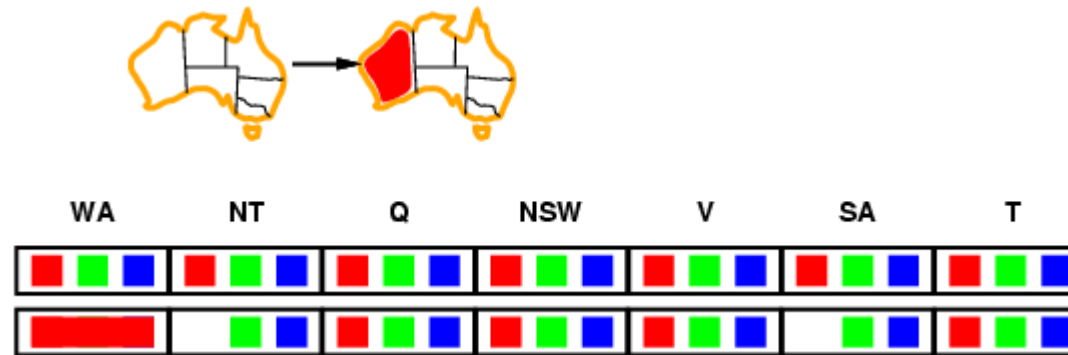
- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values



Forward Checking

Idea:

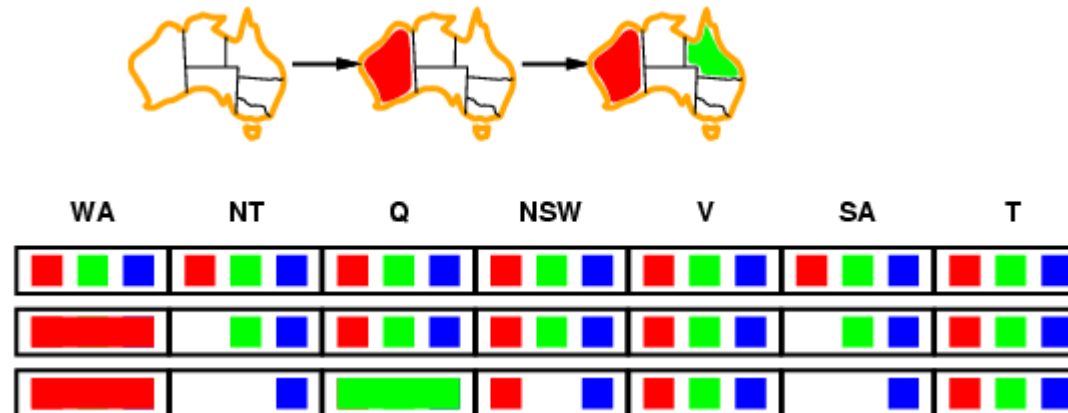
- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values



Forward Checking

Idea:

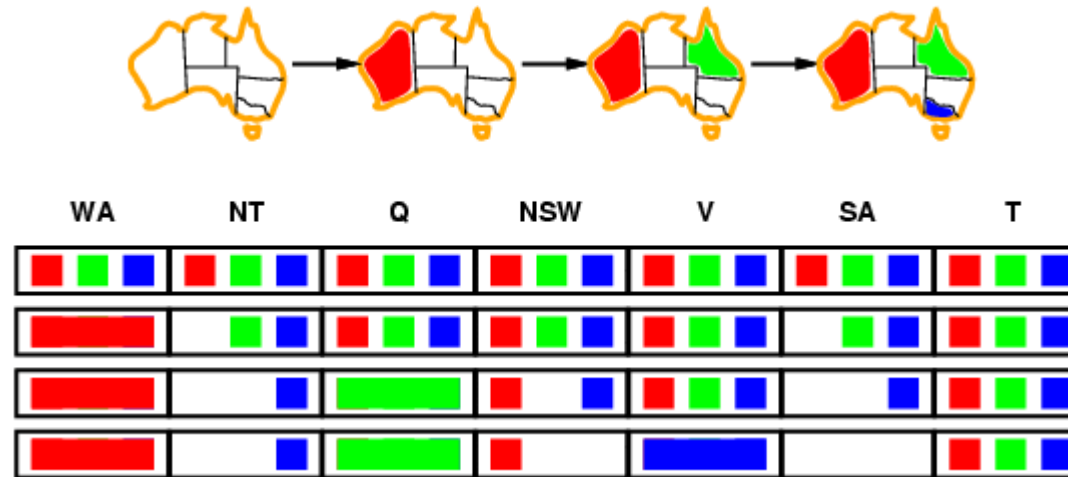
- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values



Forward Checking

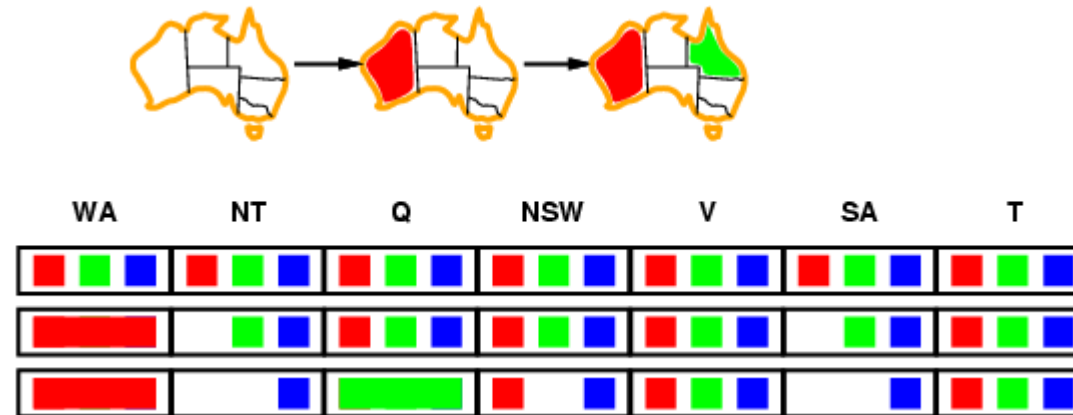
Idea:

- Keep track of remaining legal values for unassigned variables
- Terminate search when any variable has no legal values



Constraint Propagation

- Problem: Forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures:

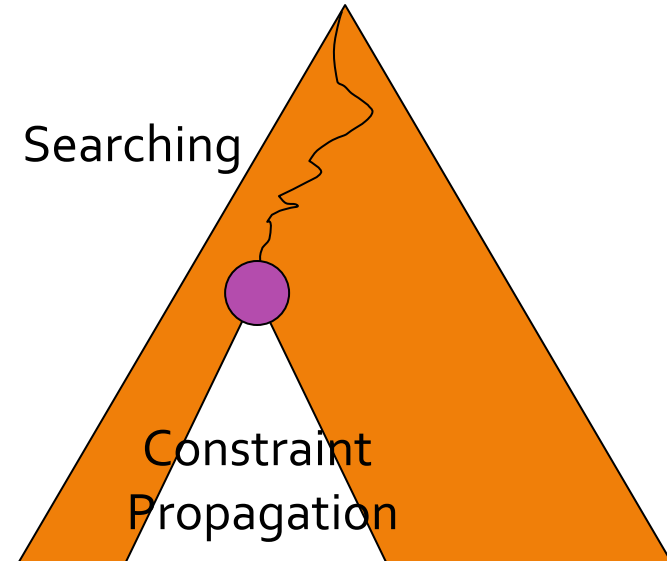


- NT and SA cannot both be blue!
- Solution: **Constraint propagation** repeatedly locally enforces constraints.

Inference in CSPs

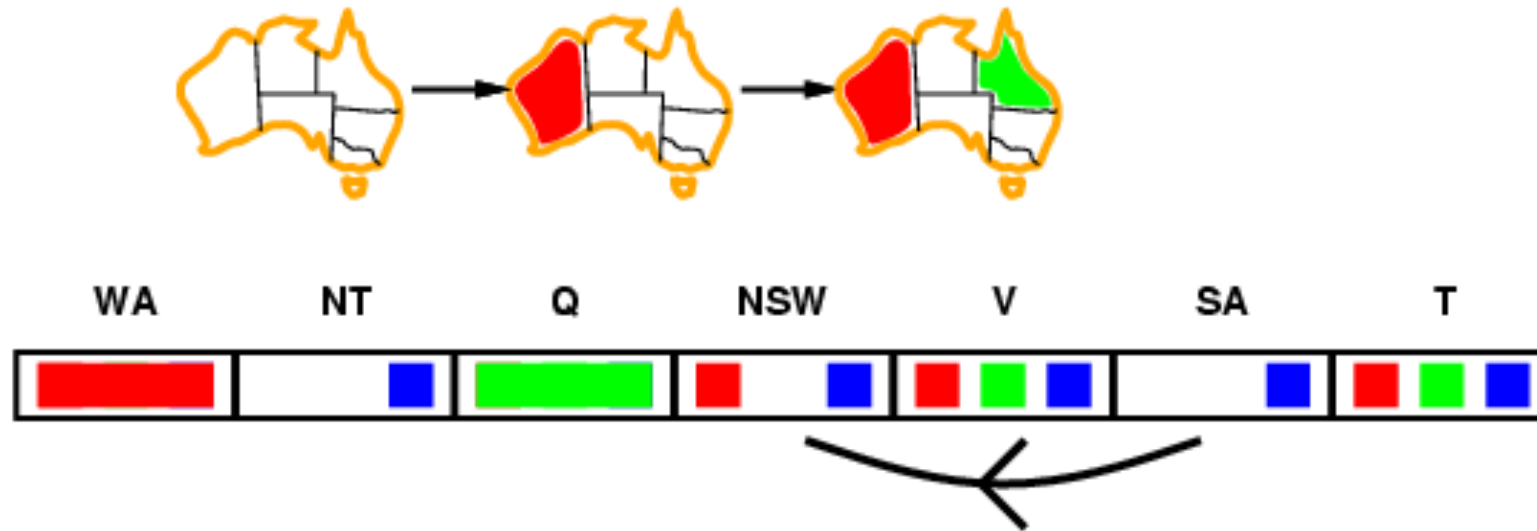
Try to infer illegal values for variables by performing constraint propagation

- Node consistency for unary constraints
- Arc consistency for binary constraints
- ...
- Can be interleaved with search, or run as a preprocessing step



Arc Consistency

X_i is arc-consistent wrt X_j (the arc (X_i, X_j) is consistent) iff for every value $x \in D_i$ there exists some value $y \in D_j$ that satisfies the binary constraint on the arc (X_i, X_j)

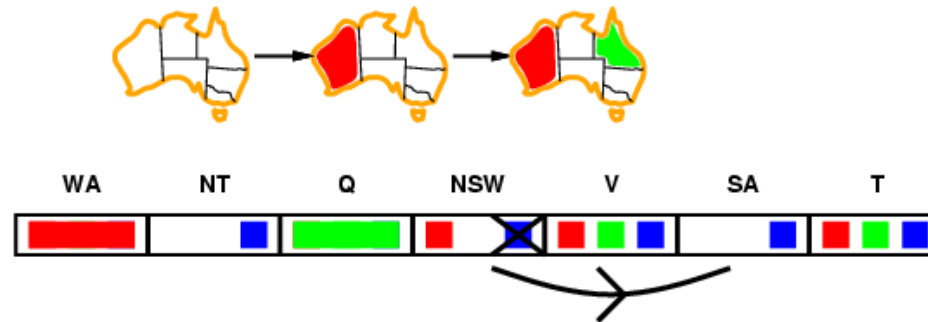


More on Arc Consistency

- If $X \leftarrow x$ makes a constraint impossible to satisfy, remove x from consideration!
- A value is impossible: the constraint provides ***no support*** for the value.
- E.g. $D_i = \{1,4,5\}$, $D_j = \{1,2,3\}$ and we have $X_i > X_j$; then $X_i \leftarrow 1$ is impossible.

Arc Consistency

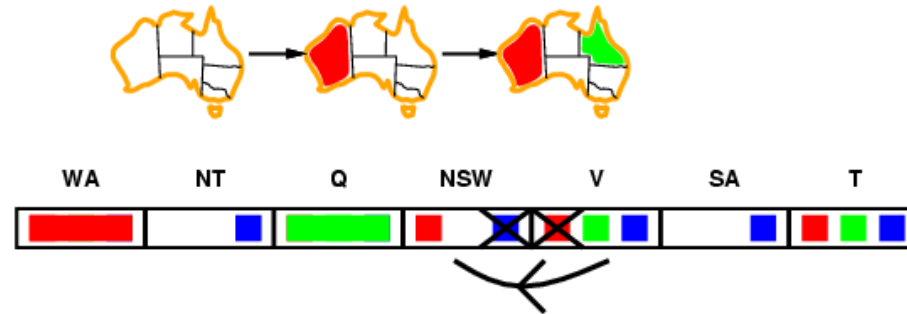
- Simplest form of propagation makes each arc **consistent**
- X_i is arc-consistent wrt X_j (the arc (X_i, X_j) is consistent) iff for every value $x \in D_i$ there exists some value $y \in D_j$ that satisfies the binary constraint on the arc (X_i, X_j)



- Arcs are directed, a binary constraint becomes two arcs
- Arc $NSW \rightarrow SA$ is originally not consistent, but is consistent after deleting $NSW = \text{blue}$

Arc Consistency

- Simplest form of propagation makes each arc **consistent**
- X_i is arc-consistent wrt X_j (the arc (X_i, X_j) is consistent) iff for every value $x \in D_i$ there exists some value $y \in D_j$ that satisfies the binary constraint on the arc (X_i, X_j)



- If X_i loses a value, neighbors of X_i need to be (re)checked
- Arc $V \rightarrow NSW$ is originally not consistent, but is consistent after deleting $V = \text{red}$

Arc Consistency Propagation

- Reducing D_i may result in domain reductions for others.
- E.g. $D_1 = \{1, 4, 5\}$, $D_2 = \{1, 2, 3\}$, $D_3 = \{2, 3, 4, 5\}$
 - Constraints: $X_3 > X_1$, $X_1 > X_2$
 - Remove 1 from D_1 so $D_1 = \{4, 5\}$
 - No support for $X_3 = 2, 3, 4$
 - We can remove those values: $D_3 = \{5\} \Rightarrow X_3 = 5$
 - Before applying AC to arc (X_1, X_2) , we cannot reduce D_3
- A chain reaction of domain reductions.

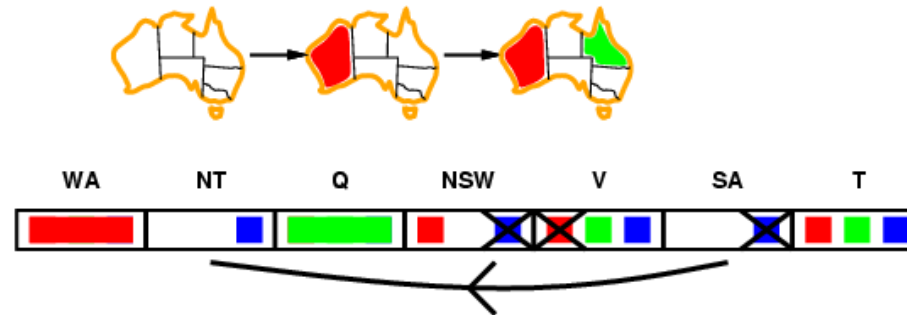
Sudoku Chain Reaction

		3		2		6		
9			3		5			1
		1	8		6	4		
		8	1		2	9		
								8
		6	7		8	2		
		2	6		9	5		
8			2		3			9
		5		1		3		

- *Alldiff* constraint on middle box makes domain of red square {3, 4, 5, 6, 9}. Column constraint reduces domain to {4}.
- Consider orange square. Original column and box constraints yield domain of {4,7}. Red square forces {7}.
- Blue box must be {1} as column already has eight values.

Arc Consistency

- Simplest form of propagation makes each arc **consistent**
- X_i is arc-consistent wrt X_j (the arc (X_i, X_j) is consistent) iff for every value $x \in D_i$ there exists some value $y \in D_j$ that satisfies the binary constraint on the arc (X_i, X_j)



- Arc consistency propagation detects failure earlier than forward checking
- Can be run as a preprocessing step or after each assignment

Arc Consistency Algorithm AC-3

function AC-3(*csp*) **returns** false if an inconsistency is found and true otherwise

inputs: *csp*, a binary CSP with components (X , D , C)

local variables: *queue*, a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty **do**

 (X_i , X_j) $\leftarrow$ REMOVE-FIRST(*queue*)

if REVISE(*csp*, X_i , X_j) **then**

if size of D_i = 0 **then return** false

for each X_k **in** X_i .NEIGHBORS - $\{X_j\}$ **do**

 add (X_k , X_i) to *queue*

return true

function REVISE(*csp*, X_i , X_j) **returns** true iff we revise the domain of X_i

revised $\leftarrow$ false

for each x **in** D_i **do**

if no value y in D_j allows (x, y) to satisfy the constraint between X_i and X_j **then**

 delete x from D_i

revised $\leftarrow$ true

return *revised*

Time complexity: $\mathcal{O}(n^2 d^3)$

Plan for this week:

■ Today:

- ❑ Finish Lecture 9 and Lecture 11 (because Lecture 10 is very long)
- ❑ Start on Lecture 10

■ Make-up Lecture (Come if able, otherwise it is recorded):

- ❑ Tomorrow 12 April (Wednesday) 2 pm to 4 pm
- ❑ On Zoom, same as previous Zoom lecture.
- ❑ Finish Lecture 10

■ Tutorial:

- ❑ Pre-recorded, covering lectures 10 and 11.
-

Time Complexity of AC-3

- CSP has at most n^2 directed arcs
- Each arc (X_i, X_j) can be inserted at most d times because X_i has at most d values to delete.
- REVISE: Checking the consistency of an arc takes $\mathcal{O}(d^2)$ time
- $\mathcal{O}(n^2 \times d \times d^2) = \mathcal{O}(n^2 d^3)$

```
function AC-3(csp) returns false if an inconsistency is found and true otherwise
inputs: csp, a binary CSP with components ( $X$ ,  $D$ ,  $C$ )
local variables: queue, a queue of arcs, initially all the arcs in csp

while queue is not empty do
  ( $X_i$ ,  $X_j$ )  $\leftarrow$  REMOVE-FIRST(queue)
  if REVISE(csp,  $X_i$ ,  $X_j$ ) then
    if size of  $D_i$  = 0 then return false
    for each  $X_k$  in  $X_i$ .NEIGHBORS - { $X_j$ } do
      add ( $X_k$ ,  $X_i$ ) to queue
return true

function REVISE(csp,  $X_i$ ,  $X_j$ ) returns true iff we revise the domain of  $X_i$ 
  revised  $\leftarrow$  false
  for each  $x$  in  $D_i$  do
    if no value  $y$  in  $D_j$  allows ( $x, y$ ) to satisfy the constraint between  $X_i$  and  $X_j$  then
      delete  $x$  from  $D_i$ 
      revised  $\leftarrow$  true
  return revised
```

Maintaining AC (MAC)

- Like any other propagation, we can use AC in search
- i.e., search proceeds as follows:
 - establish AC at the root
 - when AC-3 terminates, choose a new variable/value
 - re-establish AC given the new variable choice (i.e. maintain AC)
 - repeat;
 - backtrack if AC gives empty domain
- Exception: initially, insert into queue only arcs of neighboring unassigned variables/nodes
- The hard part of implementation is undoing effects of AC

Special Types of Consistency

- Some types of constraints lend themselves to special types of arc-consistency
- Consider the *AllDiff* constraint
 - the named variables must all take different values
 - not a binary constraint
 - can be expressed as $\frac{n(n-1)}{2}$ not-equal binary constraints
- We can apply, e.g., AC-3 as usual
- But there is a much better option

Generalized Arc Consistency and k -Consistency

- Suppose $D_1 = D_2 = \{2,3\}$ and $D_3 = \{1,2,3\}$.
- The constraints $X_i \neq X_j$ are arc-consistent
 - e.g., $X_2 = 2$ supports the value $X_3 = 3$
- The single ternary constraint $Alldiff(X_1, X_2, X_3)$ is not!
 - We must reduce $D_3 = \{1\}$.
- A special-purpose algorithm exists for $Alldiff$ constraint to establish generalized arc consistency in efficient time
 - Special-purpose propagation algorithms are vital
- Arc Consistency (2-consistency) can be extended to k -consistency

Local Search for CSPs

- Hill-climbing, simulated annealing typically work with “complete” states, i.e., all variables assigned
- To apply to CSPs:
 - allow states that violate constraints
 - operators **reassign** variable values
- Variable selection: randomly select any conflicted variable
- Value selection by **min-conflicts** heuristic:
 - choose value that violates the fewest constraints
 - i.e., hill-climb with $h(n)$ = total # of violated constraints

Min-Conflicts

function MIN-CONFLICTS(*csp*, *max_steps*) **returns** a solution or failure

inputs: *csp*, a constraint satisfaction problem

max_steps, the number of steps allowed before giving up

current $\leftarrow$ an initial complete assignment for *csp*

for *i* = 1 to *max_steps* **do**

if *current* is a solution for *csp* **then return** *current*

var $\leftarrow$ a randomly chosen conflicted variable from *csp*.VARIABLES

value $\leftarrow$ the value *v* for *var* that minimizes CONFLICTS(*var*, *v*, *current*, *csp*)

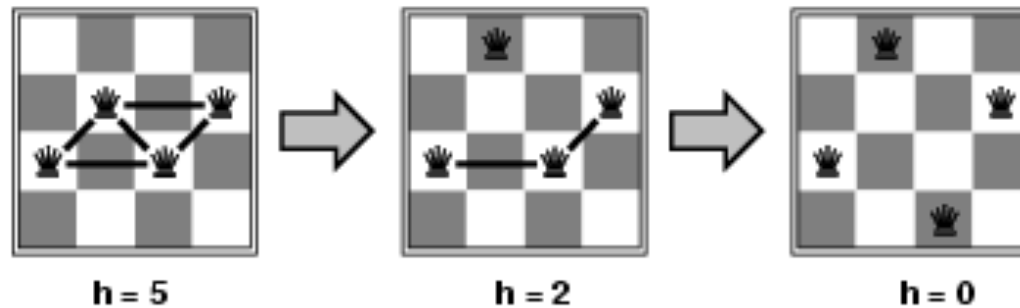
 set *var* = *value* in *current*

return *failure*

Figure 6.8 The MIN-CONFLICTS algorithm for solving CSPs by local search. The initial state may be chosen randomly or by a greedy assignment process that chooses a minimal-conflict value for each variable in turn. The CONFLICTS function counts the number of constraints violated by a particular value, given the rest of the current assignment.

Example: 4-Queens

- **States:** 4 queens in 4 columns ($4^4 = 256$ states)
- **Actions:** move queen in column
- **Goal test:** no attacks
- **Evaluation:** $h(n)$ = number of attacks

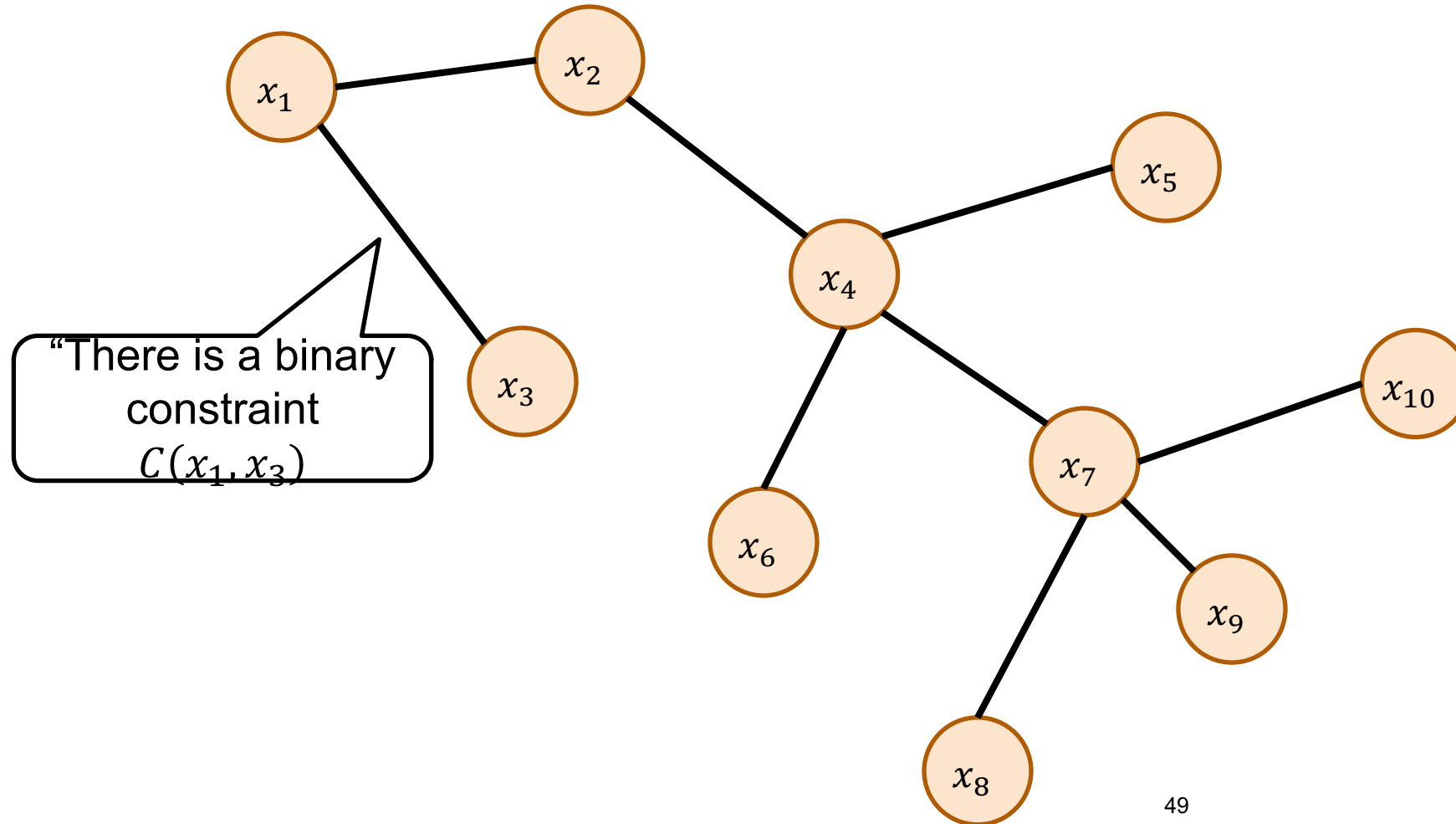


- Given random initial state, can solve n -queens in almost constant time for arbitrary n with high probability (e.g., $n = 10^6$)

Constraint Satisfaction Problems – Structured CSPs

AIMA Chapter 6.5

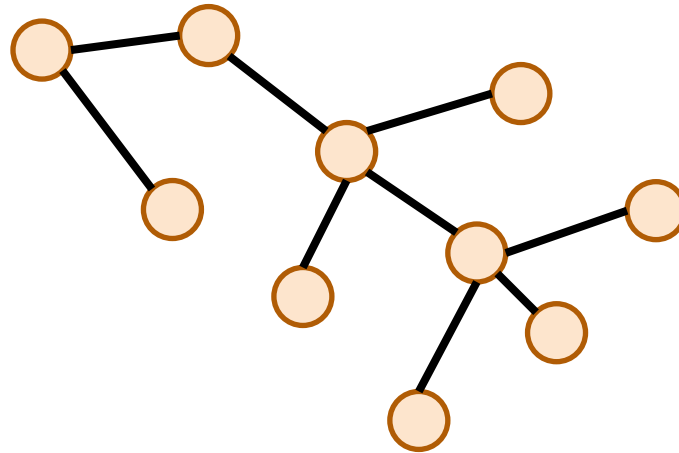
CSPs With Binary Constraints as Graphs



CSPs With Binary Constraints as Graphs

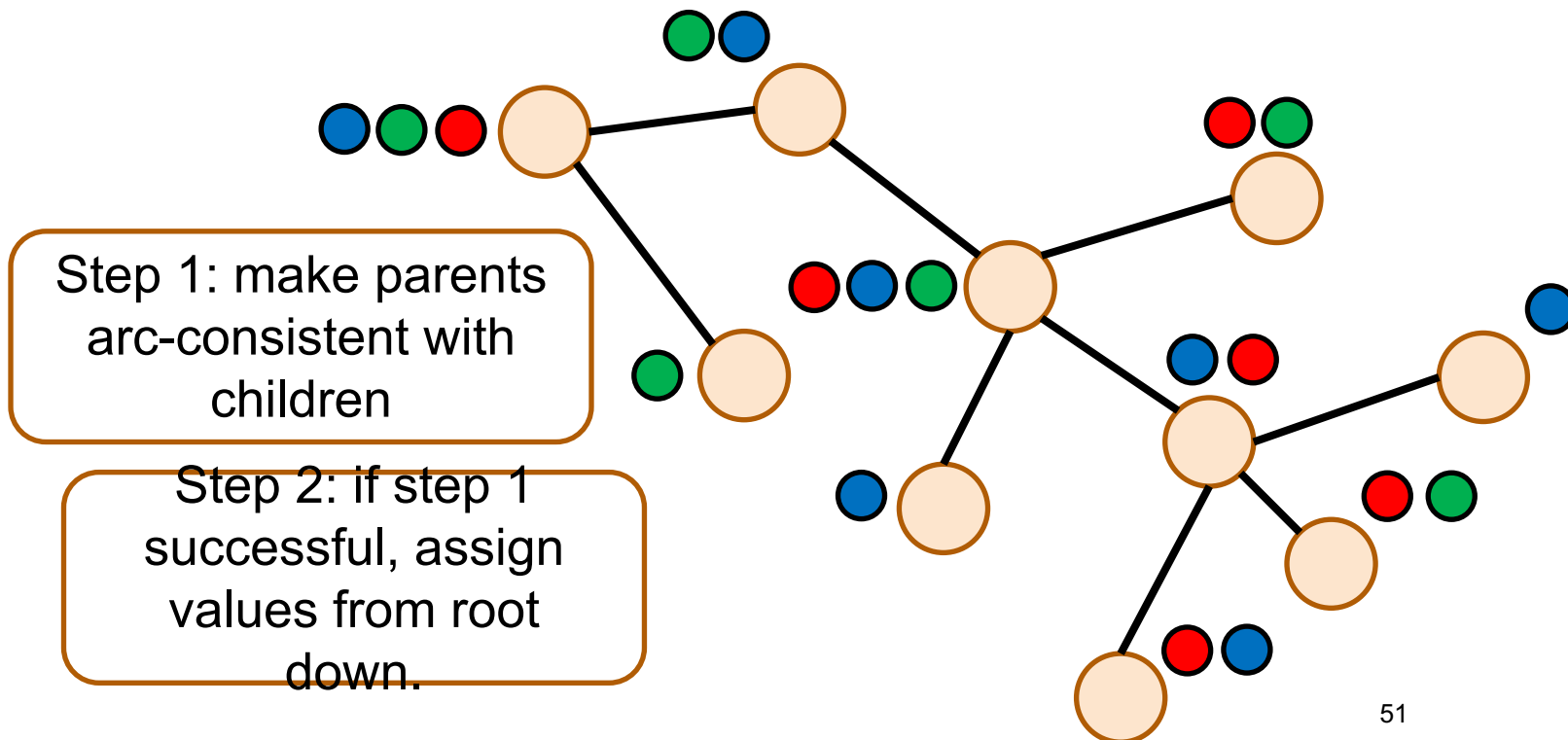
Theorem: if CSP constraint graph is a tree, then we can compute a satisfying assignment (or decide that one does not exist) in $\mathcal{O}(nd^2)$ time

No cycles!



CSPs With Binary Constraints as Graphs

Theorem: if CSP constraint graph is a tree, then we can compute a satisfying assignment (or decide that one does not exist) in $\mathcal{O}(nd^2)$ time

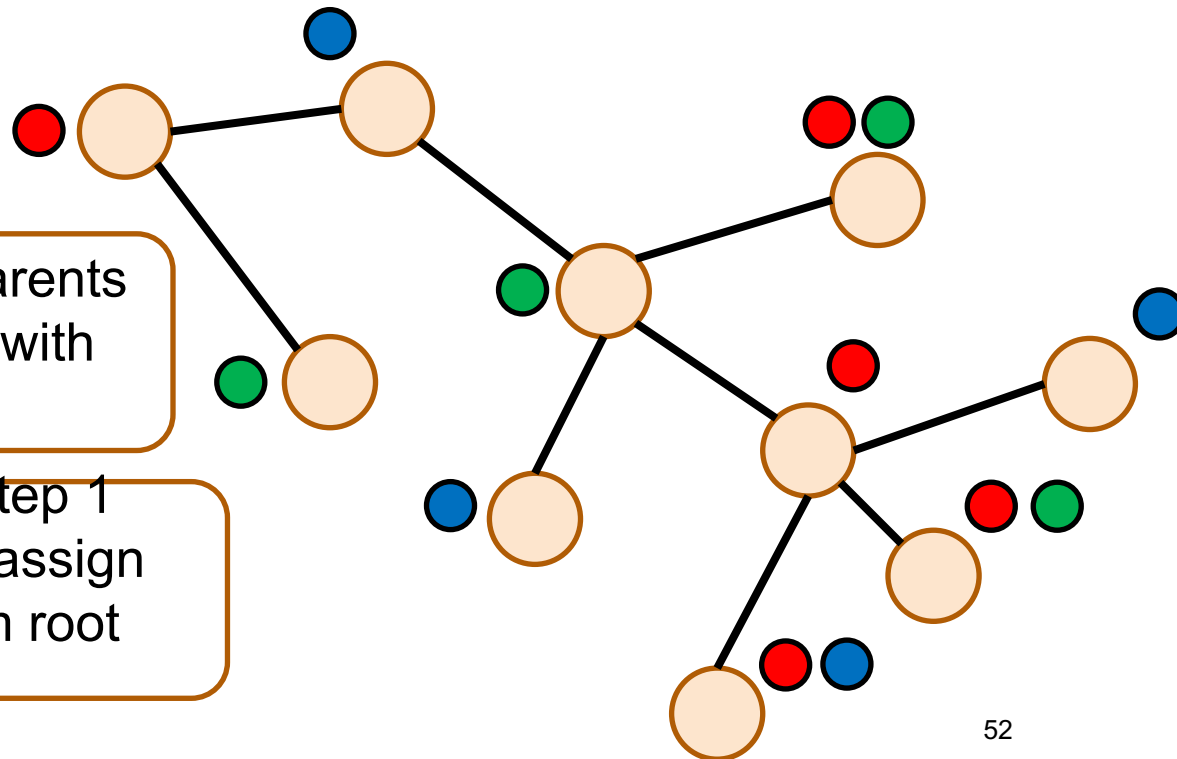


CSPs With Binary Constraints as Graphs

Theorem: if CSP constraint graph is a tree, then we can compute a satisfying assignment (or decide that one does not exist) in $\mathcal{O}(nd^2)$ time

Step 1: make parents arc-consistent with children

Step 2: if step 1 successful, assign values from root down.



CSPs With Binary Constraints as Graphs

Theorem: if CSP constraint graph is a tree, then we can compute a satisfying assignment (or decide that one does not exist) in $\mathcal{O}(nd^2)$ time

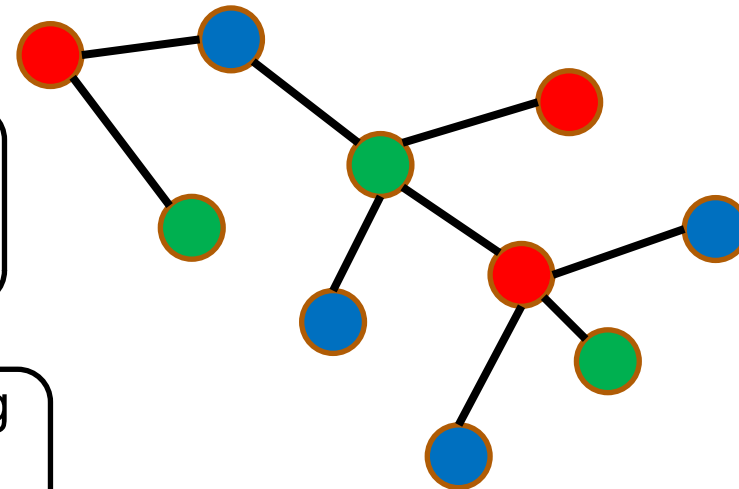
Topological sort takes $\mathcal{O}(n)$ time on tree.

Step 1: make parents arc-consistent with children

n steps taking $\mathcal{O}(d^2)$ time each

Step 2: if step 1 successful, assign values from root down.

n steps taking $\mathcal{O}(d)$ time each



CSPs With Binary Constraints as Graphs

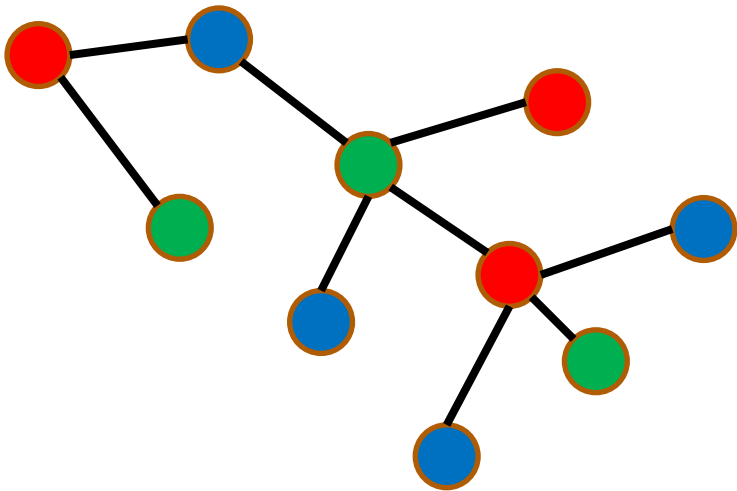
Theorem: if CSP constraint graph is a tree, then we can compute a satisfying assignment (or decide that one does not exist) in $\mathcal{O}(nd^2)$ time

Could it be that this step removes options for a valid CSP assignment?
That would be very bad...

Step 1: make parents arc-consistent with children

Step 2: if step 1 successful, assign values from root down.

Because constraint graph is a tree, no need to backtrack when assigning!
Assignment for node affects only its subtree.



IT5005

Artificial Intelligence

Lecture 9: Uncertainty

Outline

- Uncertainty
- Probability
- Syntax and Semantics
- Inference
- Independence and Bayes' Rule

Uncertainty: Motivating Example

Let taxi agent's action A_t = leave for airport t minutes before flight. **Will A_t get me there on time?**

- **Sources** of uncertainty:

1. Partial observability (e.g., road state, other drivers' plans, ...)
2. Noisy sensors (e.g., traffic reports, fuel sensor, ...)
3. Uncertainty in action outcomes (e.g., flat tire, accident, ...)
4. Complexity in modeling and predicting traffic (e.g., congestion)

- Logical agent either

1. risks falsehood: " A_{25} will get me there on time", or
2. reaches weaker conclusion: " A_{25} will get me there on time **if** there's no accident on the bridge **and** it doesn't rain **and** my tires remain intact..."

Random Variables

Domains

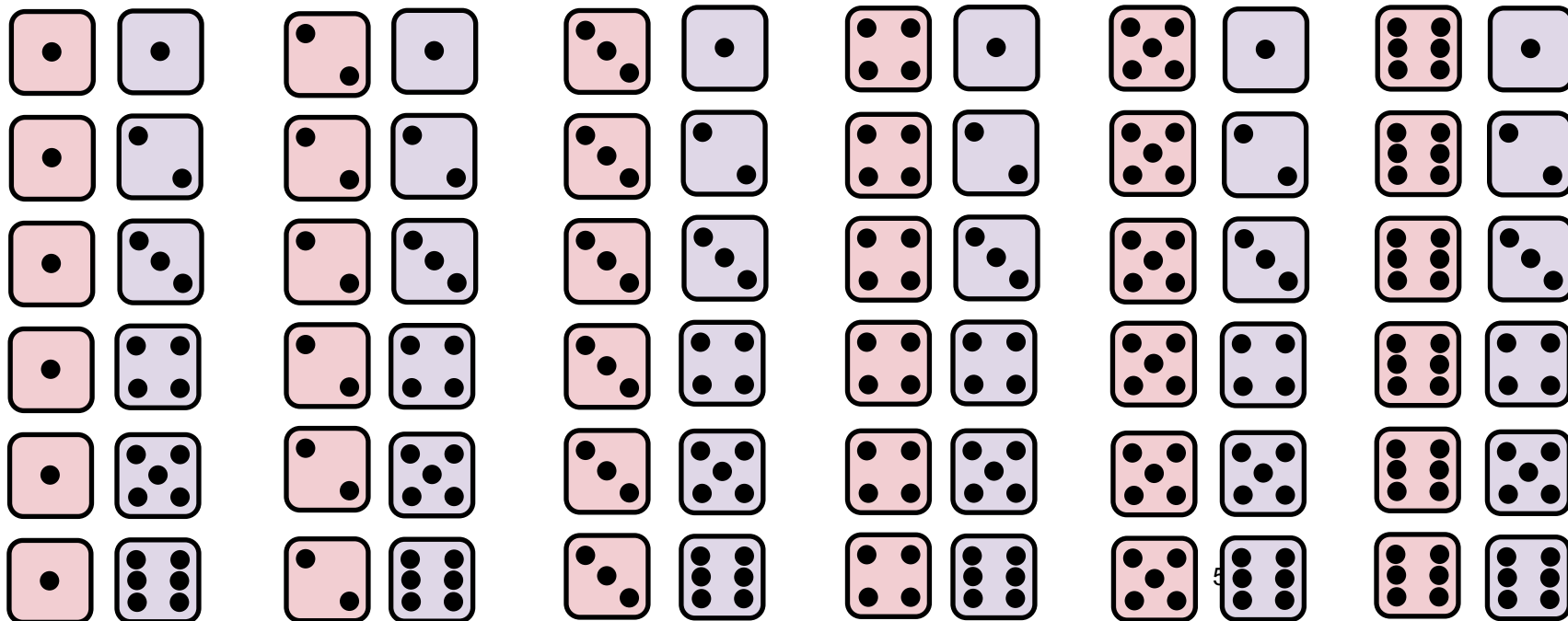
- Boolean: coin is either heads or tails (true or false)
- Discrete: a die can have values $\{1, \dots, 6\}$

Events: subsets of domains

- $Heads(X)$ the coin flipped to heads
- $Even(X)$ the die has value $\in \{2, 4, 6\}$

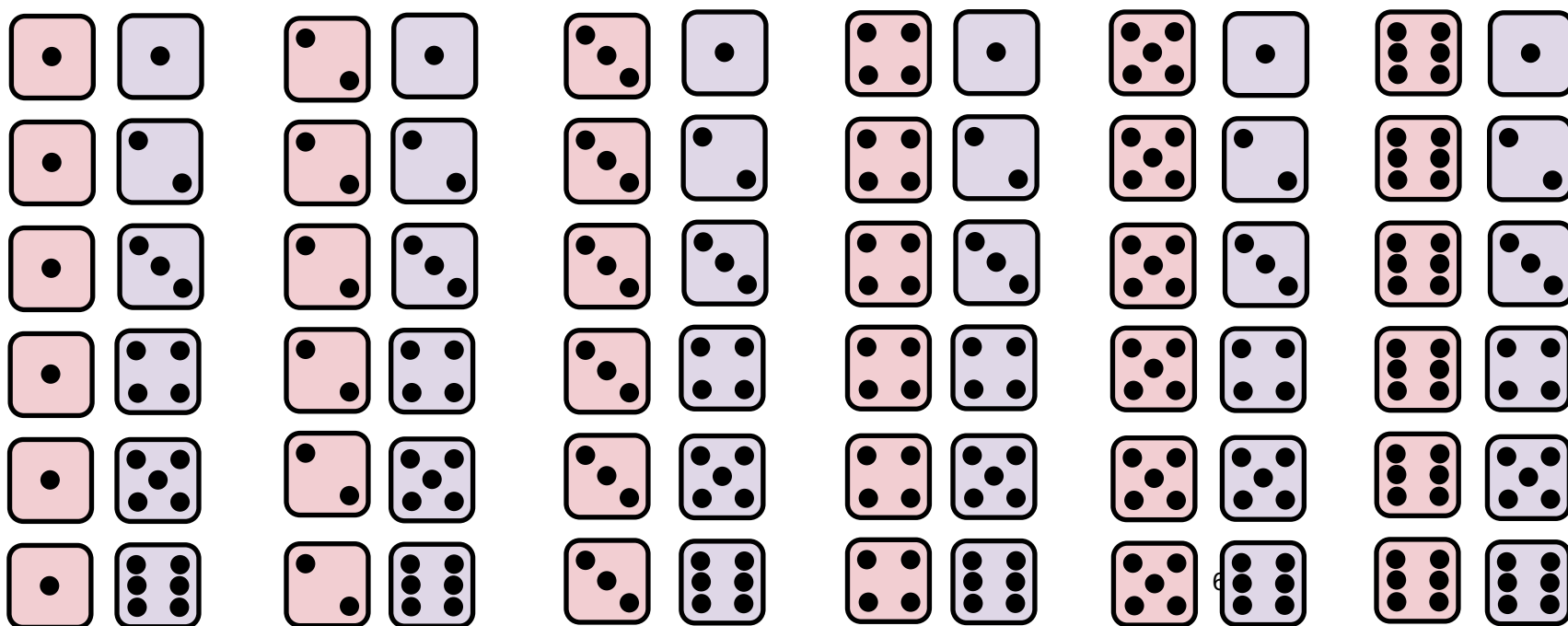
Events

- Given a random variable X , let D_X be its domain.
- **Atomic event (possible world):** an assignment of a value to each random variable; a singleton event
 - We roll two different dice



Events

- Red die = X_1 , blue die = X_2
- Event: $X_1 + X_2 = 8$



Axioms of Probability

- Let X be a random variable with finite domain D_X .
- A probability distribution over D_X assigns a value $p_X(x) \in [0,1]$ to every $x \in D_X$ s.t.

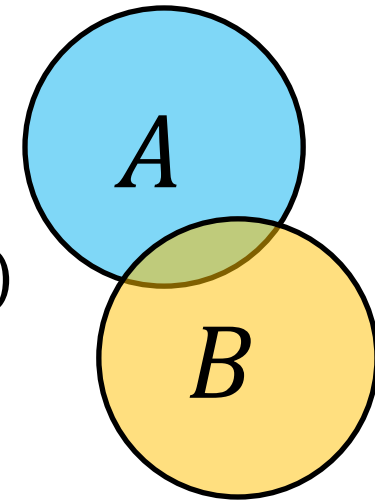
$$\sum_{x \in D_X} p_X(x) = 1$$

- For any event $A \subseteq D_X$ we have

$$\Pr[X \subseteq A] \equiv \Pr_X[A] = \sum_{x \in A} p_X(x)$$

- In particular

$$\Pr[A] + \Pr[B] = \Pr[A \cap B] + \Pr[A \cup B]$$



Joint Probability

- Given two random variables X and Y , the **joint probability** of an atomic event $(x, y) \in D_X \times D_Y$ is $p_{X,Y}(x, y) = \Pr[X = x \wedge Y = y]$
- In particular $p_X(x) = \sum_{y \in D_Y} p_{X,Y}(x, y)$

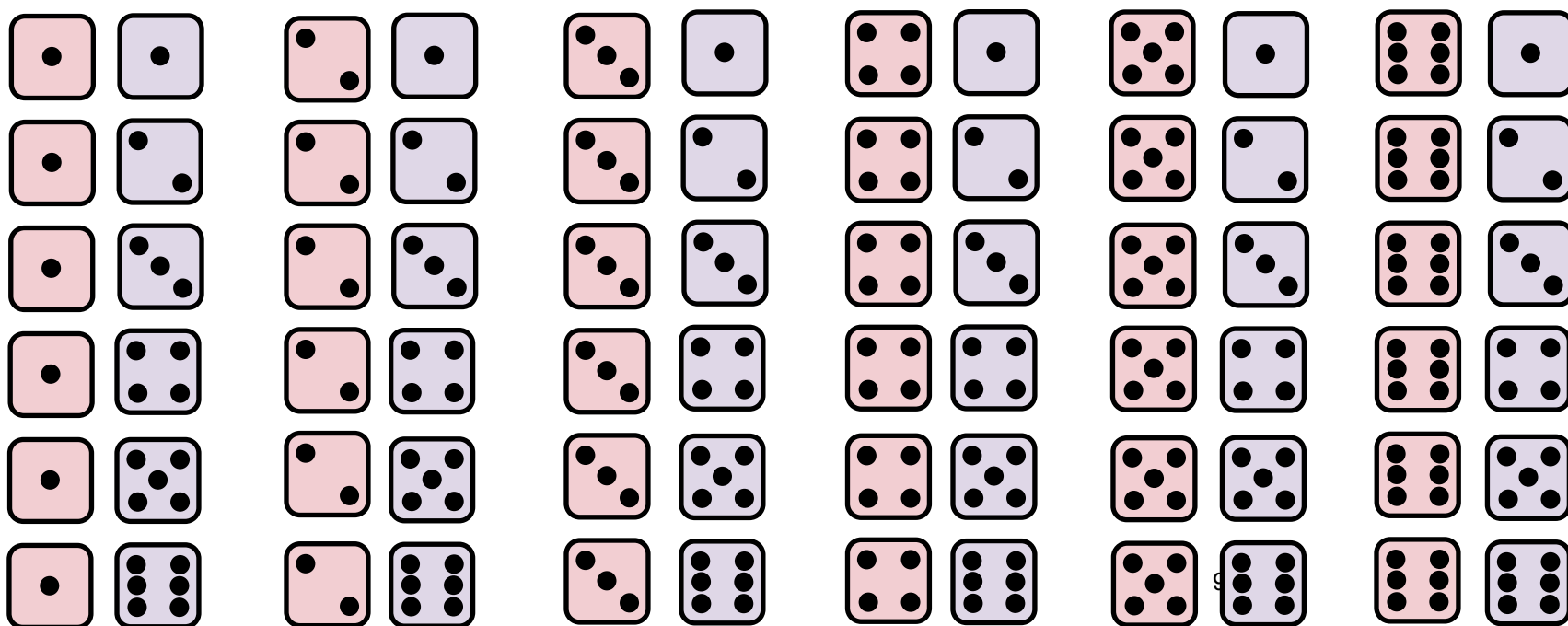
Income (in SGD)	15-24	25-34	35-44	45-54	55-64	65+
< S\$2500	0.062	0.051	0.037	0.019	0.015	0.039
S\$2500 – S\$5000	0.078	0.068	0.061	0.057	0.031	0.053
> S\$5000	0.015	0.051	0.094	0.119	0.111	0.039

$$\Pr[\text{Age} = (25 - 34)] = 0.051 + 0.068 + 0.051 = 0.17$$

Posterior/Conditional Probability

Probability that an event occurs, given that some other event occurs.

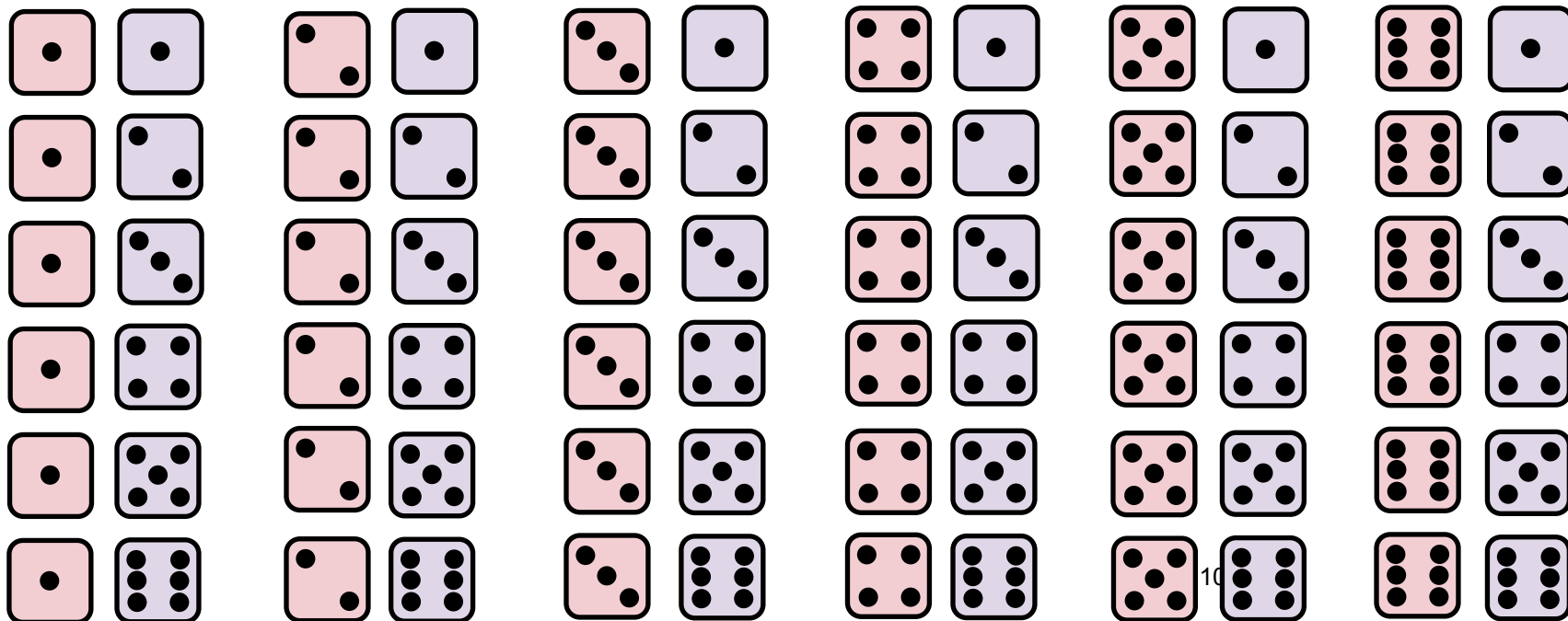
$$\Pr[X_1 = 2]$$



Posterior/Conditional Probability

Probability that an event occurs, given that some other event occurs.

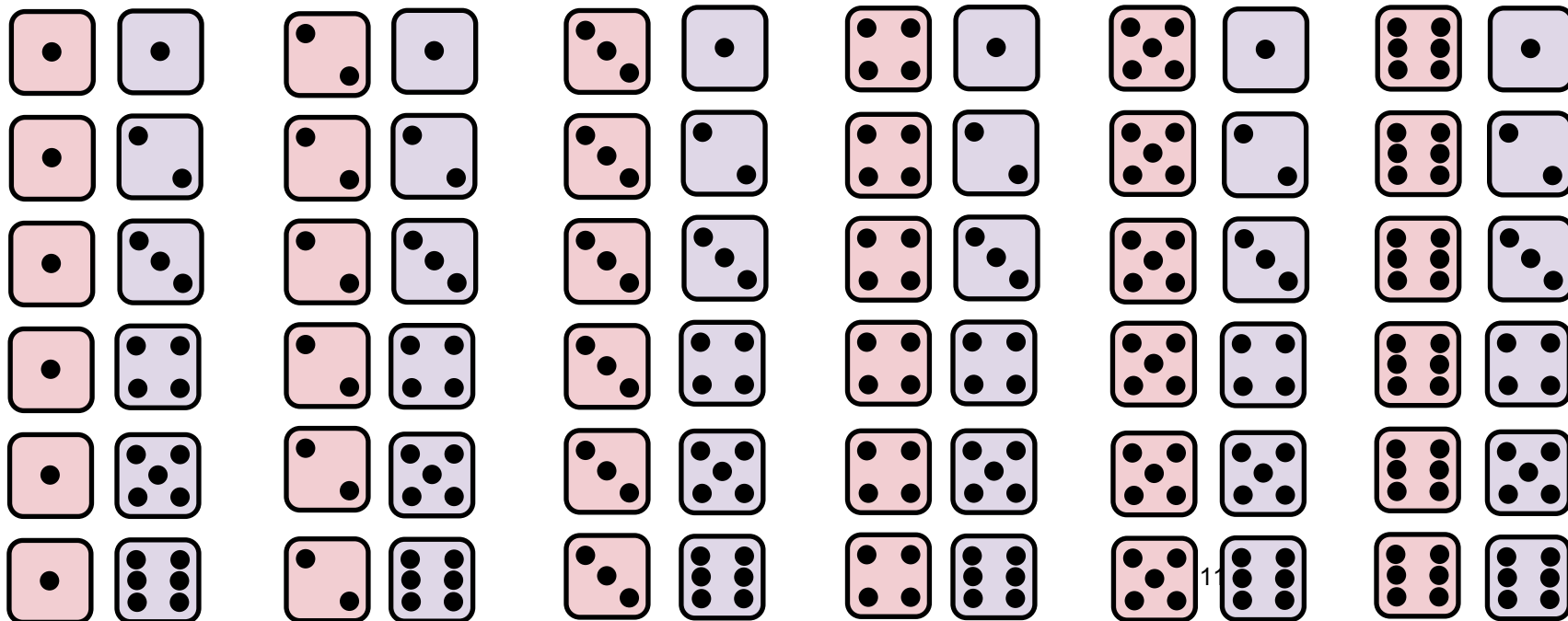
$$\Pr[X_1 = 2 \mid X_1 + X_2 = 8]$$



Posterior/Conditional Probability

Probability that an event occurs, given that some other event occurs.

$$\Pr[X_1 + X_2 = 8 \mid X_1 = 2]$$



Posterior/Conditional Probability

$$\Pr[A \mid B] = \frac{\Pr[A \wedge B]}{\Pr[B]} \text{ assuming that } \Pr[B] > 0$$

Bayes rule: $\Pr[A \mid B] = \frac{\Pr[B|A] \Pr[A]}{\Pr[B]}$

$$\Pr[X_1 = 2 \mid X_1 + X_2 = 8] = ?$$

Chain rule: derived by successive application of Bayes' rule:

$$\Pr[X_1 \wedge X_2 \wedge \cdots \wedge X_k] = \prod_{j=1, \dots, k} \Pr[X_j \mid X_1 \wedge \cdots \wedge X_{j-1}]$$

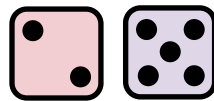
Independence

A and B are independent if $\Pr[A \wedge B] = \Pr[A] \cdot \Pr[B]$. Equivalent to $\Pr[A | B] = \Pr[A]$.

“Knowing B adds no information about A ”

Rolling two dice

$$\Pr[X_1 = 2 | X_1 + X_2 = 7] = ?$$



Bayesian Inference

Instead of inferring statements of the form

‘is X true given knowledge base?’

$$Y_1 \wedge \cdots \wedge Y_k \Rightarrow X?$$

we infer statements of the form

‘What is the likelihood of an event X given the probabilities of other events?’

$$\Pr[X \mid Y_1, \dots, Y_k] = ?$$

Inference by Enumeration

- Start with the joint probability distribution:

	toothache		$\neg$ toothache	
	catch	$\neg$ catch	catch	$\neg$ catch
cavity	0.108	0.012	0.072	0.008
$\neg$ cavity	0.016	0.064	0.144	0.576

- For any proposition (event) X , sum the atomic events y where X holds: $\Pr[X] = \sum_{y \in X} \Pr[X = y]$
- $\Pr[\text{toothache}] = 0.108 + 0.016 + 0.012 + 0.064 = 0.2$

Inference by Enumeration

- Start with the joint probability distribution:

	toothache		$\neg$ toothache	
	catch	$\neg$ catch	catch	$\neg$ catch
cavity	0.108	0.012	0.072	0.008
$\neg$ cavity	0.016	0.064	0.144	0.576

- For any proposition (event) X , sum the atomic events y where X holds: $\Pr[X] = \sum_{y \in X} \Pr[X = y]$
- $\Pr[\text{toothache} \vee \text{cavity}] =$

Inference by Enumeration

- Start with the joint probability distribution:

	toothache		$\neg$ toothache	
	catch	$\neg$ catch	catch	$\neg$ catch
cavity	0.108	0.012	0.072	0.008
$\neg$ cavity	0.016	0.064	0.144	0.576

- For any proposition (event) X , sum the atomic events y where X holds: $\Pr[X] = \sum_{y \in X} \Pr[X = y]$
- $\Pr[\neg \text{cavity} \mid \text{toothache}] = \frac{\Pr[\neg \text{cavity} \wedge \text{toothache}]}{\Pr[\text{toothache}]}$

The Power of Independence

- We have n random variables $X_1, \dots, X_n$; domains of size d . How big is their joint distribution table?
- Suppose that $X_1, \dots, X_n$ are independent: maintain only dn values!
- Independence is good (if you can find it)

Conditional Independence

Suppose that we test for pneumonia using two tests

- ❑ Blood Test: B
- ❑ Throat Swab: T
- Are they fully independent?
- BUT: B, T independent given knowledge of **underlying cause** $S = \text{sick!}$

$$\Pr[B \wedge T \mid S] = \Pr[B \mid S] \Pr[T \mid S]$$

“Tests were conducted independently, only related by the underlying sickness”

Conditional Independence

- Write out full joint distribution using chain rule:

$$\begin{aligned} & \Pr[T_1 \wedge T_2 \wedge \cdots \wedge T_k \wedge S] \\ &= \Pr[T_1 \mid S] \Pr[T_2 \mid S] \cdots \Pr[T_k \mid S] \Pr[S] \end{aligned}$$

- Joint distribution of n Boolean RVs: $2^n - 1$ entries.
- Conditional independence: linear!
- Conditional independence is more robust and common than absolute independence

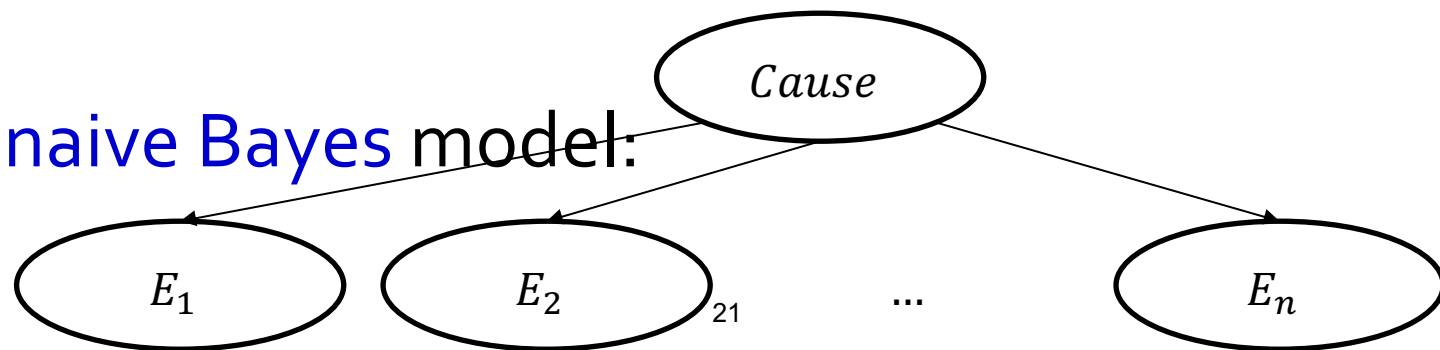
Bayes' Rule and Conditional Independence

A cause (heavy rain) can have several conditionally independent effects (Alice takes umbrella, Bob takes umbrella, Claire takes umbrella...)

$$\Pr[Cause \mid E_1 \dots E_n] = \frac{\Pr[Cause] \Pr[E_1, \dots, E_n \mid Cause]}{\Pr[E_1, \dots, E_n]}$$

$$= \alpha P[Cause] \prod_i \Pr[E_i \mid Cause]$$

- This is an example of a **naive Bayes** model:



Normalization

- We are trying to diagnose the disease X . 70% of the population is healthy, 20% are carriers, and 10% are sick.
- A blood test will come back positive with the following probability:
 - ▣ $\Pr[T = 1 \mid X = \textit{healthy}] = 0.1$
 - ▣ $\Pr[T = 1 \mid X = \textit{carrier}] = 0.7$
 - ▣ $\Pr[T = 1 \mid X = \textit{sick}] = 0.9$
- We run a test three times (independently) and obtain two positive (on tests 1 and 2) and one negative (on test 3). What is the likeliest value for X ?

Normalization

$$\begin{aligned} & \Pr[X \mid T_1 = T_2 = 1, T_3 = 0] \\ &= \frac{\Pr[X]}{\Pr[T_1 = T_2 = 1, T_3 = 0]} \Pr[T_1 = T_2 = 1, T_3 = 0 \mid X] \end{aligned}$$

We don't care about $\frac{1}{\Pr[T_1=T_2=1, T_3=0]}$! Set it to α .

$$\begin{aligned} & \alpha \Pr[X] \times \Pr[T_1 = 1, T_2 = 1, T_3 = 0 \mid X] \\ &= \alpha \Pr[X] \times \Pr[T_1 = 1 \mid X] \times \Pr[T_2 = 1 \mid X] \times \Pr[T_3 = 0 \mid X] \end{aligned}$$

$$\Pr[X = \textit{healthy} \mid A] = \alpha \times 0.7 \times 0.1 \times 0.1 \times 0.9 = 0.0063\alpha$$

$$\Pr[X = \textit{carrier} \mid A] = \alpha \times 0.2 \times 0.7 \times 0.7 \times 0.3 = 0.0294\alpha$$

$$\Pr[X = \textit{sick} \mid A] = \alpha \times 0.1 \times 0.9 \times 0.9 \times 0.1 = 0.0081\alpha$$

BAYESIAN NETWORKS

AIMA Chapter 14.1 – 14.2

Bayesian Networks

- A graphical way of writing joint distributions
- Nodes are random variables
- Edge from X to Y : X directly influences Y
- a conditional distribution for each node given its parents:

$$\Pr[X \mid Parents(X)]$$

- In the simplest case, conditional distribution can be represented as a **conditional probability table** (CPT): the distribution over X for each combination of parent values

Bayesian Networks

Given $X_1, \dots, X_n$, write

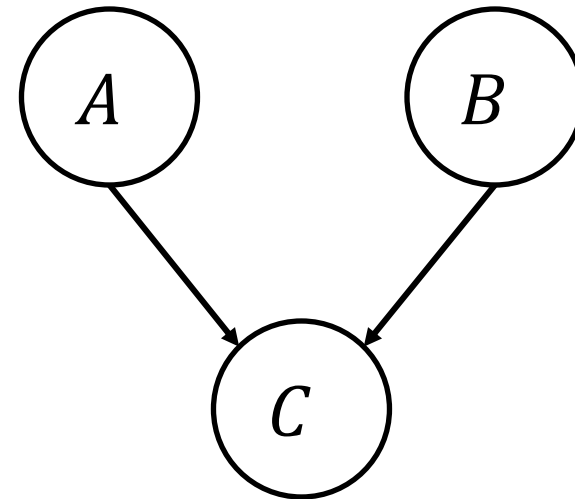
$$\Pr[X_1 \wedge \dots \wedge X_n] = \prod_i \Pr[X_i \mid \textit{Parents}(X_i)]$$

Examples

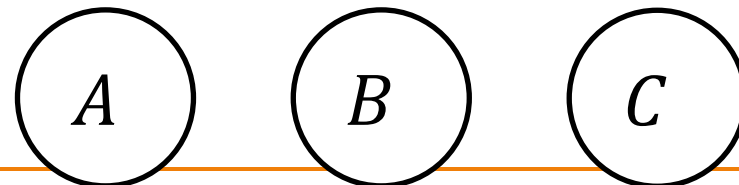
- $\Pr[A \wedge B \wedge C] = \Pr[C \mid A, B] \Pr[A] \Pr[B]$

Independent causes:
“I can be late either because of rain
or because I was sick”

(in logic: $A \vee B \rightarrow C$)



- $\Pr[A \wedge B \wedge C] = \Pr[C] \Pr[A] \Pr[B]$

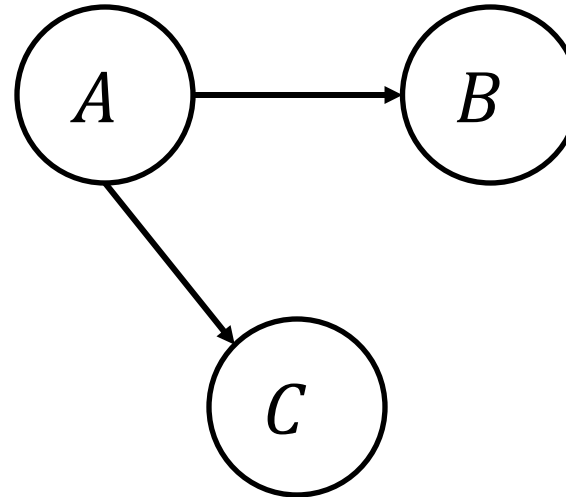


Examples

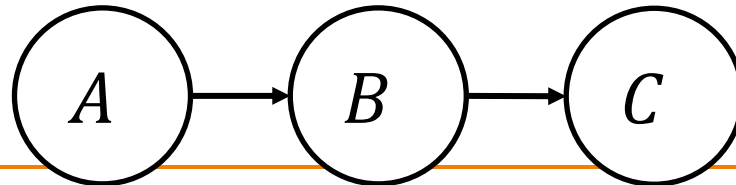
- $\Pr[A \wedge B \wedge C] = \Pr[C \mid A] \Pr[B \mid A] \Pr[A]$

Conditionally independent effects:
“A disease can cause two
independent tests to be positive”

(in logic: $A \rightarrow B; A \rightarrow C$)

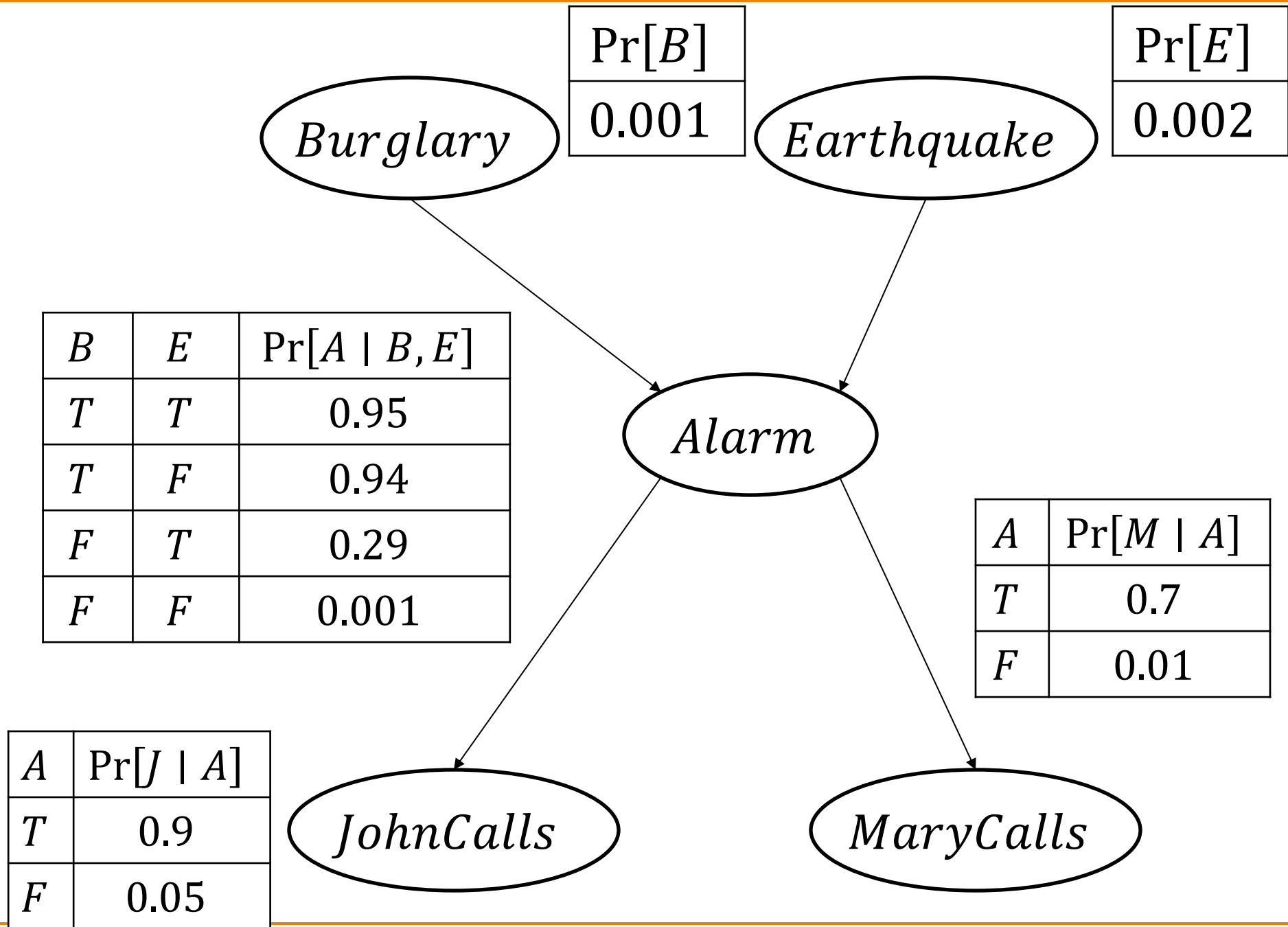


- $\Pr[A \wedge B \wedge C] = \Pr[C \mid B] \Pr[B \mid A] \Pr[A]$



Example With More Variables

- I'm at work
 - ❑ neighbor John calls to say my house alarm is ringing
 - ❑ neighbor Mary doesn't call
 - ❑ Alarm sometimes set off by minor earthquake.
 - ❑ Is there a burglar?
- Variables: B, E, A, J, M
- 5 binary variables: joint distribution table size $2^5 - 1$
- Exploit domain knowledge → smaller representation.



Bayesian Networks – Compactly Representing Joint Distributions

- Conditional probability table for Boolean X with k Boolean parents has 2^k rows: **all** possible parent values
- Each row requires one number p for $X = \text{True}$
- If each variable has $\leq k$ parents, network representation requires $\mathcal{O}(n2^k)$ values, vs. $\mathcal{O}(2^n)$ for full joint distribution.
- For burglary network, $1 + 1 + 2 + 2 + 4 = 10$ numbers as compared to $2^5 - 1 = 31$ numbers for full joint distribution

Inference in Bayesian Networks

A Bayesian Network represents the full joint distribution; can infer any query.

$$\Pr[B = 1 \mid J = 1, M = 0] = \frac{\Pr[B = 1, J = 1, M = 0]}{\Pr[J = 1, M = 0]} = ?$$

$$\Pr[J, M, A, B, E] = \Pr[J \mid A] \Pr[M \mid A] \Pr[A \mid B, E] \Pr[B] \Pr[E]$$

e.g.

$$\begin{aligned} & \Pr[B = 1, J = 1, M = 0, A = 1, E = 0] \\ &= \Pr[j \mid a] \Pr[\neg m \mid a] \Pr[a \mid b, \neg e] \Pr[b] \Pr[\neg e] \\ &= 0.9 \times 0.3 \times 0.94 \times 0.001 \times 0.998 \simeq 0.000253 \end{aligned}$$

Need to compute the cases $A = 0, E = 0$; $A = 1, E = 1$; $A = 1, E = 0$.

Constructing Bayesian Networks

1. Choose an ordering of variables $X_1, \dots, X_n$
2. For $i = 1, \dots, n$:
 - ❑ Add node X_i to the network
 - ❑ Select minimal set of parents from $X_1, \dots, X_{i-1}$ such that
$$\Pr[X_i \mid \text{Parents}(X_i)] = \Pr[X_i \mid X_1, \dots, X_{i-1}]$$
 - ❑ Link every parent to X_i
 - ❑ Write down CPT for $\Pr[X_i \mid \text{Parents}(X_i)]$

Constructing Bayesian Networks

This construction guarantees

$$\Pr[X_1, \dots, X_n] = \prod_i \Pr[X_i \mid X_1, \dots, X_{i-1}]$$

Consequence of
chain rule,
generally true!

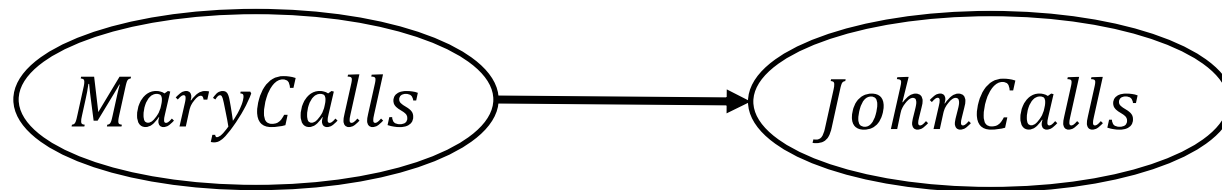
$$= \prod_i \Pr[X_i \mid \text{Parents}(X_i)]$$

By choice of
parents

Network is acyclic (why??), and has no
redundancies

Variable Order Matters

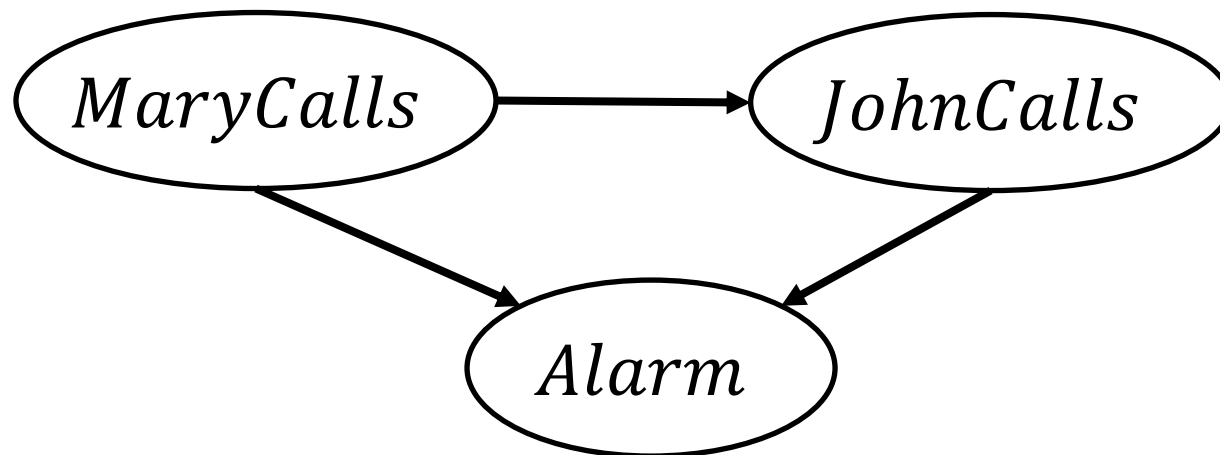
We choose the ordering M, J, A, B, E
(originally was B, E, A, M, J)



Is it true that $\Pr[J \mid M] = \Pr[J]$?

Variable Order Matters

We choose the ordering M, J, A, B, E



Is it true that

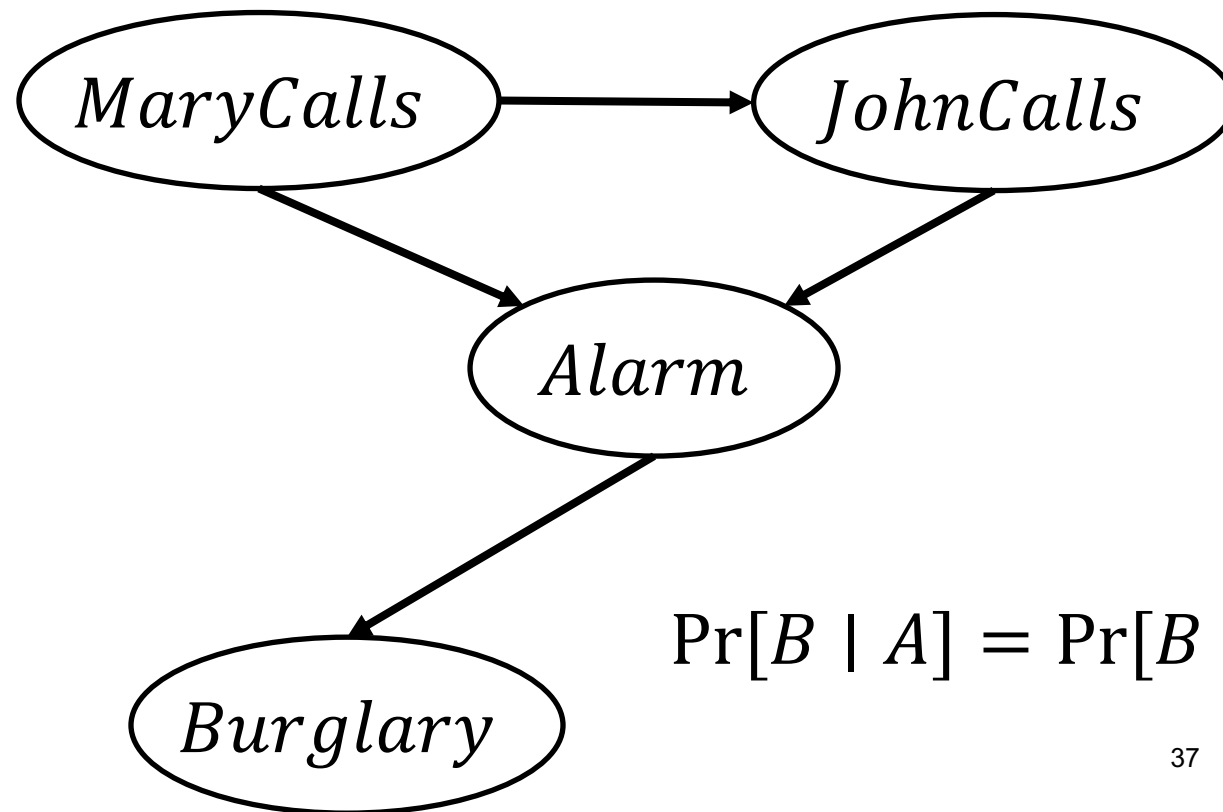
$$\Pr[A \mid M, J] = \Pr[A]$$

$$\Pr[A \mid M, J] = \Pr[A \mid J]$$

$$\Pr[A \mid M, J] = \Pr[A \mid M]$$

Variable Order Matters

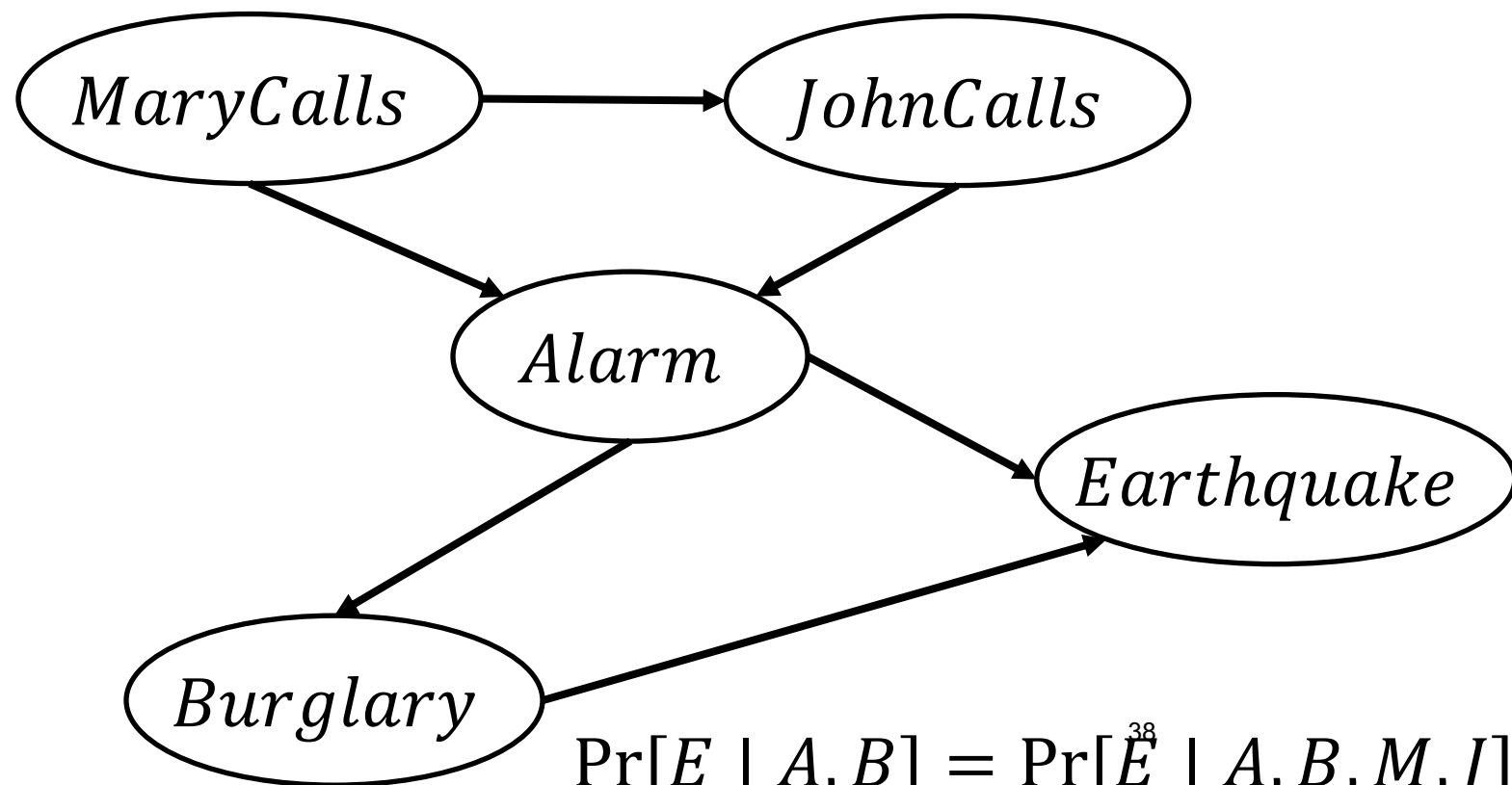
We choose the ordering M, J, A, B, E



$$\Pr[B \mid A] = \Pr[B \mid A, M, J]?$$

Variable Order Matters

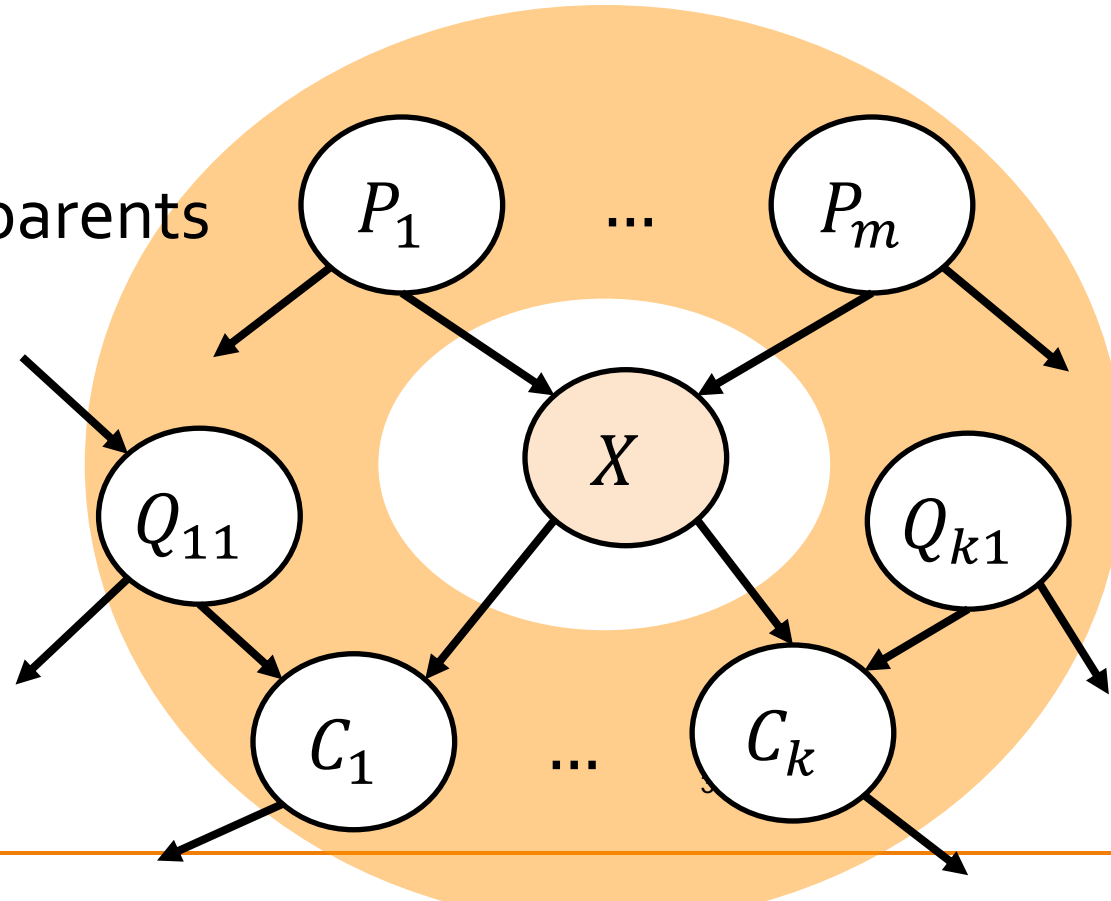
We choose the ordering M, J, A, B, E



The Markov Blanket

A node is conditionally independent of everything else **given the values** of its:

- parents
- children
- its children's parents



Putting it All Together

We want to compute $\Pr[X_1 = a \mid X_2 = b]$

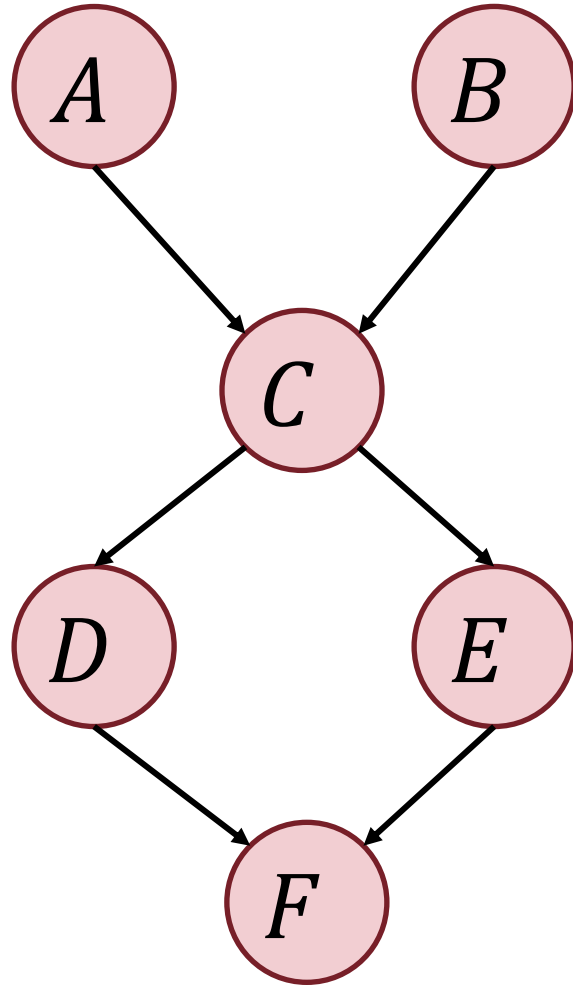
1. Bayes' rule: $\Pr[a \mid b] = \frac{\Pr[a,b]}{\Pr[b]} = \alpha \Pr[a, b]$

2. Total Probability: $\Pr[a, b] = \sum_{x_3 \in X_3} \cdots \sum_{x_n \in X_n} \Pr[a, b, x_3, \dots, x_n]$

3. Bayesian Network Factoring:

$$\sum_{x_3 \in X_3} \cdots \sum_{x_n \in X_n} \prod_j \Pr[x_j \mid Parents(X_j)]$$

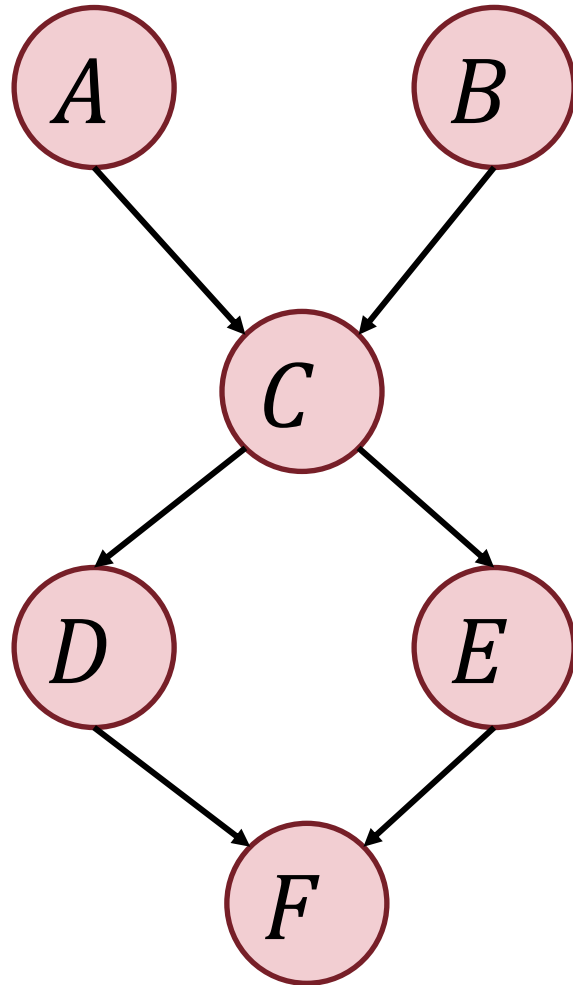
Bayesian Networks



A good way to factor joint distributions of random variables:

A variable is only conditionally dependent on its parents.

Conditional Independence in BN



Given variables X, Y and
known variables

$$\mathcal{E} = \{E_1, \dots, E_k\}$$

are X and Y independent
given knowledge of $\mathcal{E}$?

Conditional Independence in BN

Given variables X, Y and known variables
 $\mathcal{E} = \{E_1, \dots, E_k\}$
are X and Y independent given knowledge of $\mathcal{E}$?

Can be shown

- using algebra (annoying and tedious):

$$\Pr[X \mid \mathcal{E}] = \dots = \Pr[X \mid \mathcal{E}, Y]$$

- via counterexample (computing via the CPTs)

Can we show that two nodes are necessarily independent?

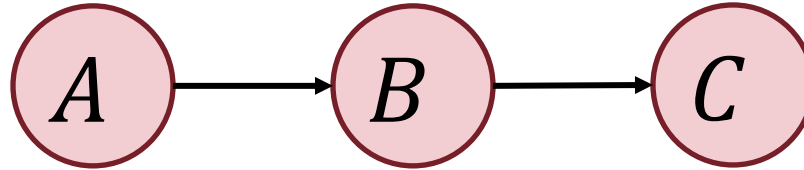
Motivation

- Given a Bayesian Network that shows how random variables are conditionally related we want to ask:
 - Given two random variables $X_1, X_2 \in X$ and $Z \subseteq X$, are X_1 and X_2 independent?
- Independence: Knowing X_1 does not give me additional information about X_2 (and vice versa)
 - Necessarily independent: $P(X_2|X_1) = P(X_2)$
 - Conditionally independent: X_1, X_2 independent *given* X_3
 - E.g. results of two tests for a particular disease are influenced by whether the person has the disease, and are thus not independent.
 - However, if we know that the person has the disease, then the two tests are independent; knowing the result of one doesn't give additional information about the result of the other.

Motivation

- In this supplement we look at a simple principle to determine this:
 - First disregard the direction of the edges.
 - If there exists some path from X_1 to X_2 via Z , we say that X_1 and X_2 are *d-connected* and X_1 and X_2 may not be independent.
 - Otherwise X_1 and X_2 are either unconditionally or conditionally independent.

Exploring Independence: Causal Chains

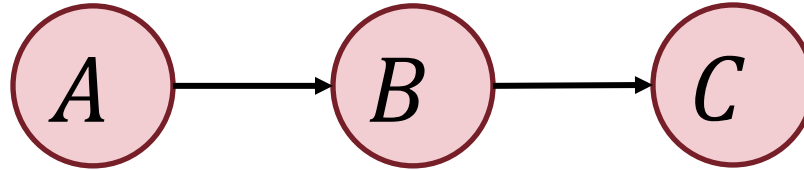


“Rain (A) causes traffic (B) which causes me to be late (C)”

Question: are A and C necessarily independent?

Question: are A and C conditionally independent, given B ?

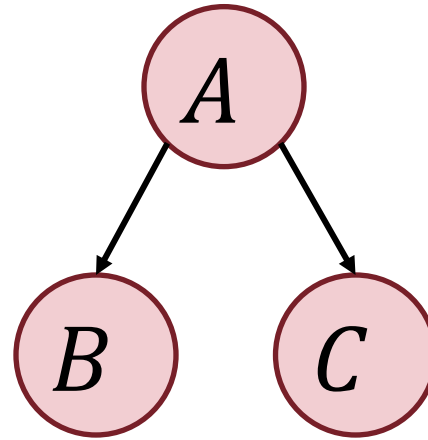
Exploring Independence: Causal Chains



$$\Pr[C \mid A, B] = \frac{\Pr[A \wedge B \wedge C]}{\Pr[A \wedge B]} = \frac{\Pr[A] \Pr[B \mid A] \Pr[C \mid B]}{\Pr[A] \Pr[B \mid A]} = \Pr[C \mid B]$$

$\Pr[C \mid A, B] = \Pr[C \mid B]$: given B , knowing A does not update my beliefs on C !

Exploring Independence: Common Cause

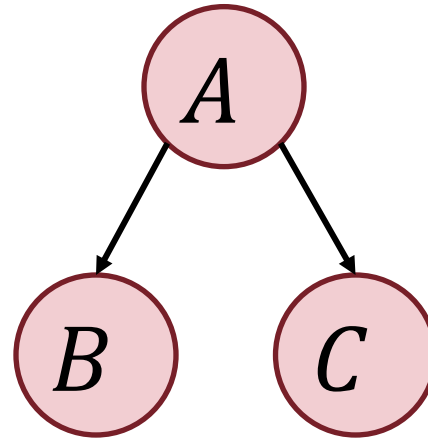


“If Batman is awake (A), he catches the Joker (B) and Bane (C)”

Question: are B and C necessarily independent?

Question: are B and C conditionally independent, given A ?

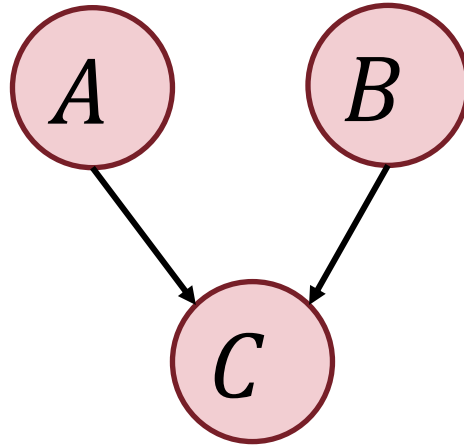
Exploring Independence: Common Cause



$$\Pr[B \mid A, C] = \frac{\Pr[A \wedge B \wedge C]}{\Pr[A \wedge C]} = \frac{\Pr[A] \Pr[B \mid A] \Pr[C \mid A]}{\Pr[A] \Pr[C \mid A]} = \Pr[B \mid A]$$

$\Pr[B \mid A, C] = \Pr[B \mid A]$: given A , knowing C does not update my beliefs on B !

Exploring Independence: Common Effect

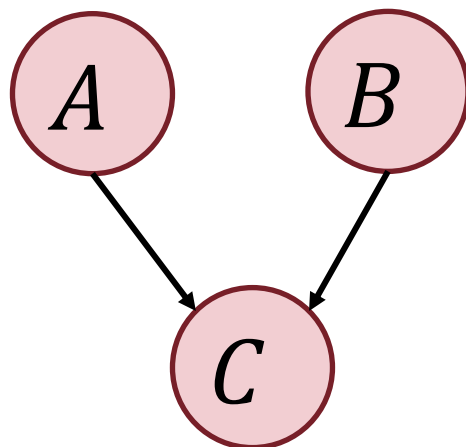


“The Joker (A) and Bane (B) could both rob the bank (C)”

Question: are A and B necessarily independent?

Question: are A and B conditionally independent, given C ?

Exploring Independence: Common Effect



Observing an effect makes two causes dependent

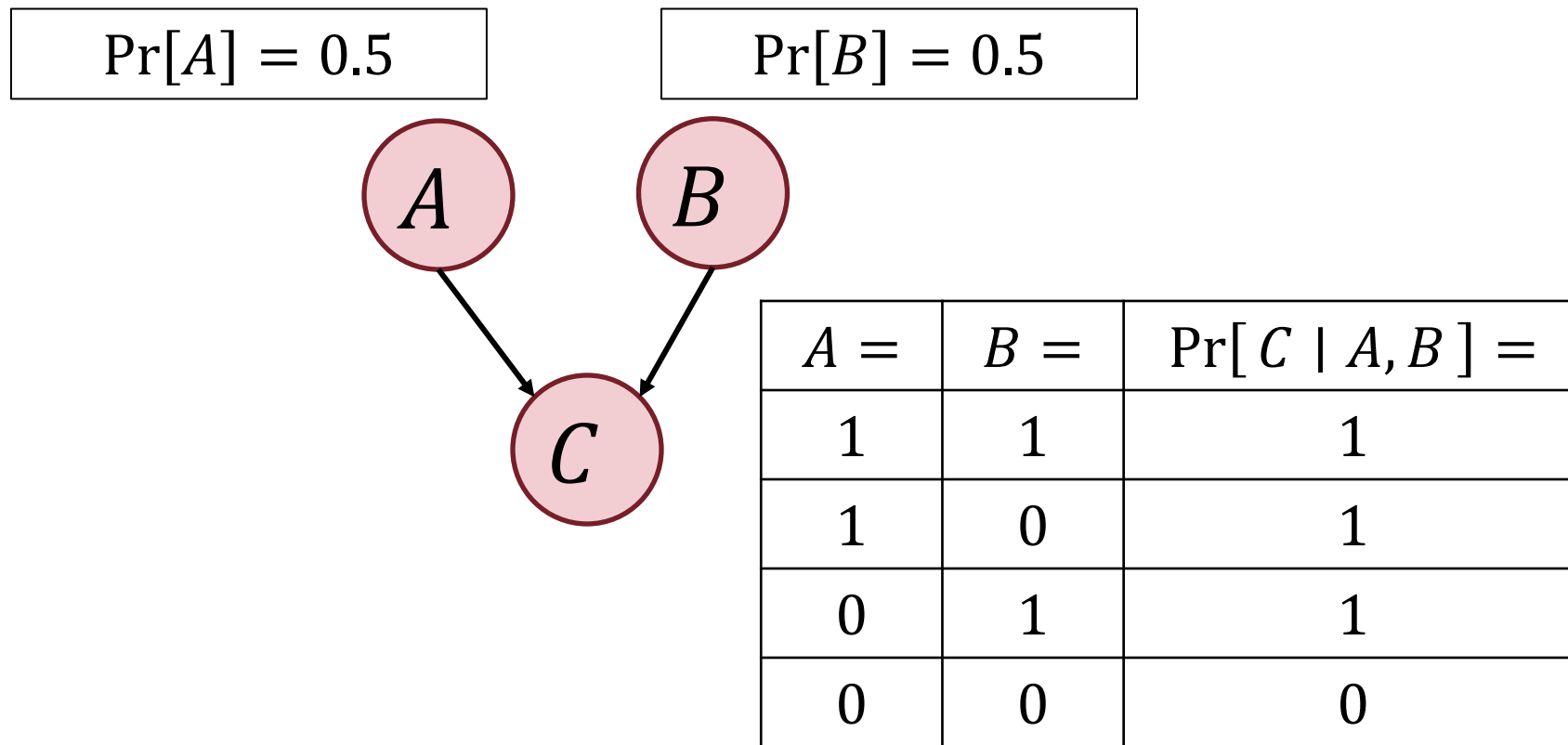
- I know that the bank was robbed ($C = 1$)
- It could be either the Joker or Bane.
- If I know the Joker didn't do it –my belief about Bane doing it is higher!

$$\Pr[A \mid C, B] \neq \Pr[A \mid C]$$

but

$$\Pr[A \mid B] = \Pr[A]$$

It's All About the CPTs



$$\Pr[A = 1] = \Pr[A = 1 | B = 0] = 0.5$$

but

$$\Pr[A = 1 | B = 0, C = 1] = 1$$

Rules for d-Connectedness and d-Separation

■ Rule 1:

- X_1, X_2 are *d-connected* if there is an unblocked path between them:
 - We can traverse a path along the arrow and not encounter a colliding arrow.

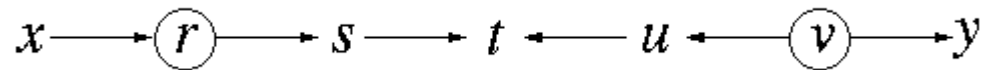
$$x \longrightarrow r \longrightarrow s \longrightarrow t \longleftarrow u \longleftarrow v \longrightarrow y$$

- Here x is d-connected to t , y is d-connected to t , but x is not d-connected to y because of the collision at t . I.e. x is d-separated from y .
- (Note: Also d-connected to nodes in between if there's no collision)

Rules for d-Connectedness and d-Separation

■ Rule 2:

- If X_1, X_2 are conditioned on a set of variables in Z , and if there are no paths from X_1 to X_2 that do not pass through any member of Z , then X_1 and X_2 are d-separated by Z :

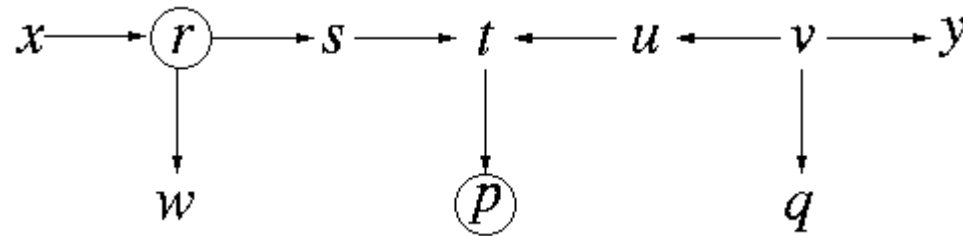


- Here $Z = \{r, v\}$, and x and s are d-separated by r , and likewise u and y are d-separated by v . x and y are d-separated by Z .

Rules for d-Connectedness and d-Separation

■ Rule 3:

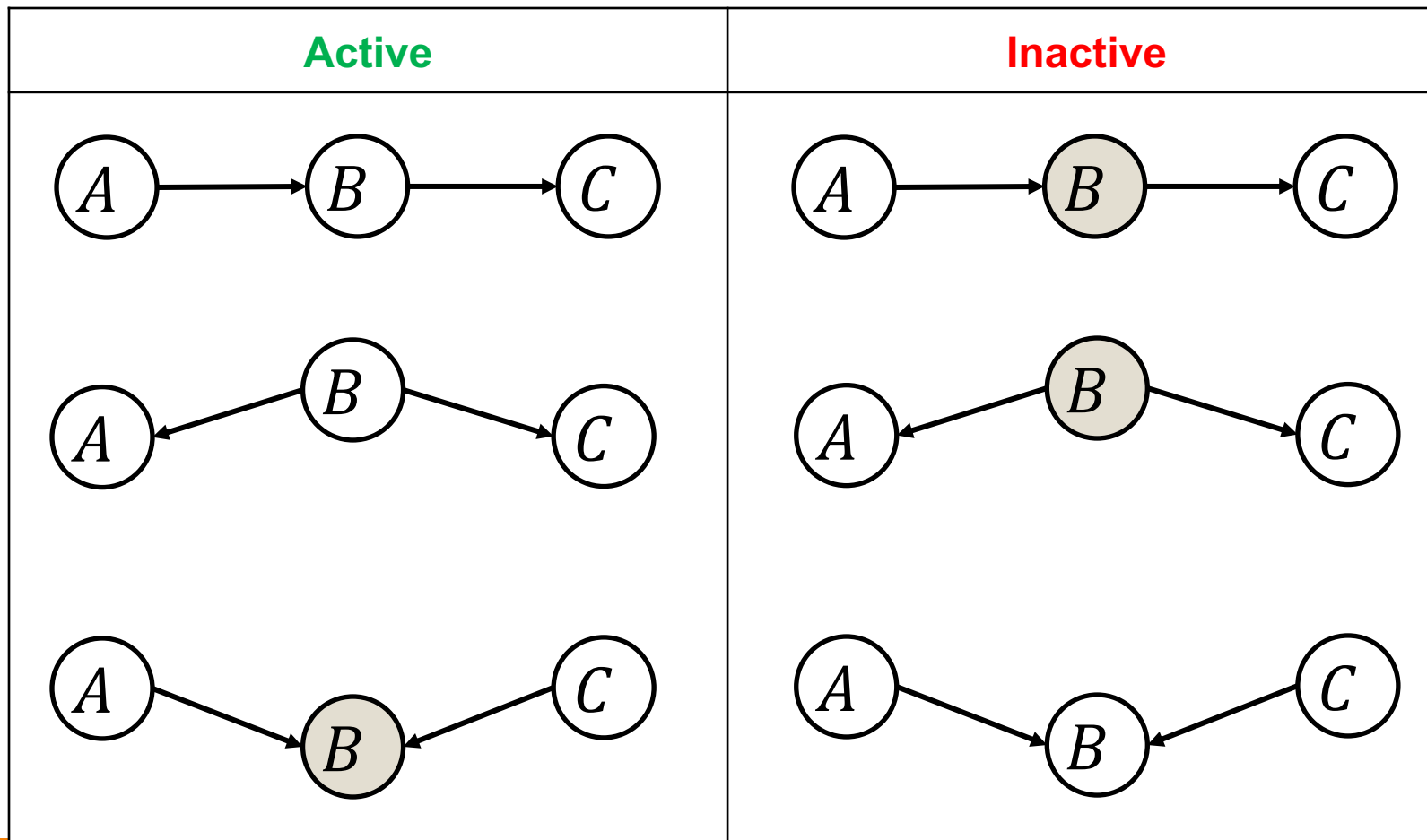
- ❑ Common Effect: If X_1 and/or X_2 have a common effect X_3 , and X_3 is known, then knowing X_1 affects our knowledge of X_2
- ❑ Thus a collider node X_3 no longer blocks the path of X_1 and X_2 if its state, or the state of one of its children is known. Thus X_1 and X_2 are no longer definitively or conditionally independent.



- ❑ Here s and u were formerly blocked by t , but once p is known, t no longer blocks s and u and they are now d-connected.

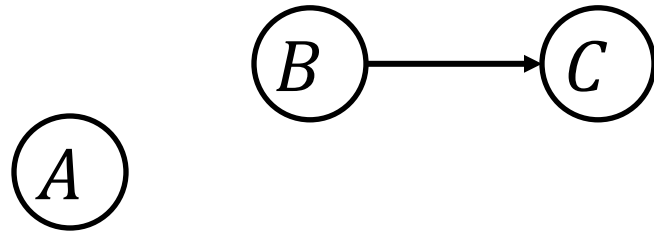
General Case – d Separation

- A path is active iff every triple on path is active
- One inactive triple $\Rightarrow$ path is inactive!



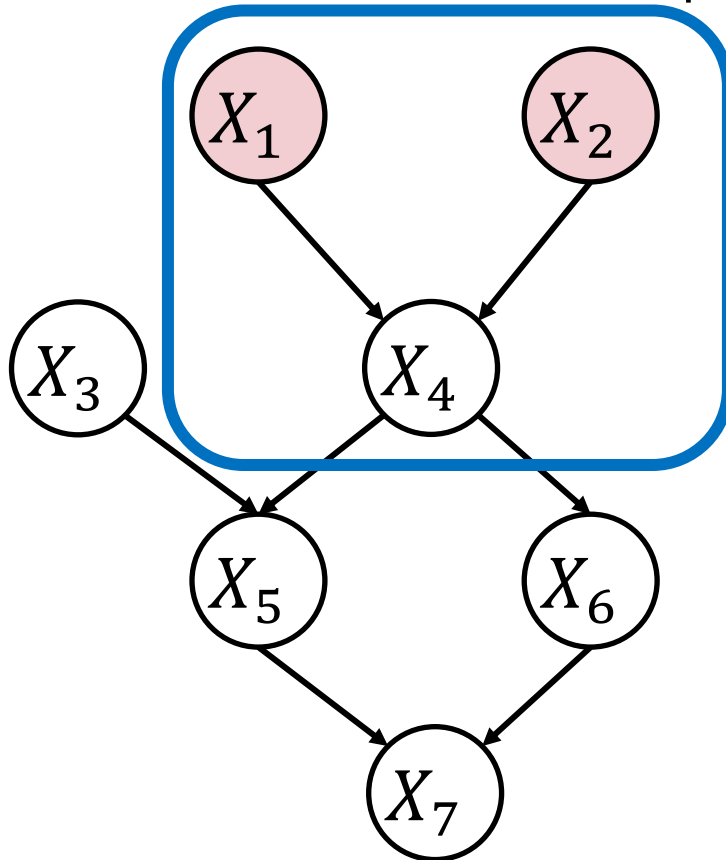
General Case – Analyze the Graph

- Degenerate cases:
 - Disconnected variables: always independent.
 - Directly connected variables: never surely independent.



General Case – Analyze the Graph

- All paths must be inactive.
- A path is active iff every triple on path is active
- One inactive triple $\Rightarrow$ path is inactive!



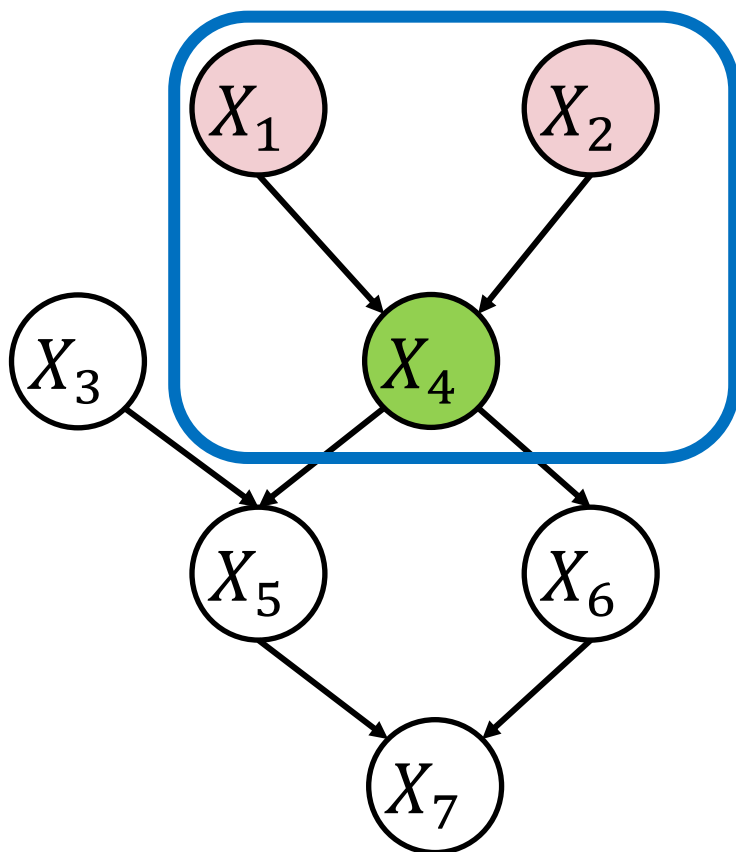
Active	Inactive

Is X_1 independent of X_2 ?

Yes!

General Case – Analyze the Graph

- All paths must be inactive.
- A path is active iff every triple on path is active
- One inactive triple $\Rightarrow$ path is inactive!



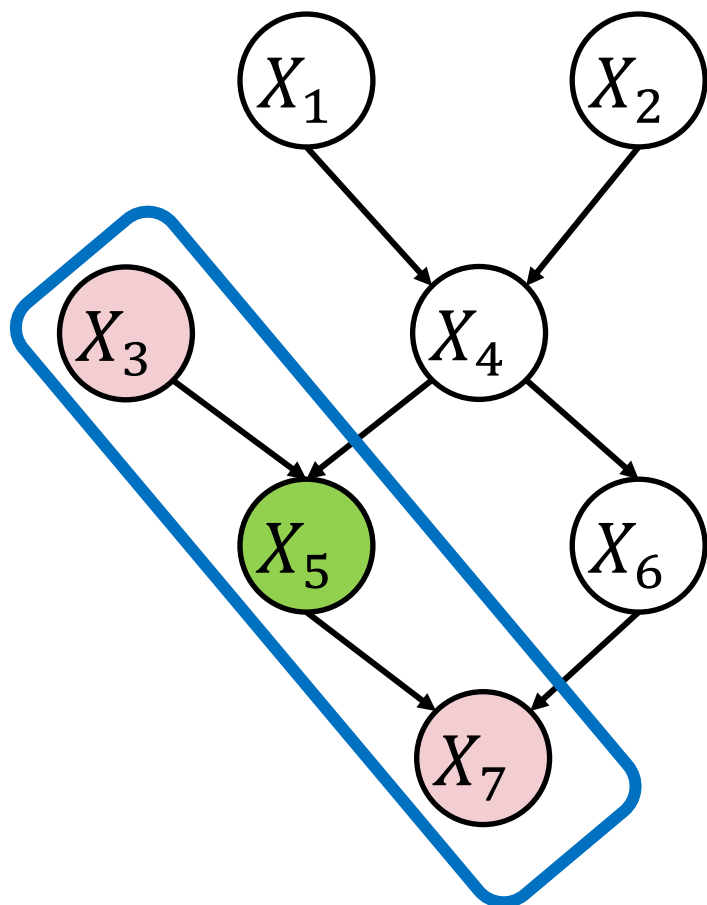
Active	Inactive

Is X_1 independent of X_2 given X_4 ?

No!

General Case – Analyze the Graph

- All paths must be inactive.
- A path is active iff every triple on path is active
- One inactive triple $\Rightarrow$ path is inactive!

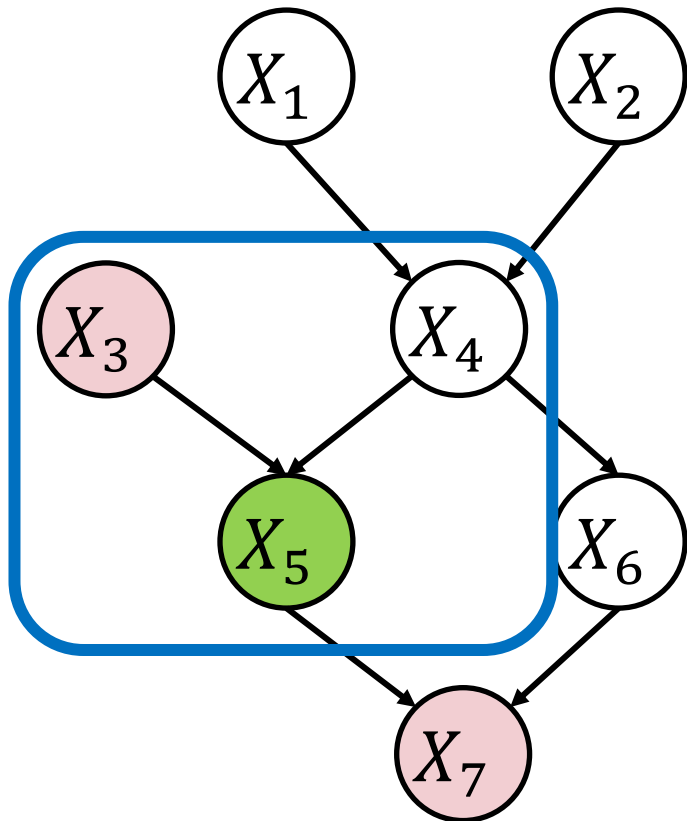


Active	Inactive

Is X_3 independent of X_7 given X_5 ?

General Case – Analyze the Graph

- All paths must be inactive.
- A path is active iff every triple on path is active
- One inactive triple $\Rightarrow$ path is inactive!

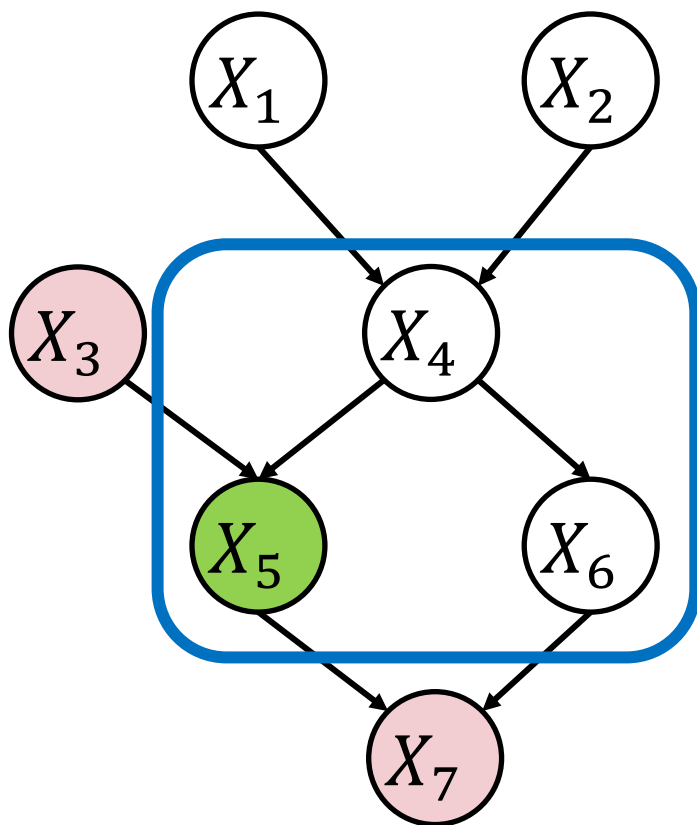


Active	Inactive

Is X_3 independent of X_7 given X_5 ?

General Case – Analyze the Graph

- All paths must be inactive.
- A path is active iff every triple on path is active
- One inactive triple $\Rightarrow$ path is inactive!

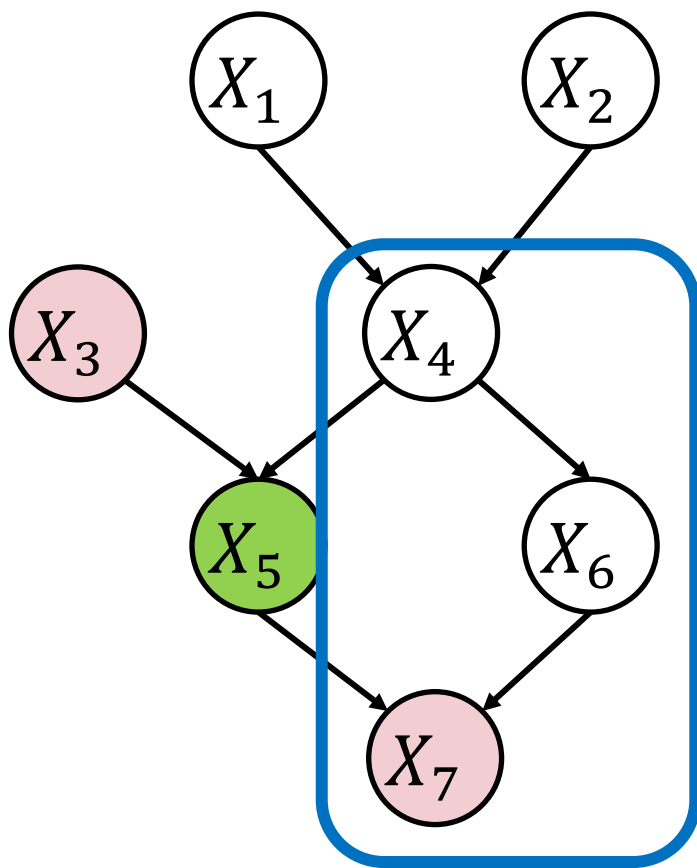


Active	Inactive

Is X_3 independent of X_7 given X_5 ?

General Case – Analyze the Graph

- All paths must be inactive.
- A path is active iff every triple on path is active
- One inactive triple $\Rightarrow$ path is inactive!



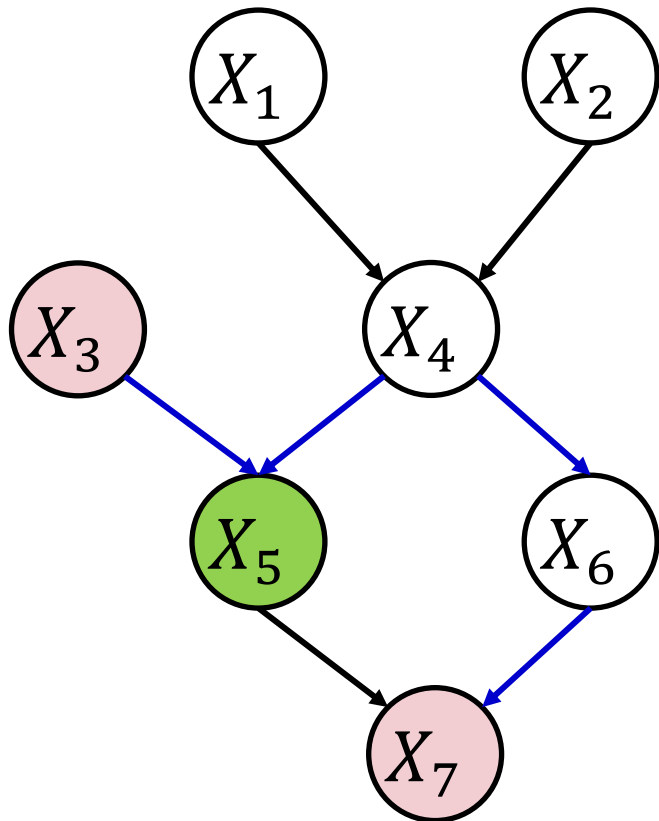
Active	Inactive

Is X_3 independent of X_7 given X_5 ?

No!

General Case – Analyze the Graph

- All paths must be inactive.
- A path is active iff every triple on path is active
- One inactive triple $\Rightarrow$ path is inactive!

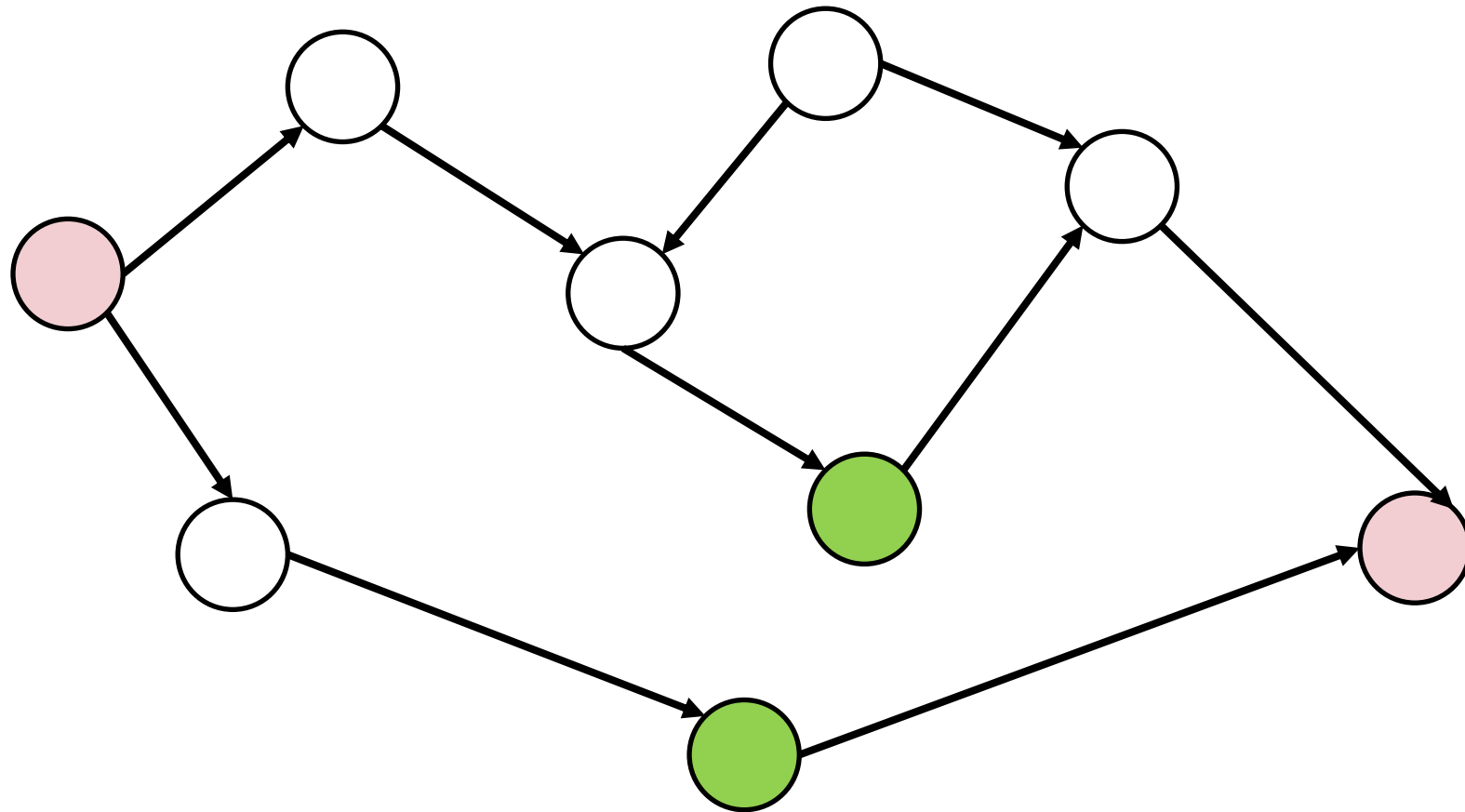


Active	Inactive

Is X_3 independent of X_7 given X_5 ?

No!

X_3, X_5, X_4, X_6, X_7 form an active path



IT5005

Artificial Intelligence

Lecture 10: Reinforcement Learning

Outline

- Introduction to Learning Agents
- Reinforcement Learning Formulation
- Agent policy and optimal policies
- Learning an optimal policy
- Q-learning

Supervised Learning

Agent



=dog



=dog



=cat



=cat



=cat



=dog



=?

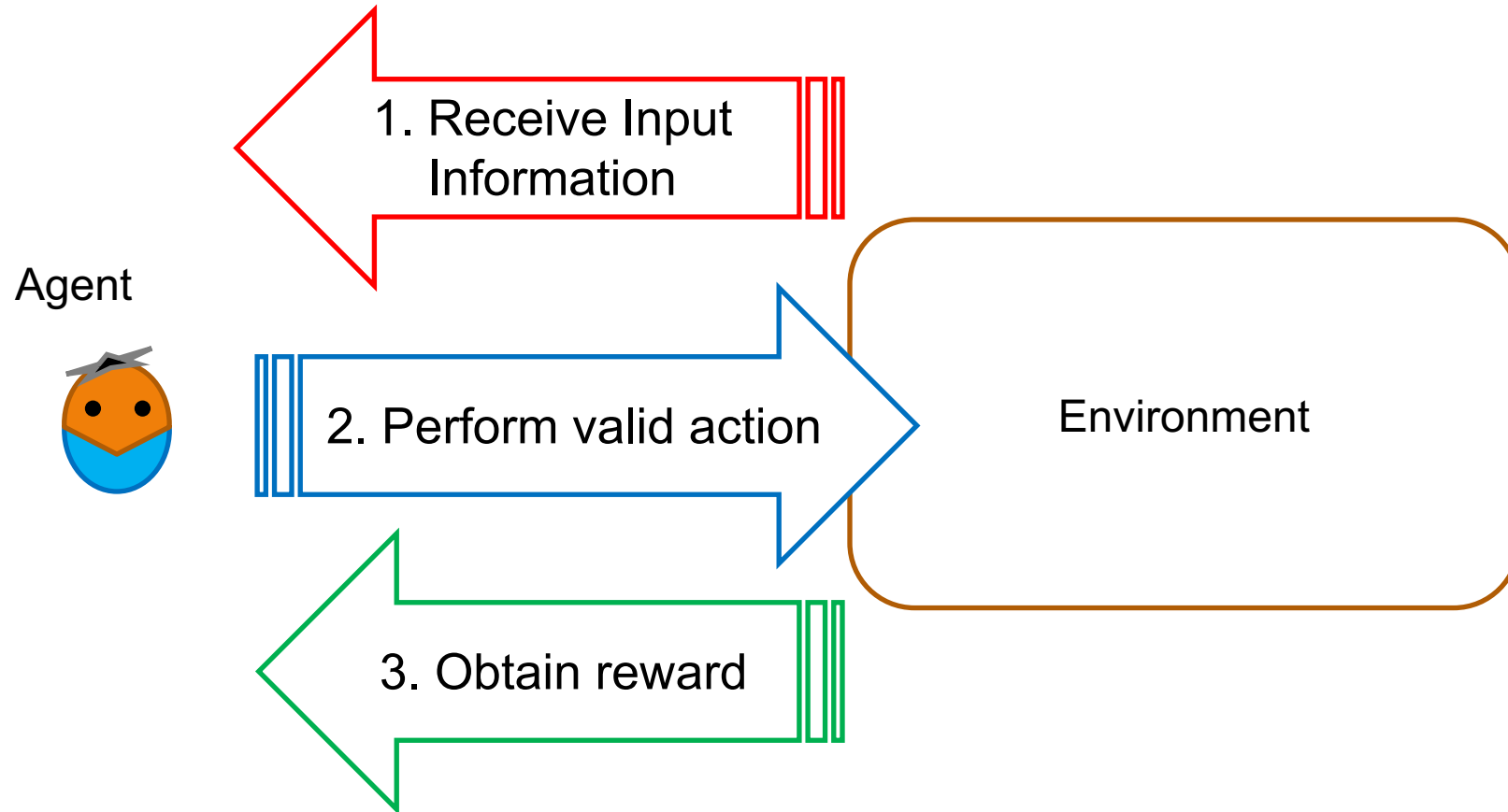
Unsupervised Learning



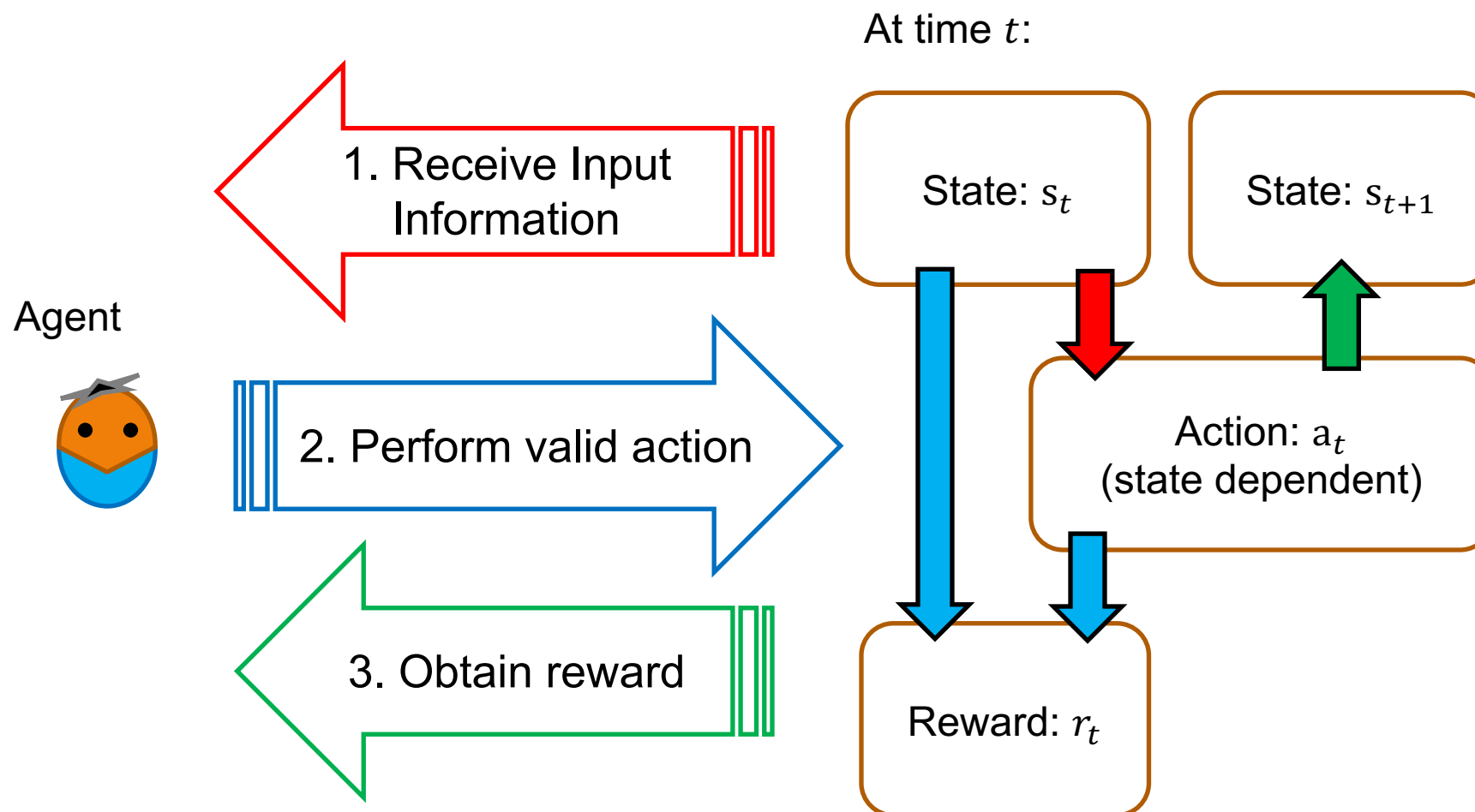
Agent



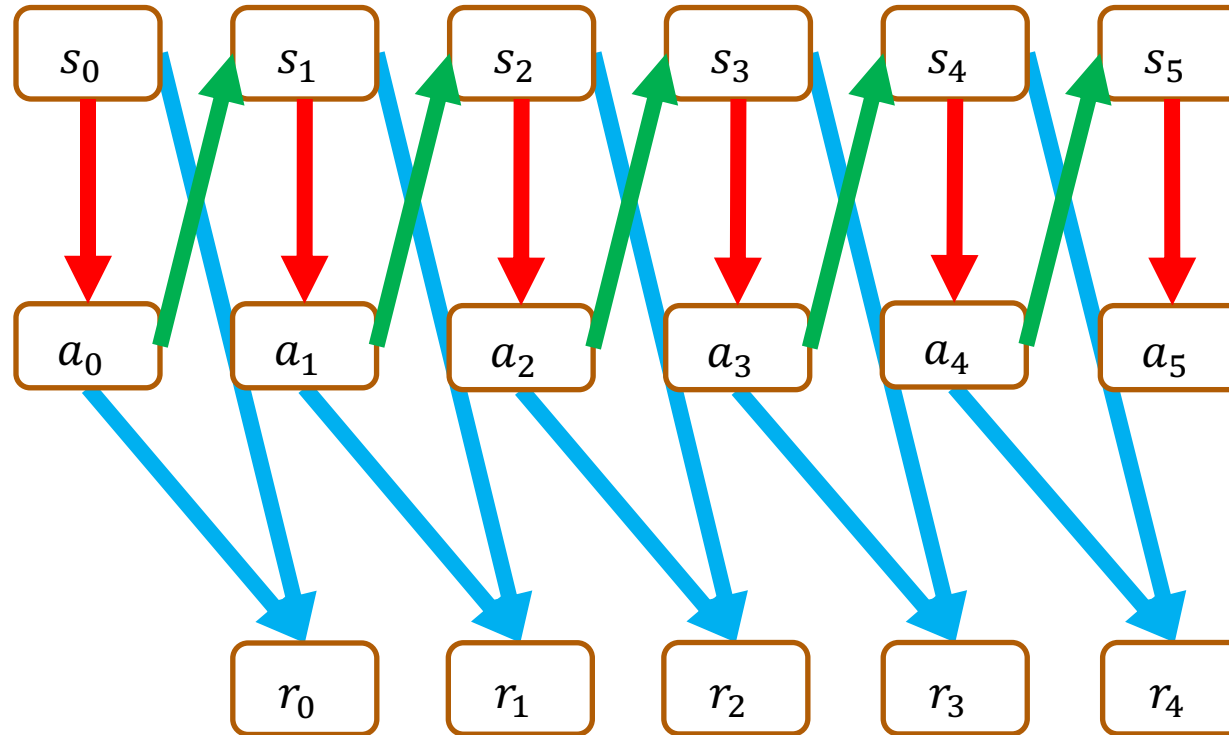
Reinforcement Learning



Reinforcement Learning

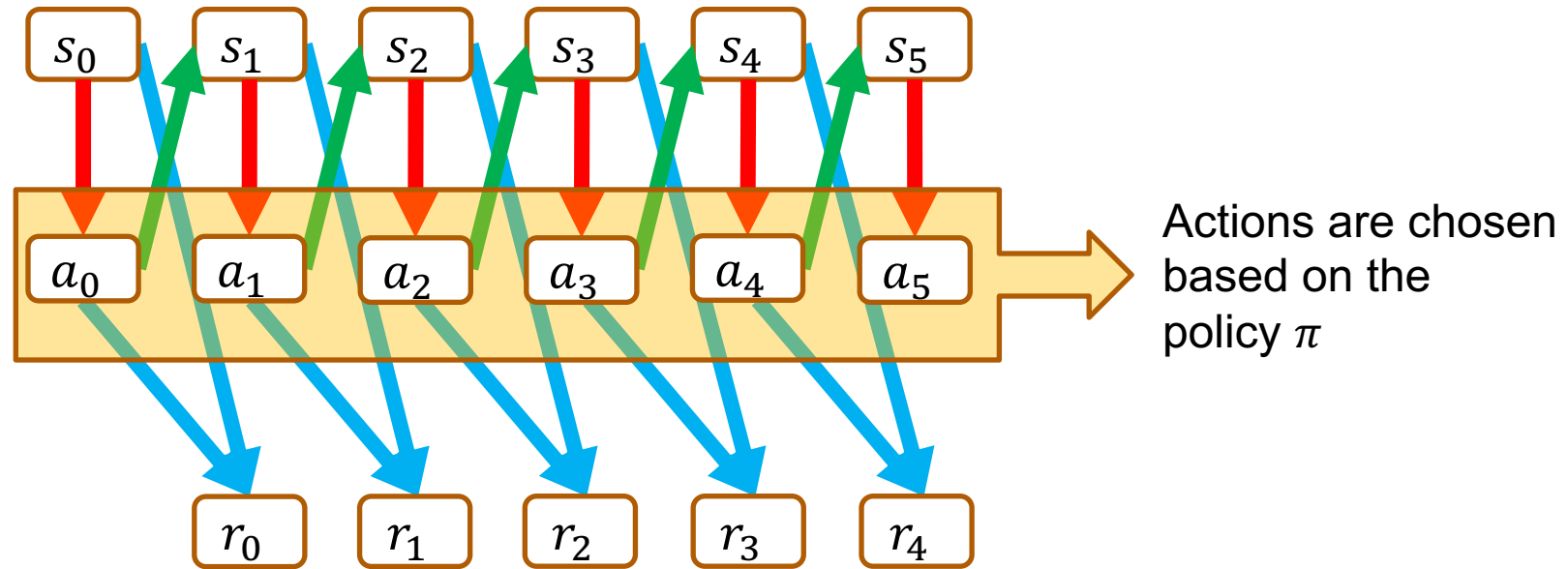


Reinforcement Learning



Markov property: next state is determined only based on current action and state, not entire sequence

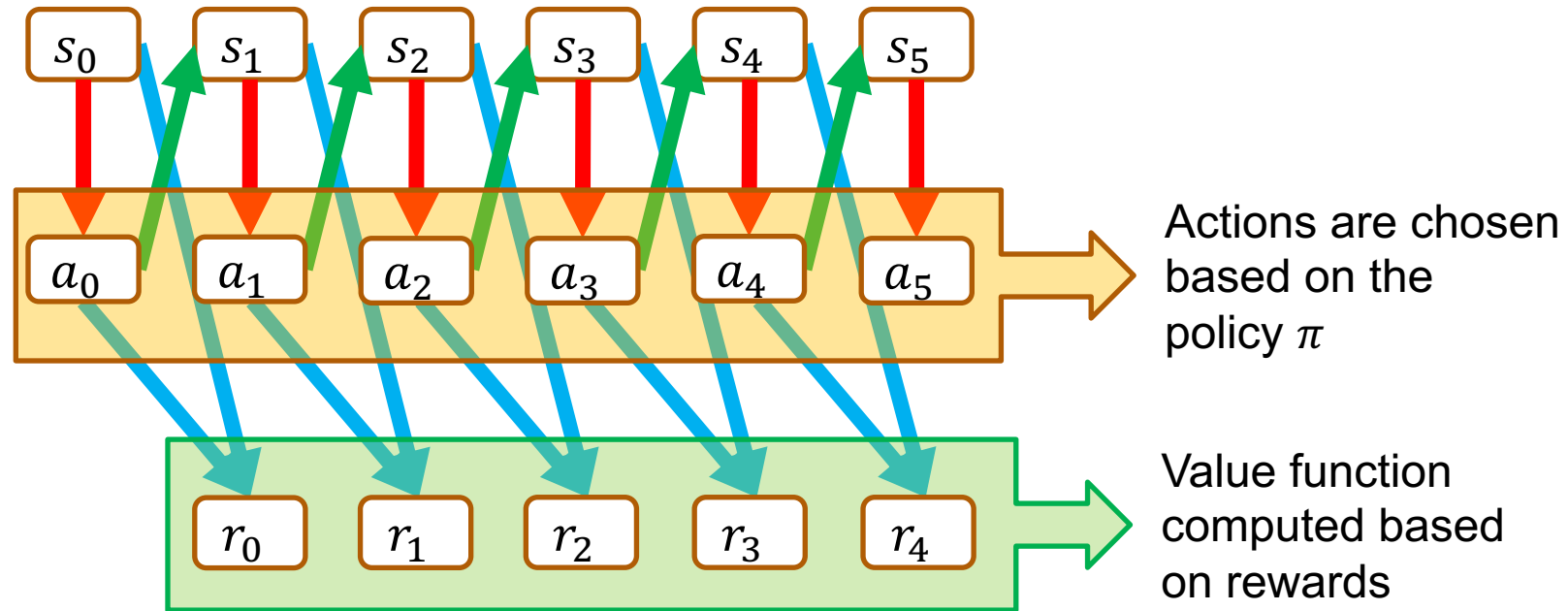
Policy



Policy π determines agent's behavior, i.e. actions

- Deterministic policy: $a_t = \pi(s_t)$
- Stochastic policy: $\pi(a \mid s_t) = \Pr[a_t = a \mid s_t]$

Value Function



Value function tries to predict how good a state is, given the rewards.

$$\begin{aligned} V^\pi(s_t) &= r_t(a_t, s_t) + \gamma r_{t+1}(a_{t+1}, s_{t+1}) + \gamma^2 r_{t+2}(a_{t+2}, s_{t+2}) + \dots \\ &= \sum_{\ell=0}^{\infty} \gamma^\ell r_{t+\ell}(a_{t+\ell}, s_{t+\ell}) \end{aligned}$$

Value Function

Value function tries to predict how good a state is, given the rewards.

$$\begin{aligned} V^\pi(s_t) &= r_t(a_t, s_t) + \gamma r_{t+1}(a_{t+1}, s_{t+1}) + \gamma^2 r_{t+2}(a_{t+2}, s_{t+2}) + \dots \\ &= \sum_{\ell=0}^{\infty} \gamma^\ell r_{t+\ell}(a_{t+\ell}, s_{t+\ell}) \end{aligned}$$

The value γ (a value between 0 and 1) is called the discount rate.

- High value of γ – long-sighted agent, cares about future rewards.
- Low value of γ – agent is greedy, who cares about the future?

Value Function – The Challenge

We want to choose a value maximizing policy.

However, we are missing two key bits of information:

The rewards of unobserved states.

What the next state will be when we take an action.

The challenge:

Infer rewards/state transitions as we go along

...while maximizing revenue.

State:

(x, y) position

Actions:

Up/Down/Left/Right

Reward:

-1 per time step

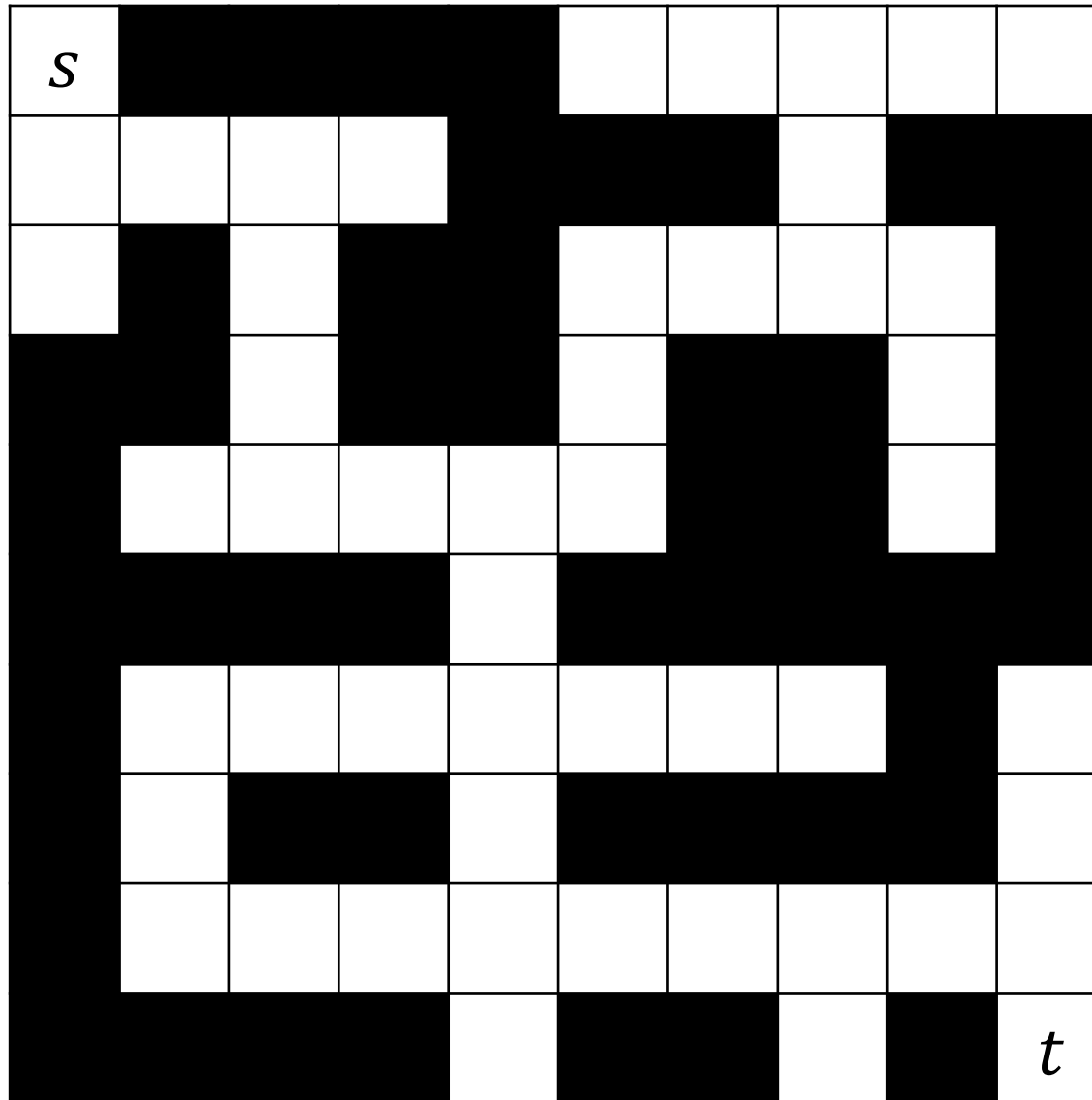
(+10 for reaching goal)

Policy:

Direction to go from each position (can be randomized).

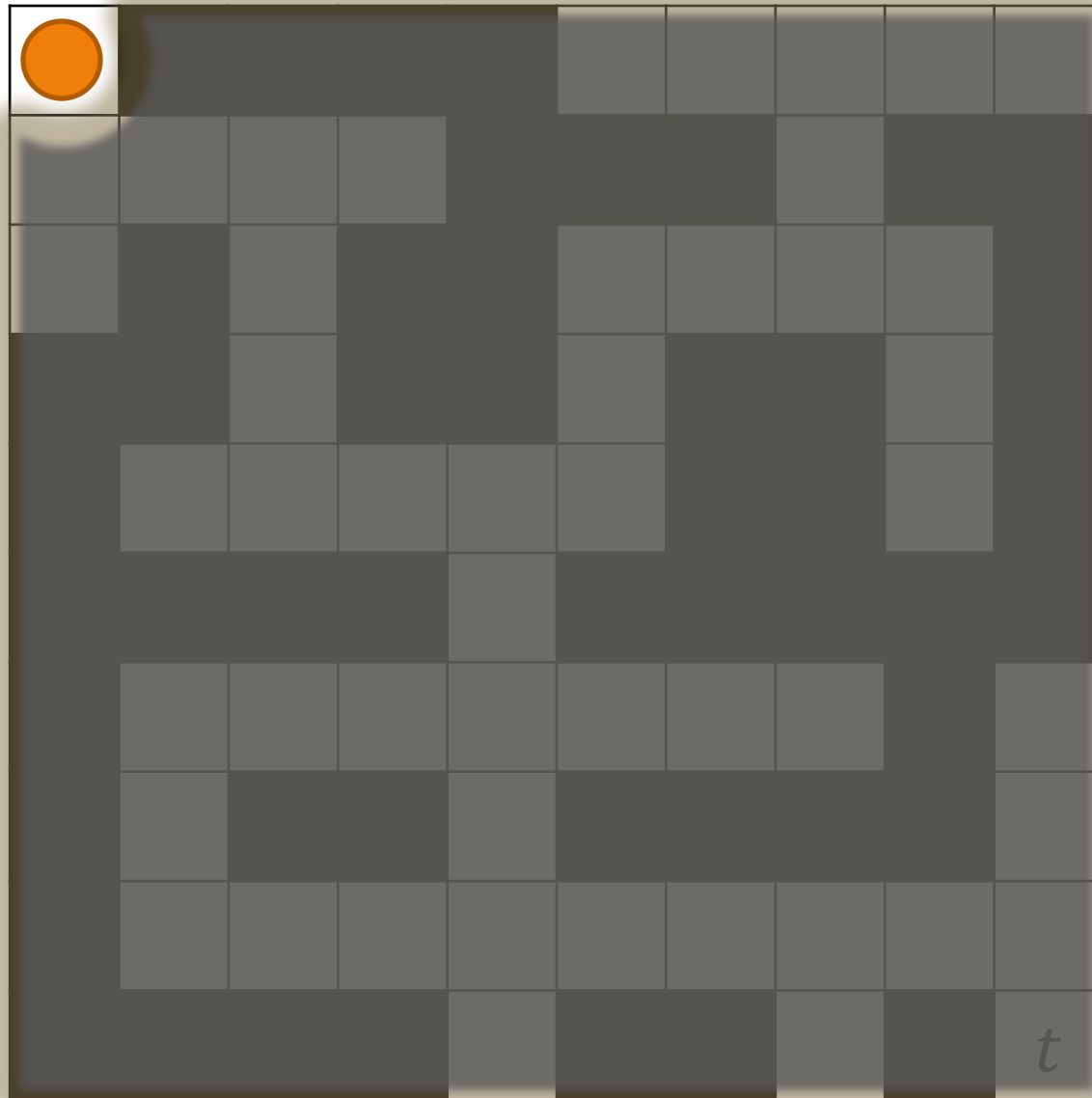
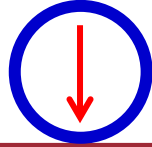
Value Function:

Total reward of policy execution from given state.



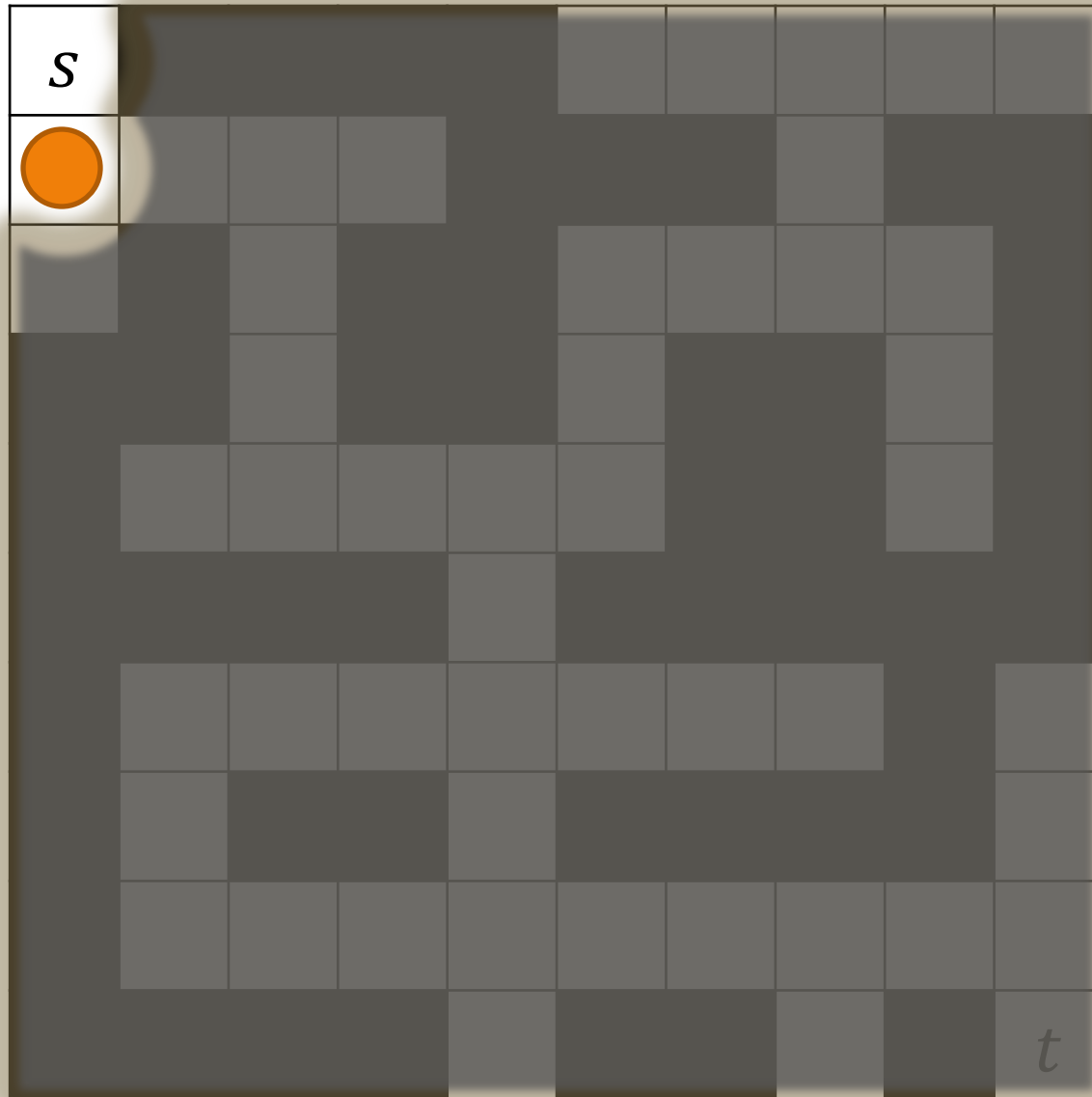
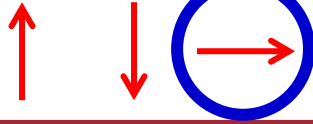
State:
(1,1) position

Actions:



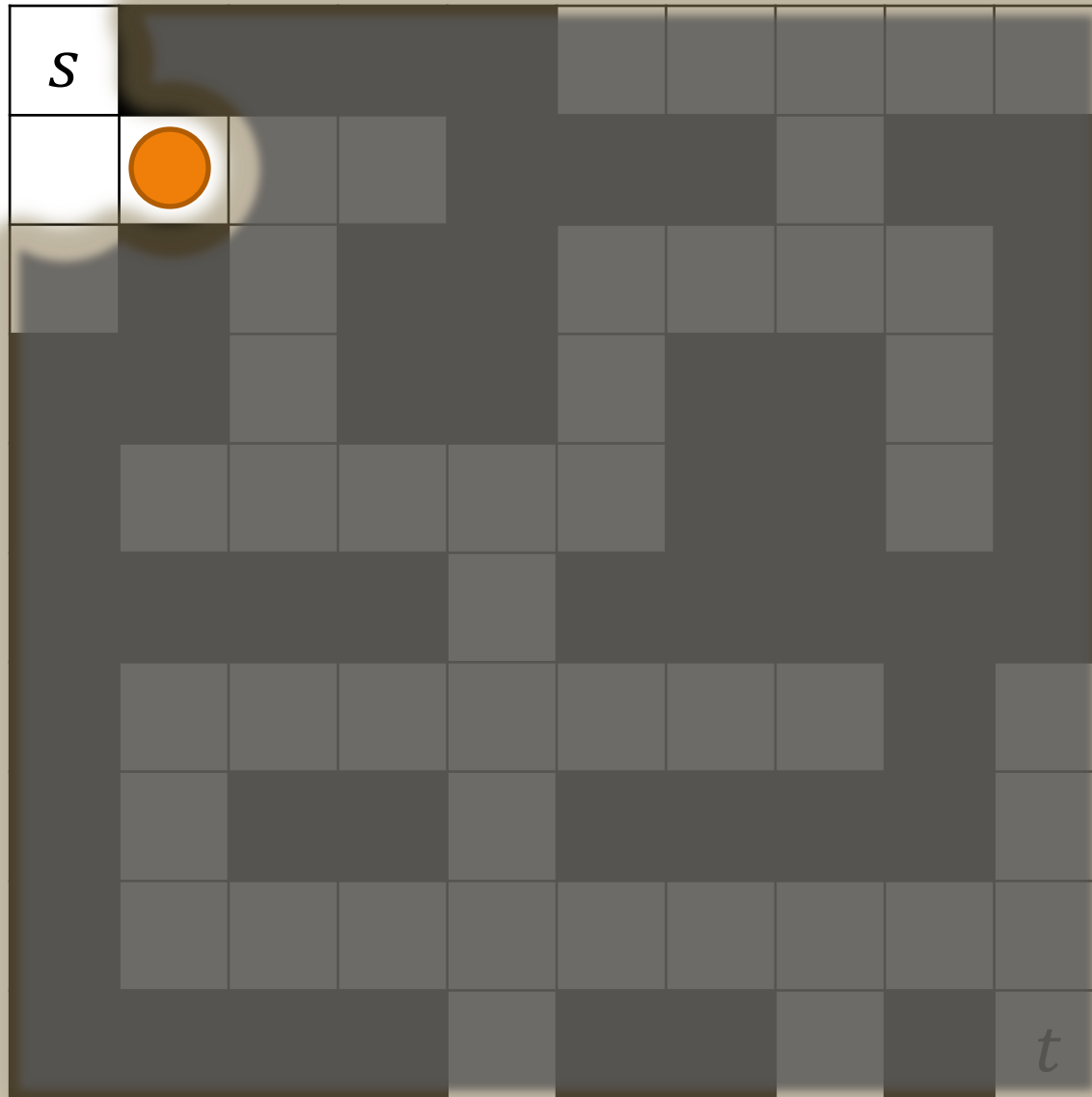
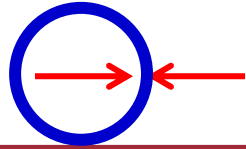
State:
(2,1) position

Actions:



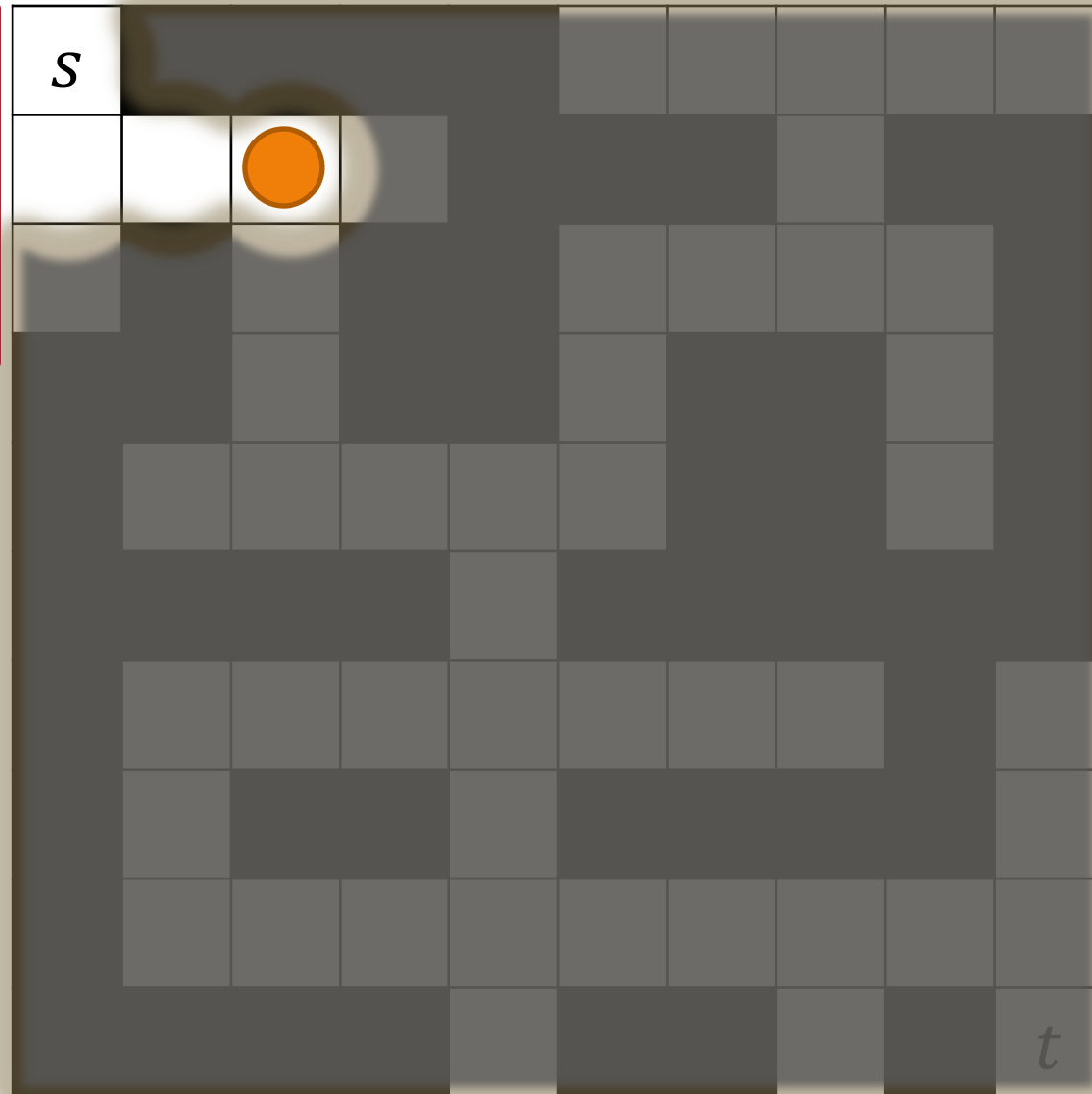
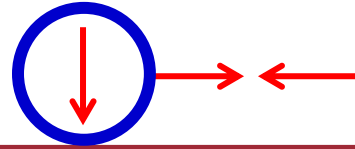
State:
(2,2) position

Actions:



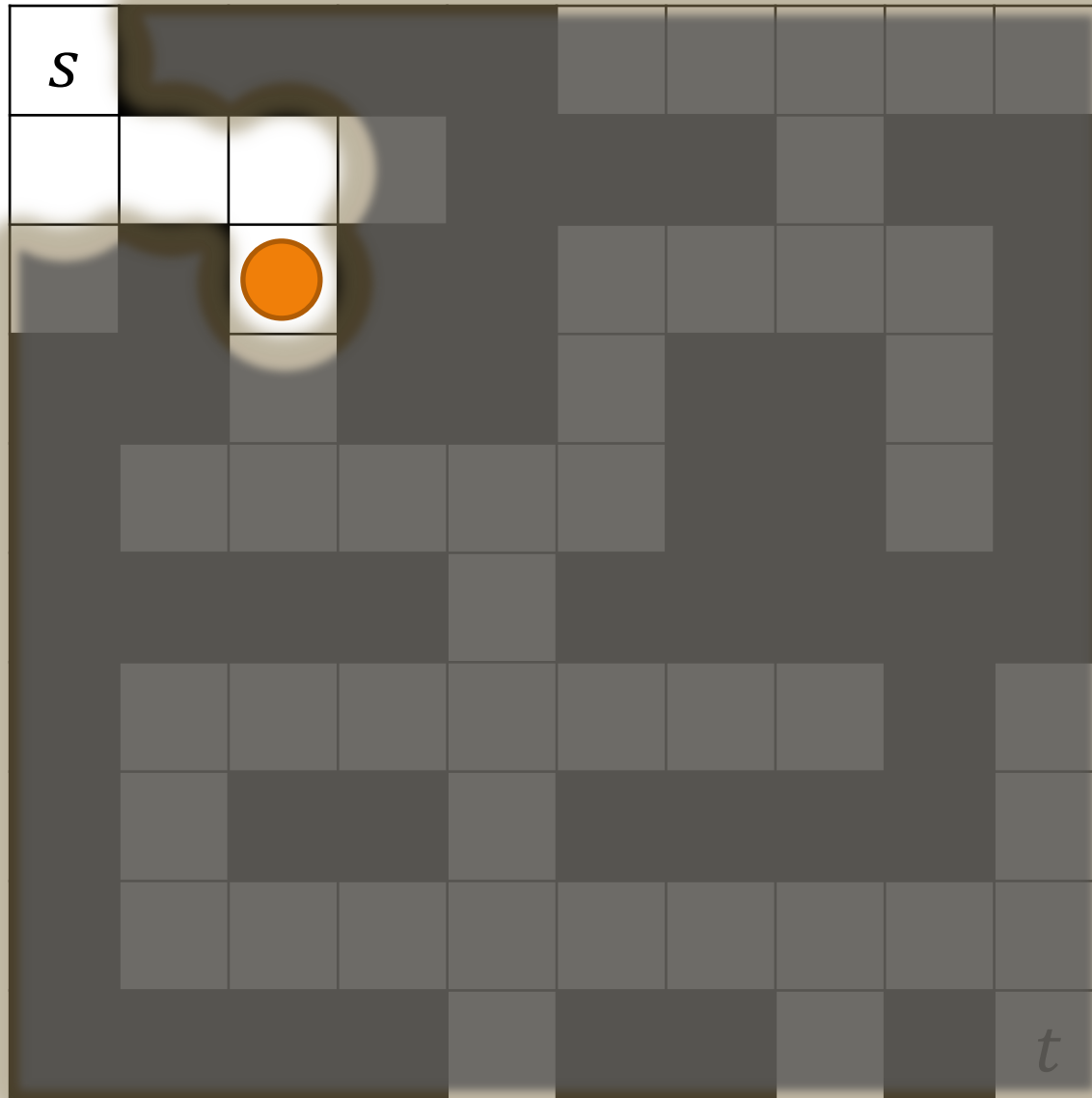
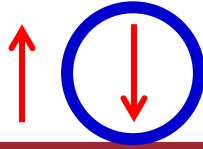
State:
(2,3) position

Actions:



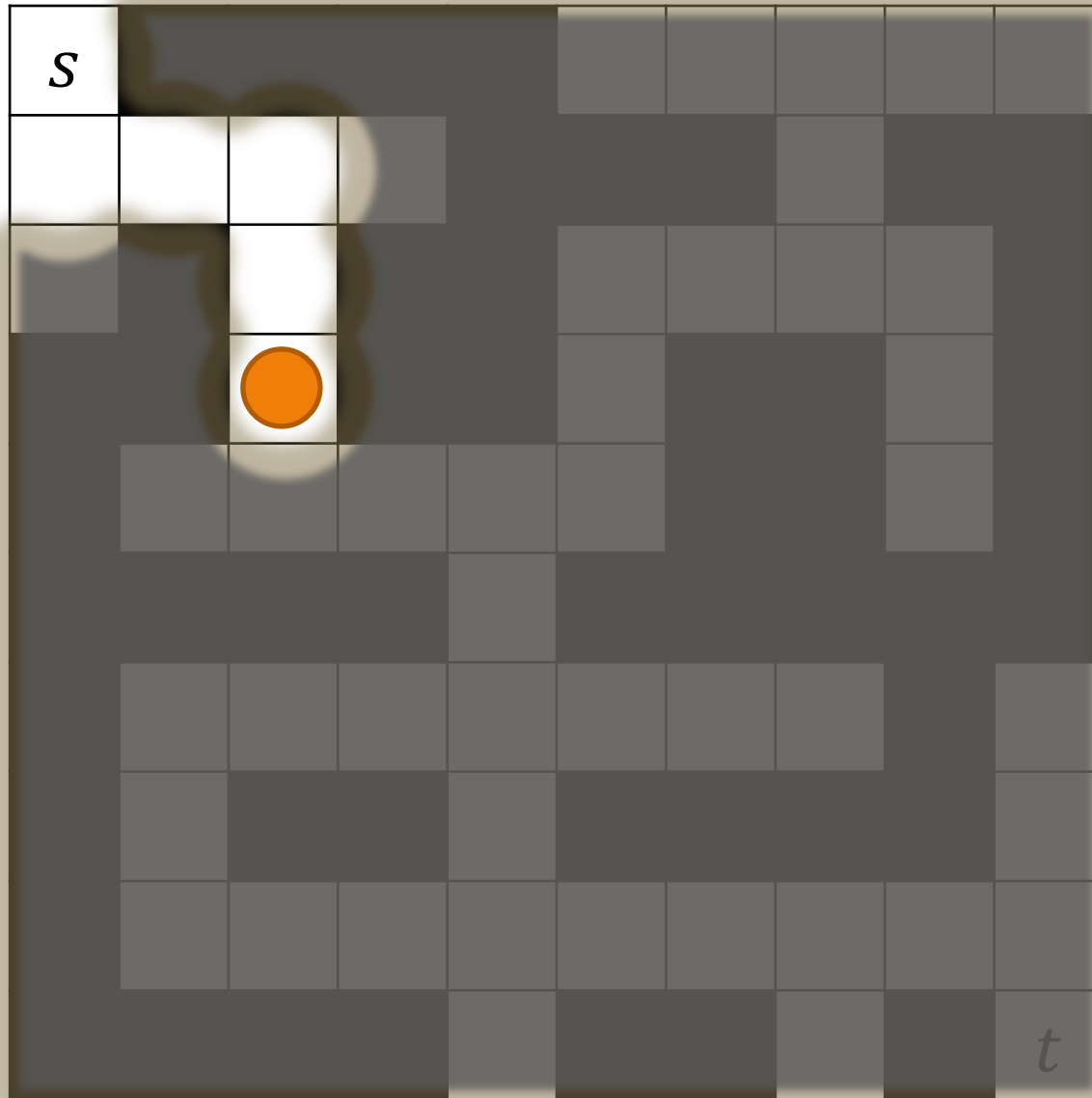
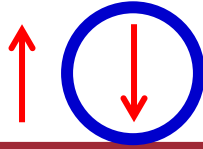
State:
(3,3) position

Actions:



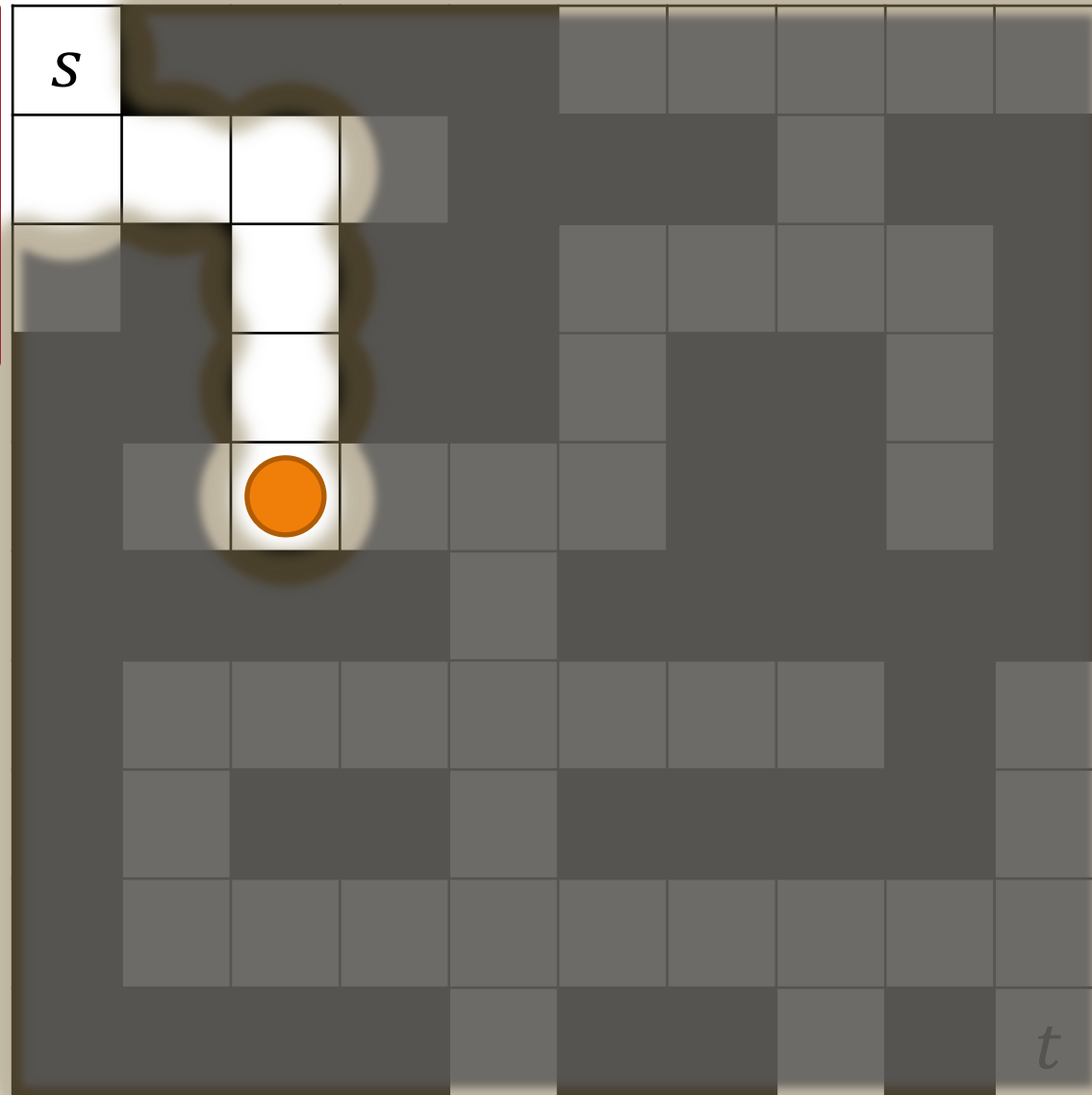
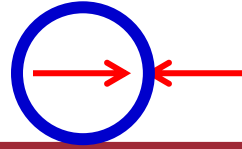
State:
(4,3) position

Actions:



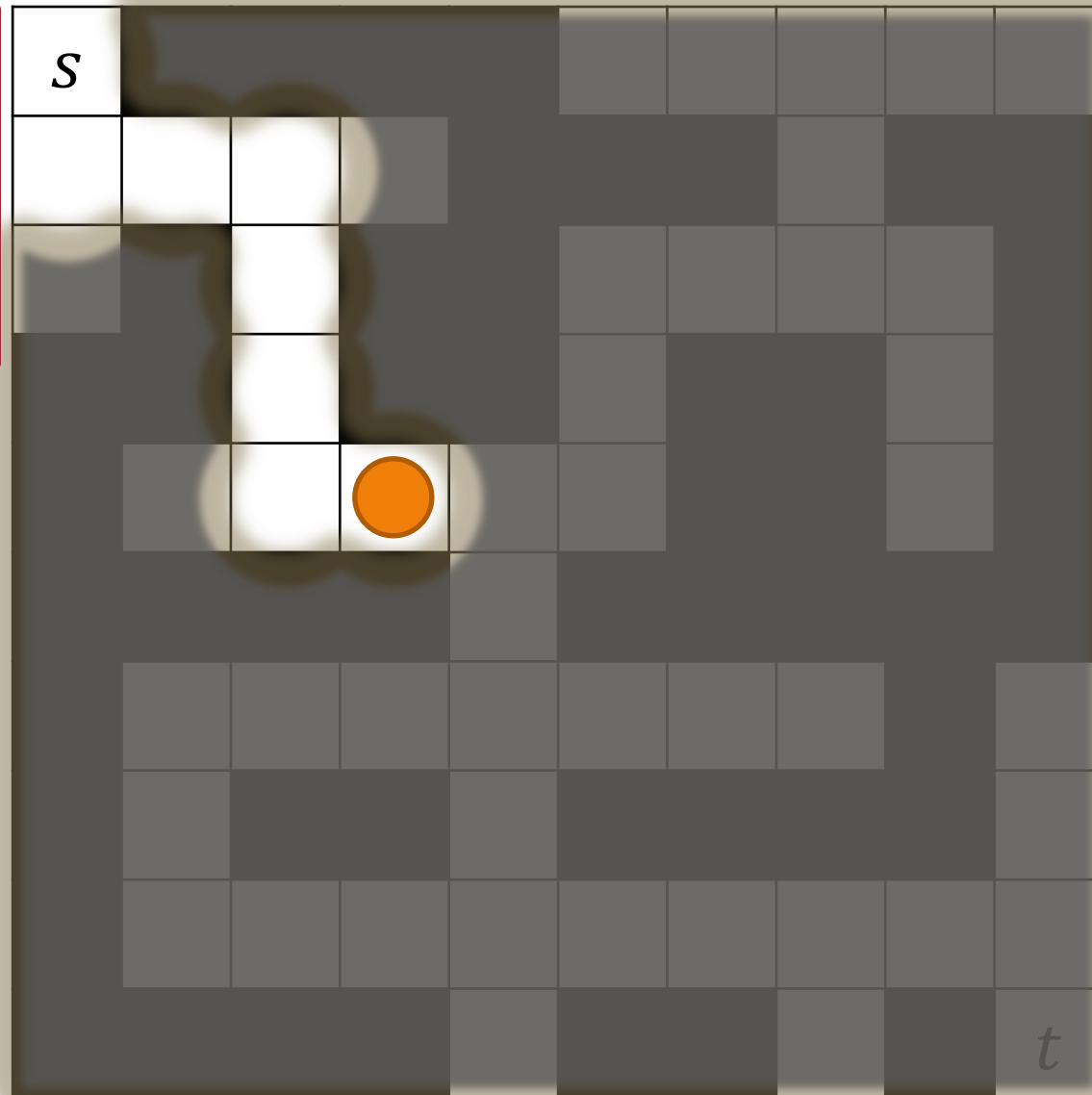
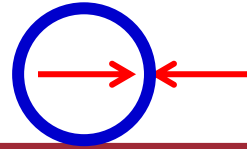
State:
(5,3) position

Actions:



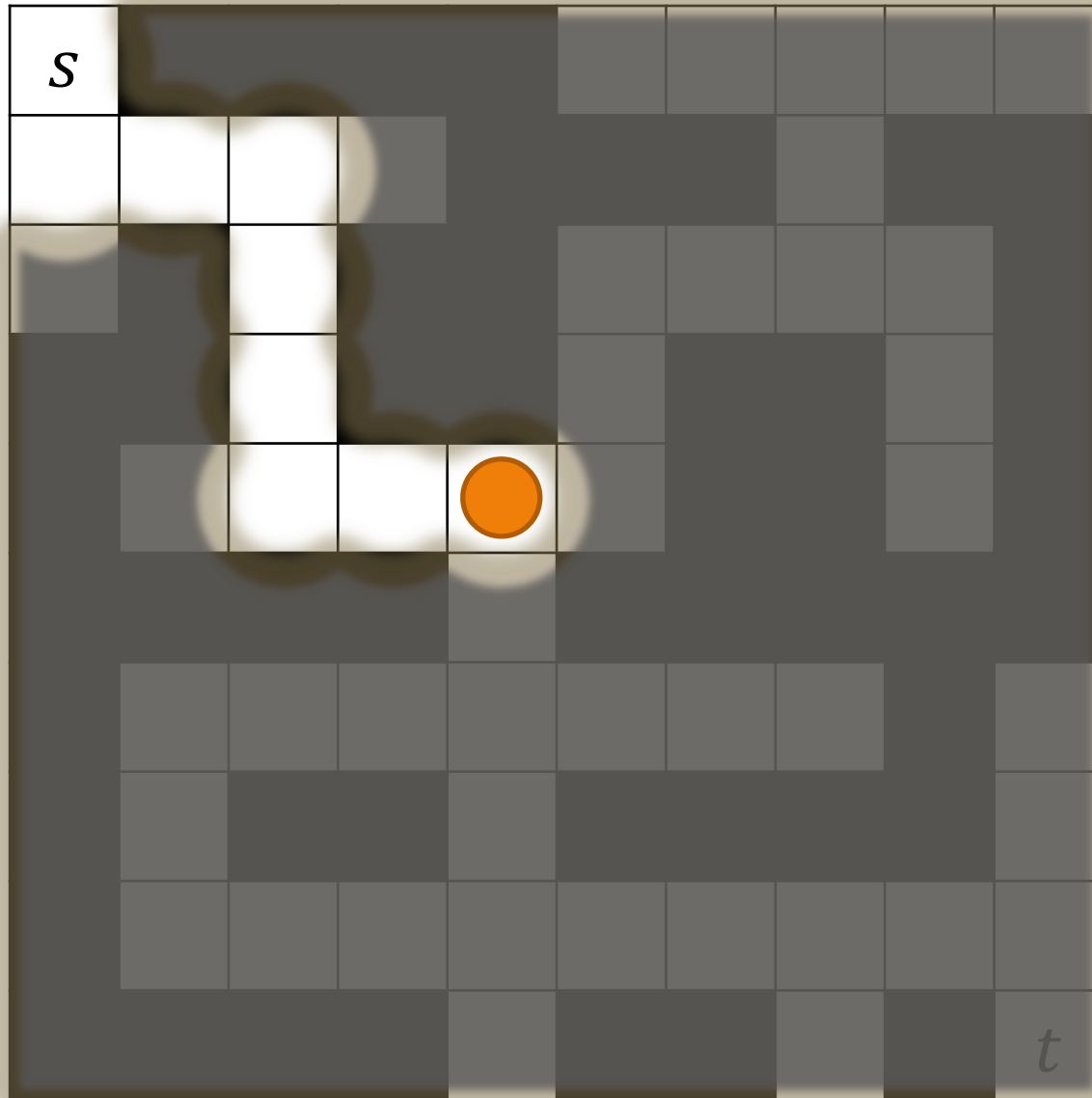
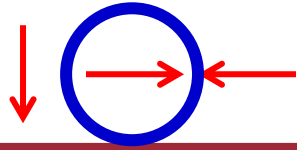
State:
(5,4) position

Actions:



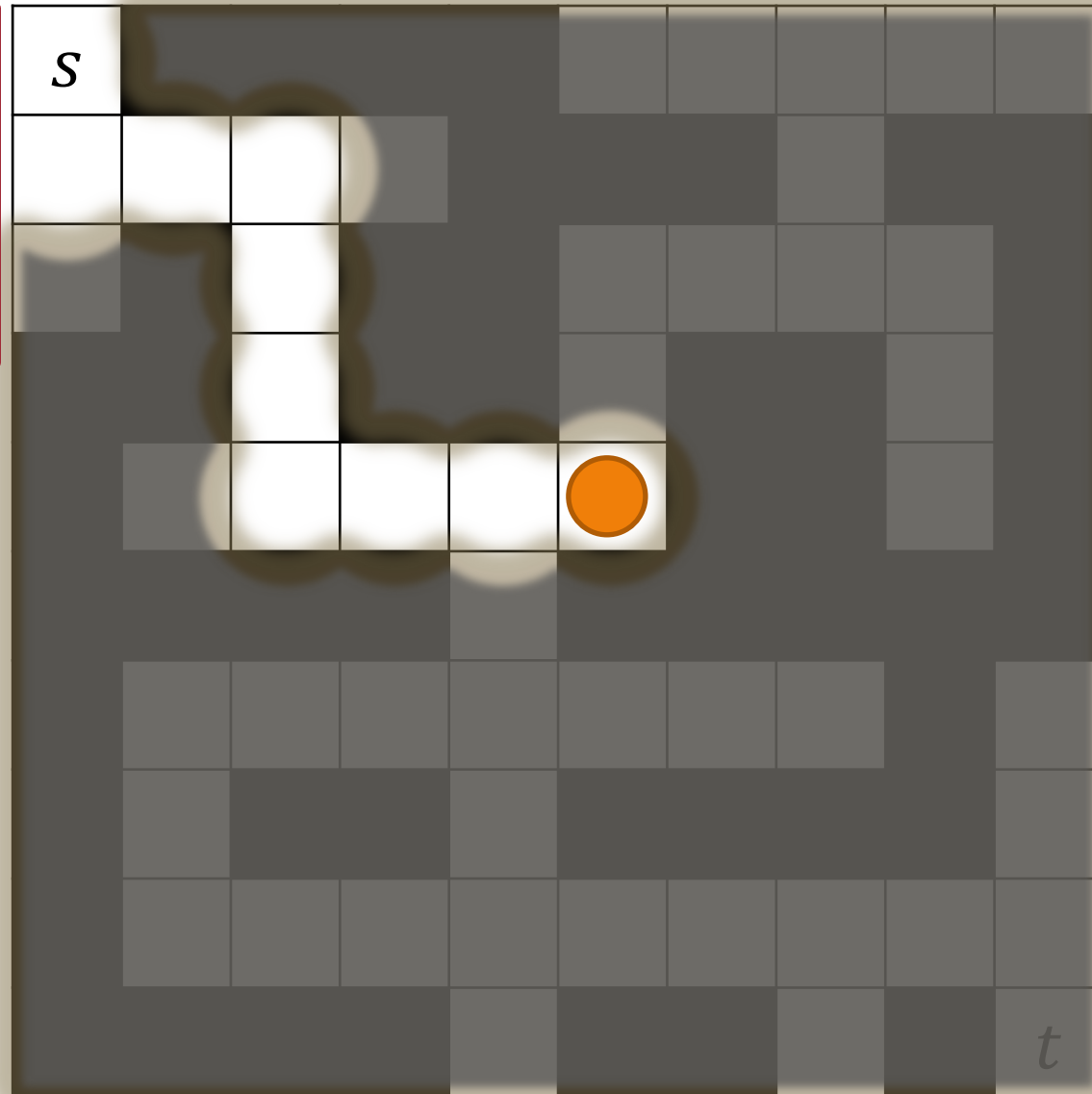
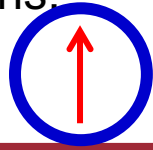
State:
(5,5) position

Actions:

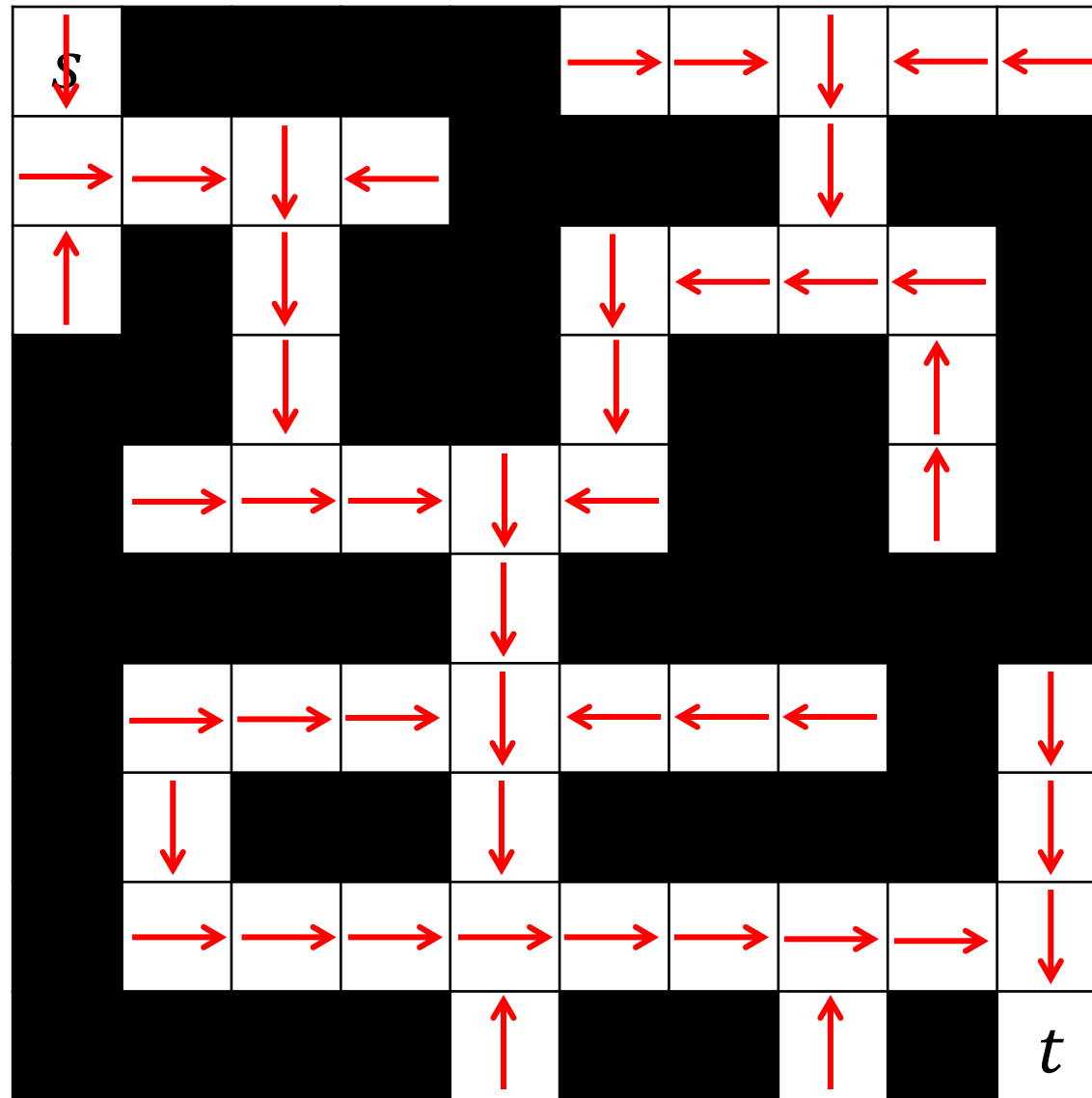


State:
(5,6) position

Actions:



Optimal policy:
spend as little time
in the maze as
possible, get to the
goal.



Value Function:
Discounted reward if
starting from this
state.

Numbers shown for
discount factor $\gamma = 1$

-8									-9	-8	-7	-8	-9
-7	-6	-5	-6								-6		
-8		-4						-3	-4	-5	-6		
		-3						-2				-7	
	-3	-2	-1	0	-1							-8	
				1									
	-1	0	1	2	1	0	-1						7
	0			3									8
	1	2	3	4	5	6	7	8	9				
				3				6					10

		X
	X	
	O	O

Reward:

-1,0,+1 lose/tie/win (seen only on final move)

State:

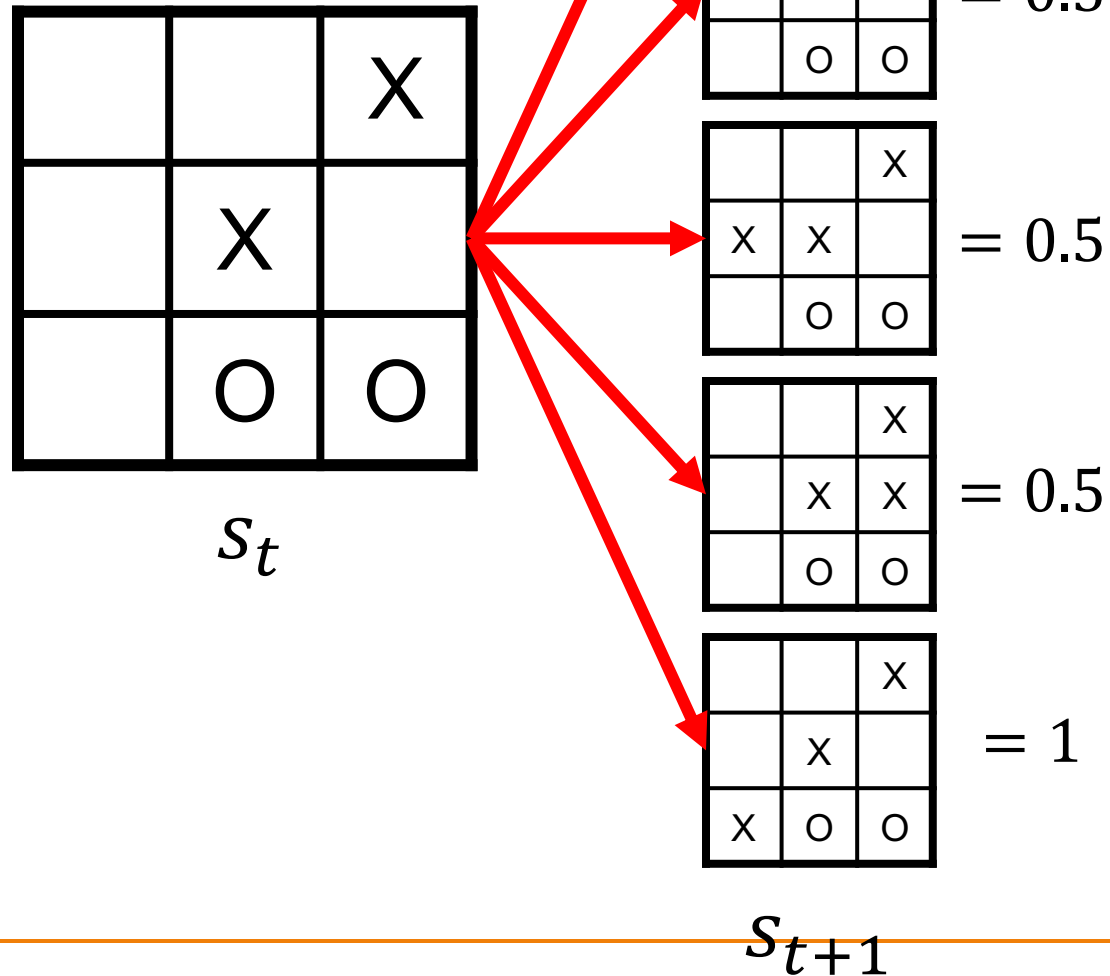
Current positions of X's and O's on board

Policy:

What moves to make in given position?

Value function:

predict future reward given state.



- All values are 0/0.5/1 initially
- At each turn choose move with highest win prob.
- Update table entries based on the game outcome.
- Value function will eventually represent true win probabilities

Alternatively:

- pick with probability proportional to win prob
- make a random choice.

Update strategy is critical:

- Necessary for convergence to optimal strategy.
- Some will work better than others.

Markov Decision Problem

Completely specified by a distribution:

$$\Pr[s_{t+1} = s; r_{t+1} = r \mid s_t, a_t]$$

“What is the next state and reward given current state and action?”

Planning: given an MDP, compute optimal policy

Learning: don't know the MDP, learn a strategy.

Markov Decision Problem

Given complete knowledge of MDP:

$$\Pr[s_{t+1} = s; r_{t+1} = r \mid s_t, a_t]$$

The optimal policy is deterministic - select optimal action in each state.

But... agent doesn't know the underlying MDP.

Needs to perform trial-and-error, interact with environment.

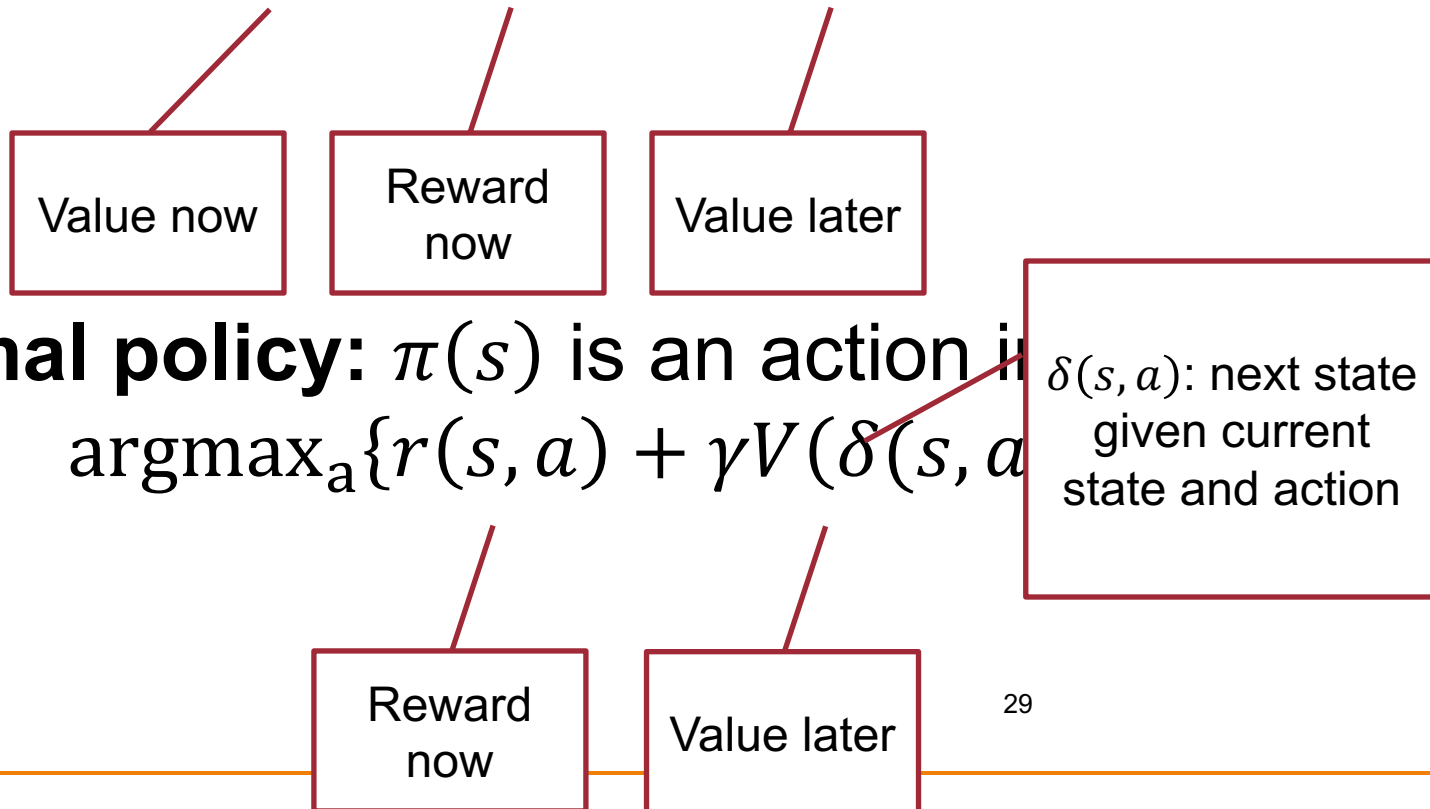
... and not lose too much reward along the way.

Learning Optimal Policies

We want to maximize (discounted) revenue.

Note that:

$$V^\pi(s_t) = r_{t+1} + \gamma V^\pi(s_{t+1})$$



Learning Optimal Policies

Optimal policy: $\pi^*(s)$ is an action in
 $\operatorname{argmax}_a \{r(s, a) + \gamma V^*(\delta(s, a))\}$

Its value is:

$$V^*(s_t) = r_{t+1} + \gamma V^*(s_{t+1})$$

We could identify optimal policy $\pi^*(s)$ if we knew $r(s, a)$ and $\delta(s, a)$.

We don't. Cannot choose optimal actions.

Q-Learning

Define:

$$Q^{\pi}(s, a) = r(s, a) + \gamma V^{\pi}(\delta(s, a))$$

$Q^{\pi}(s, a)$ is the utility we obtain if we take action a at state s , and then follow the policy π from then on.

Optimal Q function:

$$Q^*(s, a) = r(s, a) + \gamma V^*(\delta(s, a))$$

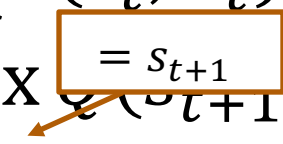
$Q^*(s, a)$ is the utility we obtain if we take action a at state s , and then follow the optimal policy from then on.

Q-Learning

Q^* and V^* are very similar:

$$V^*(s) = \max_a Q(s, a)$$

Therefore:

$$\begin{aligned} Q^*(s_t, a_t) &= r(s_t, a_t) + \gamma V^*(\delta(s_t, a_t)) \\ &= r(s_t, a_t) + \gamma \max_a Q(s_{t+1}, a) \end{aligned}$$


Q-Learning – Value Iteration Algorithm

1. Initialize $\hat{Q}(s, a) \leftarrow 0$ for all s, a .
2. Start at s_0
3. For $t = 0, \dots, \infty$:
 - a. Select action a_t
 - b. Receive reward $r(s_t, a_t)$
 - c. Set $s_{t+1} \leftarrow \delta(s_t, a_t)$
 - d. Set $\hat{Q}(s_t, a_t) \leftarrow r(s_t, a_t) + \gamma \max_{a'} \hat{Q}(s_{t+1}, a')$

Key observation: when $r(s, a) \geq 0$ and $\hat{Q} = 0$, $\hat{Q}(s, a) \leq Q^*(s, a)$ always.

In other words – we always underestimate the optimal Q values.

They always increase at every iteration, thus we converge to Q^* ... and in particular to an optimal policy!

Q-Learning – Value Iteration Algorithm

1. Initialize $\hat{Q}(s, a) \leftarrow 0$ for all s, a .
2. Start at s_0
3. For $t = 0, \dots, \infty$:
 - a. Select action a_t
 - b. Receive reward $r(s_t, a_t)$
 - c. Set $s_{t+1} \leftarrow \delta(s_t, a_t)$
 - d. Set $\hat{Q}(s_t, a_t) \leftarrow r(s_t, a_t) + \gamma \max_{a'} \hat{Q}(s_{t+1}, a')$

Problem:

We ignore current $\hat{Q}(s_t, a_t)$ value in computation.

$r(s, a)$ can be stochastic, as is $\delta(s, a)$.

One bad experience can result in bad underestimate of $Q^*(s, a)$.

Q-Learning – Value Iteration Algorithm

1. Initialize $\hat{Q}(s, a) \leftarrow 0$ for all s, a .
2. Start at s_0
3. For $t = 0, \dots, \infty$:
 - a. Select action a_t
 - b. Receive reward $r(s_t, a_t)$
 - c. Set $s_{t+1} \leftarrow \delta(s_t, a_t)$
 - d. Set $\hat{Q}(s_t, a_t) \leftarrow r(s_t, a_t) + \gamma \max_{a'} \hat{Q}(s_{t+1}, a')$

Solution:

Maintain value of $\hat{Q}$ stable as we observe it more.

Change update rule:

$$\hat{Q}(s_t, a) \leftarrow (1 - \alpha_t) \hat{Q}(s_t, a) + \alpha_t \left(r(s_t, a) + \gamma \max_{a'} \hat{Q}_i(\delta(s_t, a), a') \right)$$

Where

$$\alpha_t = \frac{1}{1 + N[s_t, a]}$$

of times action a taken at state s_t .

Q-Learning – Value Iteration Algorithm

1. Initialize $\hat{Q}(s, a) \leftarrow 0$ for all s, a .
2. Start at s_0
3. For $t = 0, \dots, \infty$:
 - a. **Select action a_t**
 - b. Receive reward $r(s_t, a_t)$
 - c. Set $s_{t+1} \leftarrow \delta(s_t, a_t)$
 - d. Set $\hat{Q}(s_t, a_t) \leftarrow r(s_t, a_t) + \gamma \max_{a'} \hat{Q}(s_{t+1}, a')$

Problem:

How do we select an action a_t ?

Greedily, deterministically, pick action a_t maximizing $\hat{Q}(s_t, a)$?

Deterministic algorithms can be ‘fooled’ by stochastic (or adversarial) inputs.

Q-Learning – Value Iteration Algorithm

1. Initialize $\hat{Q}(s, a) \leftarrow 0$ for all s, a .
2. Start at s_0
3. For $t = 0, \dots, \infty$:
 - a. **Select action a_t**
 - b. Receive reward $r(s_t, a_t)$
 - c. Set $s_{t+1} \leftarrow \delta(s_t, a_t)$
 - d. Set $\hat{Q}(s_t, a_t) \leftarrow r(s_t, a_t) + \gamma \max_{a'} \hat{Q}(s_{t+1}, a')$

Selecting the next action: pick action a_t randomly.

1. Totally randomly?
2. Randomly amongst current best actions?
3. Set $\Pr[\text{choosing action } a \mid s] = \frac{e^{\epsilon \hat{Q}(s, a)}}{\sum_{a'} e^{\epsilon \hat{Q}(s, a')}} \leftarrow$

$$\Pr[a \mid s] \sim e^{\epsilon \hat{Q}(s, a)}$$